



实验指导书

嵌入式 ARM 实验箱

STM32F407 部分

(V2.0)

华清远见教育集团 • 研发中心



170-9108-5953



2306275952/2668462267



www.fsdev.com.cn



support@farsight.com.cn



目 录

目 录.....	I
前 言.....	- 1 -
第 1 章 STM32F407 开发环境搭建.....	- 3 -
1.1 嵌入式 ARM 实验箱资源描述	- 3 -
1.2 ST_LINK V2 仿真器简介.....	- 4 -
1.3 RealView MDK 简介及安装	- 7 -
1.4 RealView MDK 中创建一个新项目	- 17 -
第 2 章 网络开源开发工具及资料获取.....	- 28 -
2.1 STM32cube 简介.....	- 28 -
2.2 STM32cubeMX 的获取.....	- 28 -
2.3 STM32cubeMX 的安装.....	- 29 -
2.4 STM32cubeMX 的使用.....	- 36 -
第 3 章 STM32F407IGTx 实验.....	- 44 -
3.1 跑马灯实验.....	- 44 -
3.2 蜂鸣器实验.....	- 59 -
3.3 直流电机驱动实验.....	- 70 -
3.4 舵机实验.....	- 86 -
3.5 步进电机实验.....	- 95 -
3.6 UART 串口 1 数据收发实验	- 103 -
3.7 继电器实验.....	- 115 -
3.8 SysTick 系统滴答实验.....	- 140 -



3.9	按键扫描数码管显示实验	- 151 -
3.10	ADC 模数转换实验	- 164 -
3.11	IR 红外线接收实验	- 175 -
3.12	4.3 寸 TFT 彩屏液晶屏实验	- 193 -
3.13	4.3 寸 TFT 彩屏触摸屏实验	- 200 -
3.14	温度采集实验	- 210 -
3.15	光电开关实验	- 218 -
3.16	UCOSII 操作系统创建 1 个任务实验	- 224 -
3.17	UCOSII 操作系统创建多任务实验	- 229 -
3.18	UCOSII 操作系统-信号量	- 235 -
3.19	UCOSIII 操作系统移植	- 242 -
3.20	UCOSIII 操作系统创建 1 个任务实验	- 248 -
3.21	UCOSIII 操作系统创建多任务实验	- 252 -
3.22	UCOSIII 操作系统-信号量	- 259 -
3.23	综合测试例程	- 265 -



前 言

作为 Cortex M3 市场的最大占有者，ST 公司在 2011 年又推出基于 Cortex M4 内核 STM32F4 系列产品，相对与 STM32F1/F2 等 Cortex M3 产品，STM32F4 最大的优势，就是新增了硬件 FPU 单元以及 DSP 指令，同时，STM32F4 的主频也提高了很多，达到 168Mhz（可获得 210DMIPS 的处理能力），这使得 STM32F4 尤其适用于需要浮点运算或 DSP 处理的应用，也被称之为：DSC，具有非常广泛的应用前景。STM32F4 相对于 STM32F1，主要优势如下：

- 1、 更先进的内核。STM32F4 采用 Cortex M4 内核，带 FPU 和 DSP 指令集，而 STM32F1 采用的是 Cortex M3 内核，不带 FPU 和 DSP 指令集。
- 2、 更多的资源。STM32F4 拥有多达 192KB 的片内 SRAM，带摄像头接口（DCMI）、加密处理器（CRYP）、USB 高速 OTG、真随机数发生器、OTP 存储器等。
- 3、 增强的外设功能。对于相同的外设部分，STM32F4 具有更快的模数转换速度、更低 ADC/DAC 工作电压、32 位定时器、带日历功能的实时时钟（RTC）、IO 复用功能大大增强、4K 字节的电池备份 SRAM 以及更快的 USART 和 SPI 通信速度。
- 4、 更高的性能。STM32F4 最高运行频率可达 168Mhz，而 STM32F1 只能到 72Mhz；STM32F4 拥有 ART 自适应实时加速器，可以达到相当于 FLASH 零等待周期的性能，STM32F1 则需要等待周期；STM32F4 的 FSMC 采用 32 位多重 AHB 总线矩阵，相比 STM32F1 总线访问速度明显提高。
- 5、 更低的功耗。STM32F40x 的功耗为：238uA/Mhz，其中低功耗版本的 STM32F401 更是低到：140uA/Mhz，而 STM32F1 则高达 421uA/Mhz。

STM32F4 家族目前拥有：STM32F40x、STM32F41x、STM32F42x 和 STM32F43x 等几个系列，数十个产品型号，不同型号之间软件和引脚具有良好的兼容性，可方便客户迅速升级产品。其中，STM32F42x/43x 系列带了 LCD 控制器和 SDRAM 接口，对于想要驱动大屏或需要大内存的朋友来说，是个不错的选择。目前 STM32F4 这些芯片型号都已量产，可以方便的购买到，不过目前来说，性价比最高的是 STM32F407，



本手册我们将以 STM32F407 为例，向大家讲解 STM32F4 的学习。

这些实例涵盖了 STM32F4 的绝大部分内部资源，并且提供很多实用级别的程序，如：内存管理、拼音输入法、手写识别、图片解码、IAP、音乐播放、视频播放等。所有实例在 MDK 编译器下编译通过，大家只需下载程序到华清远见 STM32 开发板，即可验证实验。

不管你是一个 STM32 初学者，还是一个老手，本手册都非常适合。尤其对于初学者，本手册将手把手的教你如何使用 MDK，包括新建工程、编译、仿真、下载调试等一系列步骤，让你轻松上手。

本手册的实验平台是华清远见 FS4412/6818M4 实验箱，有这款实验箱的朋友则直接可以拿本手册配套的光盘上的例程在开发板上运行、验证。而没有这款开发板而又想要的朋友，可以上淘宝购买。当然你如果有了一款自己的开发板，而又不想买再买，也是可以的，只要你的板子上有华清远见 FS4412M4 实验箱上的相同资源（需要实验用到的），代码一般都是可以通用的，你需要做的就只是把底层的驱动函数（比如 IO 口修改）稍做修改，使之适合你的开发板即可。

作者力求将本手册的内容写好，由于能力有限，手册中但难免会有出错的地方，如果大家发现手册中有什么错误的地方，希望能及时向我们反馈，再次感谢！



第 1 章 STM32F407 开发环境搭建

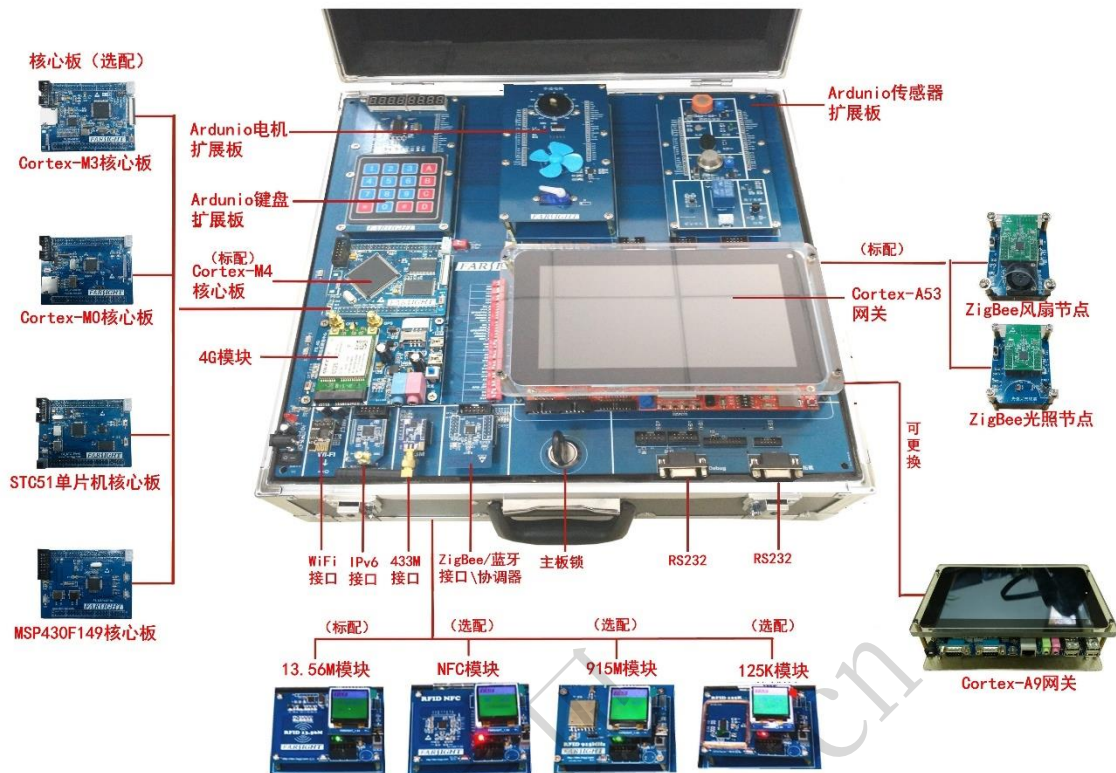
学习 STM32 的第一件事就是搭建编程环境，如今有非常多的 IDE 及调试软件/仿真硬件，像 IAR、keil、eclipse 等，这里笔者将提供 Realview MDK 方案给予学习者。Realview MDK 环境以其优越的性价比得到了快速的推广。越来越多的开发人员开始选择 MDK 作为自己的开发环境。尤其当意法半导体公司推出图形化编程软件 STM32CubeMx 之后，它跟 MDK 的完美结合，使得开发者能够相比之前的开发更快速，更高效。

本章主要介绍它的安装、使用及配置方法，内容主要有：

- (1) 仿真器简介。
- (1) 开发环境 MDK 简介及安装。
- (2) 创建新项目工程。
- (3) 工程基本配置。

1.1 嵌入式 ARM 实验箱资源描述

华清远见 FS4412/6818M4 是由华清远见研发中心专为培训教学和项目研发定制的高性能综合嵌入式开发平台。这里主要介绍该平台上的所有资源，能让开发者快速的找到对应设备的位置，方便开发。



1.2 ST_LINK V2 仿真器简介

1.2.1 ST_LINK V2 简介

ST-LINK/V2 是 STM8 和 STM32 微控制器系列的**在线调试器和编程器**。单线接口模块 (SWIM) 和串行线调试 (SWD) 接口用于与应用板上的 STM8 和 STM32 微控制器通讯。

STM32 的应用使用 USB 全速接口与 Atollic, IAR, Keil or TASKING 等集成开发环境通讯。

功能特征:

- 5V 电源, 通过 USB 连接器来供电
- USB2.0 全速兼容接口
- USB standard A 到 mini B 排线
- SWIM 特殊特征:
 - ✧ SWIM 接口支持 1.65V 到 5.5V 应用电压
 - ✧ 支持 SWIM 低速和高速模式
 - ✧ SWIM 编程速率: 在低速模式, 9.7Kbytes/s; 在高速模式, 12.8Kbytes/s
- JTAG 特殊特征:



- ✧ JTAG 接口支持 1.65V 到 3.6V 应用电压，并能承受 5V 的输入
- 支持直接固件升级 (UFU)
- 与 PC 通讯的状态 LED 指示
- 操作温度 0 到 50°C

详细信息可参见网址: <http://www.st.com/internet/evalboard/product/251168.jsp>

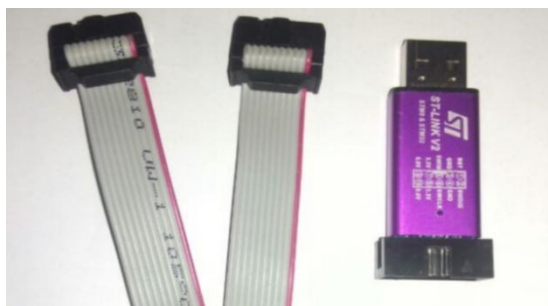


图 ST-LINK 仿真器

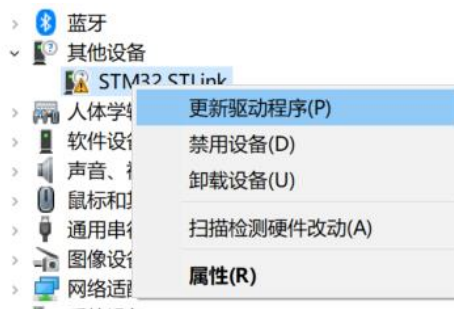
1.2.2 ST_LINK V2 驱动安装

驱动的存放路径: 【华清远见-嵌入式 ARM 实验箱资料-III\华清远见-STM 系列资料\Cortex-M4 部分\工具软件\ST-Link 驱动及工具】

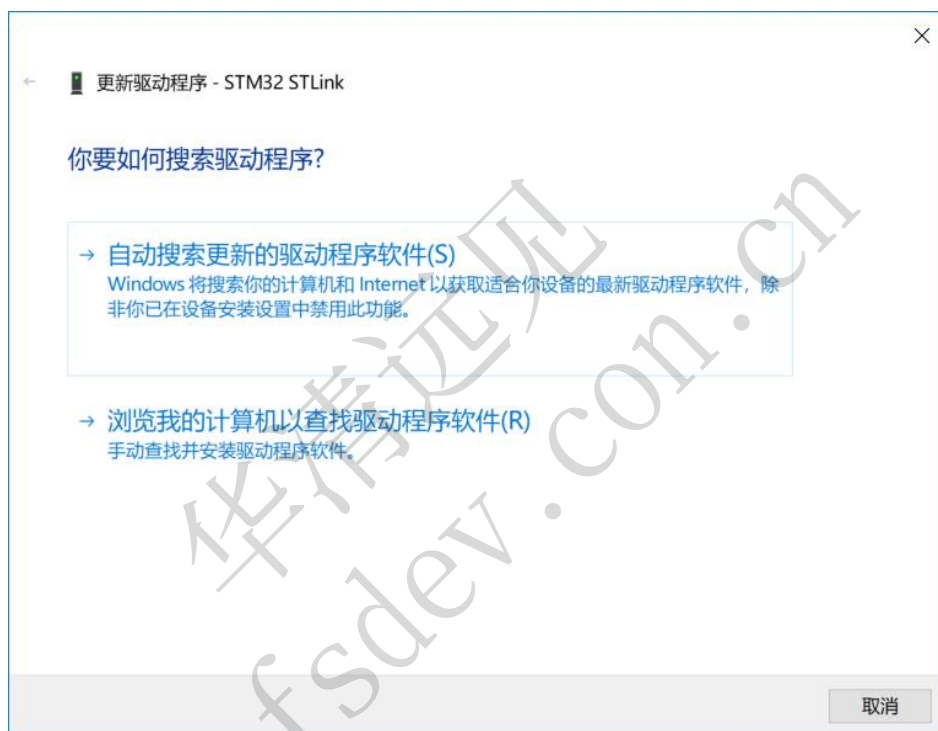
- (1) 在电脑上插入 ST_LINK, 打开设备管理器, 会看到叹号标志



- (2) 右键点击 STM32 STLink, 选择更新驱动程序



- (3) 浏览我的计算机以查找驱动程序软件，找到光盘中的驱动文件，按照电脑系统来实际选择驱动程序版本





1.3 RealView MDK 简介及安装

1.3.1 RealView MDK 简介

RealView MDK 是 ARM 公司最先推出的基于微控制器的专业嵌入式开发工具。它采用了 ARM 的最



新技术工具 RVCT，集成了享誉全球的 μ Vision IDE，因此特别易于使用，同时具备非常高的性能。uVision IDE 是一个窗口化的软件开发平台，它集成了功能强大的编辑器、工程管理器以及各种编译工具（包括 C 编译器、宏汇编器、链接/装载器和十六进制文件转换器）。uVision IDE 包含以下功能组件，能加速嵌入式应用程序开发过程。

功能强大的源代码编辑器：

- (1) 可根据开发工具配置的设备数据库
- (2) 用于创建和维护工程的工程管理器
- (3) 集汇编、编译和链接过程于一体的编译工具
- (4) 用于设置开发工具配置的对话框
- (5) 真正集成高速 CPU 及片上外设模拟器的源码级调试器
- (6) 高级 GDI 接口，可用于目标硬件的软件调试和 Keil ULINK 仿真器的连接
- (7) 用于下载应用程序到 Flash ROM 中的 Flash 编程器
- (8) 完善的开发工具手册、设备数据手册和用户向导

uVisionIDE 开发工具环境:

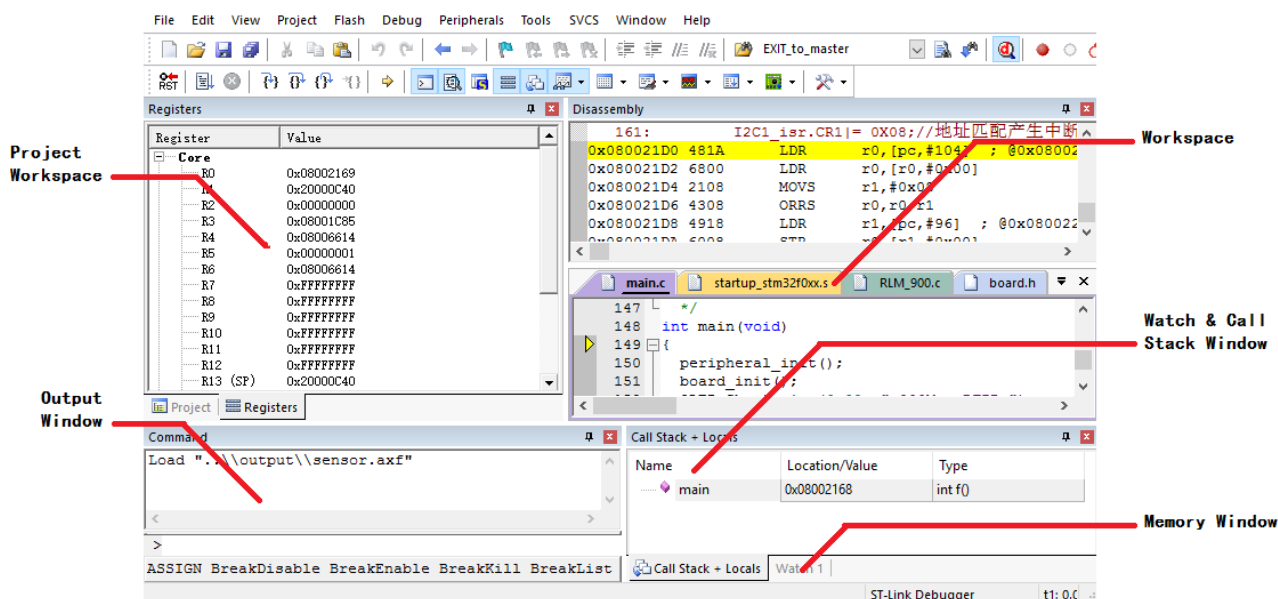


图 MDK 工作界面解析图

uVisionIDE 提供两种工作模式：

(4) 编译模式 (Build Mode)：用于维护工程文件和生成应用程序；

(5) 调试模式 (Debug Mode)：既可以用功能强大的 CUP 和外设仿真器测试程序，也可以使用调试器经 Keil ULINK USB- JTAG 适配器 (或其他 AGDI 驱动器) 连接目标系统来测试程序。例如：colink 仿真器能用于下载应用程序到目标系统的 Flash ROM 中并且可以进行仿真操作。

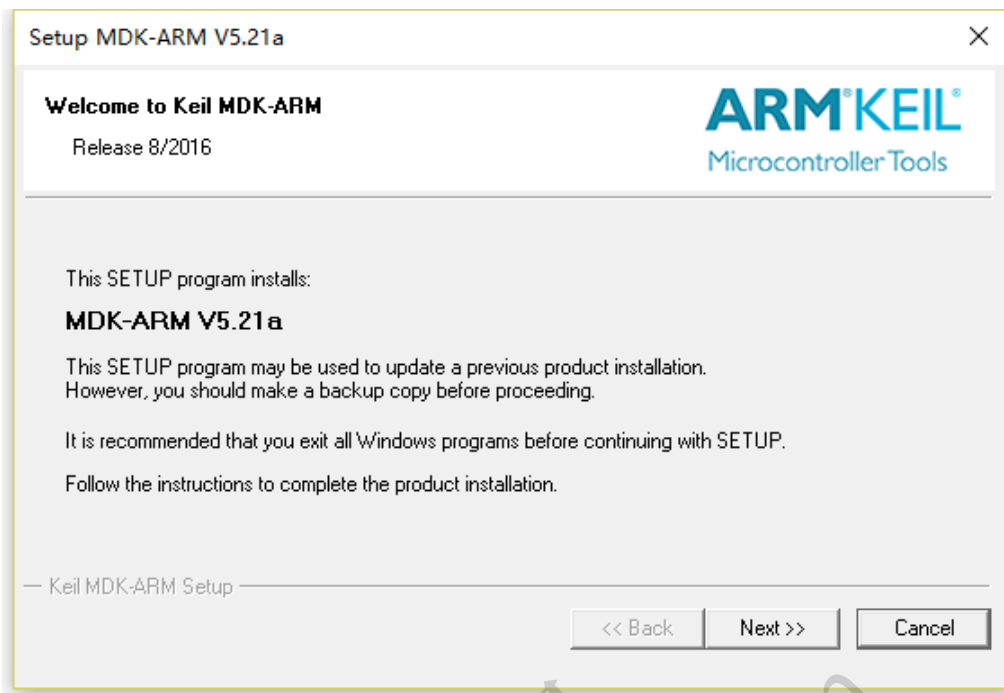
1.3.2 RealView MDK 安装

本节主要详细讲述 RealView MDK 的安装步骤，以及在安装过程中遇到的问题的解决方法。

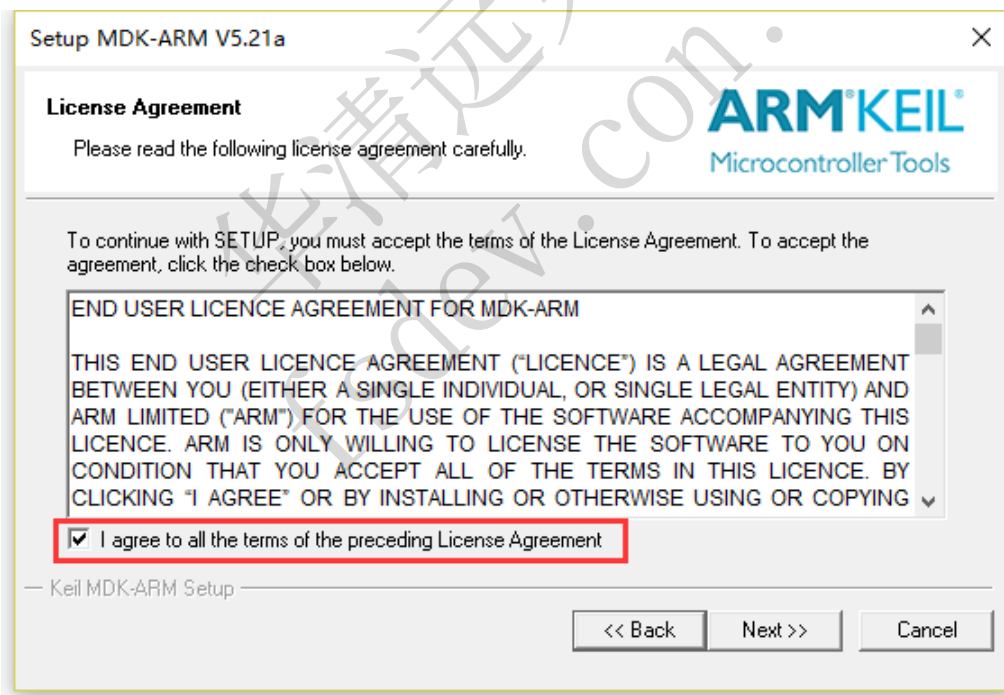
安装的软件路径是：

【华清远见-嵌入式 ARM 实验箱资料-III\华清远见-STM 系列资料\Cortex-M4 部分\工具软件\keil5】

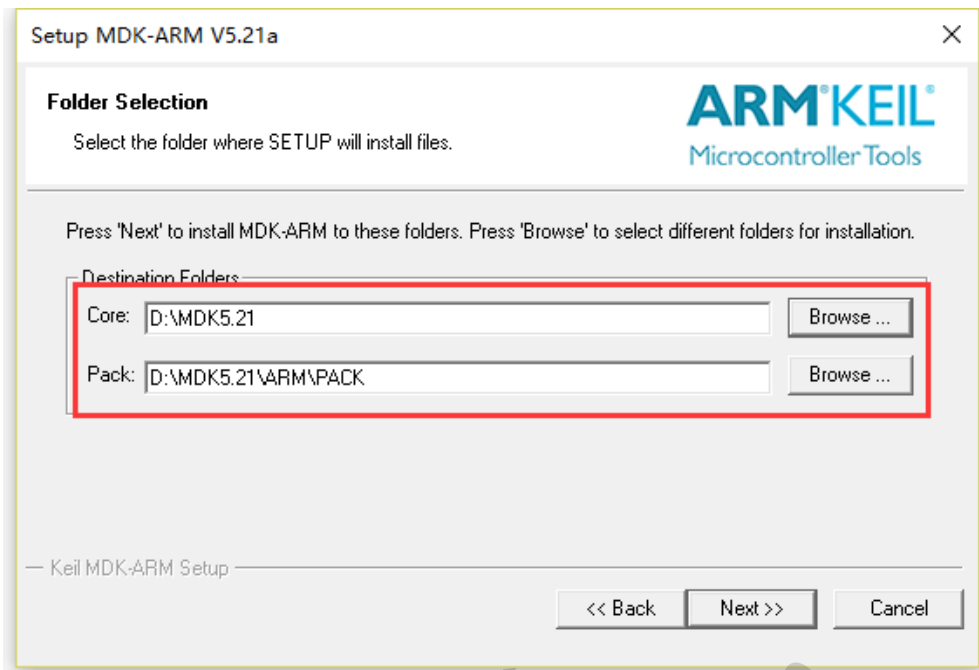
(4) 以管理员权限运行 mdk521a.exe



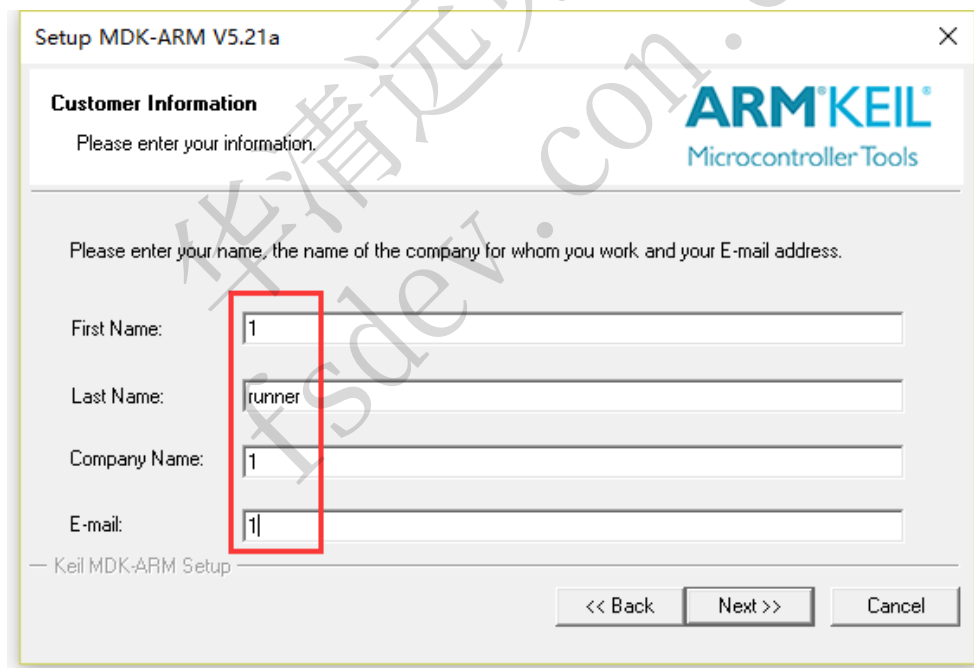
- (5) 选择“同意” 点击 “Next”



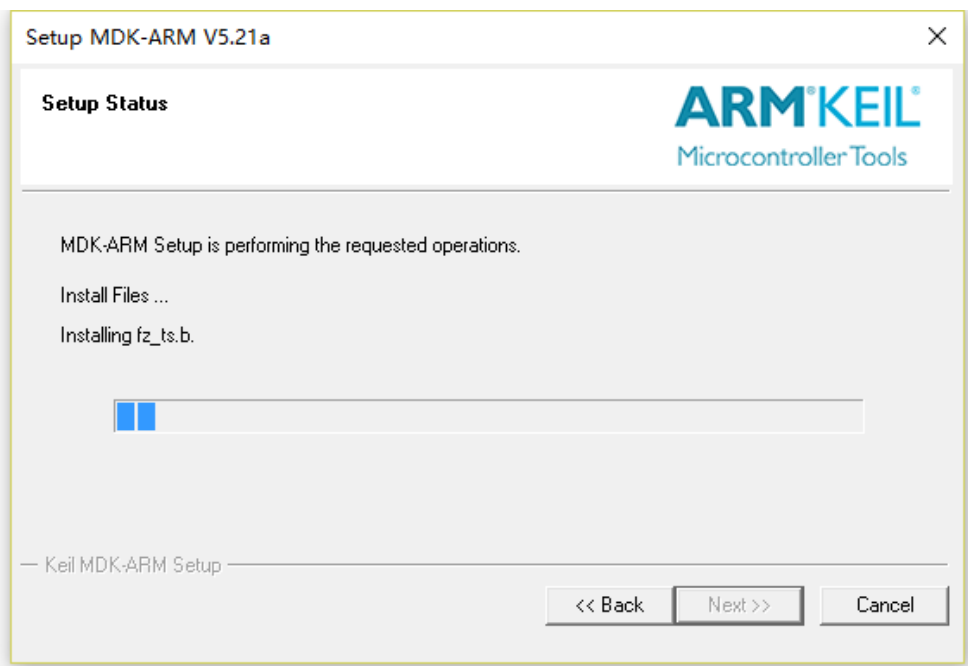
- (6) 根据自己的需要选择对应的安装路径，点击 “Next”



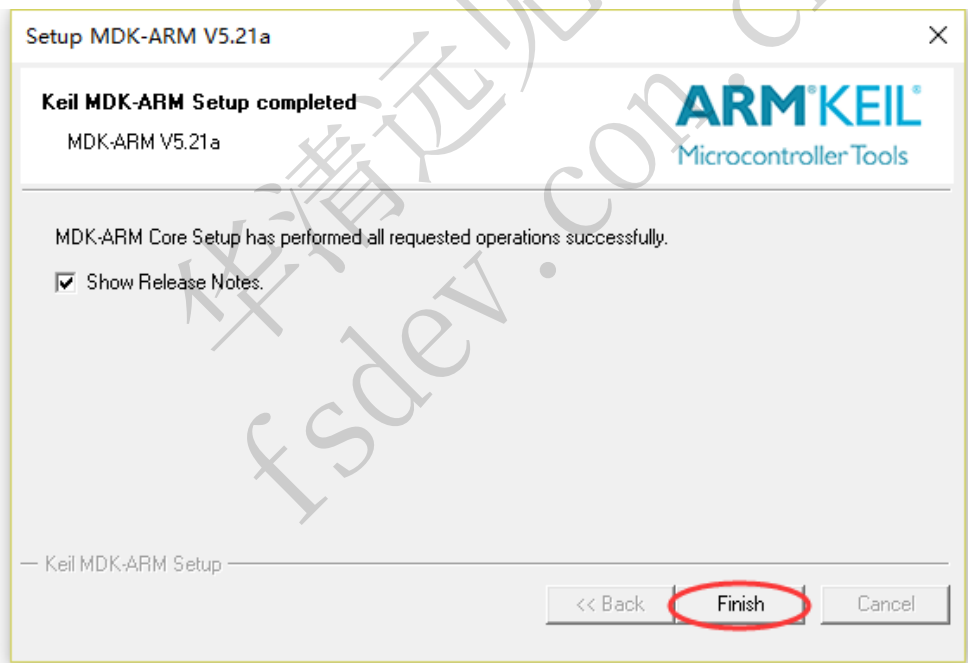
(7) 随意填写相关信息，点击“Next”



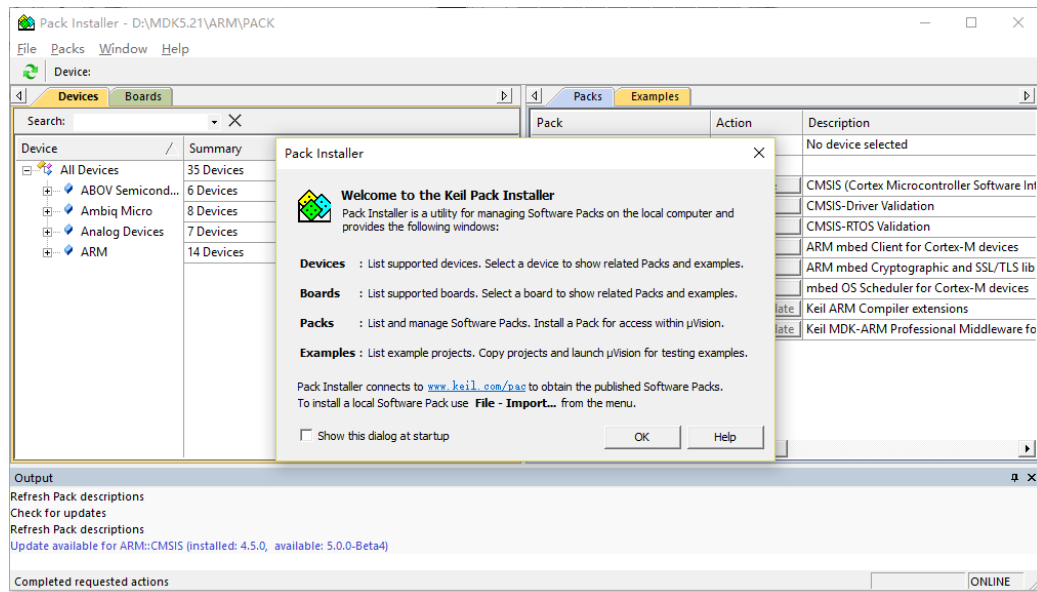
(8) 等待安装成功



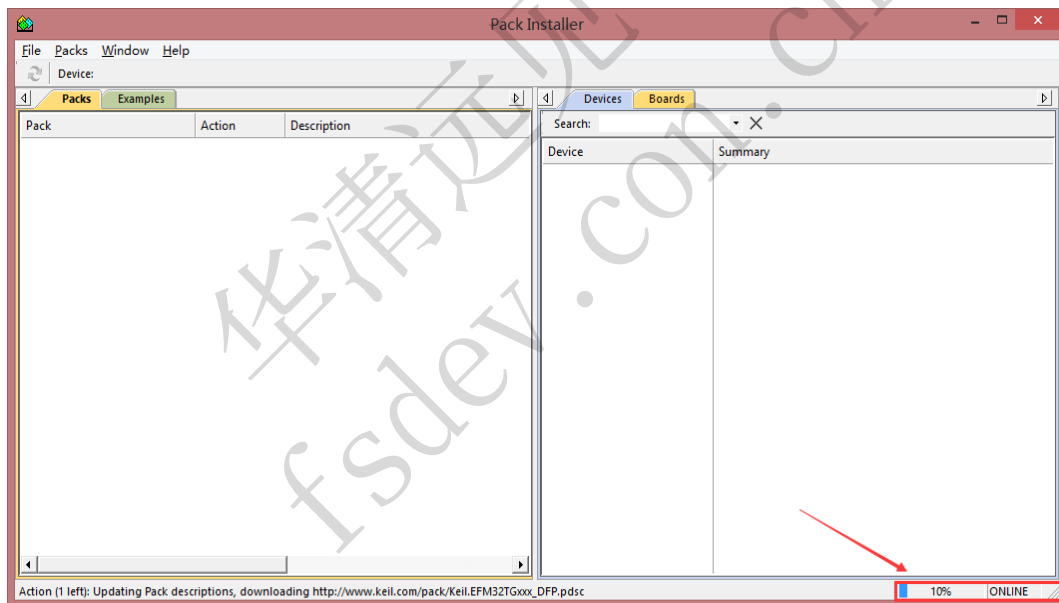
- (9) 这里的“发布信息显示”可选可不选，点击“Finish”



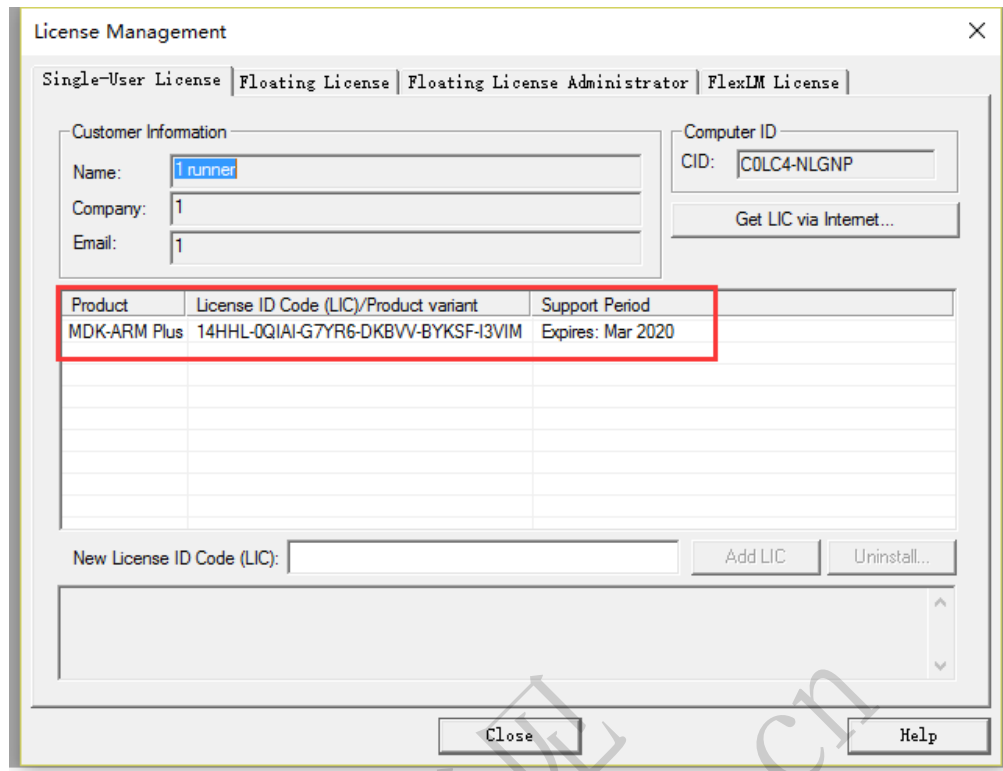
- (10) 完成安装后，会自动下载 DFP 包，如果网速允许的话，可以等待其安装完成，如果网速不允许，直接关闭该界面，我们会提供 M4 和 M0 的 DFP 安装包，直接安装那个就可以了。DFP (Device Family Pack)，表明软件包包含对单片机设备的支持，安装对应的 DFP 包之后，在创建工程的时候就可以选择对应的芯片了，如果没有这个 DFP 包，是选择不了芯片的。



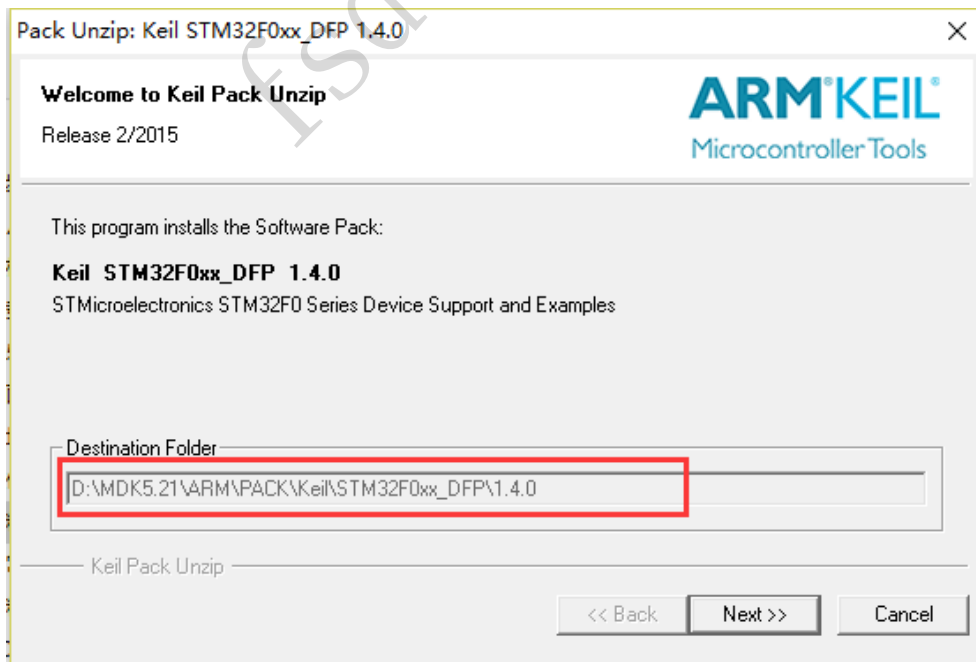
(11) 等待下载完成即可（网速允许可以操作该步骤，也可不操作）



(12) 安装完成后，需要对 MDK 进行版本注册，具体注册步骤自行百度。注册后结果如下：

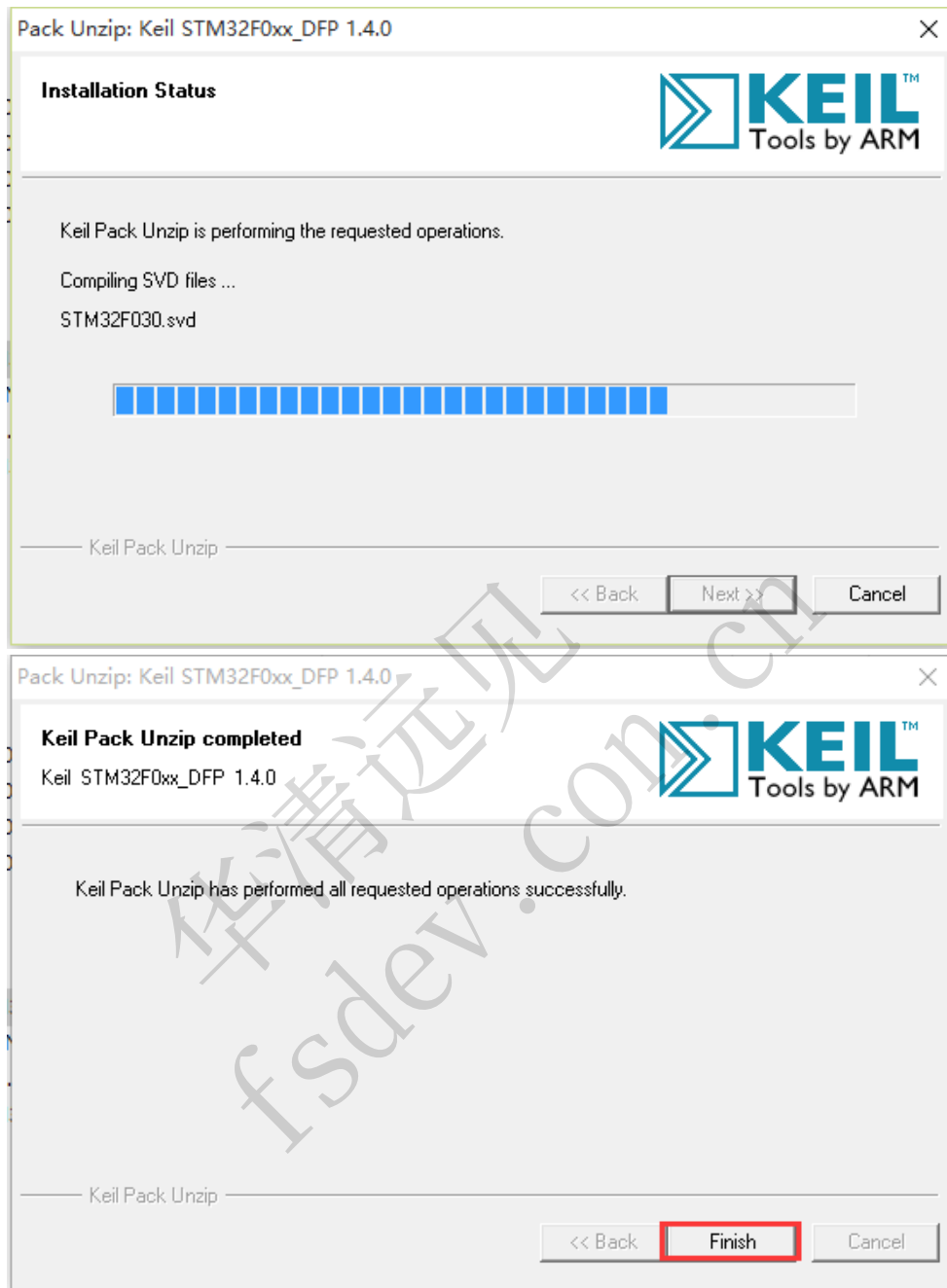


- (13) 在【光盘\华清远见-嵌入式 ARM 实验箱资料-III\华清远见-STM 系列资料\Cortex-M4 部分\工具软件\keil5】下，有为客户提供的 STM32F0xx_DFP.1.4.0.pack 和 Keil.STM32F1xx_DFP.2.0.0.pack 还有 STM32F4xx_DFP.2.7.0.pack 安装包，直接点击安装这几个文件即可。安装过程中注意，安装路径必须选择前面安装的 MDK 目录（安装时它会自己寻找路径的）。

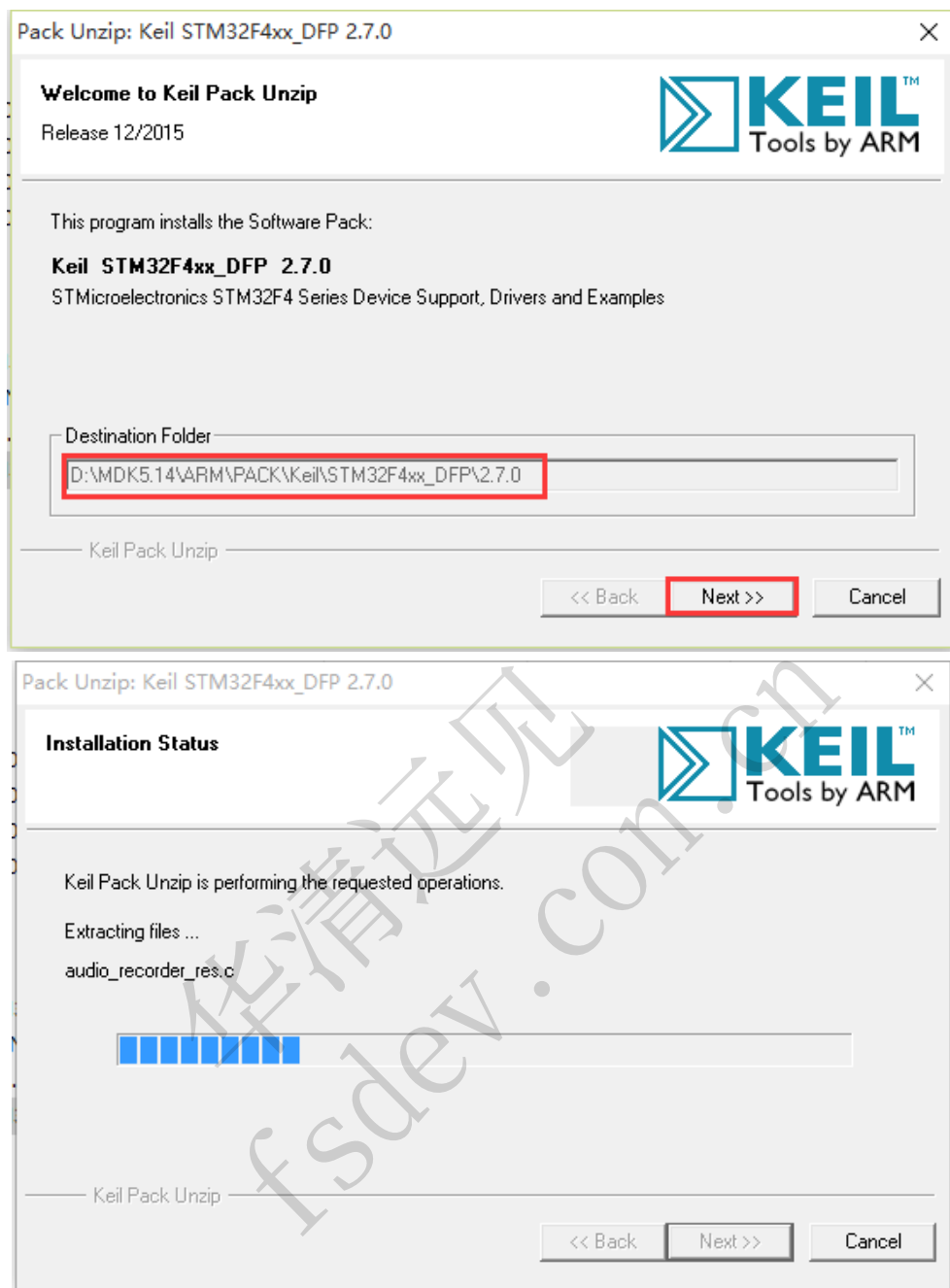


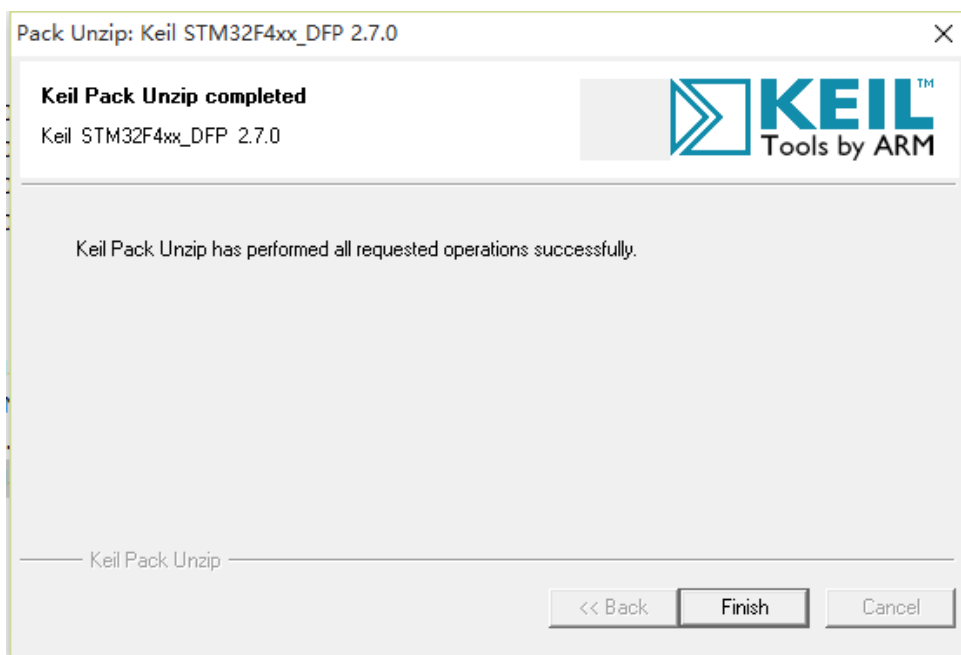


(14) 点击 Next 进行下一步操作，等待安装完成，然后点击“Finish”完成。



(15) 同样的方法安装 M4 和 M3 的 DFP 包





- (16) 至此，对 MDK 的安装工作就算完成了，安装过程中如果遇到问题请仔细核对操作步骤是否正确，同时也欢迎跟我们及时沟通，希望能尽快帮您解决问题，技术支持联系方式见首页。

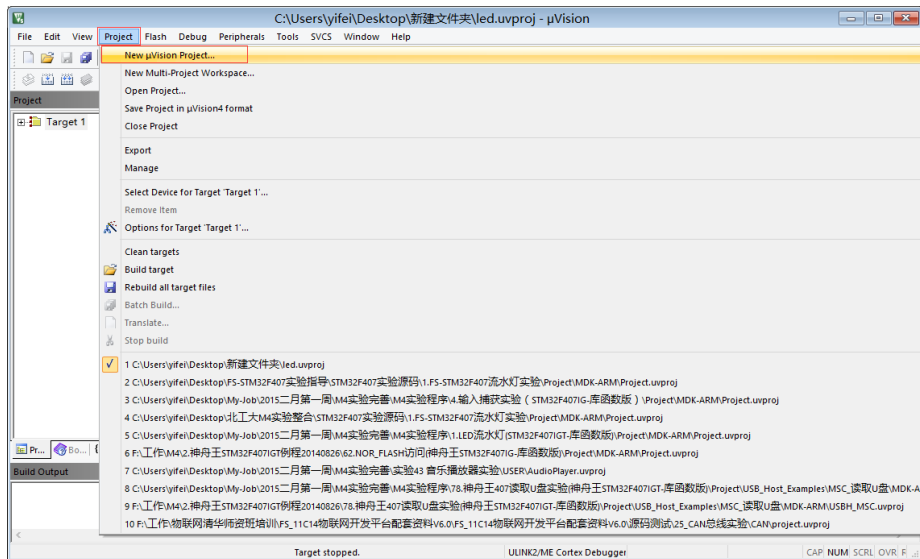
1.4 RealView MDK 中创建一个新项目

要为某个目标系统开发一个新应用程序，必须先新建一个新项目工程。新建项目具体步骤下面将作详细介绍。 μ Vision IDE 的工程开发流程与其它软件开发流程大体上相同。开发工具的开发大致流程如下：

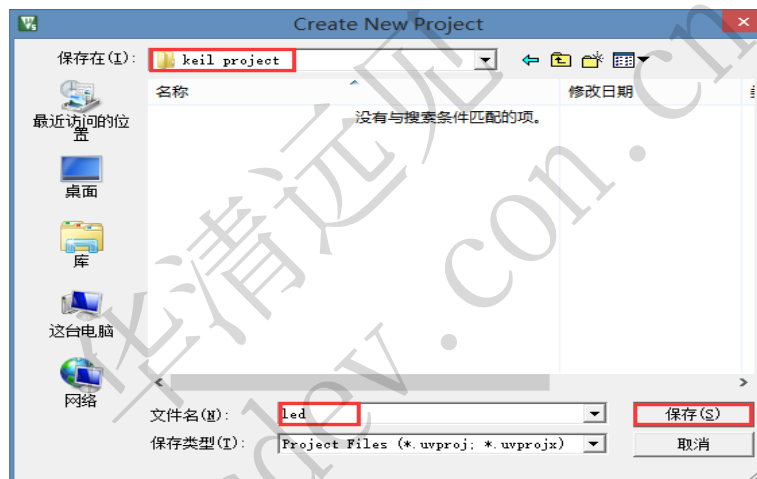
- (2) 新建一个工程，从设备库中选择目标芯片，配置工编译器环境。
- (3) 用 C 或汇编语言编写源文件。
- (4) 编译目标应用程序。
- (5) 修改源程序中的错误。
- (6) 测试链接应用程序。

1.4.1 创建新项目工程

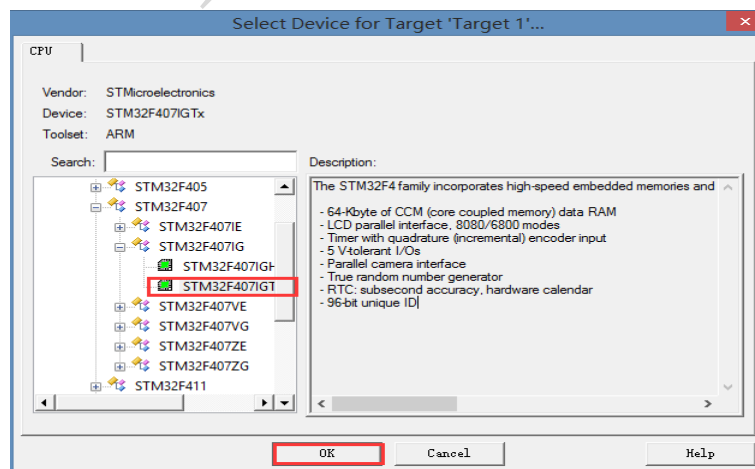
- (1) 新建一个文件夹用于存放工程文件，此处文件名为“LED”，放在 D 盘（可自行选择）
- (2) 单击 Project – New μ Vision Project.... 菜单项。



(3) 找到刚才新建的文件夹路径，并添加工程名“LED”，保存



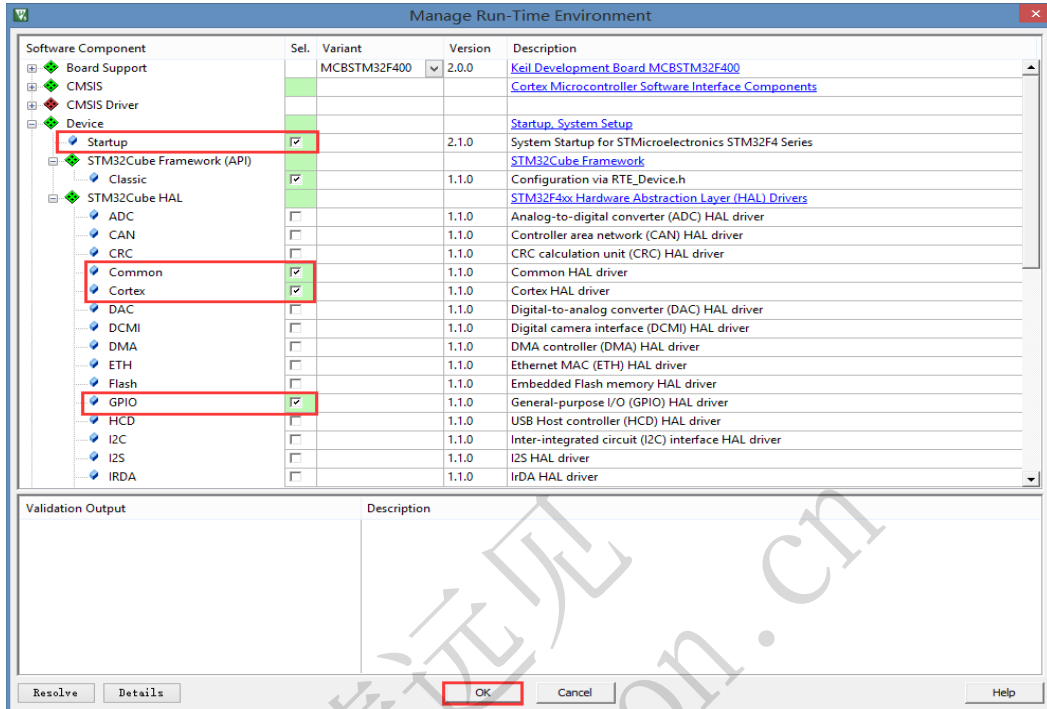
(4) 选择对应的 CPU 芯片 (STM32F407IGTx)，点击“OK”



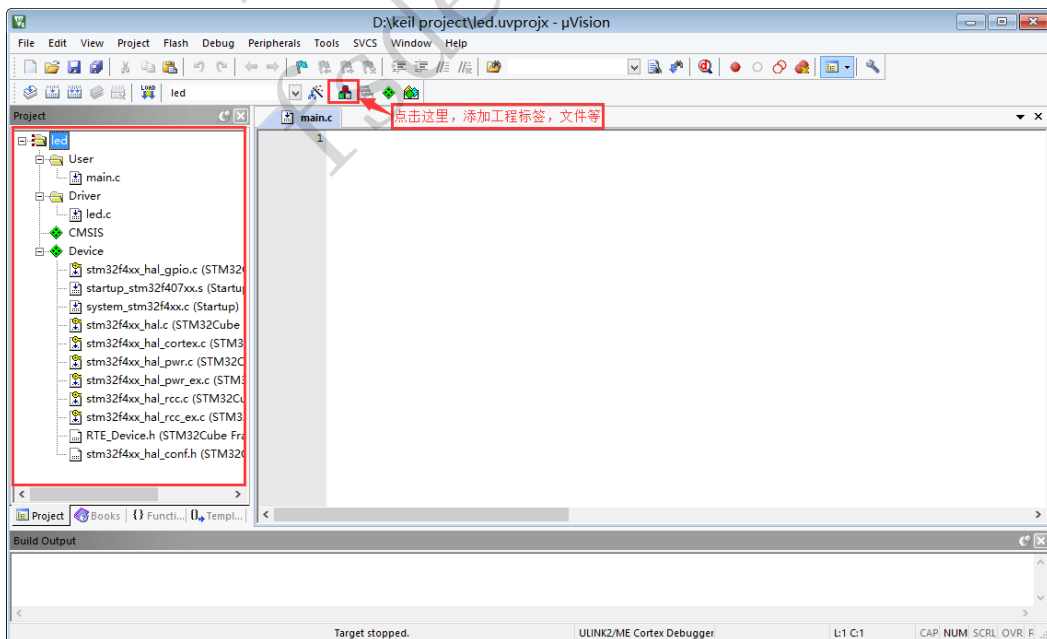
(5) 选择完 CPU 后，会弹出选择 lib 文件等，选择自己需要的文件。主要是 CMSIS 和 Device



(这里的文件根据自己的项目的需要添加, 有时候可能会有些警告, 是因为有些依赖文件没有选, 根据提示选择上去就行)。最后点击 OK, 新工程就快完成了。



(6) 接下来是, 设置一下 listing 和 Output 的路径, 添加其他源文件。这个大家都懂 (新工程默认 target 中时钟是写 72M)



工程创建大概就是这个样子。在实际的开发中, 更多的是在现有的例程基础上进行更改, 这样会避

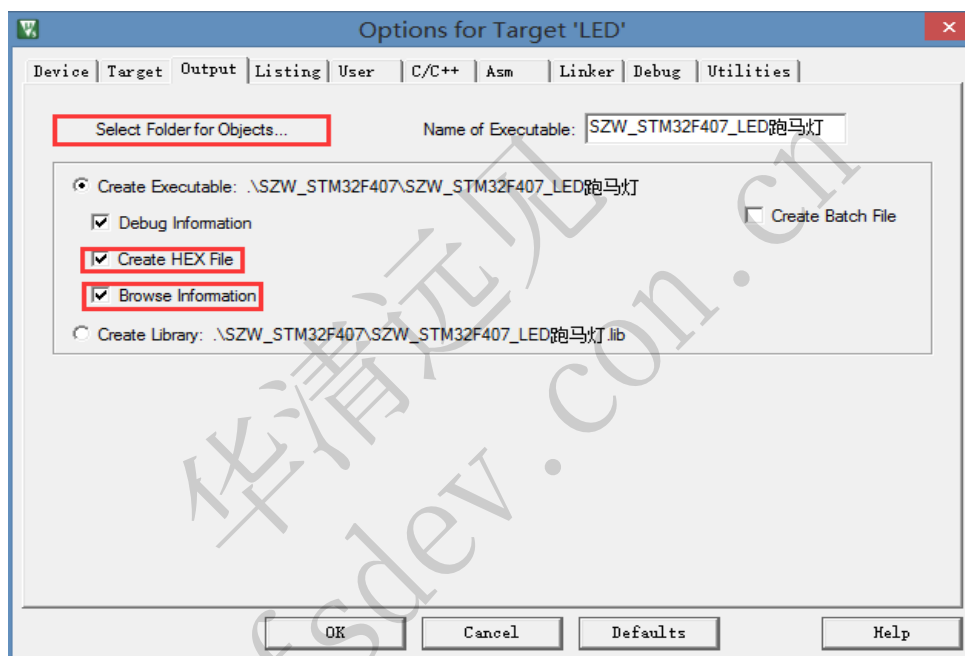


免很多没必要的疏忽，节约时间，提高开发效率。

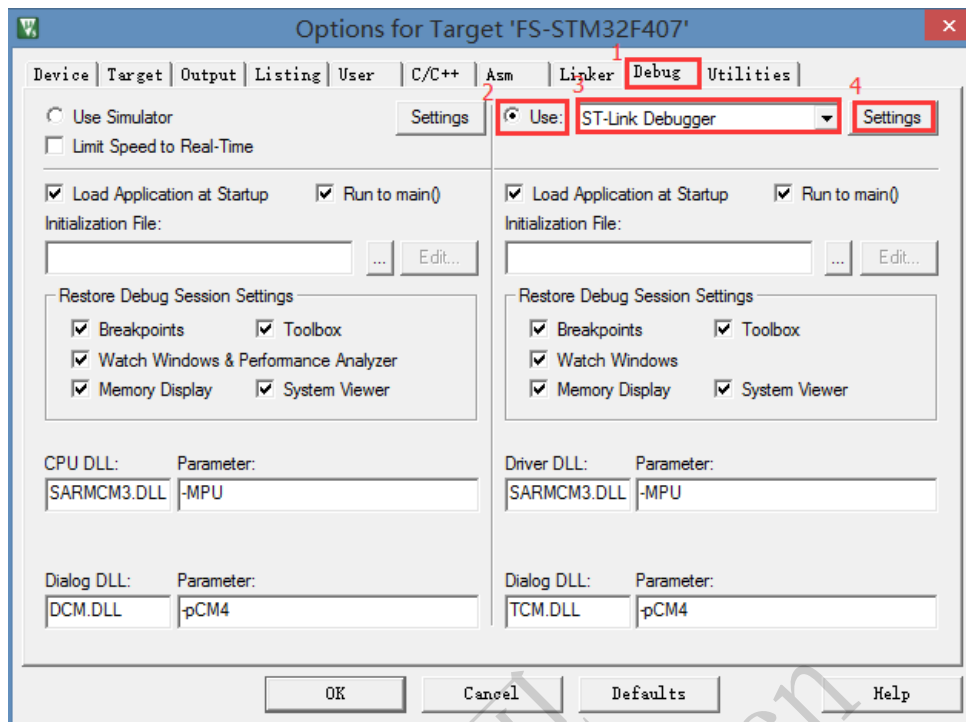
1.4.2 工程基本配置

μVision IDE 允许用户根据目标硬件的实际情况对工程进行配置。通过点击目标工具栏图标或单击菜单项 Project->Options for Target，在弹出的 Target 页面可指定目标硬件和所选择设备片内组件的相关参数，下图为本示例的相关设置。

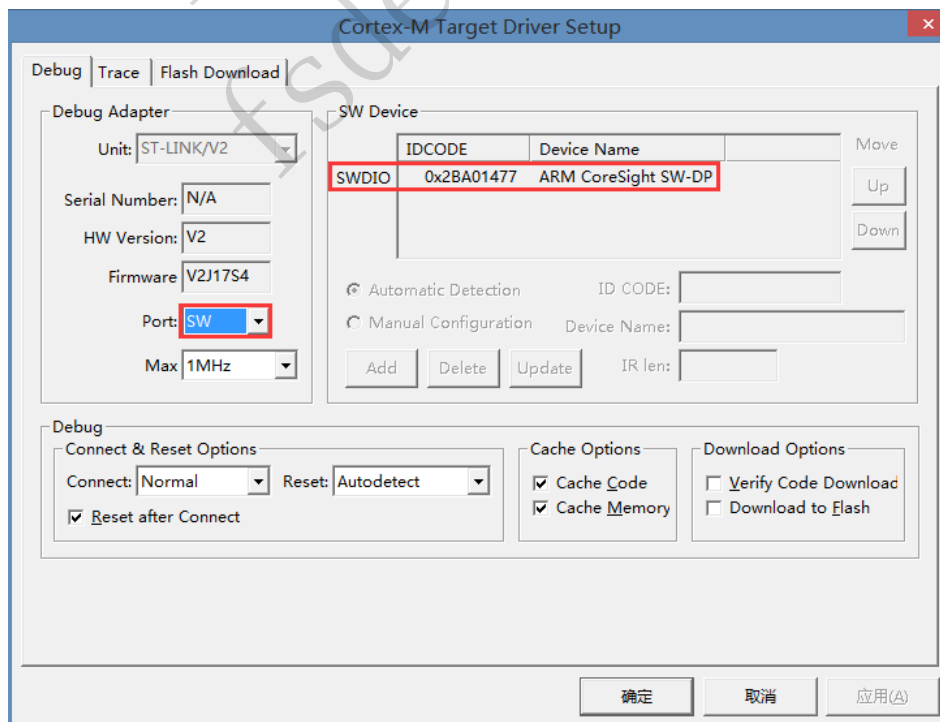
- (1) 指定输出文件存放文件夹（创建 HEX 文件）



- (2) 选择 Debug，勾选 Use，选择仿真工具为 ST-Link Debugger，点击 Setting

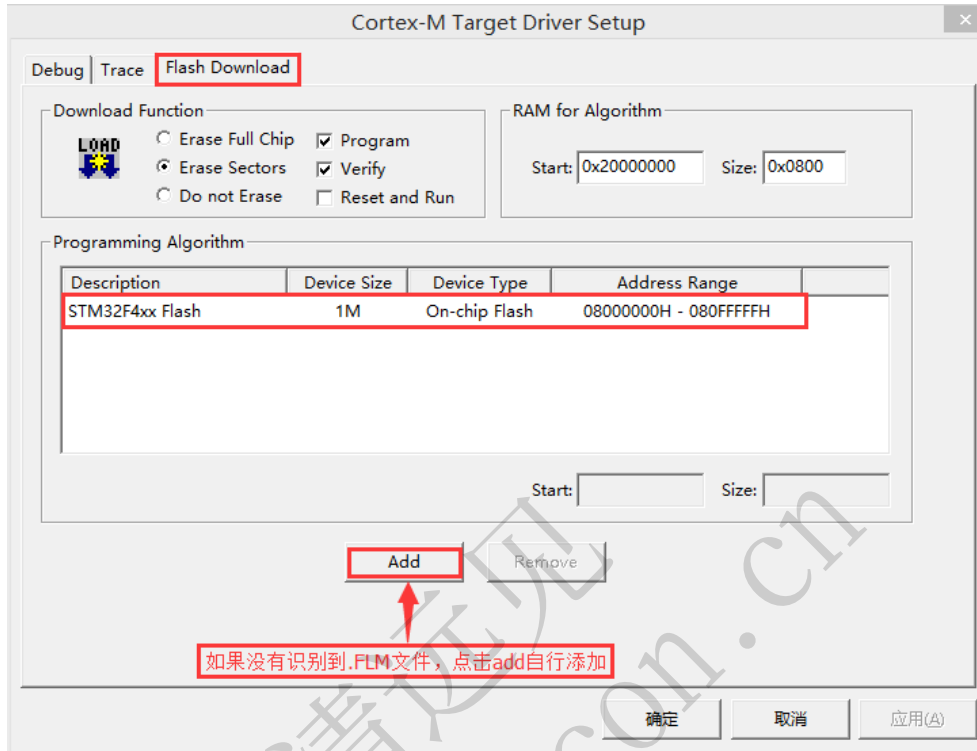


- (3) 连接开发板，并上电
- (4) Port 选择 SW，其它值默认，可以看到 SWDIO 栏，自动识别到相关信息（如果没有识别到信息，可以重新启动 Keil 软件，继续打开刚才的界面，如果还没有识别到信息请检查 ST-link 驱动是否正确安装）

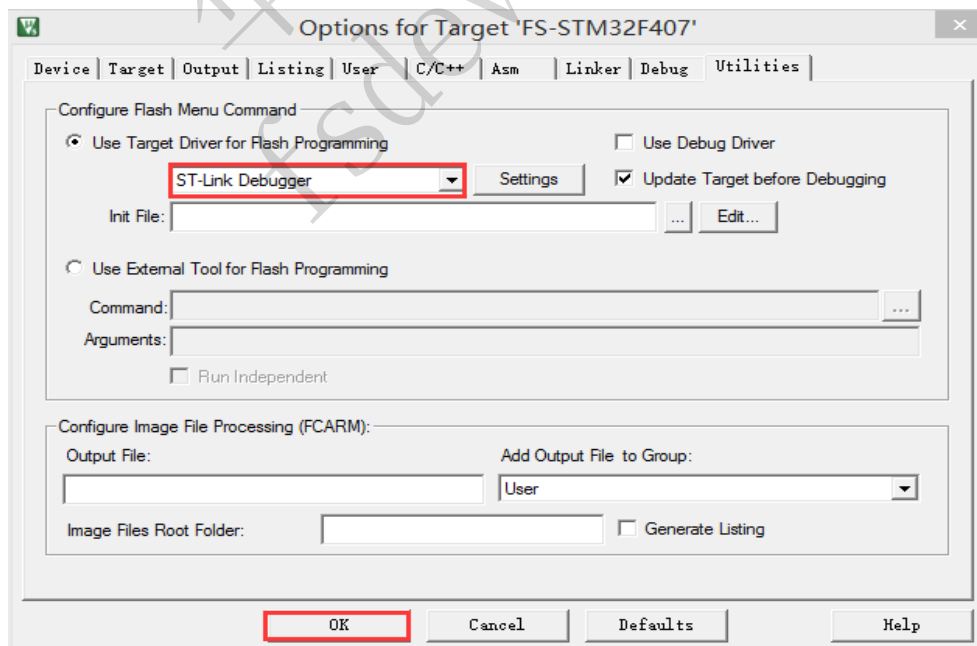




- (5) 点击 Flash Download, 可以看到已经识别到之前添加的.FLM 文件（如果没有请点击 add 自行添加），然后点击确定



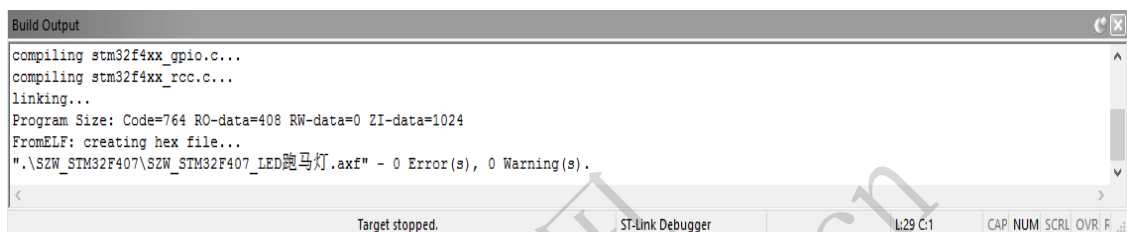
- (6) 选择 Utilities, 将目标驱动选择为 ST-Link, 点击 OK 即可



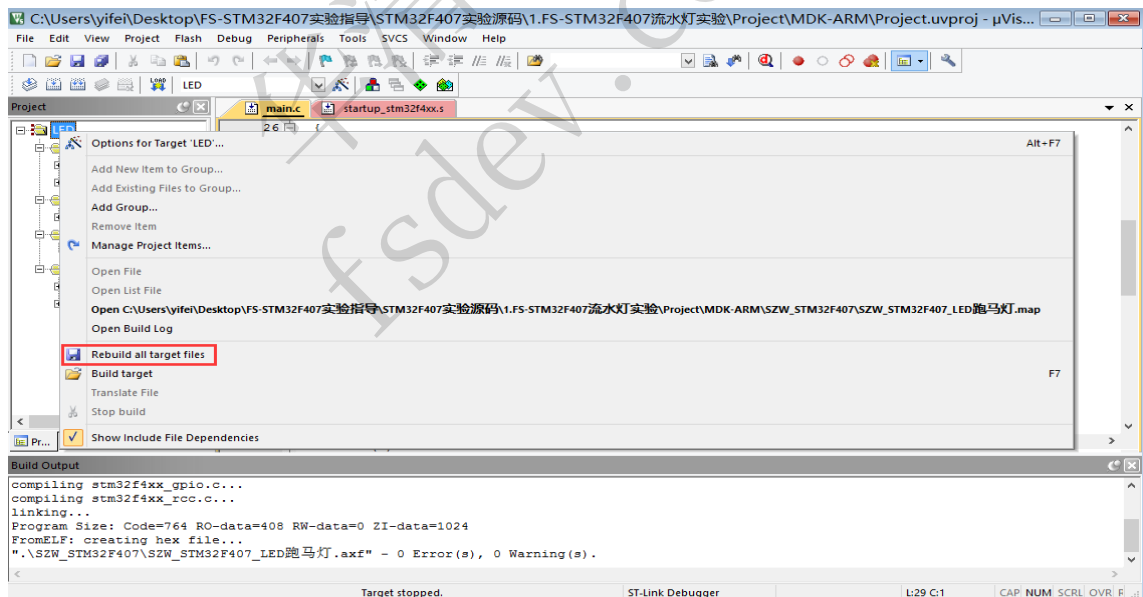


1.4.3 编译、链接

工程已经创建完成，工程文件已经添加到工程目录下，接下来的工作是编译链接工程（先打开华清示例编译一下），单击工具栏中 Build Target  图标可编译链接工程文件。如果源程序中存在语法错误，μVision 则会在 Output Window->Build 窗口中显示出错误和警告信息。双击提示信息所在行，就会在 μVision 编辑窗口里打开并显示相应的出错源文件，光标会定位在该文件的出错行上，以方便用户快速定位出错位置。



注意：如果要对工程全部重新编译和链接，选中工程名字，点右键选择“rebuild all target files”。或者点击按钮 。

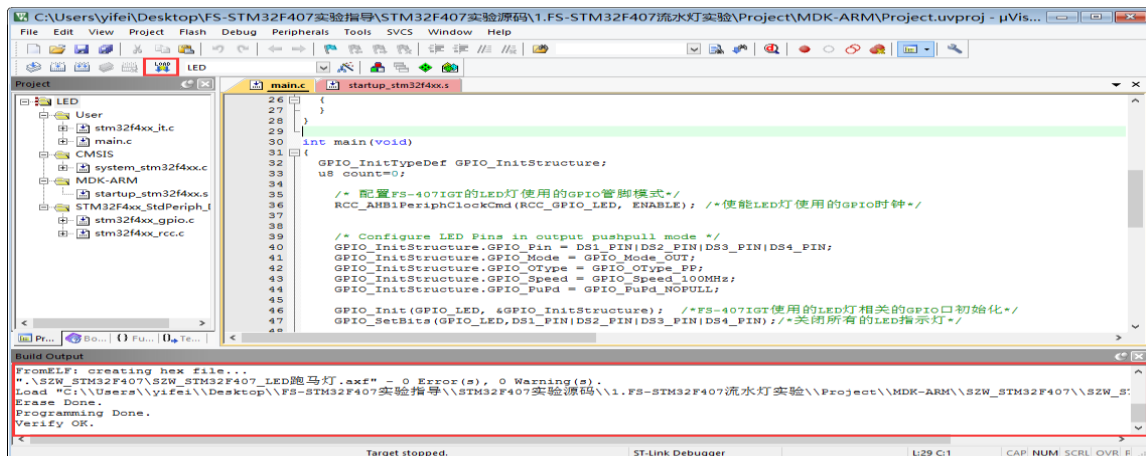


1.4.4 下载程序

μVision IDE 调试器提供了软件仿真和 GDI 驱动两种调试模式，采用 ST-Link 仿真器调试时，首先将集成环境与 ST-Link 仿真器连接，按照前面 1.4.2 小节中的工程基本配置方法对要调试的工程进行配置后，



点击 Flash -> Download 菜单项（或者点击按钮 ）可将目标文件下载到目标系统的指定存储区中，文件下载后即可进行在线仿真调试。



1.4.5 仿真调试

1. 仿真运行

 编译链接或下载完成后，就可使用μVision IDE 的调试器进行调试了。选择 Debug 菜单里的选项 **Start/Stop Debug Session** 或者点击工具栏里的对应图标进入调试模式。μVision IDE 将会初始化调试器并启动程序运行到主函数。

 选择 Debug 菜单中 GO 命令或者点击工具栏里的运行图标以启动程序运行。观察我们的开发板上的 LED1 闪烁。



选择 Debug 菜单里选择 Stop 命令或者点击工具栏里停止图标来中止程序运行。

2. 单步调试和断电



选择 Debug 菜单中的 Insert/Remove Breakpoints 选项，或者单击鼠标右键选择 Insert/Remove Breakpoints 选项，在主函数的开始设置断点。



选择 Debug 菜单或者工具栏里的 Reset CPU 命令对 CPU 进行复位。如果已经中止了程序，可使用 Run 命令启动程序运行。uVision IDE 会在断点处中止程序运行。



可以使用调试工具栏里的 Step 按钮单步运行程序。当前的指令会用黄色箭头标记出



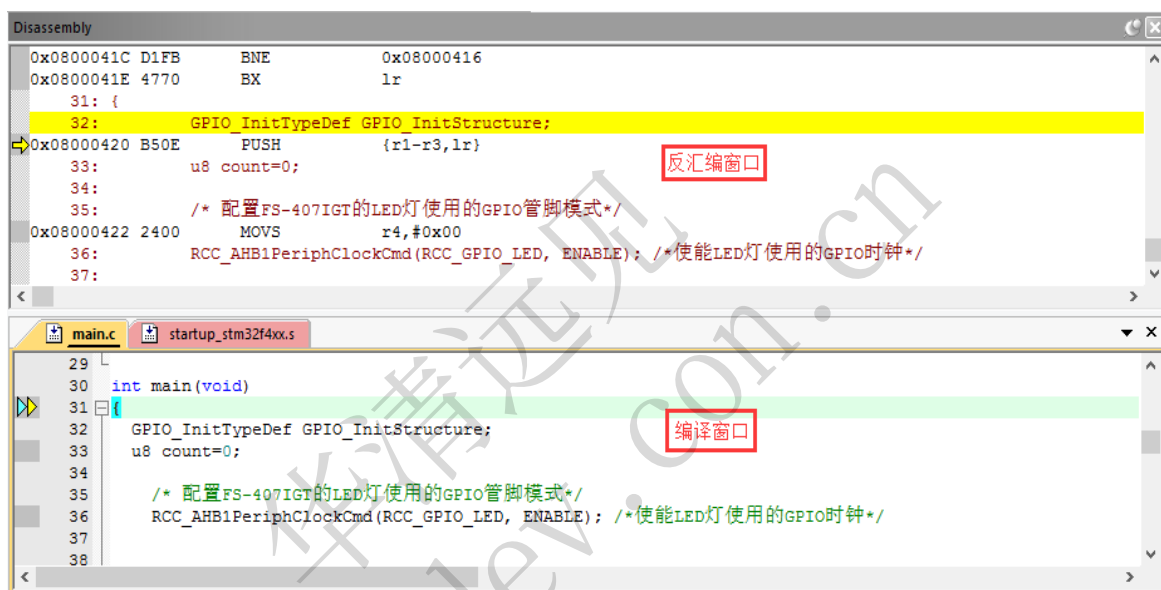
来。将鼠标指针停留在变量上可以观察其相应的值。

这四个按键分别为 Step、Step Over、Step Out、Run to Cursor Line。

注意：请大家在调试时，要逐个试一下这四个工具按钮。在实践中学习，容易理解和掌握。

3. 反汇编

使用 **Start/Stop Debug Session** 调试程序时，会自动弹出 Disassembly（反汇编）窗口。反汇编窗口（如下图所示）可以将源程序和汇编程序一起显示，也可以只显示汇编程序。



若选择反汇编窗口作为当前窗口，那么程序的执行是以 CPU 指令为单位的而不是以源程序中的行为单位的。可以用工具条上的按钮或快捷菜单命令为选中的行设置断点或对断点进行修改。

4. 逻辑分析仪

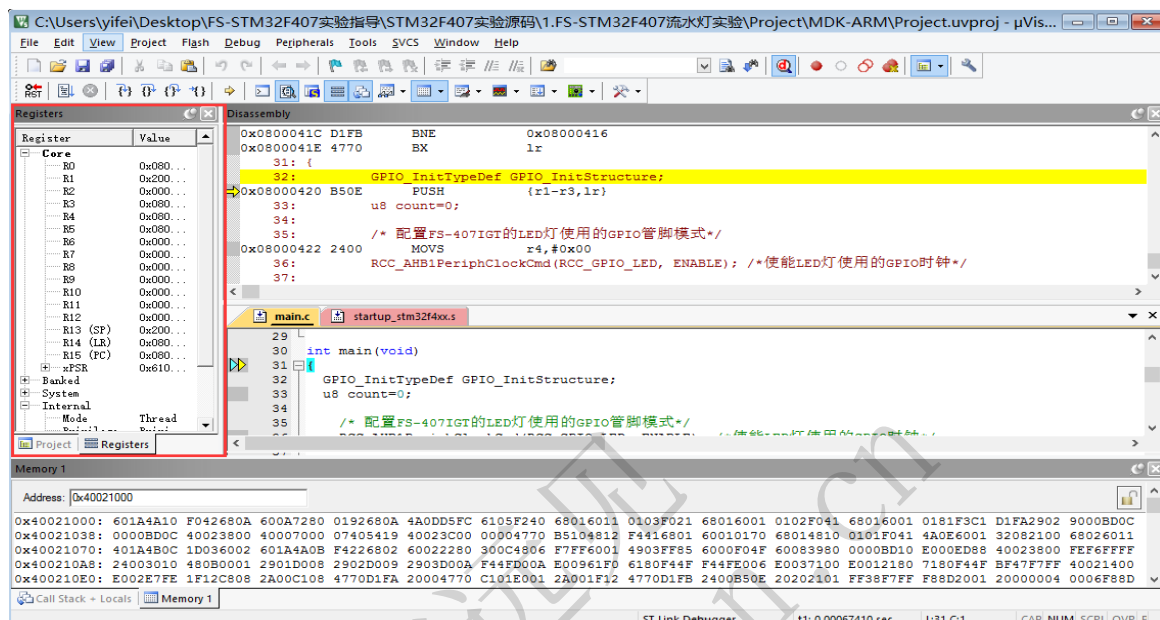
μVision IDE 逻辑分析仪可以将指定的变量或 VTREGs 值的变化以图形方式表示出来。是在软件仿真基础上应用的。

5. 寄存器窗口

是单击菜单栏 View 选择 Registers Window，弹出寄存器显示界面，在 Project Workspace → Registers 页中列出了 CPU 的所有寄存器，按模式排列共有四组，分别为 Core 模式寄存器组、System 模式寄存器

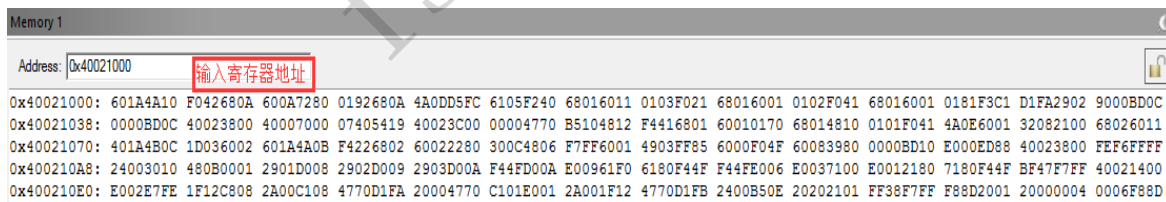


组、Banked 模式寄存器组、以及 Internal 模式寄存器组，如图所示。在每个寄存器组中又分别有相应的寄存器。在调试过程中，值发生变化的寄存器将会以蓝色显示。选中指定寄存器单击或按 F2 键便可以出现一个编辑框，从而可以改变此寄存器的值。



6. 内存窗口

通过工具栏 图标弹出内存窗口可以查看存储器内容。内存窗口可以显示不同的存储域内容，最多可将四个不同的存储域显示在不同的页中，窗口中的右键菜单可以选择输出格式。分别为分别为 Memory#1、Memory#2、Memory#3、Memory#4 存储区域。



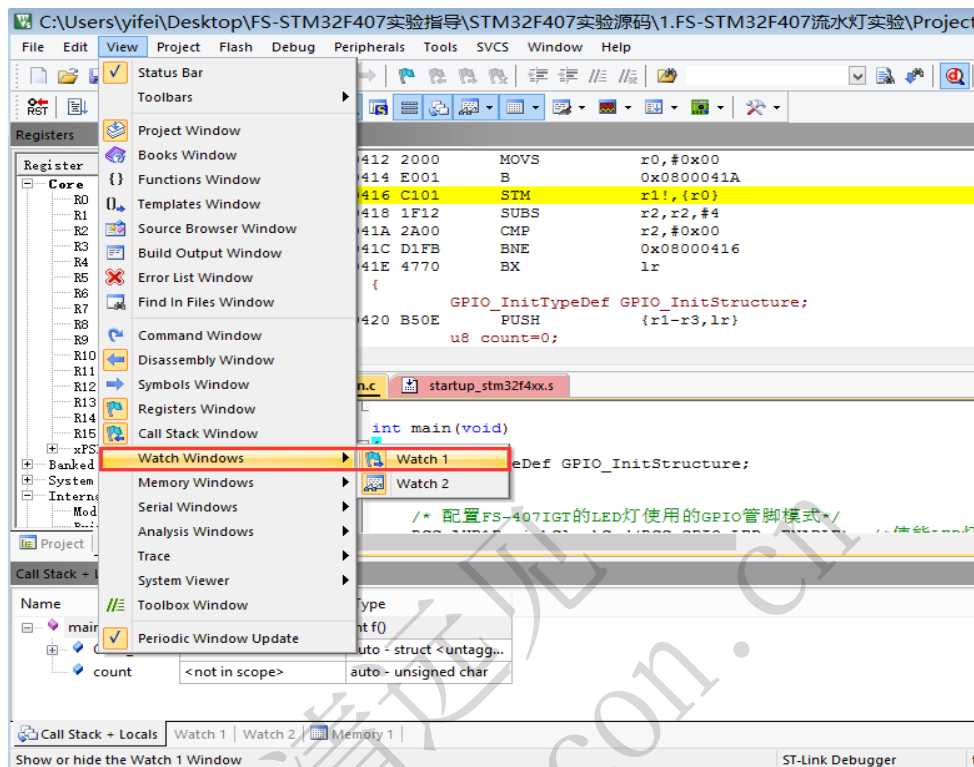
在 Address 域内，可以输入一个表达式，此表达式的值为所显示内容的地址。在某个单元的值上双击可打开一个编辑框，它允许键入一个新的存储值，即可改变存储内容。使能 View - Periodic Window Update，可以在运行目标程序时更新此窗口中的值。

7. 查看窗口

Watch 窗口用于查看和修改程序中变量的值，并可列出当前函数调用关系。在程序运行结束后 Watch



窗口中的内容自动更新。可能通过设置 View-> Periodic Window Update 来实现程序运行时实时更新变量的值。



有三种方式可以把程序变量加到 Watch 页中：

- 用鼠标选中<type F2 to edit>，等待 1 秒后再单击一次或按 F2，会出现一个编辑框，在此输入要添加的变量即可；用同样的方法，可以修改已存在的变量。
- 在工作空间中，选中要添加到 Watch 页中的变量，右键点击会出现快捷菜单，在快捷菜单中选择 Add to Watch Window，即可把选定的变量添加到 Watch 页中。
- 在 Output Window 窗口中，在 Command 页中使用 WS 命令将所要添加的变量添加到 Watch 页中。要修改变量的值，只需选中变量的值，再单击或按 F2 即可出现一个编辑框，进行修改变量的值。要删除变量，只需选中变量，按 Delete 键或用 WK 命令就可以删除变量。



第2章 网络开源开发工具及资料获取

2.1 STM32cube 简介

功能强大的 STM32Cube 新软件平台由设计工具、中间件和硬件抽象层组成，让客户能够集中精力创新，中国，2014 年 3 月 10 日 ——横跨多重电子应用领域的全球领先的半导体供应商、全球领先的 ARM Cortex-M 内核微控制器厂商意法半导体（STMicroelectronics，简称 ST）针对 STM32 微控制器推出一套免费的功能强大的设计工具及软件 STM32CubeTM。新开发平台可简化客户的开发项目，缩短项目研发周期，并进一步强化 STM32 在电子设计人员心目中解决创新难题的首选微控制器的地位。

STM32CubeMX 开发平台包括 STM32CubeMX 图形界面配置器及初始化 C 代码生成器和各种类型的嵌入式软件。配置初始化工具能够一步一步地引导用户完成微控制器配置，而嵌入式软件将为用户省去整合不同厂商软件的繁琐工作。嵌入式软件包括一个新的硬件抽象层(HAL, Hardware Abstraction Layer)，用于简化代码在 STM32 产品之间的移植过程。通过在一个软件包内整合在 STM32 微控制器上开发应用所需的全部通用软件，该平台根除了评估每个软件之间关联性的复杂任务。STM32Cube 提供数千个用例和一个软件更新功能，方便用户快捷高效的获取最新版本的软件。

STM32Cube 是一个全面的软件平台，包括了 ST 产品的每个系列。(如，STM32CubeF4 是针对 STM32F4 系列).平台包括了 STM32Cube 硬件抽象层和一套的中间件组件(RTOS, USB, FS, TCP/IP, Graphics, 等等).

C 代码项目产生，涵盖 STM32 初始化部分。兼容 IAR, KEIL 和 GCC 编译器。

2.2 STM32cubeMX 的获取

关于 STM32cubeMX 软件是在 ST 公司官网获取的，具体的下载地址：

http://www.st.com/web/en/web/catalog/tools/FM147/CL1794/SC961/SS1533/PF259242?icmp=259242_prom_s1574_jun2014

在 ST 官网上的查找路径：



Home > Development Tools > Software Development Tools > STM32 Software Development Tools > STM32 Configurators and Code Generators > STM32CubeMX

GET SOFTWARE

Part Number	Software Version	Marketing Status	Supplier	Order from ST
STM32CubeMX	4.16.1	Active	ST	Get Software

STM32 系列芯片固件库的获取：

<http://www.st.com/web/en/catalog/tools/FM147/CL1794/SC961>

在 ST 官网上的查找路径：

Home > Embedded Software > MCUs Embedded Software > STM32 Embedded Software

Last Viewed Share Bookmark Download Full screen					
unknown STM32 3rd-party Embedded Software STM32 ODE Function Pack SW STM32 Standard Peripheral Libraries STM32 SI >					
Total Parts: (254) for STM32 Embedded Software Matching Parts : (254) Customize Table Reset All					
Part Number	Marketing Status	Supplier	Supported Devices	Software Type	Software
STSW-STM32108 STM32F301x/302x/303x/334x DSP and standard periph...	Active	ST	STM32F3	Firmware	1.2.3
STSW-STM32065 STM32F4 DSP and standard peripherals library	Active	ST	STM32,STM32F4	Firmware	1.7.1
STSW-STM32048 STM32F0xx standard peripherals library	Active	ST	STM32F0	Firmware	1.5.0
STSW-STM32054 STM32F10x standard peripheral library	Active	ST	STM32,STM32F1	Firmware	3.5.0
STSW-STM32062 STM32F2xx standard peripherals library (UM1061)	Active	ST	STM32,STM32F2	Firmware	1.1.0
STSW-STM32077 STM32L1xx standard peripherals library	Active	ST	STM32L1	Firmware	1.3.1

2.3 STM32cubeMX 的安装

在安装 STM32cubeMX 之前需要先安装 Java SDK 开发工具包，没有 Java SDK 的支持 STM32cubeMX 就

无法正常工作，我们这里安装的是 JavaSetup8u51.exe，下面看一下具体的安装步骤：

- ▶ 首先双击 Java SDK 的安装文件，路径在：

【华清远见-嵌入式 ARM 实验箱资料-III\华清远见-STM 系列资料\Cortex-M4 部分\工具软件

\stm32CubeMx】



jre-8u251-windows-x64.exe	2020/10/29 11:25	应用程序	75,497 KB
SetupSTM32CubeMX-5.6.0.exe	2020/2/19 22:01	应用程序	181,482 KB
stm32cube_fw_f4_v1250.zip	2020/11/11 17:00	ZIP 压缩文件	651,656 KB
stm32cube_fw_f4_v1252.zip	2020/11/11 17:01	ZIP 压缩文件	117,601 KB

- ▶ 如果想改变工具包的安装路径，就需要将下面的选项勾选上：



- ▶ 然后点击安装，开始 Java SDK 的安装，大约两分钟的时间：



下面开始安装 STM32cubeMX：

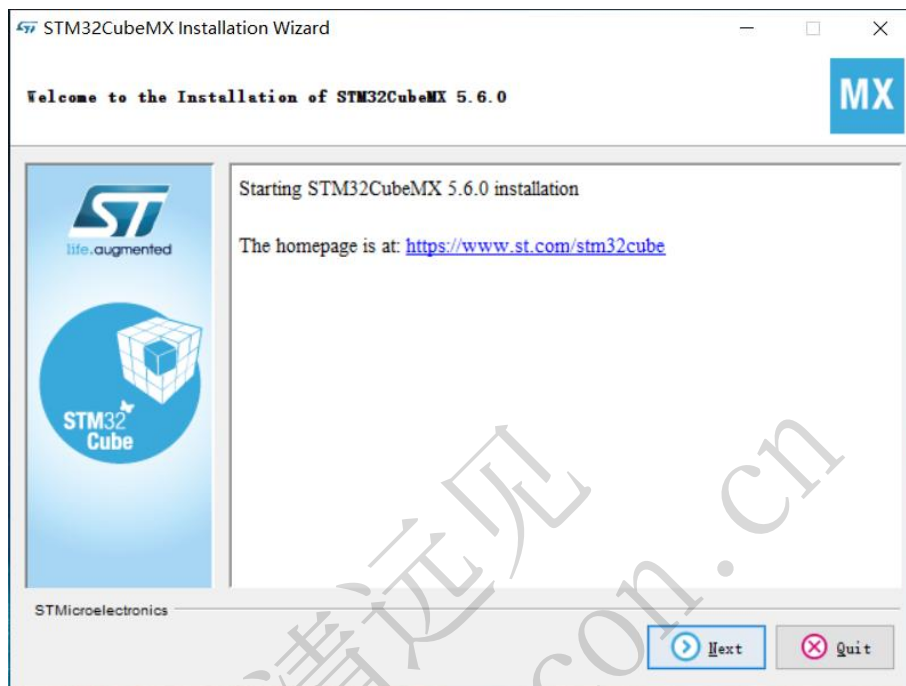
- ▶ 光盘中提供了 STM32CubeMX-5.6，路径：



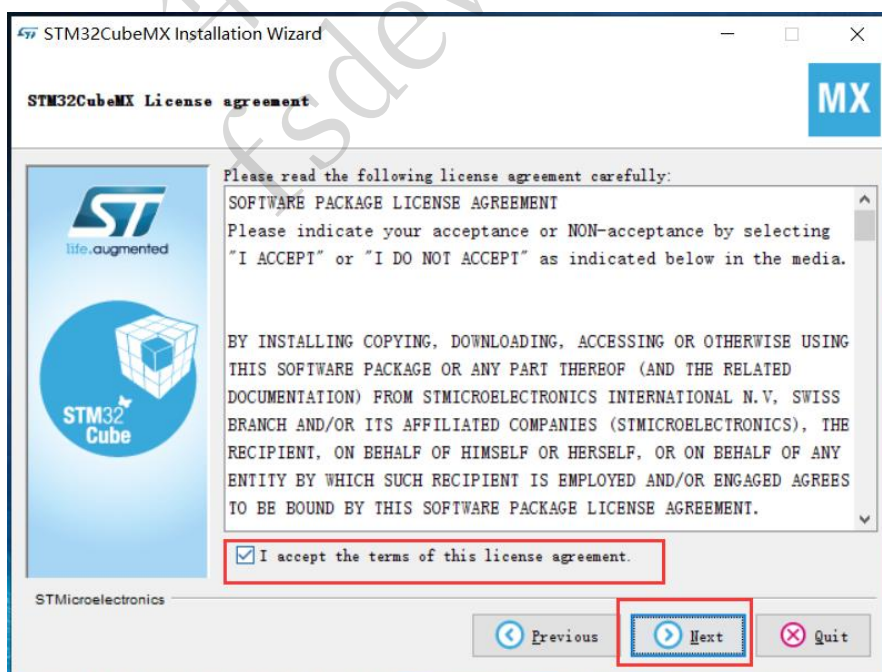
【华清远见-嵌入式 ARM 实验箱资料-III\华清远见-STM 系列资料\Cortex-M4 部分\工具软件

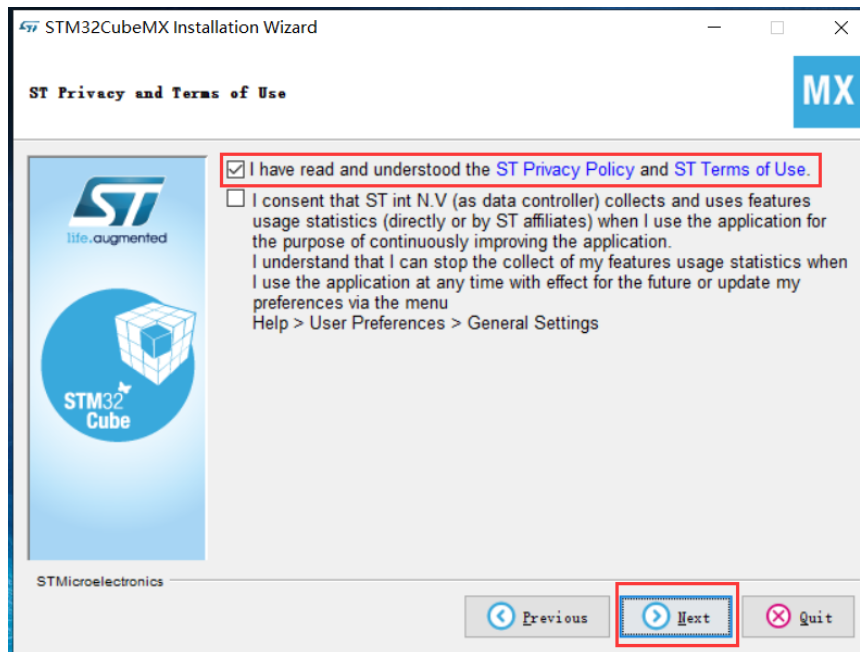
\stm32CubeMx】

- ▶ 双击 STM32CubeMX 安装包，点击 Next 进行下一步操作：

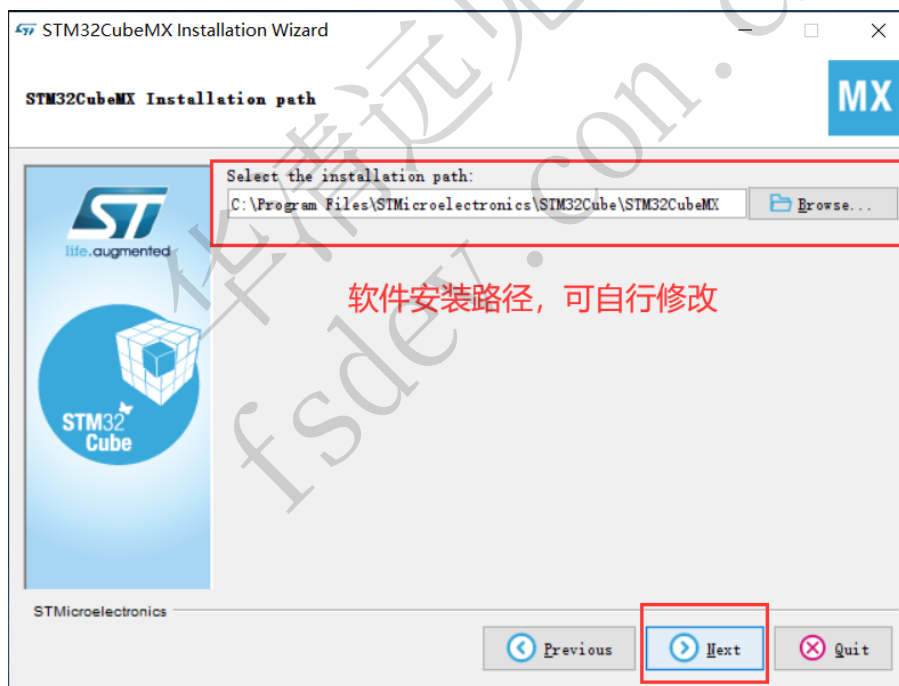


- ▶ 勾选图中对应选项，点击 Next 进行下一步操作：

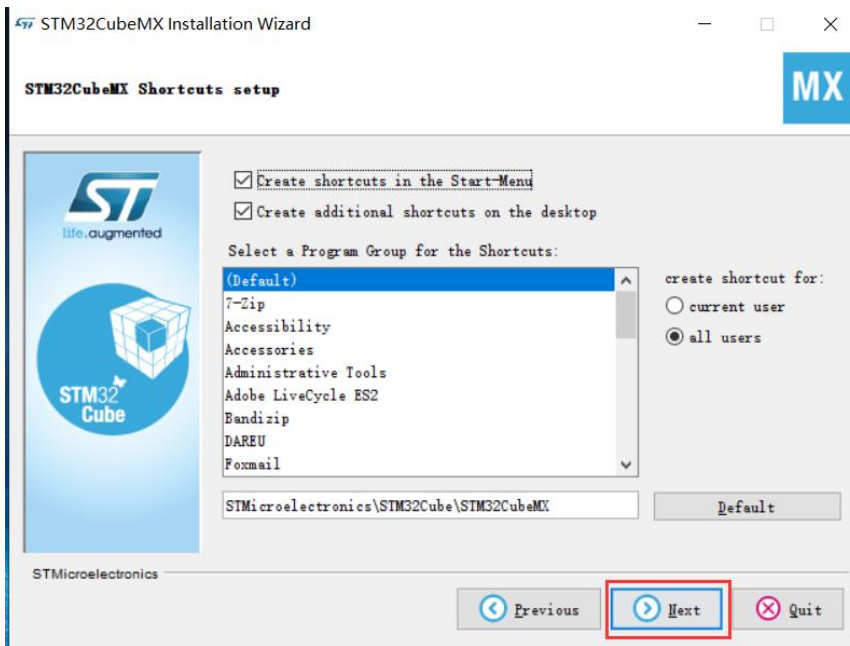




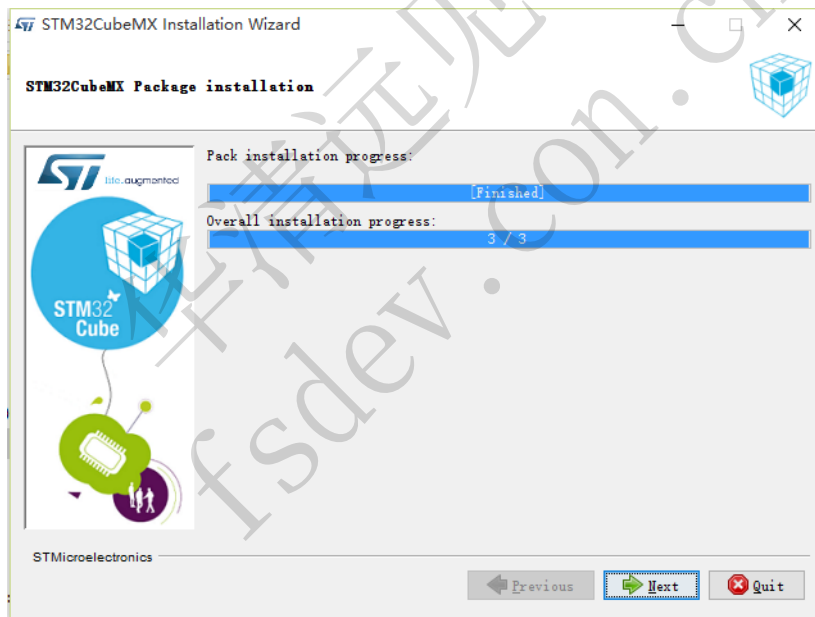
- ▶ 选择对应的安装目录, 需要注意的是安装目录中尽量不含中文, 然后点击 Next 进行下一步操作:



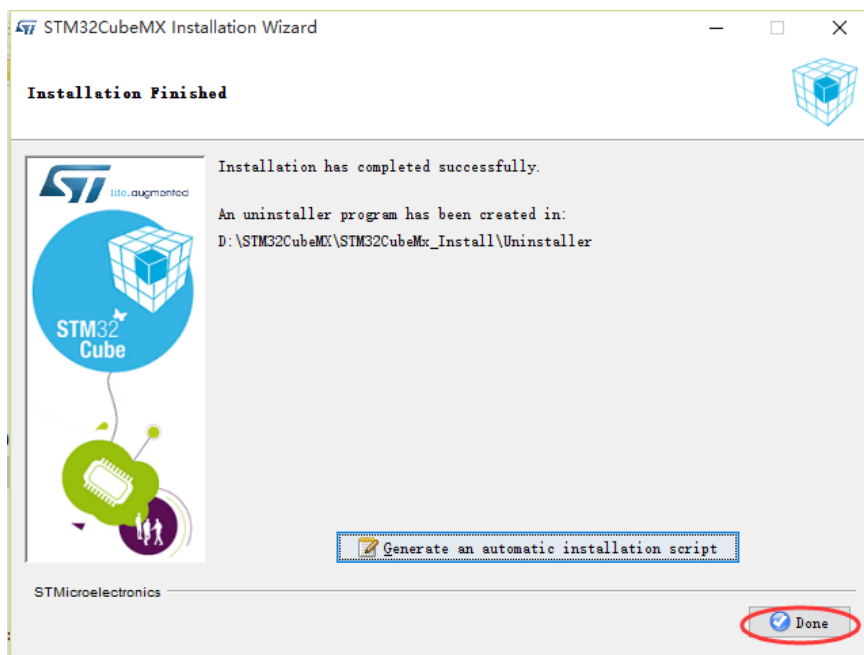
- ▶ 这里设置默认选项, 直接点击 Next, 进行下一步操作即可



- ▶ 等待安装完成，点击 Next 进行下一步操作：



- ▶ 点击 Done，完成 STM32cubeMX 的安装工作：



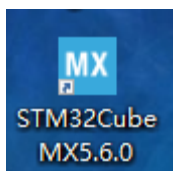
这样 STM32cubeMX 软件就安装好了，接下来需要为该图形化软件配置对应的固件库，固件库的下载地址在一节中已经提到了，具体的配置过程如下：

- ▶ 接下来进行安装固件库，固件库简单说就是相应芯片的配置及各种封装函数，对应的压缩包如下，固件库的位置在：

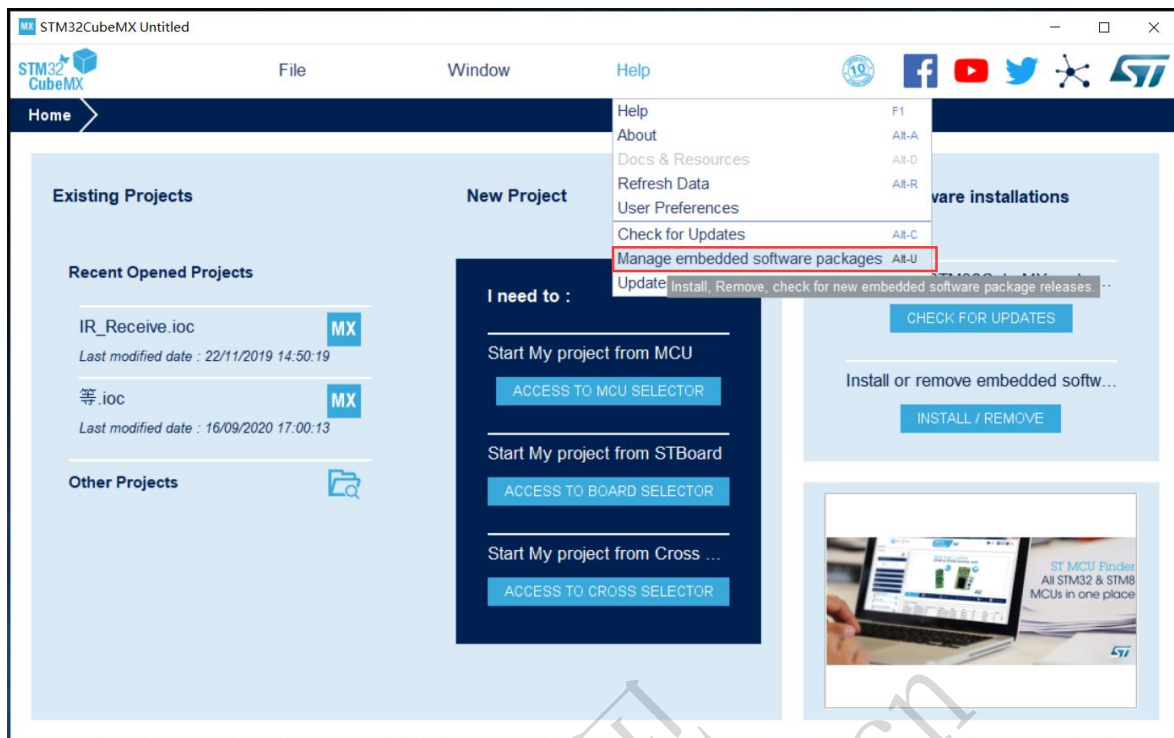
【华清远见-嵌入式 ARM 实验箱资料-III\华清远见-STM 系列资料\Cortex-M4 部分\工具软件\stm32CubeMx】

jre-8u251-windows-x64.exe	2020/10/29 11:25	应用程序	75,497 KB
SetupSTM32CubeMX-5.6.0.exe	2020/2/19 22:01	应用程序	181,482 KB
stm32cube_fw_f4_v1250.zip	2020/11/11 17:00	ZIP 压缩文件	651,656 KB
stm32cube_fw_f4_v1252.zip	2020/11/11 17:01	ZIP 压缩文件	117,601 KB

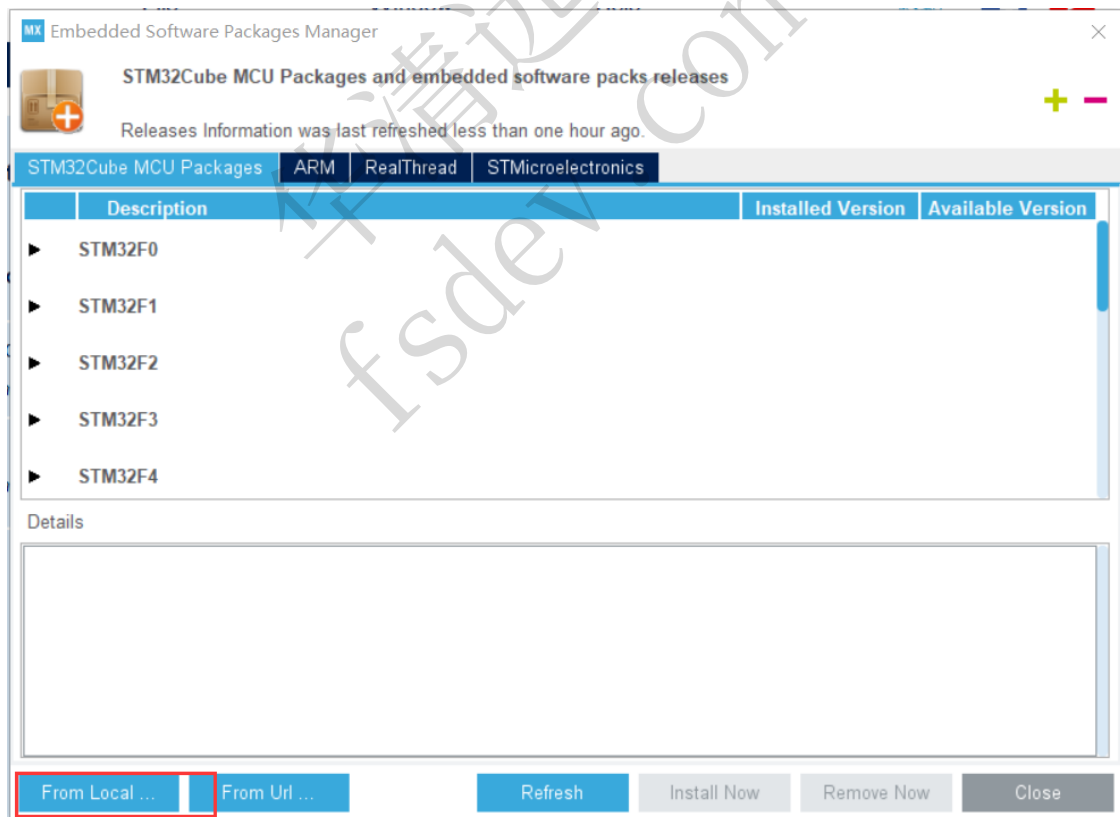
- ▶ 打开 STM32Cubemx 软件



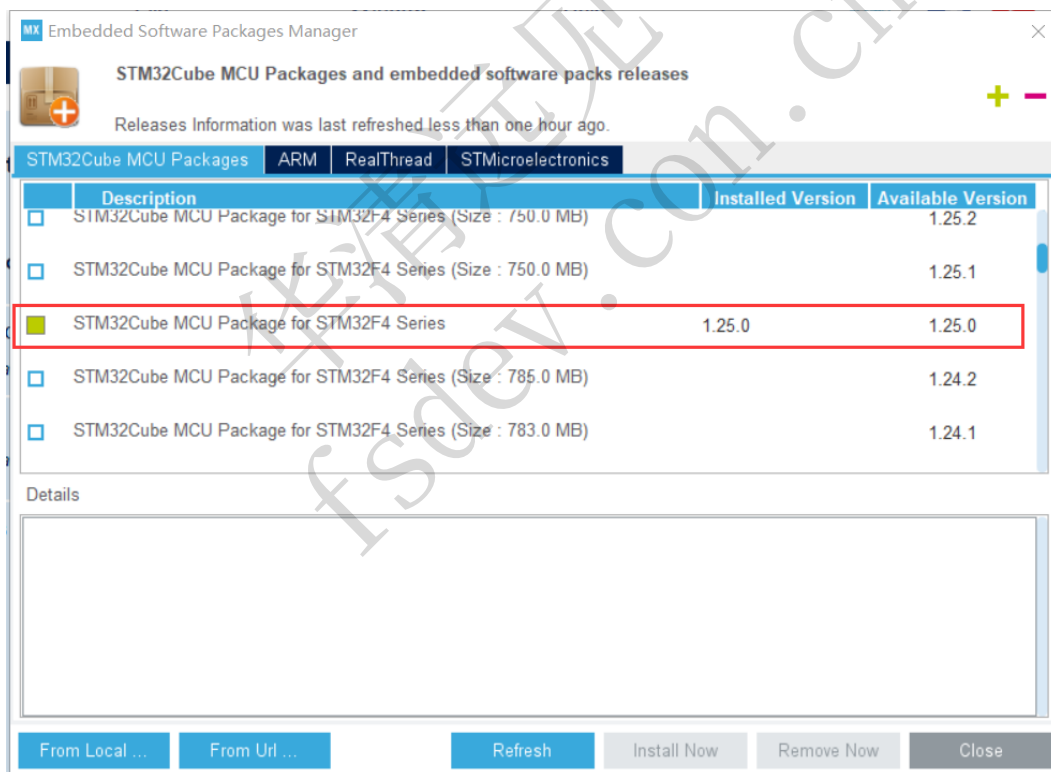
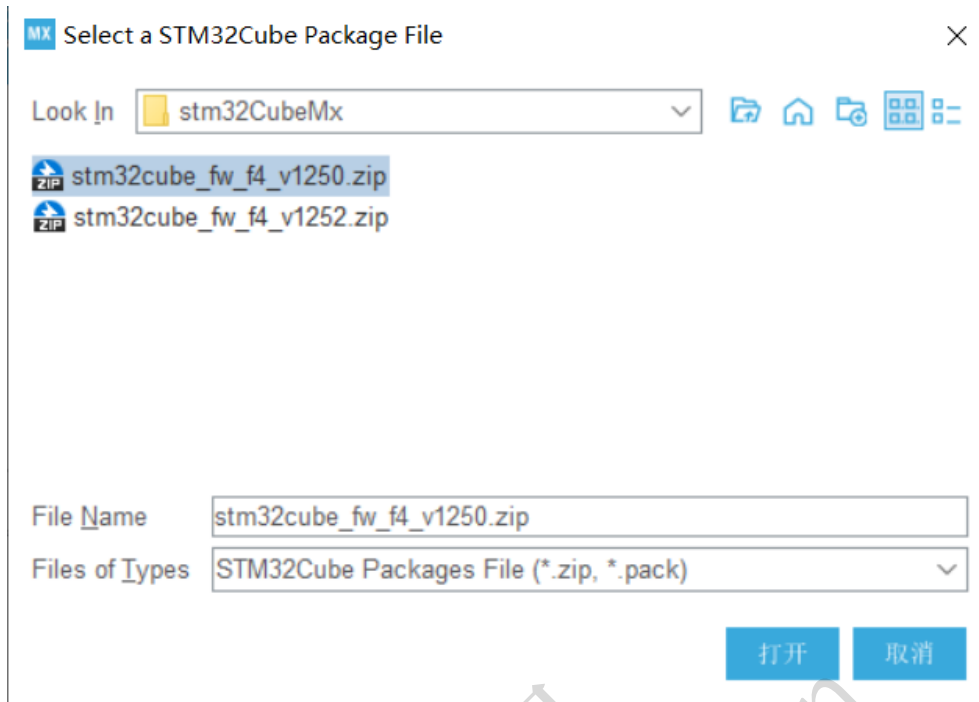
- ▶ 加载固件库，方法如下



► 安装固件库



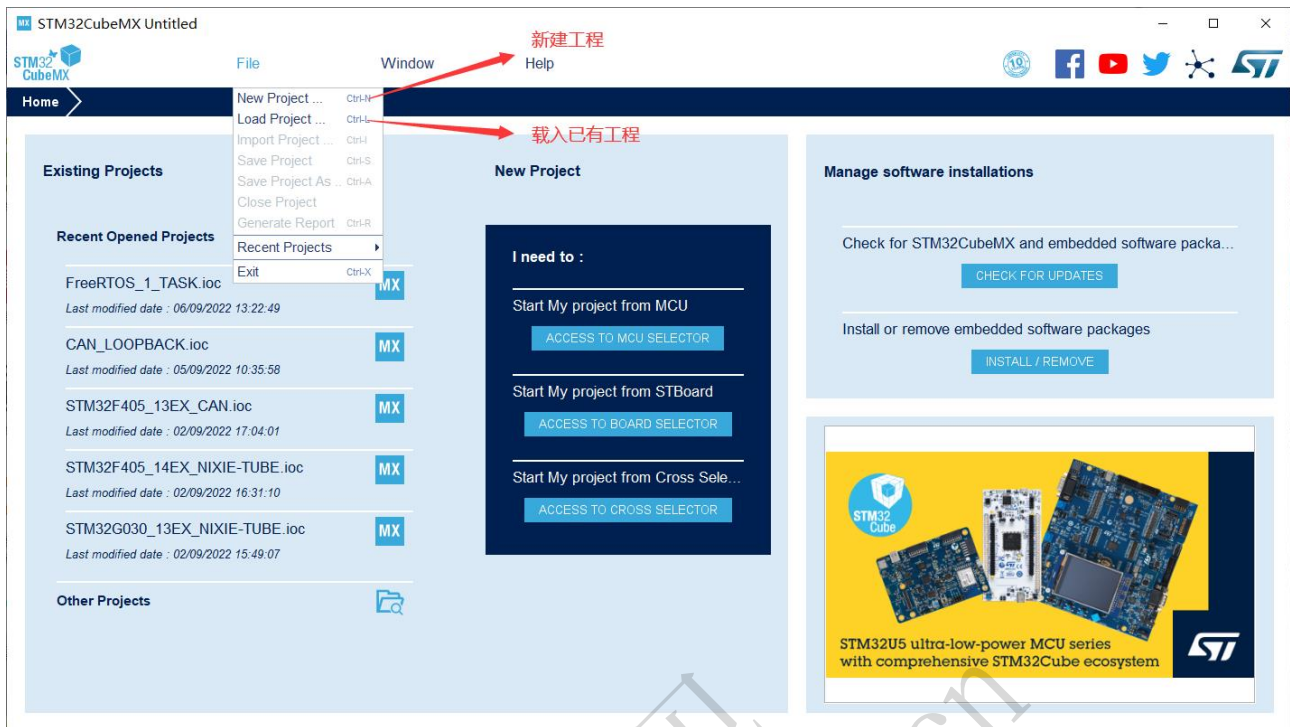
► 查找路径选择固件库压缩包进行安装



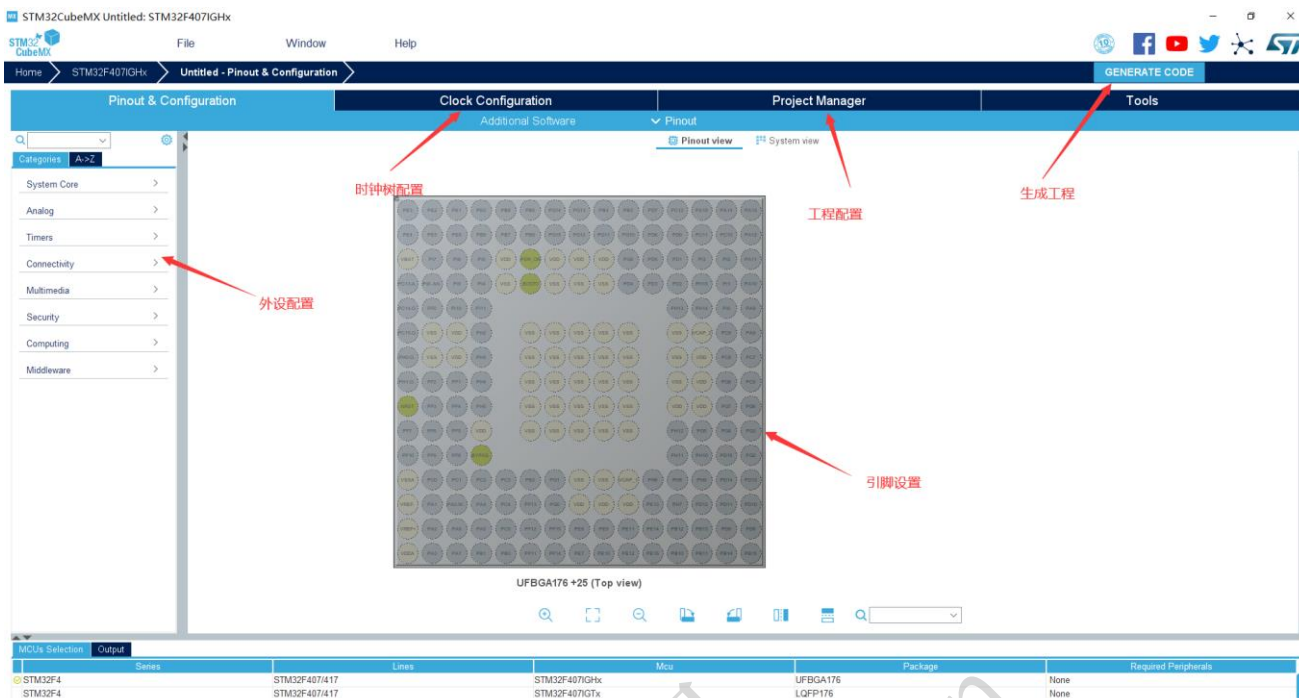
- ▶ 这样就可以进行图形化编程了。

2.4 STM32cubeMX 的使用

- 如何选择芯片：
 - ▶ 点击新建工程进入选择芯片界面

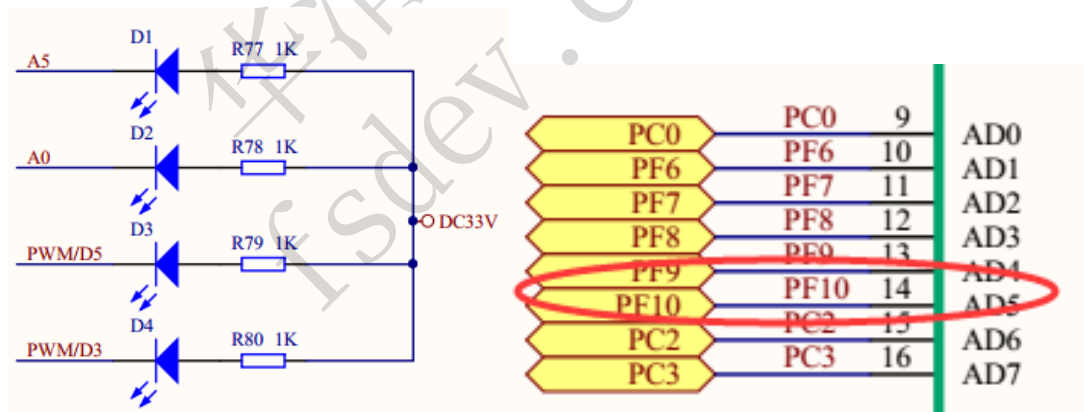


► 点击 Start Project，进入配置界面：

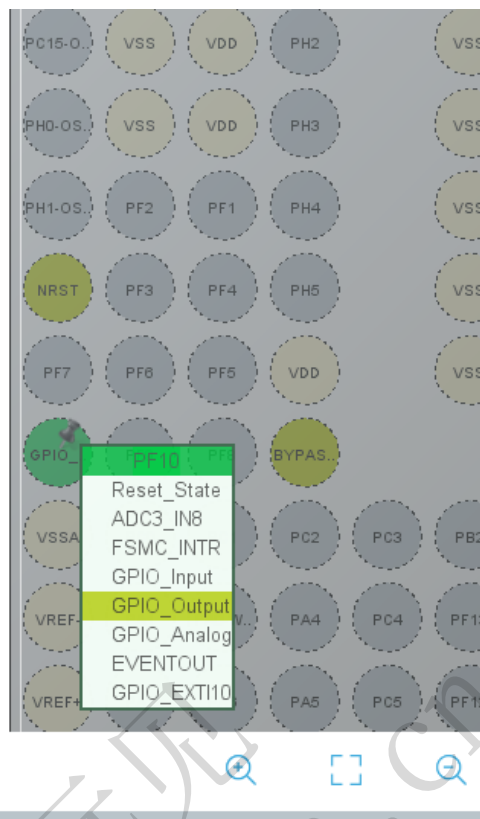


然后我们可以一起创建一个简单的 LED 灯的工程，让大家能尽快的熟悉这个软件的使用：

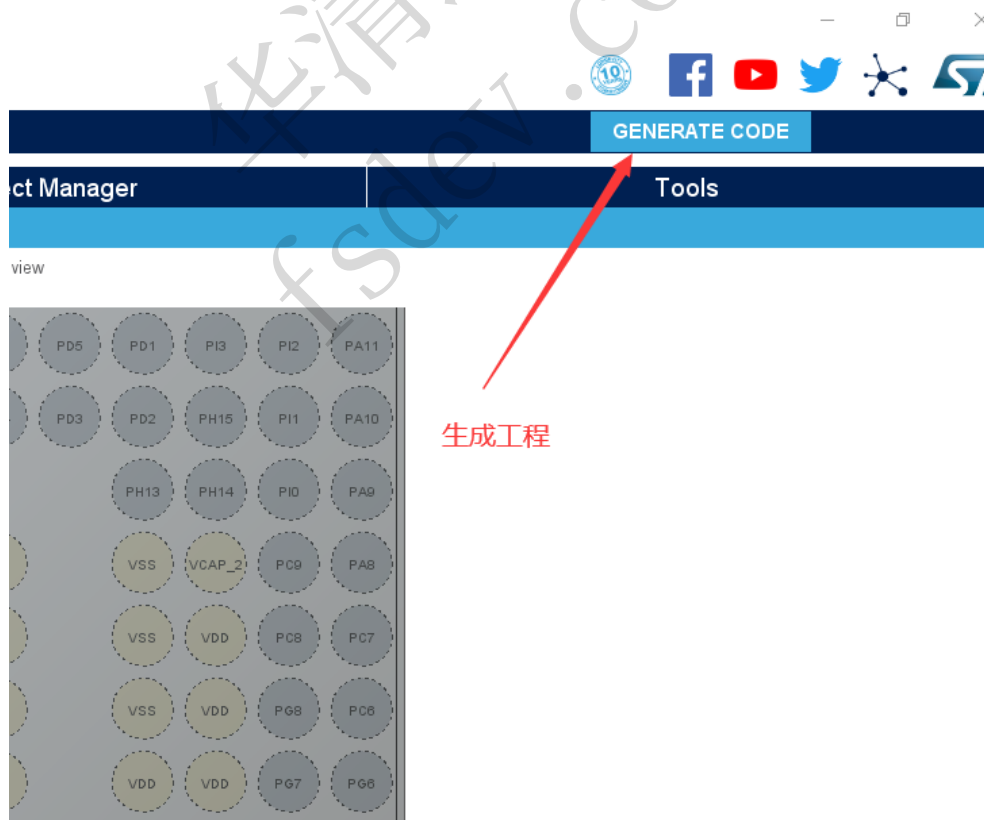
- ▶ 要创建一个点 LED 灯的工程，在创建工程之前，需要先搞清楚，哪个引脚控制 LED 灯，这就需要在原理图中去看了，下面就是 LED 灯的电路图：



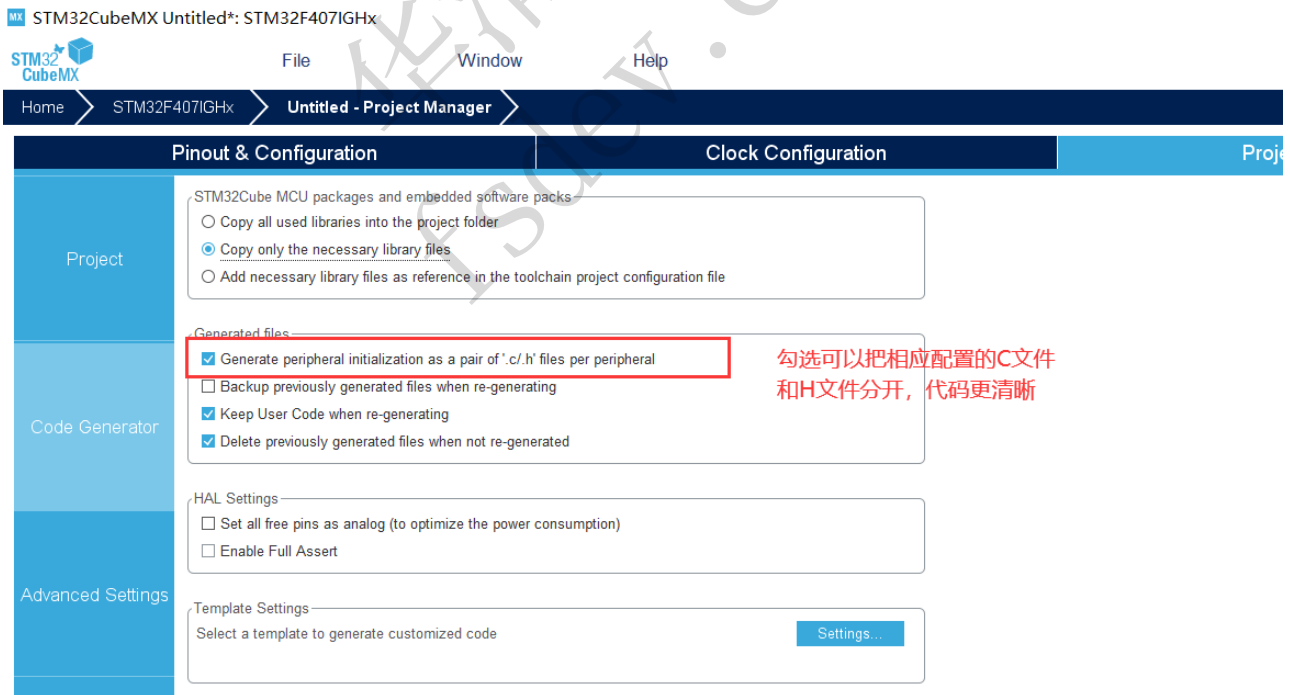
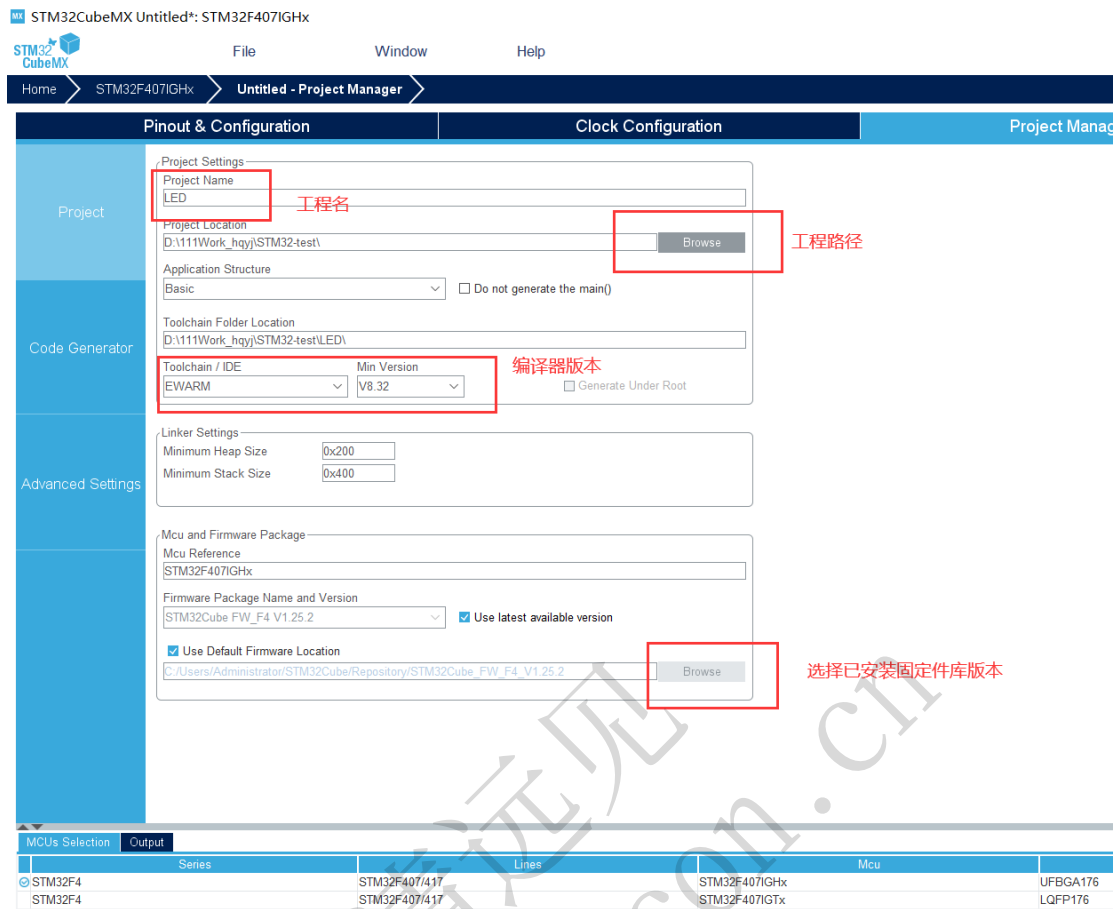
- ▶ 由上图可以看到，如果需要点亮 D1 灯的话，需要操作的引脚是 PF10。由于这款芯片的引脚比较多，所以搜索框输入想操作的引脚，找到 PF10 引脚后，单击该引脚（左键点击），可以看到能对该引脚进行的操作有这么几个选项，这里点灯，只需要将引脚设置为输出，给一个低电平即可，由上图可以看到有一个 GPIO_Output 选项，点击它，这样就将 PF10 引脚配置成输出模式了，其实只做点灯程序的话工程已经配置好了，接下来需要自动生成源代码，在源代码中添加相应的逻辑语句即可：



- ▶ 点击生成工程按钮



- ▶ 点击生成工程按钮后会弹出工程设置界面如下



再次点击生成工程按钮创建工程，完成后弹出框如下，点击 Open Project 打开工程



- ▶ 下面需要在 main.c 中加入一些逻辑代码，使 LED 灯实现闪烁的效果，但是要注意，使用 STM32CubeMx 软件生成的工程文件，添加代码时，必须要在固定的位置添加，如果在其它地方添加代码，再次重新生成工程后，添加的代码就消失了。不过在生成的工程文件中很容易会发现可以添加代码的字段标志，每个字段都是以 USER CODE BEGIN 开始，以 USER CODE END 结束，只需将代码添加到这个标志中间即可，如下图所示：



```

59
60 int main(void)
61 {
62
63     /* USER CODE BEGIN 1 */
64
65     /* USER CODE END 1 */
66
67     /* MCU Configuration-----*/
68
69     /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
70     HAL_Init();
71
72     /* Configure the system clock */
73     SystemClock_Config();
74
75     /* Initialize all configured peripherals */
76     MX_GPIO_Init();
77
78     /* USER CODE BEGIN 2 */
79
80     /* USER CODE END 2 */
81
82     /* Infinite loop */
83     /* USER CODE BEGIN WHILE */
84     while (1)
85     {
86         /* USER CODE END WHILE */
87
88         /* USER CODE BEGIN 3 */
89
90     }
91     /* USER CODE END 3 */
92
93 }
94

```

- ▶ 下面在主函数中，添加亮灯和灭灯的逻辑语句：

```

int main(void)
{
    /* USER CODE BEGIN 1 */

    /* USER CODE END 1 */

    /* MCU Configuration-----*/

    /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
    HAL_Init();

    /* Configure the system clock */
    SystemClock_Config();

    /* Initialize all configured peripherals */
    MX_GPIO_Init();

    /* USER CODE BEGIN 2 */

    /* USER CODE END 2 */

    /* Infinite loop */
    /* USER CODE BEGIN WHILE */
    while (1)
    {
        /* USER CODE END WHILE */

        /* USER CODE BEGIN 3 */
        HAL_GPIO_WritePin(GPIOF,GPIO_PIN_10,GPIO_PIN_SET);
        HAL_Delay(1000);
        HAL_GPIO_WritePin(GPIOF,GPIO_PIN_10,GPIO_PIN_RESET);
        HAL_Delay(1000);
        /* USER CODE END 3 */
    }
}

```

- ▶ 添加完程序后，保存编译，将程序下载到开发板，会看到如下现象，说明我们的工程创建成功。（程序的编译和下载工程请参考前一章的介绍）：





第3章 STM32F407IGHx 实验

3.1 跑马灯实验

3.1.1 【实验目的】

熟悉 RealView MDK 工具软件的使用方式。

能够把 ST 官方库文件添加到项目工程。

掌握 RealView MDK 软件的基本设计流程和设计步骤，能够使用工具进行设计、编程、仿真、调试。

通过本实验掌握的 GPIO 口的使用方法。

掌握如何利用 STM32F407IGTb 芯片的 I/O 口控制 LED。

3.1.2 【实验环境】

嵌入式 ARM 实验箱

ST-Link 仿真器

RealView MDK5.21 集成开发软件

STM32CUBEMX 图形开发软件

PC 机 XP、Window7/8 (32/64bit)

3.1.3 【实验内容】

编写 LED 程序，实现嵌入式 ARM 实验箱上的 LED 循环闪烁。调试编译好固件程序后，将程序下载到

STM32F407 核心板，按复位键观察实验现象。

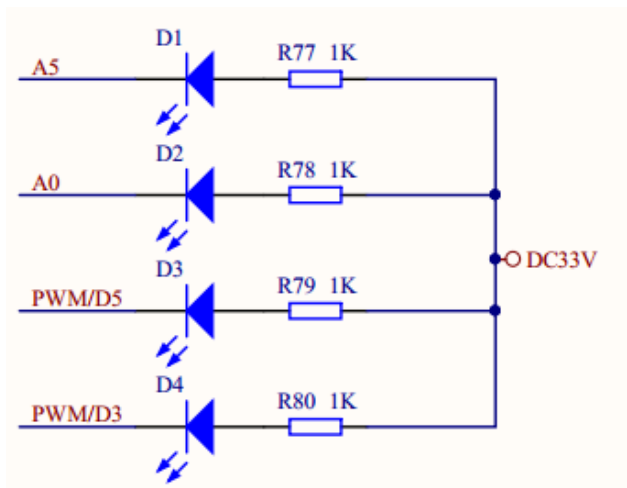
3.1.4 【实验原理】

只要是对硬件操作，就要首先查看原理图。查看外设是给模块的 MCU 的哪个引脚相连。嵌入式 ARM 实验箱上的 LED 的亮灭状态，与芯片上的引脚 I/O 输出电平有关。



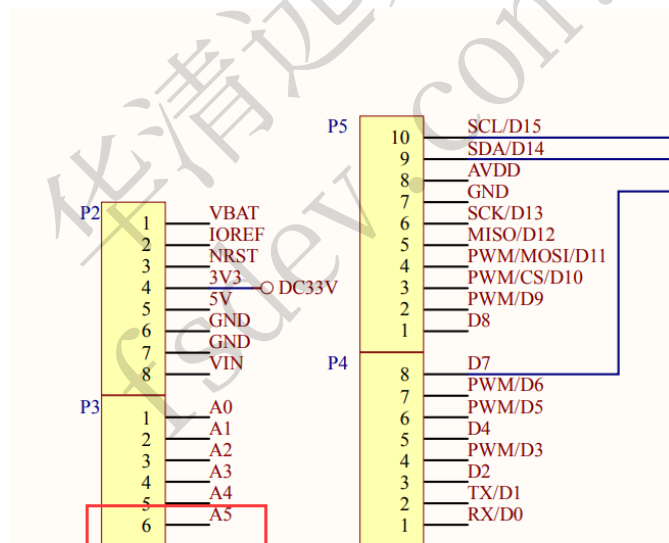
首先打开【华清远见-嵌入式 ARM 实验箱资料-III/华清远见-STM 系列资料/Cortex-M4 部分/原理图

/Arduino】目录下的按键数码管板.pdf，我们以 A5 为例，由下图可知，当 A5 为低电平时，D1 亮。



查看按键数码管和实验箱大底板的接口，确定 A5 的位置。从下图可以看出，A5 的连接处是 P3 的编

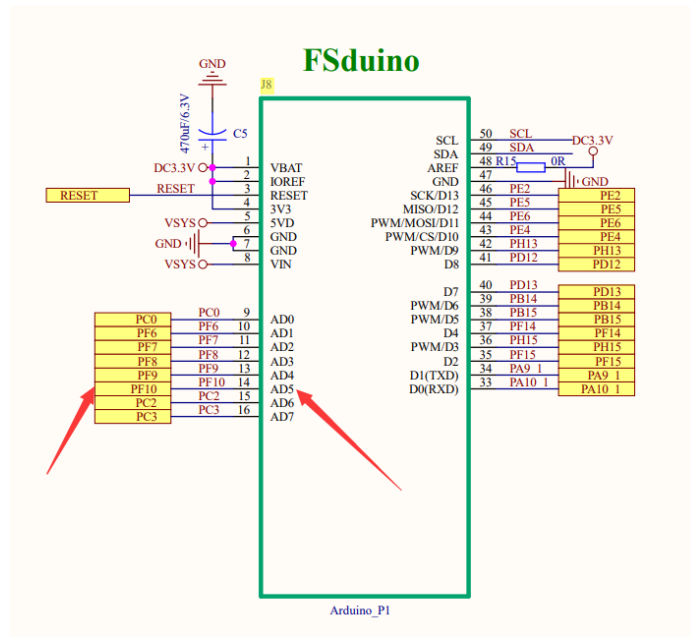
号 6。

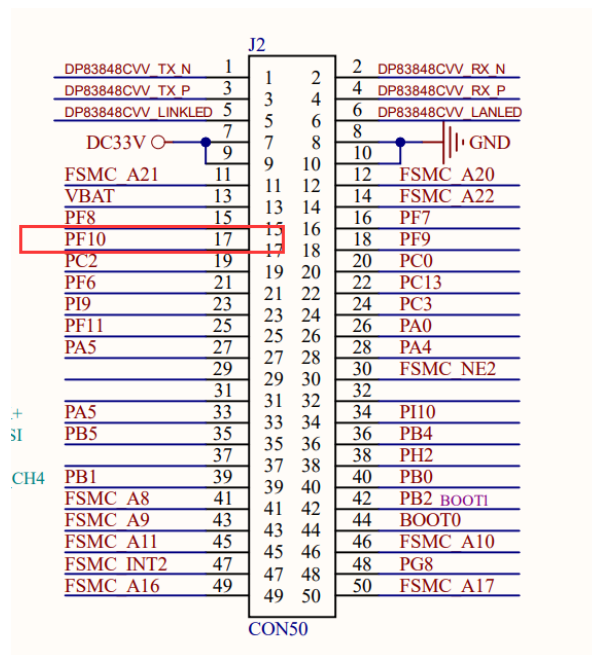


接下来，我们需要查看，打开【华清远见-嵌入式 ARM 实验箱资料-III/华清远见-STM 系列资料/Cortex-

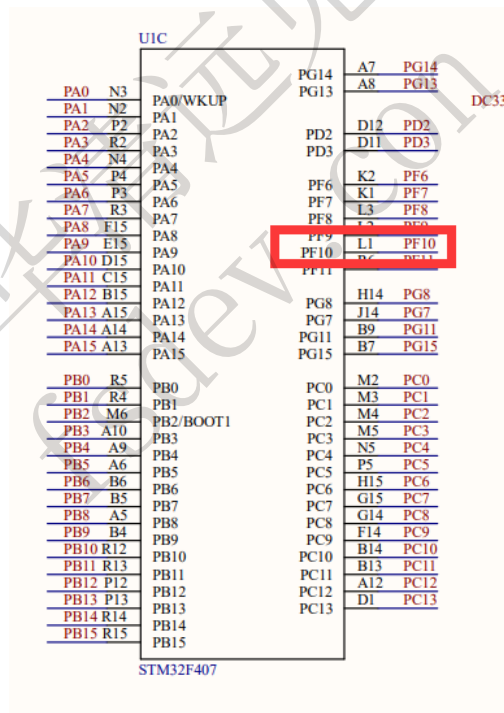
M4 部分/原理图】目录下的实验箱底板图.pdf，大底板很大，我们不确定，原理图的内容也很多，我们不容易找到，可以看下按键数码管在实验箱底板上的丝印是 J8，接下来我们在实验箱大底板.pdf 上搜索 J8

即可，由下图我们可以看出，A5 连接到了实验箱底板的 PF10。





最终确定，我们要控制的 STM32F407 芯片的引脚为 PF10



IO 操作重要结构体：GPIO_InitTypeDef;



```

1      /**
2      * @brief GPIO Init structure definition
3      */
4      typedef struct
5      {
6          uint32_t Pin;          /*!< Specifies the GPIO pins to be configured.
7                                  This parameter can be any value of @ref GPIO_pins_define */
8
9          uint32_t Mode;         /*!< Specifies the operating mode for the selected pins.
10                                 This parameter can be a value of @ref GPIO_mode_define */
11
12         uint32_t Pull;         /*!< Specifies the Pull-up or Pull-Down activation for the selected pins.
13                                 This parameter can be a value of @ref GPIO_pull_define */
14
15         uint32_t Speed;         /*!< Specifies the speed for the selected pins.
16                                 This parameter can be a value of @ref GPIO_speed_define */
17
18         uint32_t Alternate;     /*!< Peripheral to be connected to the selected pins.
19                                 This parameter can be a value of @ref GPIO_Alternate_function_selection */
20     } GPIO_InitTypeDef;

```

第一个成员变量 Pin 是所操作的管脚,第二个 Mode 是模式选择,第三个 Pull 是上拉下拉,或者都不加,第四个 Speed 是速度选择,第五个是管脚复用成什么模式。一般我们只操作前四个。

STM32F407 的 IO 口可以由软件配置成 4 种模式,其实操作的是 GPIO 的端口模式寄存器:

- 输入（复位状态）/input（reset state）
- 通用输出模式 / general purpose output mode
- 复用功能模式 / alternate function mode
- 模拟模式 / analog mode

上面给两个寄存器赋值了, 1~4 位是 GPIO 端口模式寄存器, 5~8 位是端口输出类型寄存器（决定是推挽输出还是开漏输出）有关于寄存器这边不详细解释,有兴趣的同学可以参考《STM32F4XX 中文参考手册》187 页。

STM32 的 GPIO 端口在作为输出时,可以软件配置端口最大支持的时钟速率,下图是端口输出速度寄存器,有以下几种速度选择:



31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
OSPEEDR15[1:0]	OSPEEDR14[1:0]	OSPEEDR13[1:0]	OSPEEDR12[1:0]	OSPEEDR11[1:0]	OSPEEDR10[1:0]	OSPEEDR9[1:0]	OSPEEDR8[1:0]								
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OSPEEDR7[1:0]	OSPEEDR6[1:0]	OSPEEDR5[1:0]	OSPEEDR4[1:0]	OSPEEDR3[1:0]	OSPEEDR2[1:0]	OSPEEDR1[1:0]	OSPEEDR0[1:0]								
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

位 2y:2y+1 **OSPEEDRy[1:0]**: 端口 x 配置位 (Port x configuration bits) (y = 0..15)

这些位通过软件写入, 用于配置 I/O 输出速度。

00: 2 MHz (低速)

01: 25 MHz (中速)

10: 50 MHz (快速)

11: 30 pF 时为 100 MHz (高速) (15 pF 时为 80 MHz 输出 (最大速度))

对应到 STM32 库中的定义如下:

```
1 #if defined(STM32L0) || defined(STM32F4)
2 #define GPIO_SPEED_LOW          GPIO_SPEED_FREQ_LOW
3 #define GPIO_SPEED_MEDIUM       GPIO_SPEED_FREQ_MEDIUM
4 #define GPIO_SPEED_FAST         GPIO_SPEED_FREQ_HIGH
5 #define GPIO_SPEED_HIGH         GPIO_SPEED_FREQ_VERY_HIGH
6 #endif /* STM32L0 || STM32F4 */
```

为了降低 STM32 处理器的功耗, 可以依据实际的需求, 选择合适的时钟速率, GPIO 口支持的最大时钟速率越低, 它产生的功耗也越低。

下面的初始化 FS-STM32F407 板载的 4 个 LED 灯的代码。对应的 GPIO 端口初始化的子函数。

```
1 void MX_GPIO_Init(void)
2 {
3
4     GPIO_InitTypeDef GPIO_InitStruct;
5
6     /* GPIO Ports Clock Enable */
7     __GPIOF_CLK_ENABLE();
8     __GPIOH_CLK_ENABLE();
9     __GPIOC_CLK_ENABLE();
10    __GPIOB_CLK_ENABLE();
11
12    /*Configure GPIO pin : PF10 */
13    GPIO_InitStruct.Pin = GPIO_PIN_10;
14    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
```




```

15     GPIO_InitStruct.Pull = GPIO_NOPULL;
16     GPIO_InitStruct.Speed = GPIO_SPEED_LOW;
17     HAL_GPIO_Init(GPIOF, &GPIO_InitStruct);
18
19     /*Configure GPIO pin : PC0 */
20     GPIO_InitStruct.Pin = GPIO_PIN_0;
21     GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
22     GPIO_InitStruct.Pull = GPIO_NOPULL;
23     GPIO_InitStruct.Speed = GPIO_SPEED_LOW;
24     HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);
25
26     /*Configure GPIO pin : PB15 */
27     GPIO_InitStruct.Pin = GPIO_PIN_15;
28     GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
29     GPIO_InitStruct.Pull = GPIO_NOPULL;
30     GPIO_InitStruct.Speed = GPIO_SPEED_LOW;
31     HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
32
33     /*Configure GPIO pin : PH15 */
34     GPIO_InitStruct.Pin = GPIO_PIN_15;
35     GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
36     GPIO_InitStruct.Pull = GPIO_NOPULL;
37     GPIO_InitStruct.Speed = GPIO_SPEED_LOW;
38     HAL_GPIO_Init(GPIOH, &GPIO_InitStruct);
39
40 }
```

调用的 HAL 函数解析：

```
void HAL_GPIO_WritePin(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin, GPIO_PinState PinState);
```

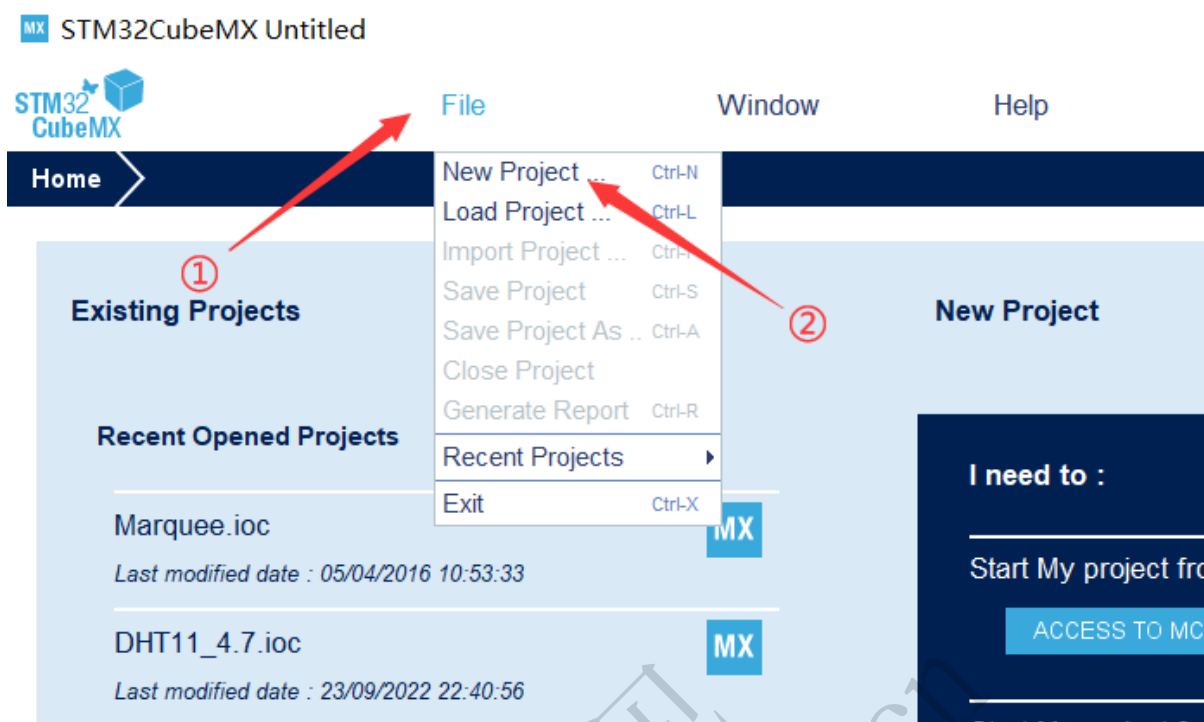
该函数其实是对 BSRR 寄存器进行操作。

第一个参数传的是 GPIO 所在的组，第二个是操作第几个管脚，第三个是对管脚进行置位，更加详细

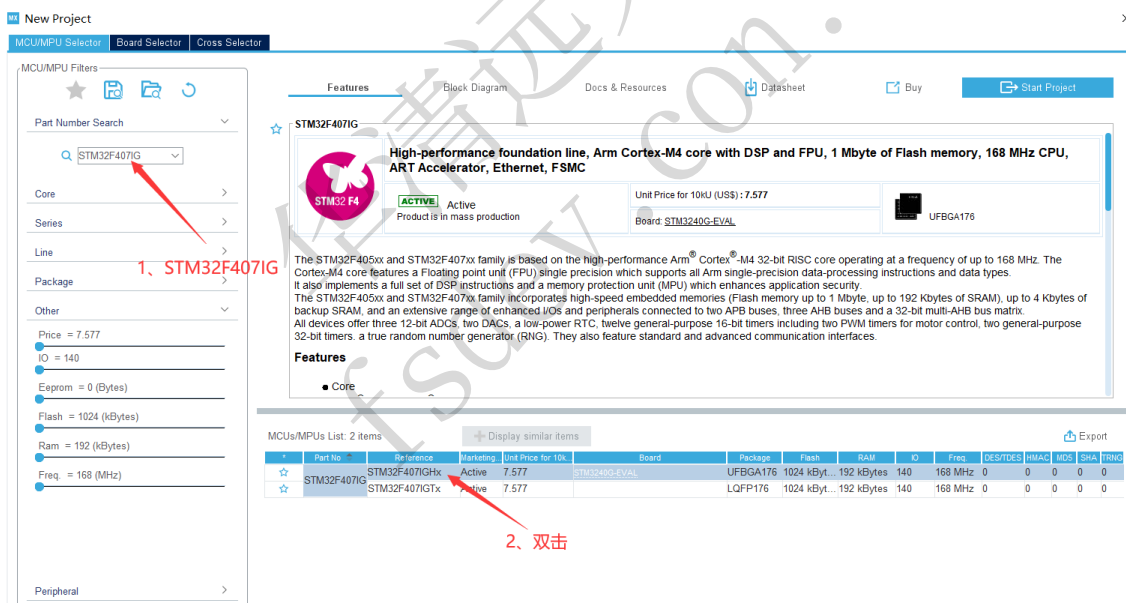
实现过程，请参考 stm32f4xx_hal_gpio.h 文件

3.1.5 【stm32cubemx 配置介绍】

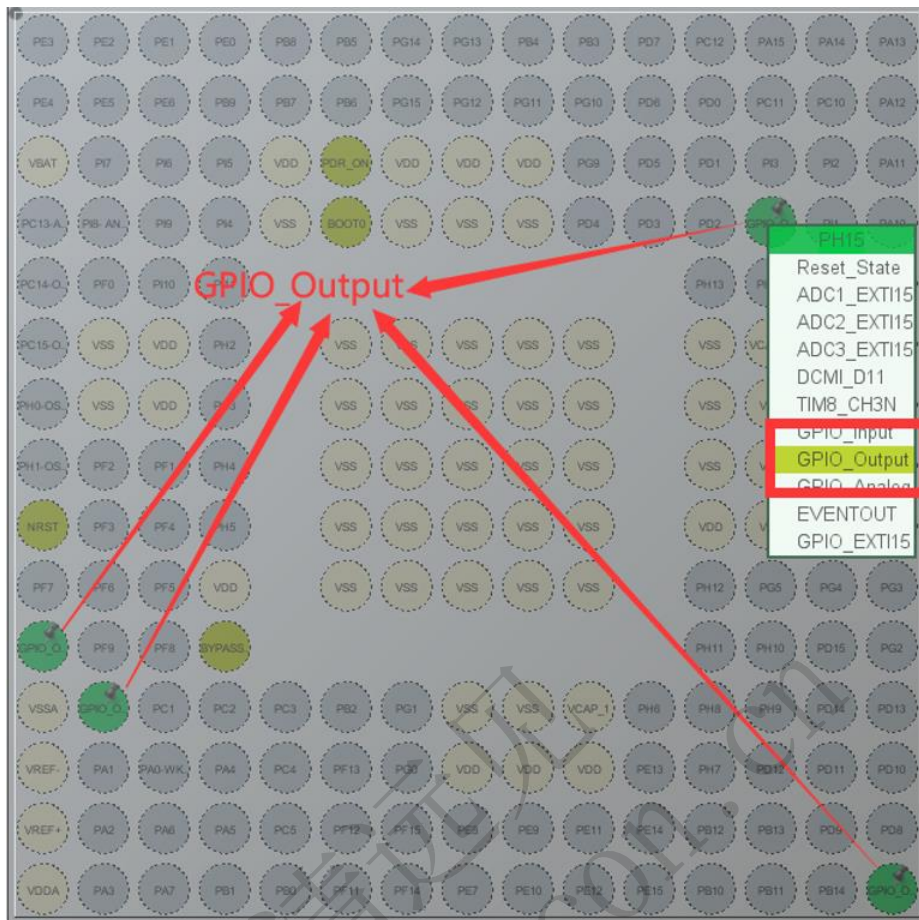
(1) 打开 STM32CubeMX->点击左上角的 File->点击 New Project



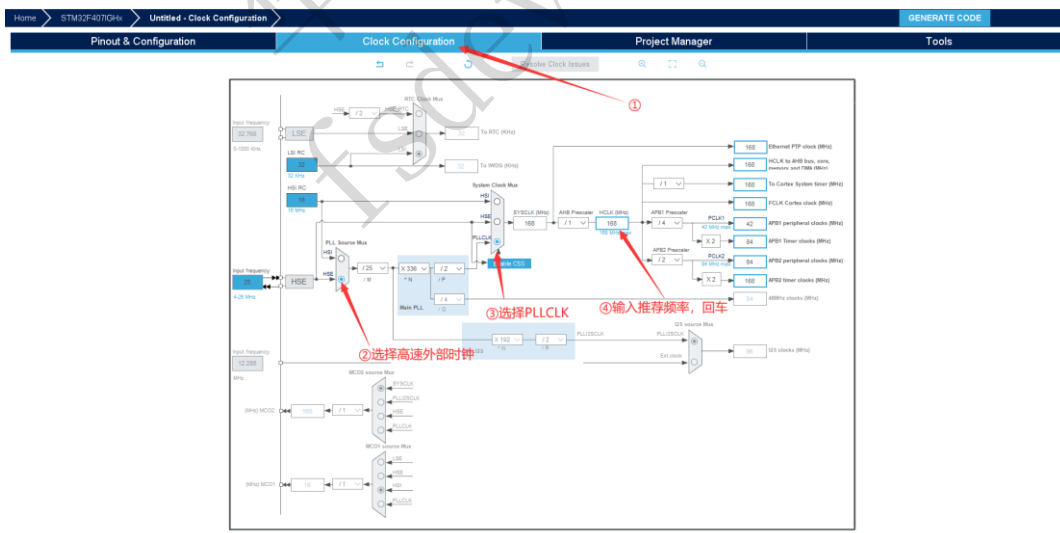
(2) 选择芯片，查看芯片的型号，在搜索框输入芯片型号，本实验中选用的芯片是 STM32F407IGHx



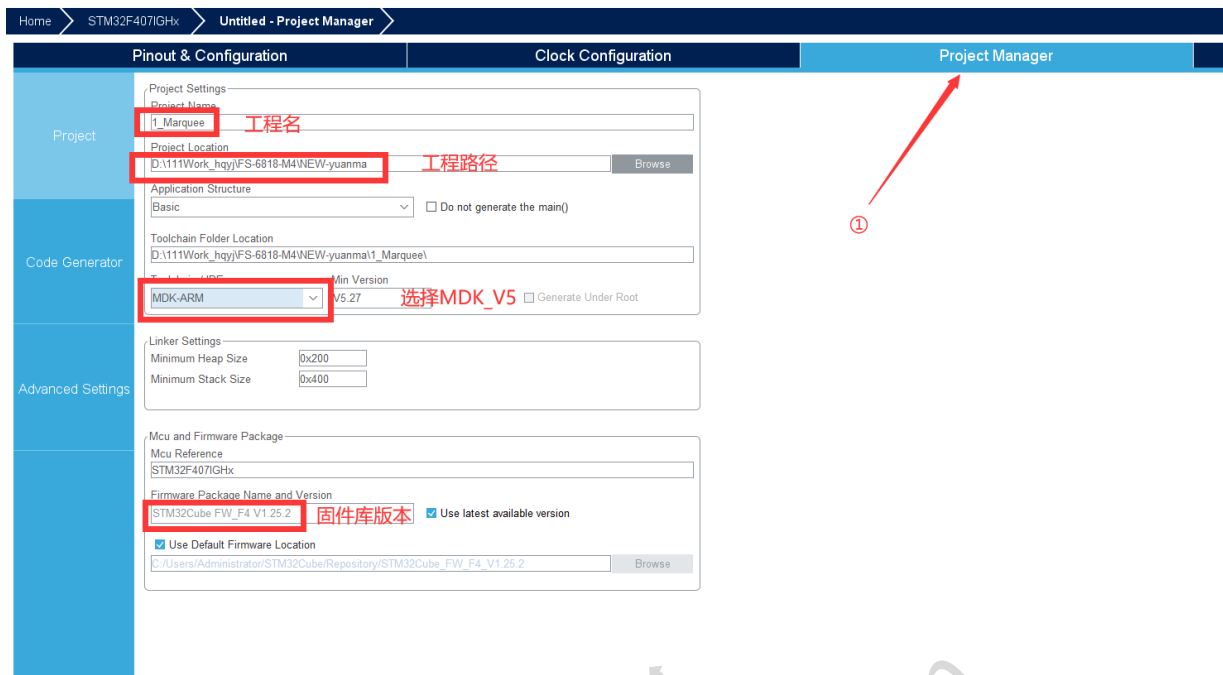
(3) 配置 PF10/PC0/PB15/PH15 为 GPIO_Output 模式，使用默认的 GPIO 即可配置即可



- (4) 配置 RCC，点击左侧的 RCC，找到 High Speed Clock (HSE)，点击选择 Crystal/Ceramic Resonator，RCC 配置完成，点击上方的 Clock Configuration，配置时钟树。



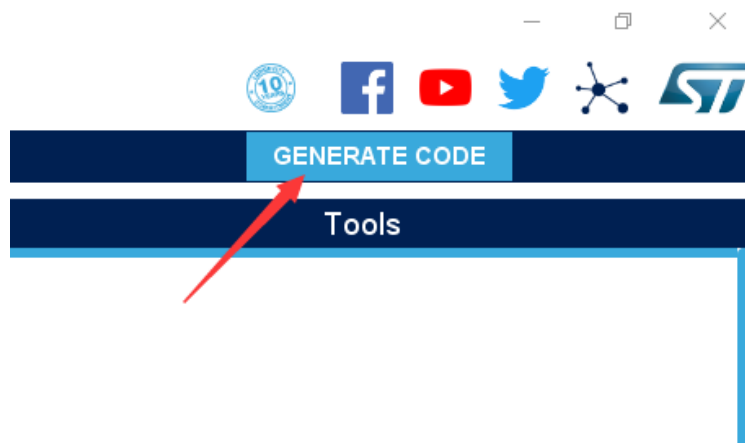
- (5) 配置工程名字，工程路径和 IDE 软件（MDK-ARM V5.27），（注意不要有中文路径）。

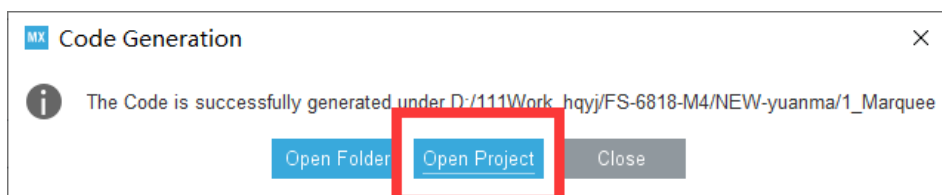


- (6) 工程软件生成配置，配置工程为 C 文件和 h 文件分开，以及拷贝使用到的 HAL 库文件，其他的文件不拷贝。



- (7) 最后点击右上角的“GENWRTE CODE”，生成工程，生成工程完成后会弹出提示框，选择 Open Project 可直接从 MDK 中打开工程代码。





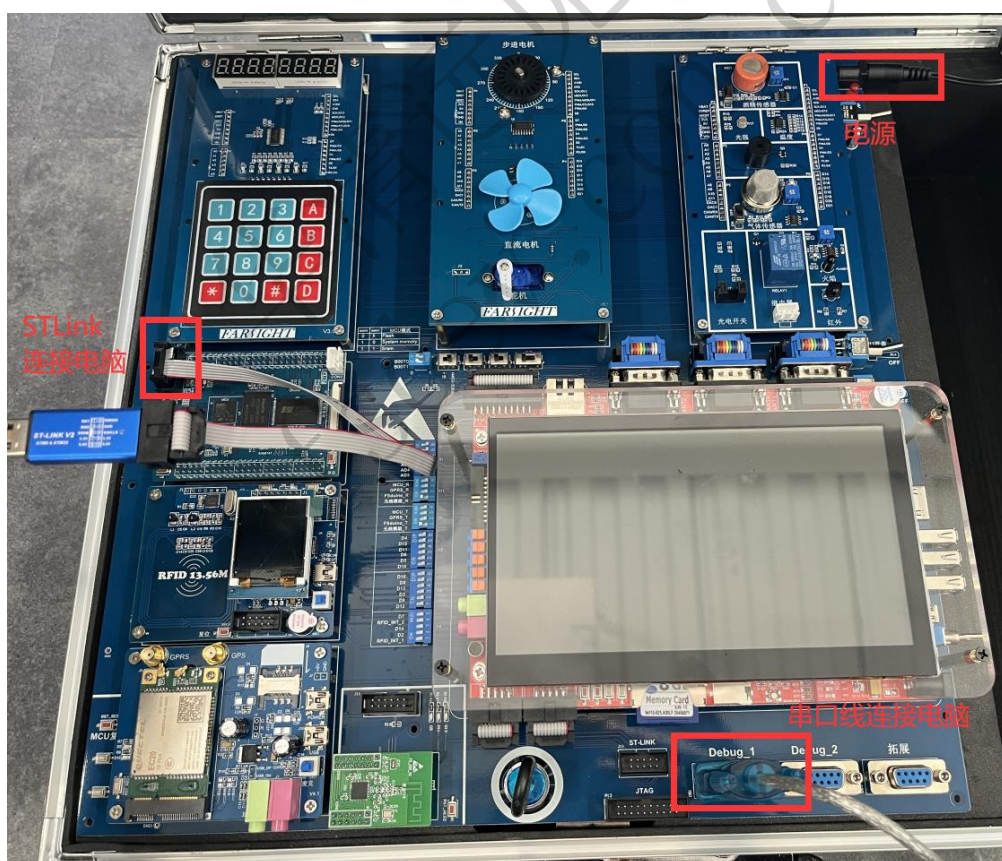
3.1.6 【实验步骤】

- (1) 打开工程文件，路径为

【华清远见-嵌入式 ARM 实验箱资料-III\华清远见-STM 系列资料\Cortex-M4 部分\程序源码

\1_Marquee】

- (2) 对工程进行编译调试
- (3) 开发板连接



- (4) 往开发板下载程序，具体步骤参照 1.4.4 节
- (5) 开发板上电查看实验结果



3.1.7 【实验结果】

可以看到开发板底板上的 LED 灯 D1、D2、D3、D4 循环亮灭



图 LED 实验现象

3.1.8 【实验参考代码】

这里只提供部分重要代码，如想查看部分函数里的详细实现，请打开工程源码，在μVision IDE for ARM 软件里进行查看和操作，这样查找方便清晰、有条理、并可以随意跟进。

```

1      /* Includes -----*/
2      #include "stm32f4xx_hal.h"
3      #include "gpio.h"
4
5      /* USER CODE BEGIN Includes */
6
7      /* USER CODE END Includes */
8
9      /* Private variables -----*/
10

```



```

11  /* USER CODE BEGIN PV */
12  /* Private variables -----*/
13
14  /* USER CODE END PV */
15
16  /* Private function prototypes -----*/
17  void SystemClock_Config(void);
18
19  /* USER CODE BEGIN PFP */
20  /* Private function prototypes -----*/
21  void Turn_On_LED(uint8_t LED_NUM);
22  /* USER CODE END PFP */
23
24  /* USER CODE BEGIN 0 */
25
26  /* USER CODE END 0 */
27
28  int main(void)
29  {
30
31      /* USER CODE BEGIN 1 */
32      uint8_t count = 0;
33      /* USER CODE END 1 */
34
35      /* MCU Configuration-----*/
36
37      /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
38      HAL_Init();
39
40      /* Configure the system clock */
41      SystemClock_Config();
42
43      /* Initialize all configured peripherals */
44      MX_GPIO_Init();
45
46      /* USER CODE BEGIN 2 */
47
48      /* USER CODE END 2 */
49
50      /* Infinite loop */
51      /* USER CODE BEGIN WHILE */
52      while (1)
53      {

```



```

54      /* USER CODE END WHILE */
55
56      /* USER CODE BEGIN 3 */
57      HAL_GPIO_WritePin(GPIOB, GPIO_PIN_15, GPIO_PIN_SET);
58      HAL_GPIO_WritePin(GPIOF, GPIO_PIN_10, GPIO_PIN_SET);
59      HAL_GPIO_WritePin(GPIOH, GPIO_PIN_15, GPIO_PIN_SET);
60      HAL_GPIO_WritePin(GPIOC, GPIO_PIN_0, GPIO_PIN_SET);
61
62      Turn_On_LED(count % 4); //点亮一个 LED 灯
63      count++;
64      HAL_Delay(500);
65  }
66  /* USER CODE END 3 */
67
68  }
69
70  /** System Clock Configuration
71  */
72  void SystemClock_Config(void)
73  {
74
75      RCC_OscInitTypeDef RCC_OscInitStruct;
76      RCC_ClkInitTypeDef RCC_ClkInitStruct;
77
78      __PWR_CLK_ENABLE();
79
80      __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE1);
81
82      RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
83      RCC_OscInitStruct.HSEState = RCC_HSE_ON;
84      RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
85      RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
86      RCC_OscInitStruct.PLL.PLLM = 25;
87      RCC_OscInitStruct.PLL.PLLN = 336;
88      RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV2;
89      RCC_OscInitStruct.PLL.PLLQ = 4;
90      HAL_RCC_OscConfig(&RCC_OscInitStruct);
91
92      RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_SYSCLK | RCC_CLOCKTYPE_PCLK1
93                                  | RCC_CLOCKTYPE_PCLK2;
94      RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
95      RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
96      RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV4;

```




```

97     RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV2;
98     HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_5);
99
100    HAL_SYSTICK_Config(HAL_RCC_GetHCLKFreq() / 1000);
101
102    HAL_SYSTICK_CLKSourceConfig(SYSTICK_CLKSOURCE_HCLK);
103
104    /* SysTick_IRQn interrupt configuration */
105    HAL_NVIC_SetPriority(SysTick_IRQn, 0, 0);
106 }
107
108 /* USER CODE BEGIN 4 */
109 void Turn_On_LED(uint8_t LED_NUM)
110 {
111     switch(LED_NUM)
112     {
113     case 0:
114         HAL_GPIO_WritePin(GPIOH, GPIO_PIN_15, GPIO_PIN_RESET); /*点亮D4灯*/
115         break;
116     case 1:
117         HAL_GPIO_WritePin(GPIOB, GPIO_PIN_15, GPIO_PIN_RESET); /*点亮D3灯*/
118         break;
119     case 2:
120         HAL_GPIO_WritePin(GPIOC, GPIO_PIN_0, GPIO_PIN_RESET); /*点亮D2灯*/
121         break;
122     case 3:
123         HAL_GPIO_WritePin(GPIOF, GPIO_PIN_10, GPIO_PIN_RESET); /*点亮D1灯*/
124         break;
125     default:
126         break;
127     }
128
129 }
130 /* USER CODE END 4 */
131
132 #ifndef USE_FULL_ASSERT
133
134 /**
135  * @brief Reports the name of the source file and the source line number
136  * where the assert_param error has occurred.
137  * @param file: pointer to the source file name
138  * @param line: assert_param error line source number
139  * @retval None

```



```

140     */
141 void assert_failed(uint8_t *file, uint32_t line)
142 {
143     /* USER CODE BEGIN 6 */
144     /* User can add his own implementation to report the file name and line number,
145        ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
146     /* USER CODE END 6 */
147
148 }
149
150 #endif
151
152 /**
153  * @}
154  */
155
156 /**
157  * @}
158  */
159
160 /***** (C) COPYRIGHT STMicroelectronics *****/
161

```

3.2 蜂鸣器实验

3.2.1 【实验目的】

熟悉 RealView MDK 工具软件的使用方式。

能够把 ST 官方库文件添加到项目工程。

掌握 RealView MDK 软件的基本设计流程和设计步骤，能够使用工具进行设计、编程、仿真、调试。

掌握如何通过一个 GPIO 管脚控制有源蜂鸣器；

掌握如何控制 STM32 的 IO 口输出；

3.2.2 【实验环境】

嵌入式 ARM 实验箱



ST-Link 仿真器

RealView MDK5.21 集成开发软件

PC 机 XP、Window7/8 (32/64bit)

3.2.3 【实验内容】

蜂鸣器是一种一体化结构的电子讯响器，采用直流电压供电，广泛应用于计算机、打印机、报警器、电子玩具、汽车电子设备、电话机、定时器等电子产品中作发声器件。用于产品的声音提醒或者警告。

编写蜂鸣器程序，实现开发板上的蜂鸣器按一定频率发出滴滴的响声。调试编译好固件程序后，将程序下载到 STM32F407 核心板，按复位键观察实验现象。

3.2.4 【实验原理】

前面流水灯的实验已经介绍过 STM32 处理器 GPIO 接口，如何配置它的模式，时钟速率等，在这一节就不再重复讲解 GPIO 接口的使用。

STM32 的单个 IO 最大可以提供 25mA 的电流。而开发板上的蜂鸣器的驱动电流是 30mA 左右。两者十分相近。如果直接用 I/O 口驱动蜂鸣器的话，其它地方用电就得省着点。所以，我们不用 I/O 口直接驱动蜂鸣器，而是用三极管驱动。

在开发板中，提供了一个蜂鸣器，由处理器的 PG6 管脚控制，当 PG6 管脚输出固定频率时，蜂鸣器发出声音。

蜂鸣器类型：有源蜂鸣器和无源蜂鸣器。

有源蜂鸣器内置振荡电路，直接加电源就可以正常发声，通常频率固定。

无源蜂鸣器则需要通过外部的正弦或方波信号驱动，直接加电源只能发出很轻微的振动声。

查阅 STM32 处理器可知，STM32 处理器 IO 口能够承受的电流为 25mA，如下表所示：



I_{IO}	Output current sunk by any I/O and control pin	25
	Output current source by any I/Os and control pin	25

查看蜂鸣器的原理图，从图中我们可以看出，D15 处的电平决定了蜂鸣器电路的通断。也就是 D15

处为低电平时，PNP 三极管导通，蜂鸣器工作。

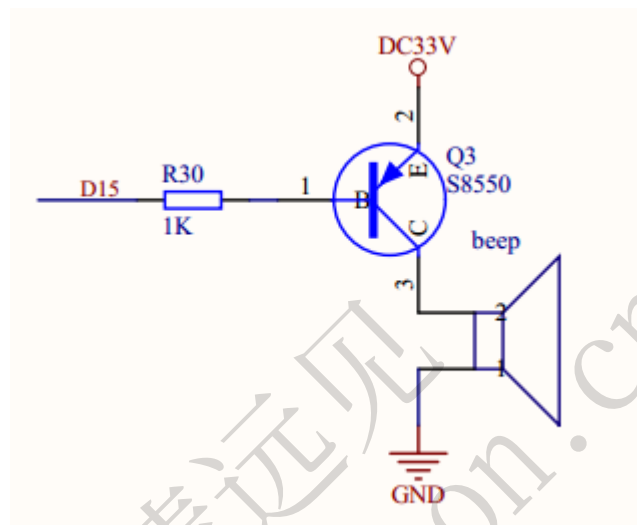
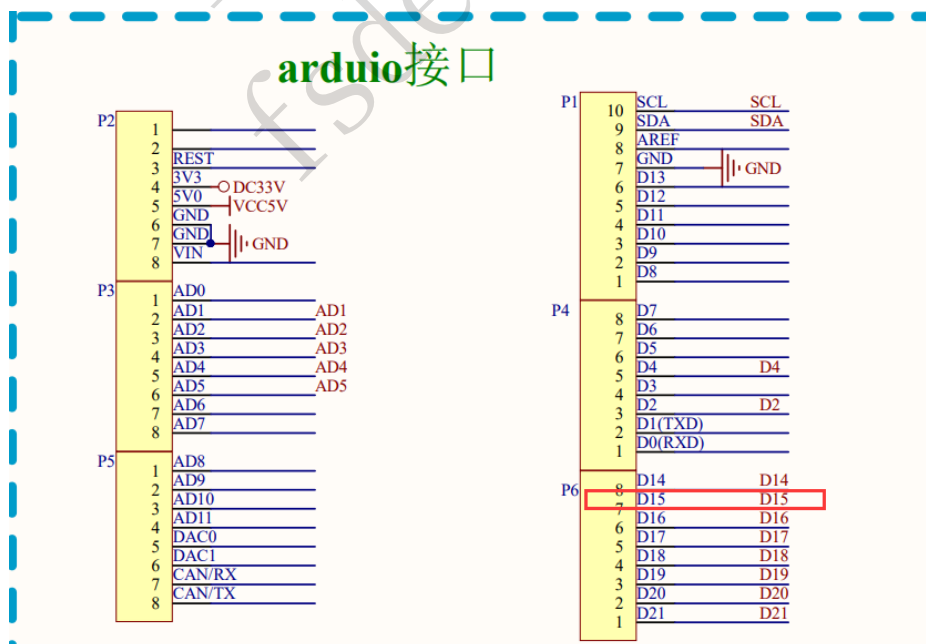
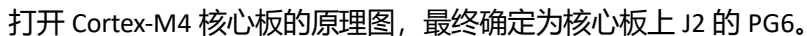
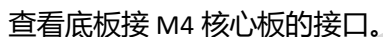


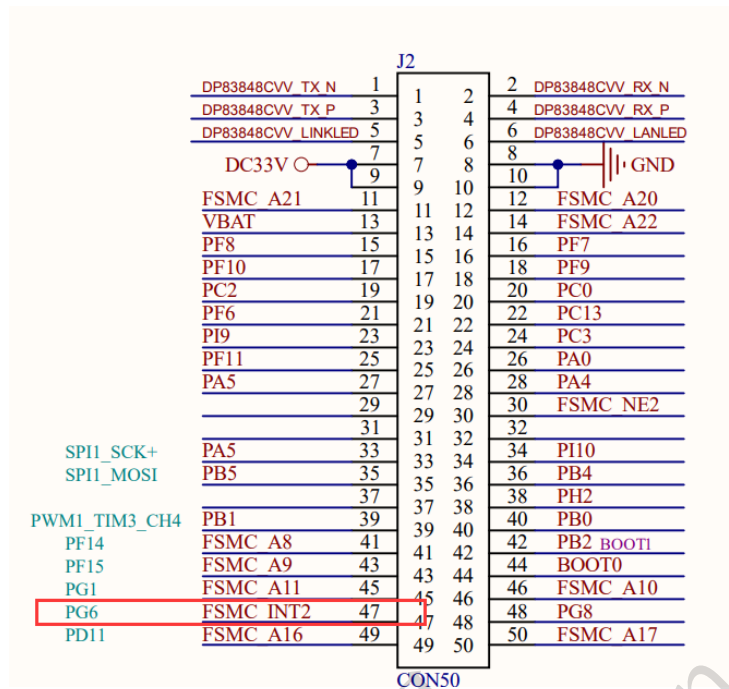
图 蜂鸣器硬件原理图

查看蜂鸣器的 arduino 接口，如下图。

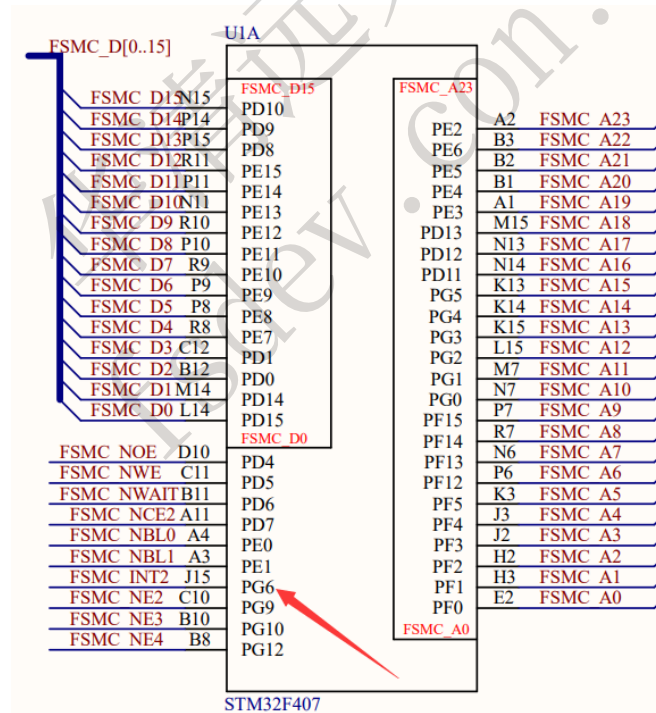


查看实验箱底板图，找到传感器板上的酒精传感器的 D15 对应底板的连接，为 PG6，如下图。



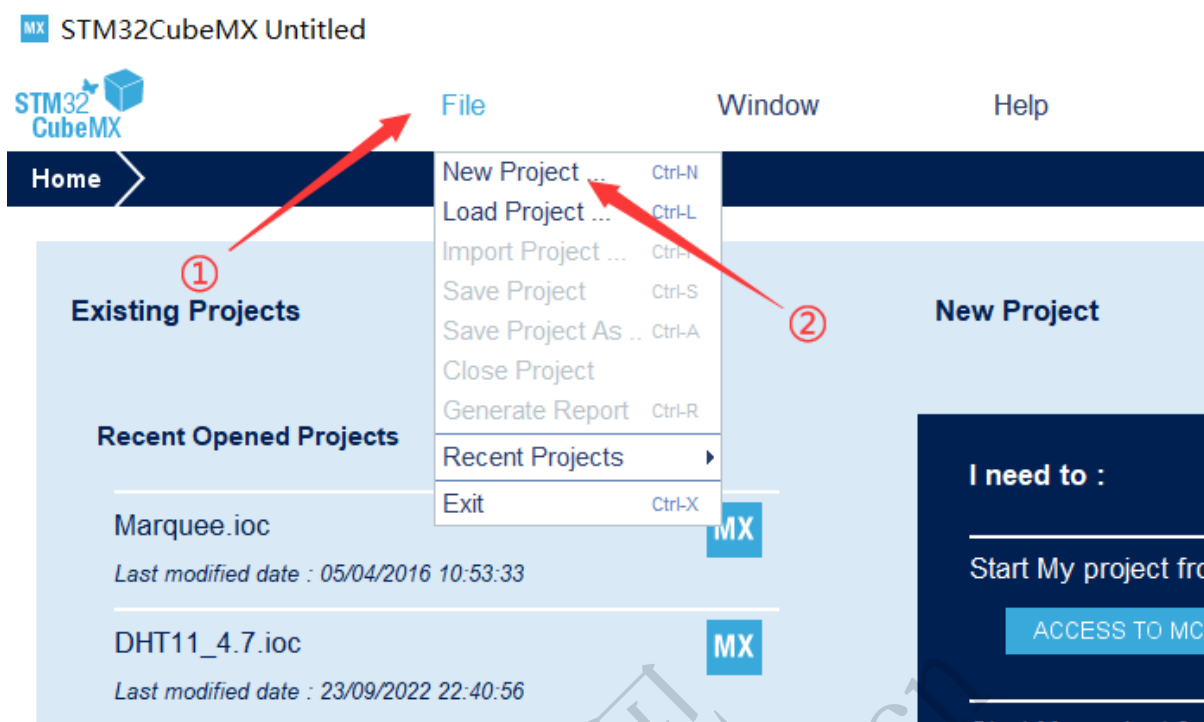


最终，我们确定控制 PG6 引脚的电平，即可控制蜂鸣器。

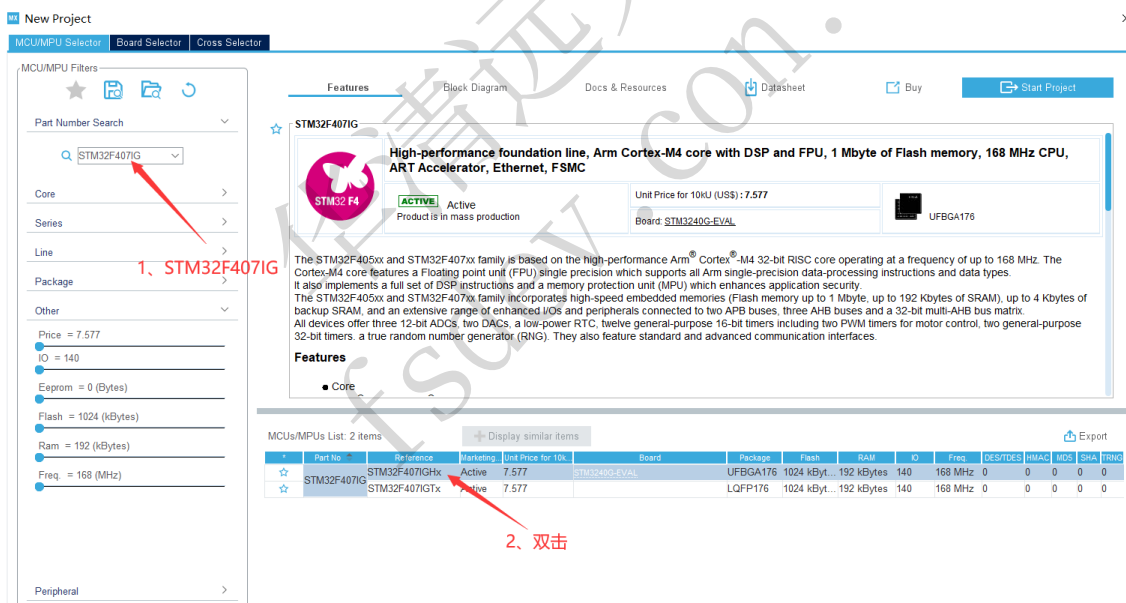


3.2.5 【stm32cubemx 配置介绍】

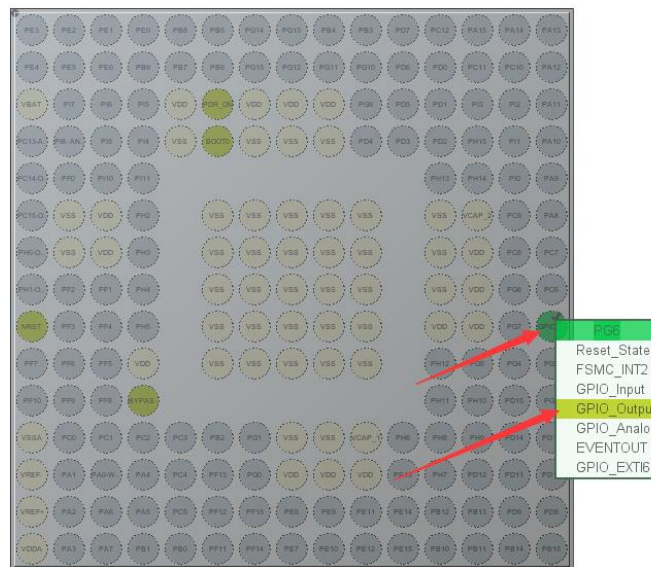
(1) 打开 STM32CubeMX->点击左上角的 File->点击 New Project



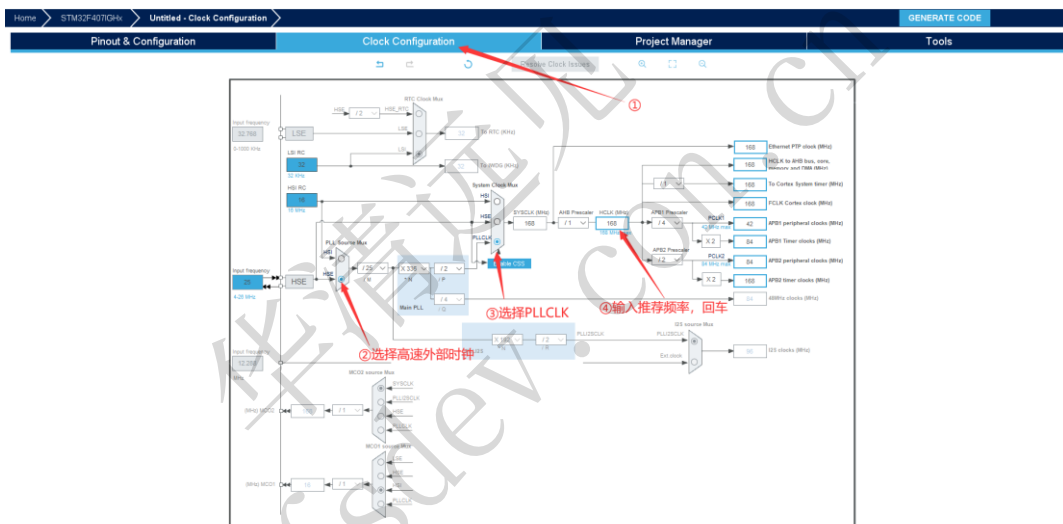
(2) 选择芯片，查看芯片的型号，在搜索框输入芯片型号，本实验中选用的芯片是 STM32F407IGHx



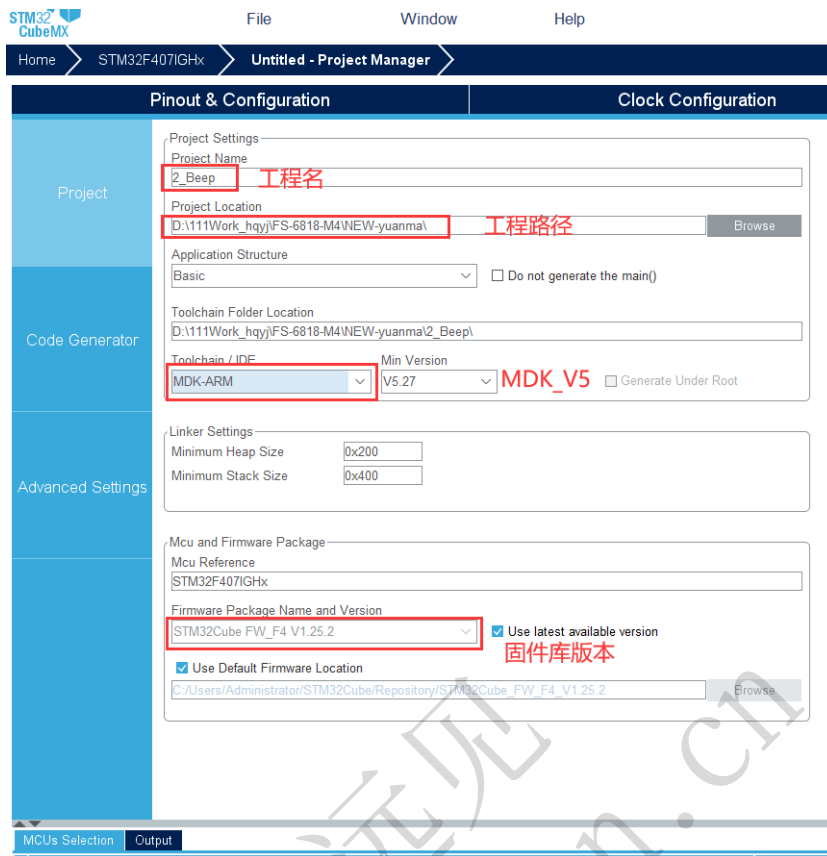
(3) 管脚进行配置，如上一节实验，找到 PG6，将管脚设为 GPIO_Output 模式



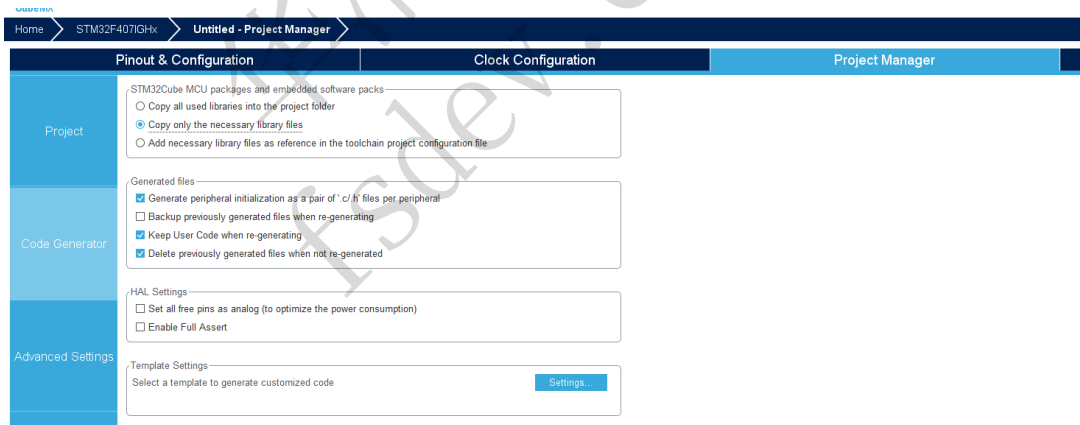
- (4) 配置 RCC，点击左侧的 RCC，找到 High Speed Clock (HSE)，点击选择 Crystal/Ceramic Resonator，RCC 配置完成，点击上方的 Clock Configuration，配置时钟树。



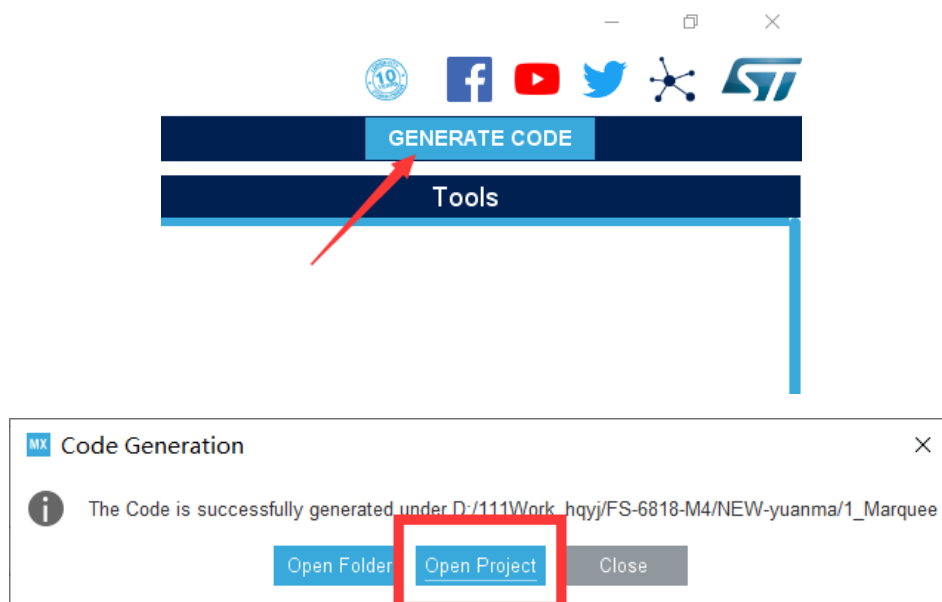
- (5) 配置工程名字，工程路径和 IDE 软件 (MDK-ARM V5.27)，(注意不要有中文路径)。



- (6) 工程软件生成配置，配置工程为 C 文件和 h 文件分开，以及拷贝使用到的 HAL 库文件，其他的文件不拷贝，最后点击右上角的“GENWRTE CODE”，生成工程。



- (7) 生成工程完成后会弹出提示框，选择 Open Project 可直接从 MDK 中打开工程代码。



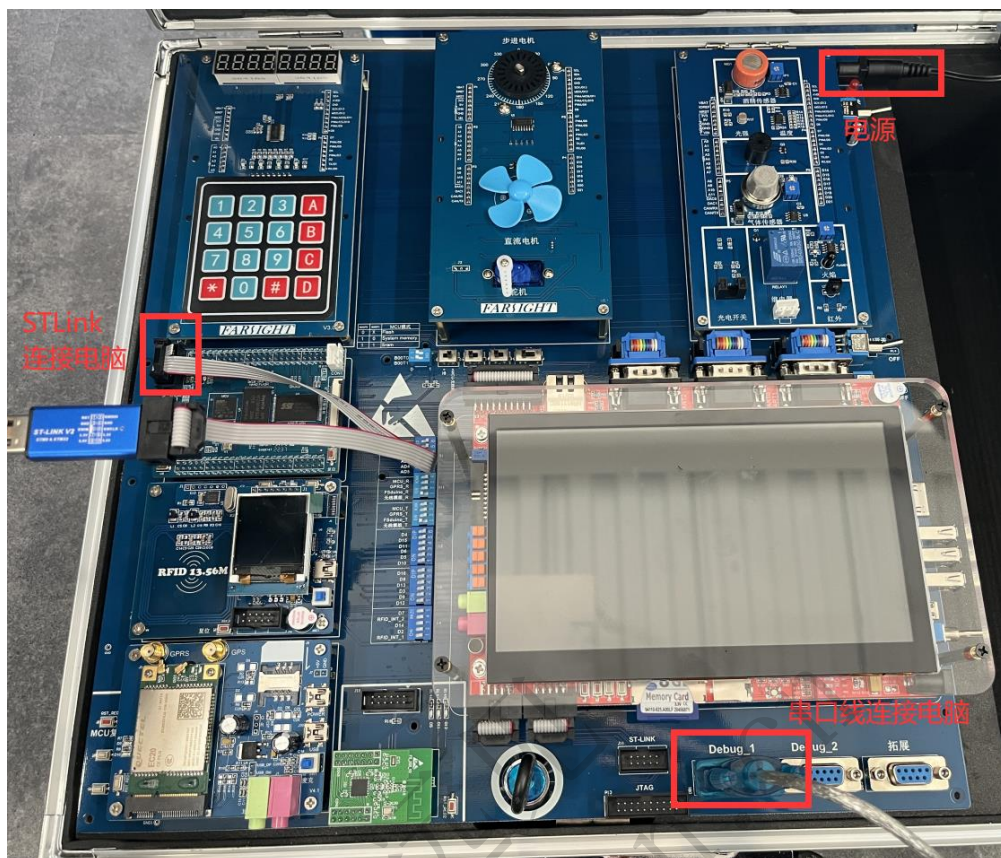
3.2.6 【实验步骤】

(1) 打开工程文件，路径为

【[华清远见-嵌入式 ARM 实验箱资料-III\华清远见-STM 系列资料\Cortex-M4 部分\程序源码\2_Beep](#)】

(2) 对工程进行编译调试

(3) 开发板连接



- (4) 往开发板下载程序，具体步骤参照 1.4.4 节
- (5) 开发板上电查看实验结果

3.2.7 【实验结果】

可以听到蜂鸣器发出的响声。蜂鸣器位置如下图红圈所示：



图 蜂鸣器位置图



3.2.8 【实验参考代码】

这里只提供部分重要代码，如想查看部分函数里的详细实现，请打开工程源码，在μVision IDE for ARM 软件里进行查看和操作，这样查找方便清晰、有条理、并可以随意跟进。

```

1  /* Includes -----*/
2  #include "stm32f4xx_hal.h"
3  #include "gpio.h"
4
5  /* USER CODE BEGIN Includes */
6
7  /* USER CODE END Includes */
8
9  /* Private variables -----*/
10
11 /* USER CODE BEGIN PV */
12 /* Private variables -----*/
13
14 /* USER CODE END PV */
15
16 /* Private function prototypes -----*/
17 void SystemClock_Config(void);
18
19 /* USER CODE BEGIN PFP */
20 /* Private function prototypes -----*/
21
22 /* USER CODE END PFP */
23
24 /* USER CODE BEGIN 0 */
25
26 /* USER CODE END 0 */
27
28 int main(void)
29 {
30
31     /* USER CODE BEGIN 1 */
32
33     /* USER CODE END 1 */
34
35     /* MCU Configuration-----*/
36
37     /* Reset of all peripherals, Initializes the Flash interface and the SysTick. */

```



```

38     HAL_Init();
39
40     /* Configure the system clock */
41     SystemClock_Config();
42
43     /* Initialize all configured peripherals */
44     MX_GPIO_Init();
45
46     /* USER CODE BEGIN 2 */
47
48     /* USER CODE END 2 */
49
50     /* Infinite loop */
51     /* USER CODE BEGIN WHILE */
52     while (1)
53     {
54         /* USER CODE END WHILE */
55
56         /* USER CODE BEGIN 3 */
57         HAL_GPIO_WritePin(GPIOG, GPIO_PIN_6, GPIO_PIN_SET); //打开蜂鸣器
58         HAL_Delay(2); //50HZ 频率
59         HAL_GPIO_WritePin(GPIOG, GPIO_PIN_6, GPIO_PIN_RESET); //关闭蜂鸣器
60         HAL_Delay(2); //50HZ 频率
61     }
62     /* USER CODE END 3 */
63
64 }
```

3.3 直流电机驱动实验

3.3.1 【实验目的】

STM32 的 IO 口在前面的流水灯实验中已经有了详细的介绍，这里我们不再细讲。我们将熟练控制直流电机的正转和反转。

3.3.2 【实验环境】

嵌入式 ARM 实验箱

ST-Link 仿真器



RealView MDK5.21 集成开发软件

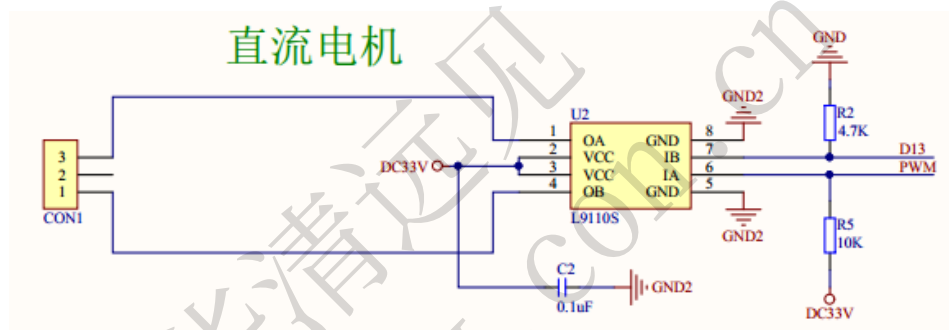
PC 机 XP、Window7/8 (32/64bit)

3.3.3 【实验内容】

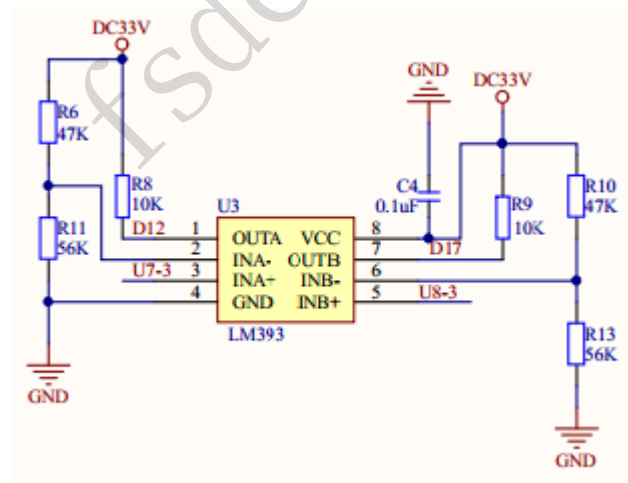
编写程序，实现嵌入式 ARM 实验箱上的直流电机的正反转实验。调试编译好固件程序后，将程序下载到 STM32F407 核心板，按复位键观察实验现象。

3.3.4 【实验原理】

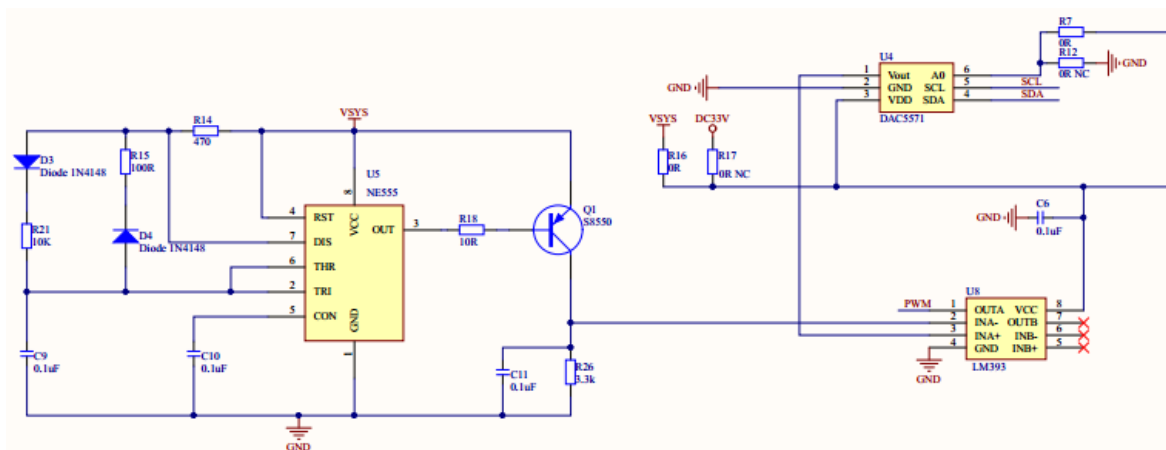
该实验需要使用到开发板上的直流电机，相关硬件电路如下：



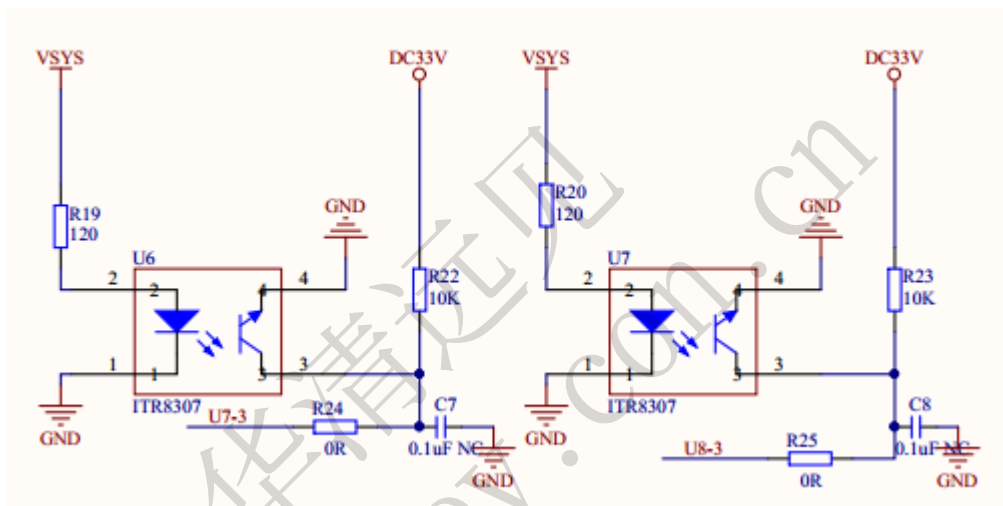
直流电机驱动电路



电压比较器电路



DAC 及 NE555 电路



光电开关电路

L9110 描述：

L9110 是为控制和驱动电机设计的两通道推挽式功率放大专用集成电路器件，将分立电路集成在单片 IC 之中，使外围器件成本降低，整机可靠性提高。该芯片有两个 TTL/CMOS 兼容电平的输入，具有良好的抗干扰性；两个输出端能直接驱动电机的正反向运动，它具有较大的电流驱动能力，每通道能通过 800mA 的持续电流，峰值电流能力可达 1.5A；（直流小电机一般需要 350mA）在驱动继电器、直流电机、步进电机或开关功率管的使用上安全可靠。L9110 被广泛应用于玩具汽车电机驱动、脉冲电磁阀驱动，步进电机驱动和开关功率管等电路上。

555 定时器成本低，性能可靠。只需要外接几个电阻，电容就可以实现多谐振荡器。单稳态触发器以



及斯密特触发器等脉冲产生与变换的电路。它也可以作为定时器广泛应用于仪器仪表，家用电器，电子测量以及自动控制等方面。

NE555 为 8 脚的时基集成电路。它主要的特点就是仅仅需要简单的电阻器，电容器即可完成特定的震荡延时作用。其延时范围极广。可以几微妙到几个小时。它的操作电源也极大，可以与 TTL、CMOS 等逻辑电路配合使用，也就是它的输出电平以及输入触发电平。均能与这些系列的逻辑电路的高低电平匹配。它的计时精度高，温度的稳定性佳，而且价格便宜。

555 集成电路开始是作定时器应用的，所以叫做 555 定时器或 555 时基电路。但后来经过开发，它除了作定时延时控制外，还可用于调光、调温、调压、调速等多种控制及计量检测。此外，还可以组成脉冲振荡、单稳、双稳和脉冲调制电路，用于交流信号源、电源变换、频率变换、脉冲调制等。由于它工作可靠、使用方便、价格低廉，目前被广泛用于各种电子产品中，555 集成电路内部有几十个元器件，有分压器、比较器、基本 R-S 触发器、放电管以及缓冲器等，电路比较复杂，是模拟电路和数字电路的混合体。

本次设计当中，利用 NE555 设计出多谐振荡器。（使用者可以查看相关资料，本节不进行详细说明）。

多谐振荡器又称为无稳态触发器。它没有稳定的输出状态，只有两个暂稳态，在电路处于某一暂稳态后，经过一段时间就可以自行触发反转到另外的暂稳态。两个暂稳态自行相互转换而输出一系列的矩形波。多谐振荡器可以作为方波发生器。

它的各个引脚的功能：

1 脚：外接电源的负端 SS 或者接地，一边情况下接地。

8 脚：外接电源 VCC, 双极型时基电路 VCC 的范围是 4.5-16V CMOS 型时基电路 VCC 的范围。

3 脚：输出端 VO

2 脚：低触发端



6 脚：TH 高触发端

4 脚：是直接清 0 端。当此端接低电平，则时基电路不工作，此时不讨论 TR TH 处于何种电平，时基电路输出为 0，该端不用的时候应接高电平。

5 脚：VC 为控制电压端。若此端外接电压，则可以改变内部两个比较器的基准电压。当该端不用的时候，应该将该端串入一只 0.01uf 的电容接地，以防止引入干扰。

7 脚：放电端。该端与放电管集电极相连，用做定时器电容的放电操作。

接通电源后，假定为高电平，电容 C9 开始充电，充电的回路是 V_{SYS}-R14-D3-R21-C9-地。按指数规律上升，当上升时，（TH 端电平大于 $>2/3V_{CC}$ ），输出反转为低电平。是低电平，T 导通，C 放电。放电回路：C9-D4-R15-R19 按指数规律下降，当下降时，（TH 端电平 $<1/3V_{CC}$ ），输出反转为高电平。

输出高电平的时间： $T = (R14+R21)C\ln 2$

输出低电平的时间： $T = R15C\ln 2$

振荡周期： $T = (R14+R21+R15)C\ln 2$

这样周而复始的产生波形。

DAC5571 是 TI 数字模拟转换器，一路电压输出的 8 位 DA。它使用 2.7v 到 5.5v 6 个引脚的供电电压。对芯片通过 IIC 进行通信。通知该芯片产生一个电压值。

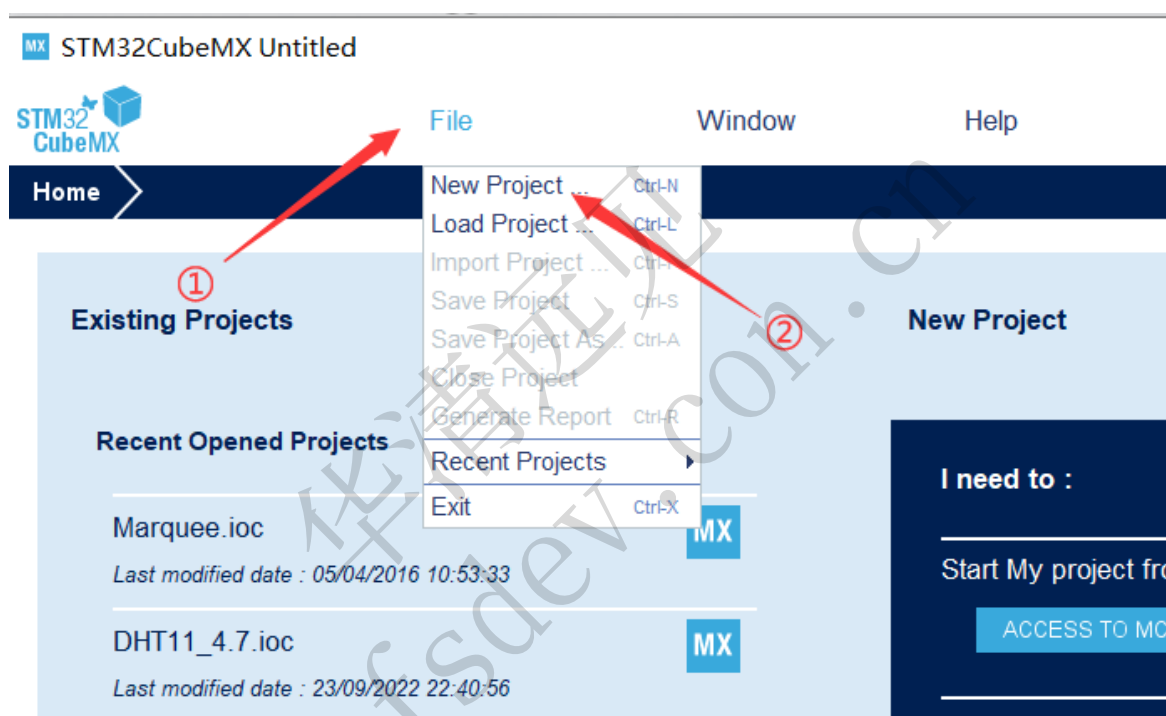
我们可以看到 RC 震荡电路产生的电压和 DAC5571 的产生的电压进入到 LM393 当中。LM393 是双电压比较器集成电路。当 $INA+$ 大于 $INA-$ 时，OUTA 输出高电平，当 $INA-$ 大于 $INA+$ 的时候，输出低电平。通过原理图，我们可以看到 DAC5571 产生的电压连接到 $INA+$ 端，NE555 控制的 RC 震荡电路产生的波形连接到 $INA-$ 端，这样就形成一种电压比较的形式。当 DAC5571 电压大于 RC 电路当前的电压的时候，产生低电平。当 RC 两端的电压大于 DAC5571 的电压的时候，产生高电平。这样电压比较器的 OUTA 端不就是输出高低电平了，从而相当于输出 PWM 波形来进行电路的控制。当我们改变 DAC5571 的电压值的过程中，LM393 的高低电平的占空比就会变化，从而改变了 PWM 波，这样就可以控制直流电机的速度了。



那么如何进行直流电机测速呢。本次设计利用的 ITR8307 反射式光电开关管。它的工作原理就是，震荡回路产生的调制脉冲经反射电路后，由光电管 GL 辐射出光电脉冲，当被测物体进行受光器作用范围时，被反射回来的光脉冲进入光敏三极管，并在接收电路中将光脉冲解调为电脉冲信号。在经过比较器进行整形，后送给单片机。

3.3.5 【stm32cubemx 配置介绍】

(1) 打开 STM32CubeMX->点击左上角的 File->点击 New Project



(2) 选择芯片，查看芯片的型号，在搜索框输入芯片型号，本实验中选用的芯片是 STM32F407IGHx



New Project

MCU/MPU Selector Board Selector Cross Selector

MCU/MPU Filters

Part Number Search

STM32F407G

Core

Series

Line

Package

Other

Price = 7.577

IO = 140

Espram = 0 (Bytes)

Flash = 1024 (KBytes)

Ram = 192 (KBytes)

Freq = 168 (MHz)

Peripheral

Features

Block Diagram

Docs & Resources

Datasheet

Buy

Start Project

STM32F407G

High-performance foundation line, Arm Cortex-M4 core with DSP and FPU, 1 Mbyte of Flash memory, 168 MHz CPU, ART Accelerator, Ethernet, FSMC

ACTIVE Active

Unit Price for 10K (US\$): 7.577

Board: STM3240G-EVAL

UFPGA176

The STM32F405xx and STM32F407xx family is based on the high-performance Arm® Cortex® M4 32-bit RISC core operating at a frequency of up to 168 MHz. The Cortex-M4 core features a Floating point unit (FPU) single precision which supports all Arm single-precision data-processing instructions and data types. It also implements a full set of DSP instructions and a memory protection unit (MPU) which enhances application security. The STM32F405xx and STM32F407xx family incorporates high-speed embedded memories (Flash memory up to 1 Mbyte, up to 192 Kbytes of SRAM), up to 4 Kbytes of backup SRAM, and an extensive range of enhanced I/Os and peripherals connected to two APB buses, three AHB buses and a 32-bit multi-AHB bus matrix. All devices offer three 12-bit ADCs, two DACs, a low-power RTC, twelve general-purpose 16-bit timers including two PWM timers for motor control, two general-purpose 32-bit timers, a true random number generator (RNG). They also feature standard and advanced communication interfaces.

Features

Core

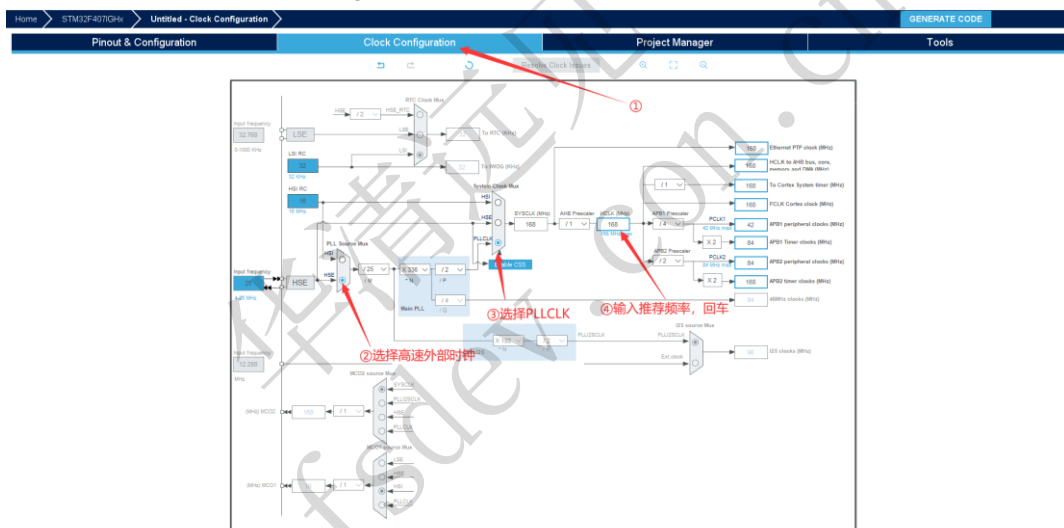
MCUs/MPUs List: 2 items

Display similar items

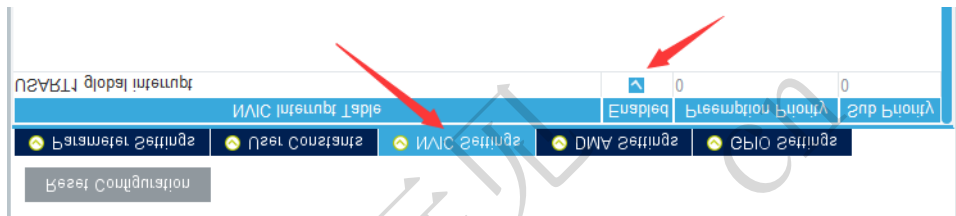
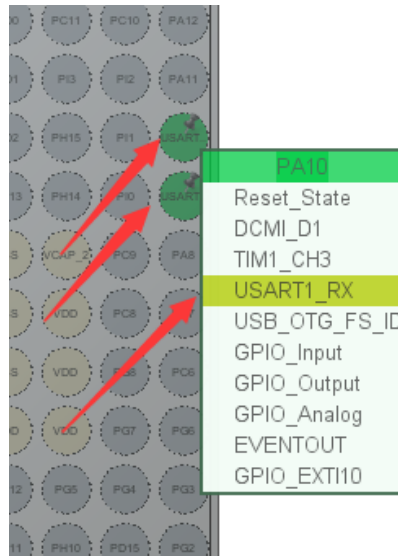
Part No.	Reference	Marketing	Unit Price for 10k	Board	Package	Flash	RAM	IO	Freq	DES/IDES	HWAC	MDS	SH4	TRNG
STM32F407G	STM32F407G	Active	7.577	STM3240G-EVAL	UFPGA176	1024 KByte	192 KBytes	140	168 MHz	0	0	0	0	0
STM32F407G	STM32F407G	Active	7.577	STM3240G-EVAL	LQFP176	1024 KByte	192 KBytes	140	168 MHz	0	0	0	0	0

Export

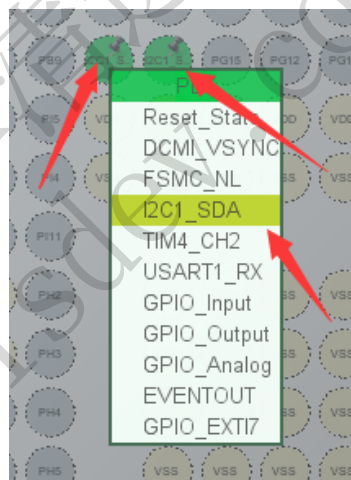
- (3) 配置 RCC，点击左侧的 RCC，找到 High Speed Clock (HSE)，点击选择 Crystal/Ceramic Resonator，RCC 配置完成，点击上方的 Clock Configuration，配置时钟树。



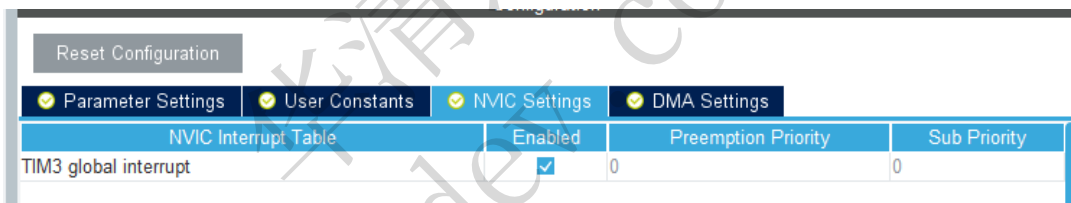
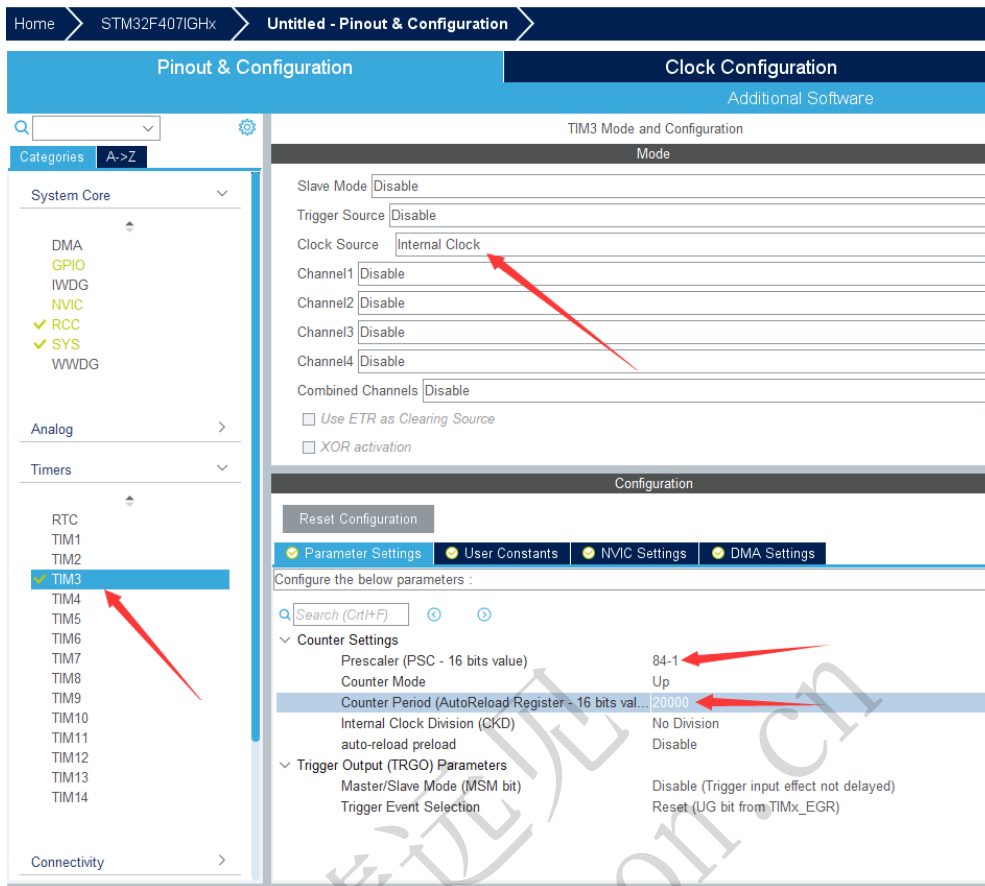
- (4) 配置 PA9 为 USART1_TX，配置 PA10 为 USART1_RX。波特率为 115200，数据长度 8 位，无校验位，一位停止位，使能中断。



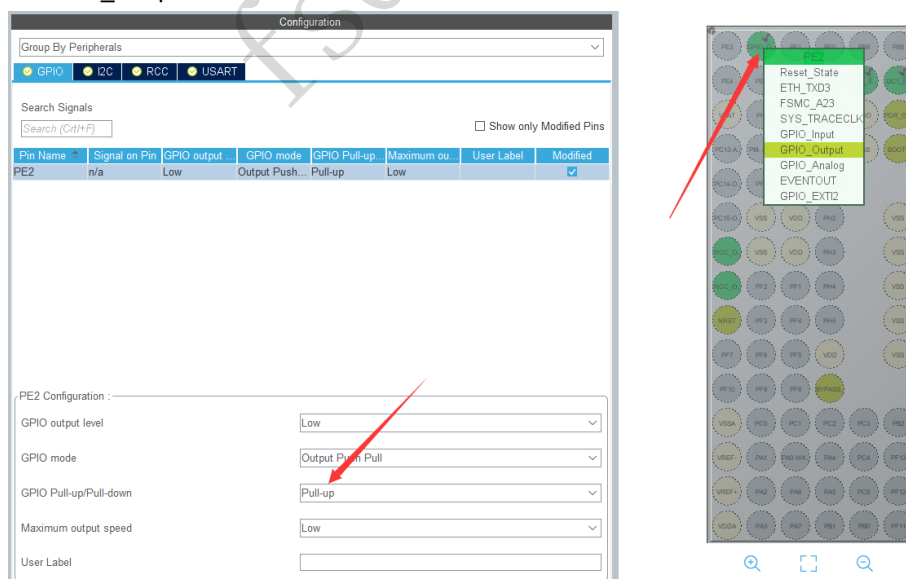
(5) 配置 PB6 为 I2C1_SCL, PB7 为 I2C1_SDA, I2C 配置使用默认配置即可。



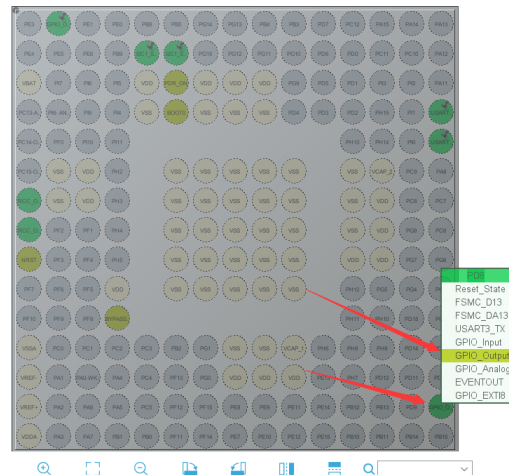
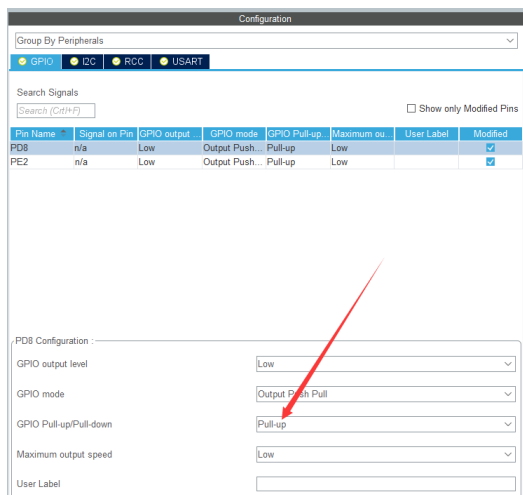
(6) 配置 TIM3, 使能中断



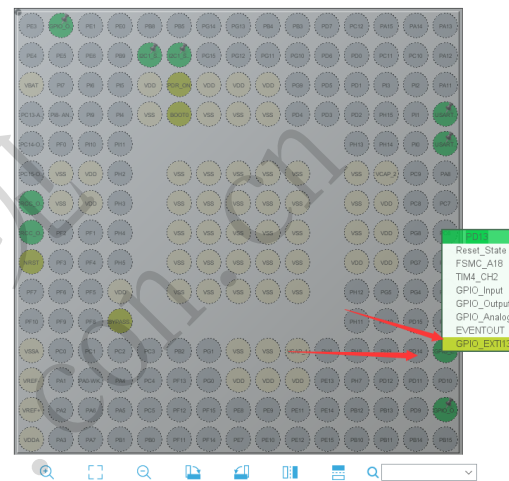
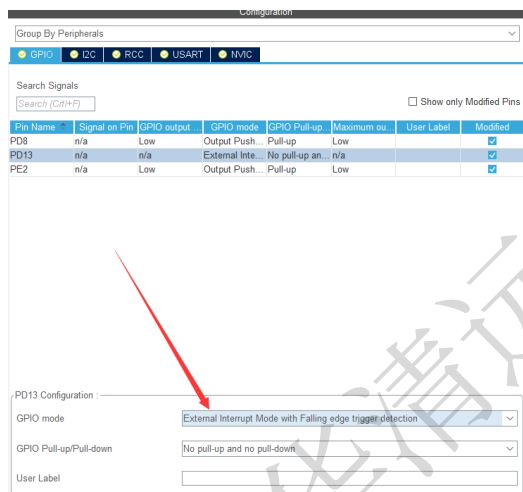
(7) 设置 PE2 为 GPIO_Output 模式，选择上拉模式。



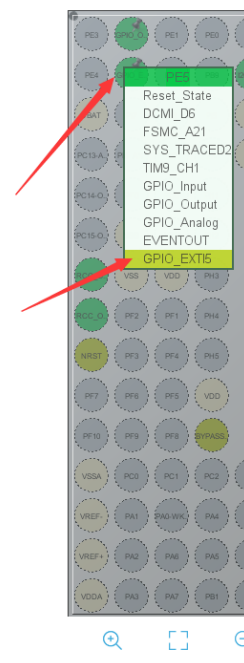
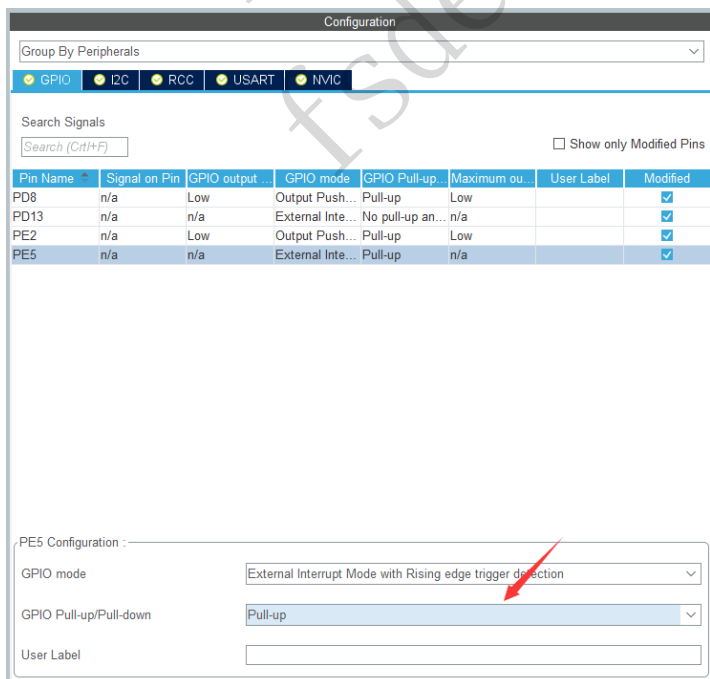
(8) 配置 PD8 为 GPIO_Output 模式，选择上拉模式。



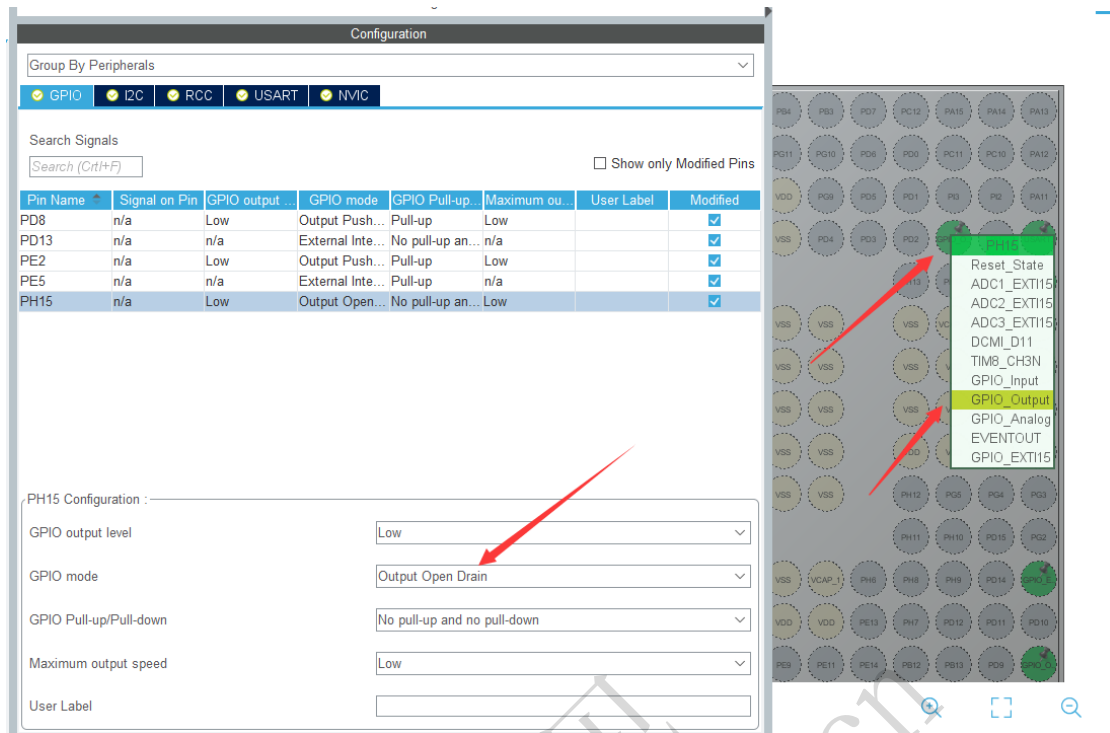
(9) 配置 PD13 引脚为 GPIO_EXTI13，下降沿触发。



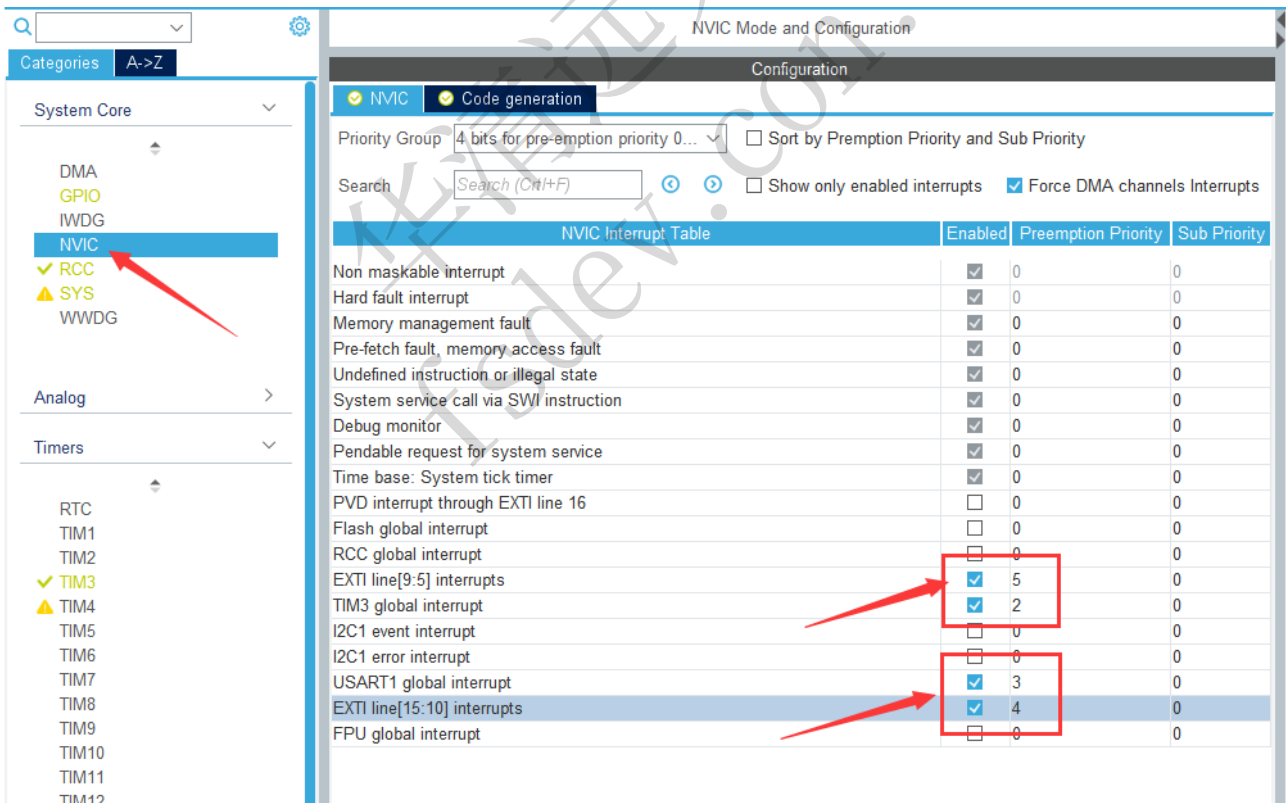
(10) 配置 PE5 为 GPIO_EXTI5，下降沿触发，上拉模式。



(11) 配置 PH15 为 GPIO_Output 模式，GPIO 模式设置为开漏输出。



(12) 使能 NVIC 的中断线，配置优先级。



(13) 配置工程名字，工程路径和 IDE 软件（MDK-ARM V5.27），（注意不要有中文路径）。



Home > STM32F407IGHx > Untitled - Project Manager

Pinout & Configuration	Clock Configuration
Project	Project Settings
Code Generator	Project Name: 3_DC_Motor 工程名
Advanced Settings	Project Location: D:\Y111Work_hqyj\FS-6818-M4\NEW-yuanma\ 工程路径
	Application Structure: Basic ☐ Do not generate the main()
	Toolchain Folder Location: D:\Y111Work_hqyj\FS-6818-M4\NEW-yuanma\3_DC_Motor\
	Toolchain / IDE: MDK-ARM Min Version: V5.27 MDK_V5 ☐ Generate Under Root
	Linker Settings
	Minimum Heap Size: 0x200
	Minimum Stack Size: 0x400
	Mcu and Firmware Package
	Mcu Reference: STM32F407IGHx
	Firmware Package Name and Version: STM32Cube_FW_F4_V1.25.2 固件库版本
	☒ Use latest available version
	☒ Use Default Firmware Location
	C:/Users/Administrator/STM32Cube/Repository/STM32Cube_FW_F4_V1.25.2

- (14) 工程软件生成配置，配置工程为 C 文件和 h 文件分开，以及拷贝使用到的 HAL 库文件，其他的 文件不拷贝。

Pinout & Configuration	Clock Configuration
Project	STM32Cube MCU packages and embedded software packs
Code Generator	☐ Copy all used libraries into the project folder ☒ Copy only the necessary library files ☐ Add necessary library files as reference in the toolchain project configuration file
Advanced Settings	Generated files
	☒ Generate peripheral initialization as a pair of '.c/.h' files per peripheral ☐ Backup previously generated files when re-generating ☒ Keep User Code when re-generating ☒ Delete previously generated files when not re-generated
	HAL Settings
	☐ Set all free pins as analog (to optimize the power consumption) ☐ Enable Full Assert
	Template Settings
	Select a template to generate customized code Settings...

- (15) 最后点击右上角的“GENWRTE CODE”，生成工程。生成工程完成后会弹出提示框，选择 Open Project 可直接从 MDK 中打开工程代码。

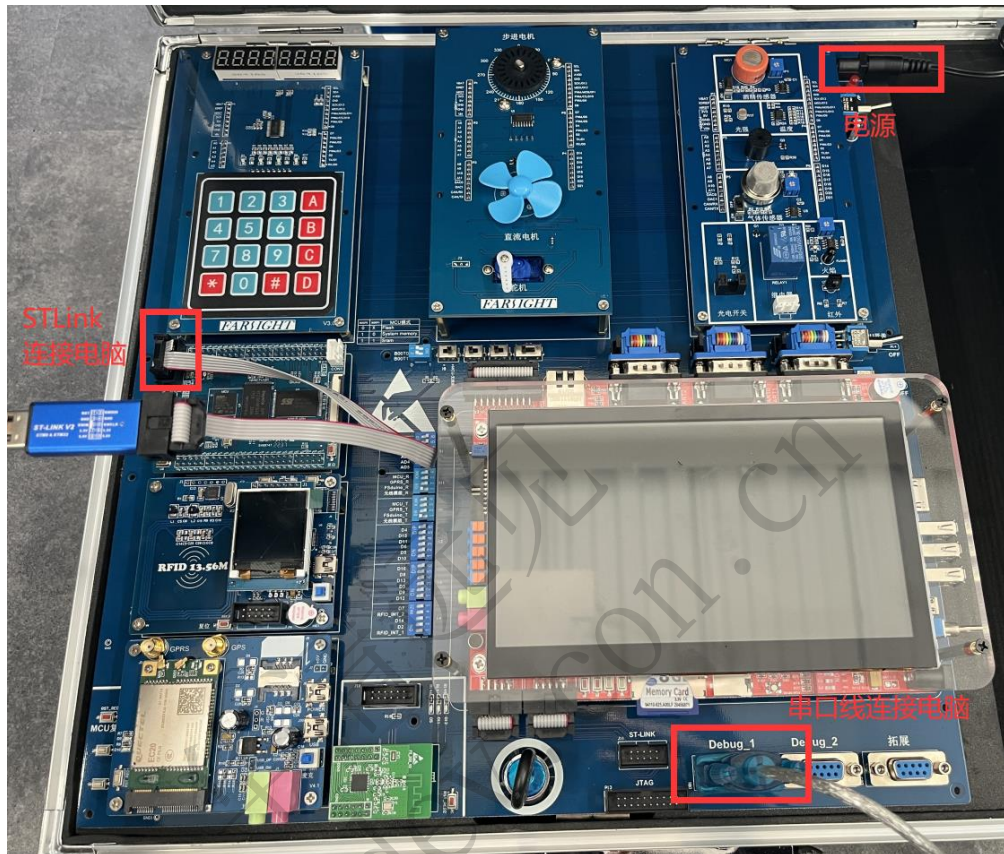
3.3.6 【实验步骤】

- (1) 打开工程文件，路径为【华清远见-嵌入式 ARM 实验箱资料-III\华清远见-STM 系列资料



【Cortex-M4 部分\程序源码\3_DC_Motor】

- (2) 对工程进行编译调试
- (3) 开发板连接



- (4) 往开发板下载程序，具体步骤参照 1.4.4 节
- (5) 开发板上电查看实验结果

3.3.7 【实验结果】

将固件程序下载到 STM32F407 核心板后，开发板上的直流电机可以通过矩阵键盘的按键 1、2、3 来控制反转运行，4、5、6 控制正转运行，直流电机位置如下图红圈所示：

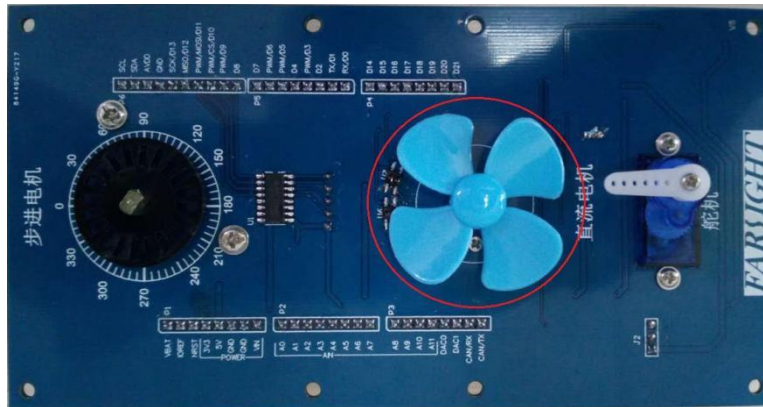


图 直流电机示意图

3.3.8 【实验参考代码】

这里只提供部分重要代码，如想查看部分函数里的详细实现，请打开工程源码，在µVision IDE for ARM 软件里进行查看和操作，这样查找方便清晰、有条理、并可以随意跟进。

```

1  int main(void)
2  {
3      /* USER CODE BEGIN 1 */
4
5      /* USER CODE END 1 */
6
7      /* MCU Configuration-----*/
8
9      /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
10     HAL_Init();
11
12     /* USER CODE BEGIN Init */
13
14     /* USER CODE END Init */
15
16     /* Configure the system clock */
17     SystemClock_Config();
18
19     /* USER CODE BEGIN SysInit */
20
21     /* USER CODE END SysInit */
22
23     /* Initialize all configured peripherals */
24     MX_GPIO_Init();

```



```

25  MX_I2C1_Init();
26  MX_TIM3_Init();
27  MX_USART1_UART_Init();
28  /* USER CODE BEGIN 2 */
29      printf("\n\r");
30      printf("\n\r-----\r\n");
31      printf("\n\r FS-STM32 开发板 直流电机测试实验\r\n");
32
33      DC_Motor_Pin_Low();
34
35      HAL_TIM_Base_Start_IT(&htim3);    //启动定时器 3.
36
37  /* USER CODE END 2 */
38
39  /* Infinite loop */
40  /* USER CODE BEGIN WHILE */
41  while (1)
42  {
43      /* USER CODE END WHILE */
44
45      /* USER CODE BEGIN 3 */
46      if(flag1 == 1)
47      {
48          //      printf("---\n");
49          flag1 = 0;
50          I2C_ZLG72128_Read(&hi2c1,0x61,0x01,Rx1_Buffer_P,1);//读键值
51          I2C_ZLG72128_Read(&hi2c1,0x61,0x03,Rx1_Buffer_T,1);//读键值
52
53          if (Rx1_Buffer_T[0] != 0xff)          //功能按键
54          {
55              printf("\n\r function key value = %#x\r\n", Rx1_Buffer_T[0]);
56              swtich_key_func();
57              DC_Task(Rx1_Buffer_T[0]);
58          }
59          if (Rx1_Buffer_P[0] != 0x0)          //普通按键
60          {
61              printf("\n\r key value = %#x\r\n", Rx1_Buffer_P[0]);
62              swtich_key();
63              DC_Task(Rx1_Buffer_P[0]);
64          }
65
66      }
67      //DC_Motor_Data

```



```

68     Led(DC_Motor_Data);
69
70     HAL_Delay(100);
71 }
72 /* USER CODE END 3 */
73 }
74
75 void DC_Task(uint8_t iKey)
76 {
77     uint8_t Buffer_DC[1]={0Xff};
78     uint8_t Buffer_DC_Zero[1]={0x00};
79     switch(iKey)
80     {
81         case 0xf7:                //1
82             DC_Motor_Pin_Low();
83             fs_flag = 1; //zheng
84             I2C_DC_Motor_Write(&hi2c1,DC_Motor_Addr,0x03,Buffer_DC_Zero,1);
85             break;
86         case 0xfb:                //2
87             DC_Motor_Pin_Low();
88             fs_flag = 1; //zheng
89             I2C_DC_Motor_Write(&hi2c1,DC_Motor_Addr,0x05,Buffer_DC,1);
90             break;
91         case 0xfd:                //3
92             DC_Motor_Pin_Low();
93             fs_flag = 1; //zheng
94             I2C_DC_Motor_Write(&hi2c1,DC_Motor_Addr,0x0f,Buffer_DC,1);
95             break;
96         case 0x14:                //4
97             DC_Motor_Pin_High();
98             fs_flag = 0; //zheng
99             I2C_DC_Motor_Write(&hi2c1,DC_Motor_Addr,0x0A,Buffer_DC,1);
100            break;
101            case 0x13:                //5
102                DC_Motor_Pin_High();
103                fs_flag = 0; //zheng
104                I2C_DC_Motor_Write(&hi2c1,DC_Motor_Addr,0x05,Buffer_DC,1);
105                break;
106            case 0x12:                //6
107                DC_Motor_Pin_High();
108                fs_flag = 0; //zheng
109                I2C_DC_Motor_Write(&hi2c1,DC_Motor_Addr,0x03,Buffer_DC,1);
110            break;

```



```

111
112     default:
113         DC_Motor_Pin_Low();
114         fs_flag = 2; //zheng
115         I2C_DC_Motor_Write(&hi2c1,DC_Motor_Addr,0x00,Buffer_DC_Zero,1);
116         break;
117     }
118 }
```

3.4 舵机实验

3.4.1 【实验目的】

通过定时器，我们能够实现 IO 口输出脉冲，进而控制舵机，通过本章的学习，对定时器有了一个更全面的认识与理解。

3.4.2 【实验环境】

嵌入式 ARM 实验箱

ST-Link 仿真器

RealView MDK5.21 集成开发软件

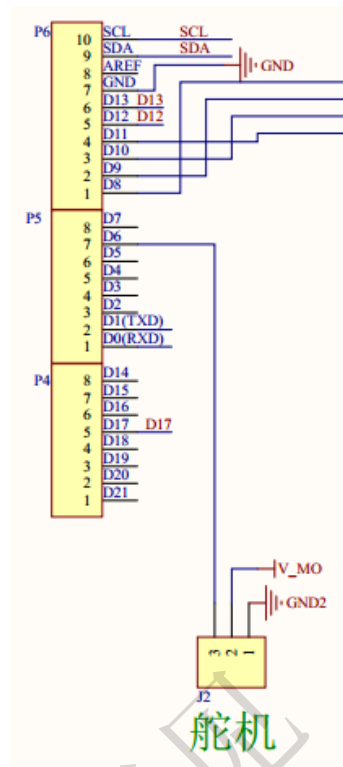
PC 机 XP、Window7/8 (32/64bit)

3.4.3 【实验内容】

编写程序，实现嵌入式 ARM 实验箱上的舵机 180 度的摆动实验。调试编译好固件程序后，将程序下载载到 STM32F407 核心板，按复位键观察实验现象。

3.4.4 【实验原理】

该实验需要使用到开发板上的舵机，相关硬件电路如下：



舵机连接电路



舵机原理图

控制信号由接收机的通道进入信号调制芯片，获得直流偏置电压。它内部有一个基准电路，产生周期为 20ms，宽度为 1.5ms 的基准信号，将获得的直流偏置电压与电位器的电压比较，获得电压差输出。最后，电压差的正负输出到电机驱动芯片决定电机的正反转。当电机转速一定时，通过级联减速齿轮带



动电位器旋转，使得电压差为 0，电机停止转动。

舵机的控制一般需要一个 20ms 左右的时基脉冲，该脉冲的高电平部分一般为 0.5ms-2.5ms 范围内的角度控制脉冲部分，总间隔为 2ms。以 180 度角度伺服为例，那么对应的控制关系是这样的：

0.5ms-----0 度；

1.0ms-----45 度；

1.5ms-----90 度；

2.0ms-----135 度；

2.5ms-----180 度；

PWM 简介：脉冲宽度调制，顾名思义，就是对脉冲宽度的控制。STM32F4 通用定时器包含一个 16 位或 32 位自动重载计数器（CNT），定时器可用于输入捕获（可用来测量输入的 PWM 波周期和脉宽），输出比较（规定的时刻输出需要的电平）和输出 PWM，单脉冲模式输出（产生一个脉冲信号），F4 通用定时器都是互相独立的，没有共享资源。

在这里我们用定时器产生输出 PWM，通过原理图查找舵机引脚为 PB14，查找 F4 数据手册 PB14 管脚对应定时器 12 的通道一。

参数设置



```
void MX_TIM12_Init(void)
{
    TIM_ClockConfigTypeDef sClockSourceConfig;
    TIM_OC_InitTypeDef sConfigOC;

    htim12.Instance = TIM12; // 定时器地址
    htim12.Init.Prescaler = 83; // 定时器预分频
    htim12.Init.CounterMode = TIM_COUNTERMODE_UP; // 向上计数模式
    htim12.Init.Period = 20000; // 计数值
    htim12.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1; // 时钟分频
    HAL_TIM_Base_Init(&htim12); // 地址赋值初始化

    sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL; // 内部时钟源
    HAL_TIM_ConfigClockSource(&htim12, &sClockSourceConfig); // 时钟源初始化

    HAL_TIM_PWM_Init(&htim12); // PWM初始化

    sConfigOC.OCMode = TIM_OCMODE_PWM1; // PWM模式1
    sConfigOC.Pulse = 1000; // 选定计数初值
    sConfigOC.OCpolarity = TIM_OCPOLARITY_HIGH; // 输出极性高
    sConfigOC.OCFastMode = TIM_OCFAST_DISABLE; // 快速模式不使能
    HAL_TIM_PWM_ConfigChannel(&htim12, &sConfigOC, TIM_CHANNEL_1); // 连接通道1并赋值
}
```

注意：PWM 模式有模式一和模式二，模式二和模式一输出极性相反。在手册里可以看到预分频实际计算是要+1，所以我们预分频要减一才是真正的值。

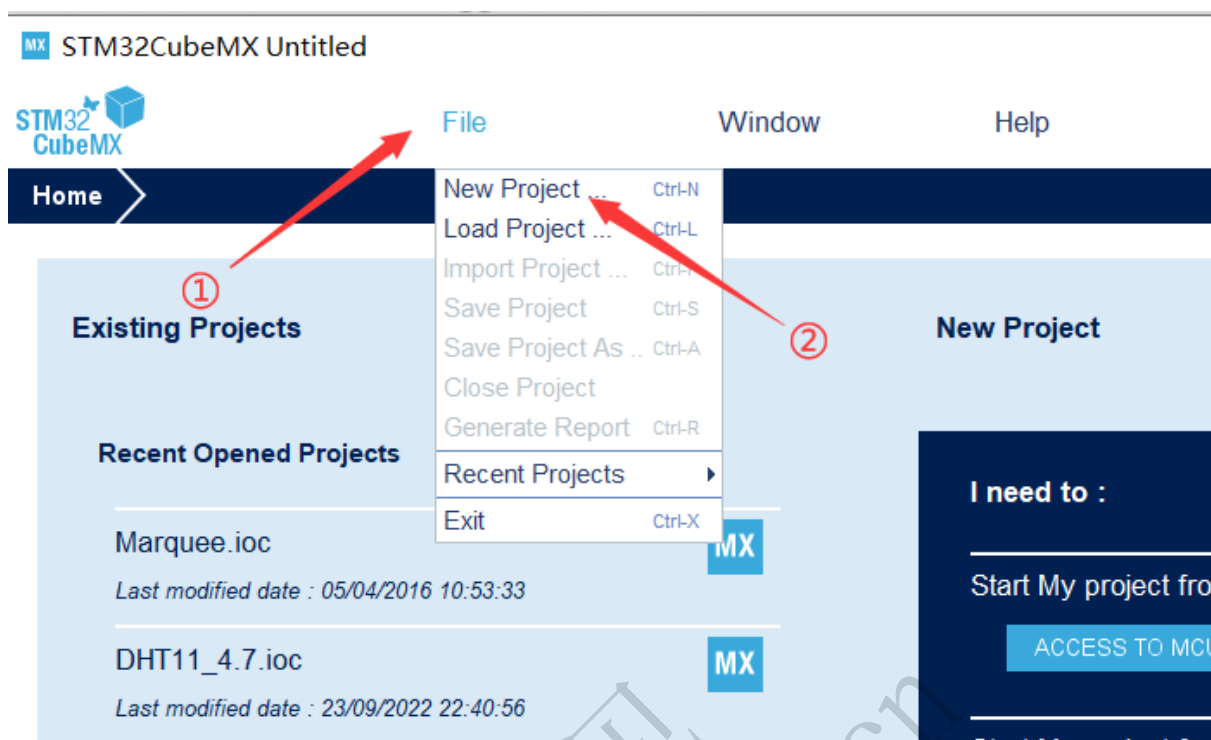
周期计算： $20000 \times 1 / (\text{定时器时钟} / 83 + 1) = 20\text{ms}$;

改变脉冲宽度的函数调用：__HAL_TIM_SetCompare，这是一个宏定义函数，真正实现是在
__HAL_TIM_SET_COMPARE(__HANDLE__, __CHANNEL__, __COMPARE__)

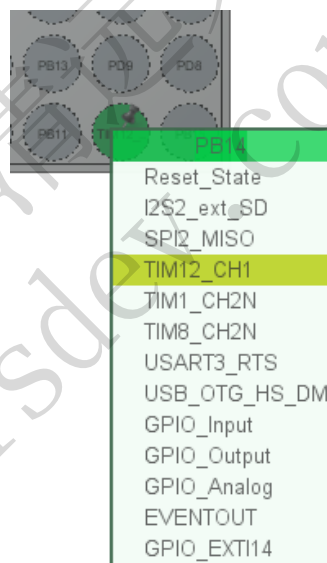
第一个参数传的定时器地址，第二个是通道，第三个是脉冲宽度。

3.4.5 【stm32cubemx 配置介绍】

- (1) 打开 STM32CubeMX->点击左上角的 File->点击 New Project
- (2) 选择芯片，查看芯片的型号，在搜索框输入芯片型号，本实验中选用的芯片是 STM32F407IGHx



(3) 配置 PB14 引脚为 TIM12_CH1 模式



(4) 对 TIM12 进行初始化配置



Functional Software

TIM12 Mode and Configuration

Mode

Slave Mode

Trigger Source

☒ Internal Clock

Channel1

Channel2

Combined Channels

☐ One Pulse Mode

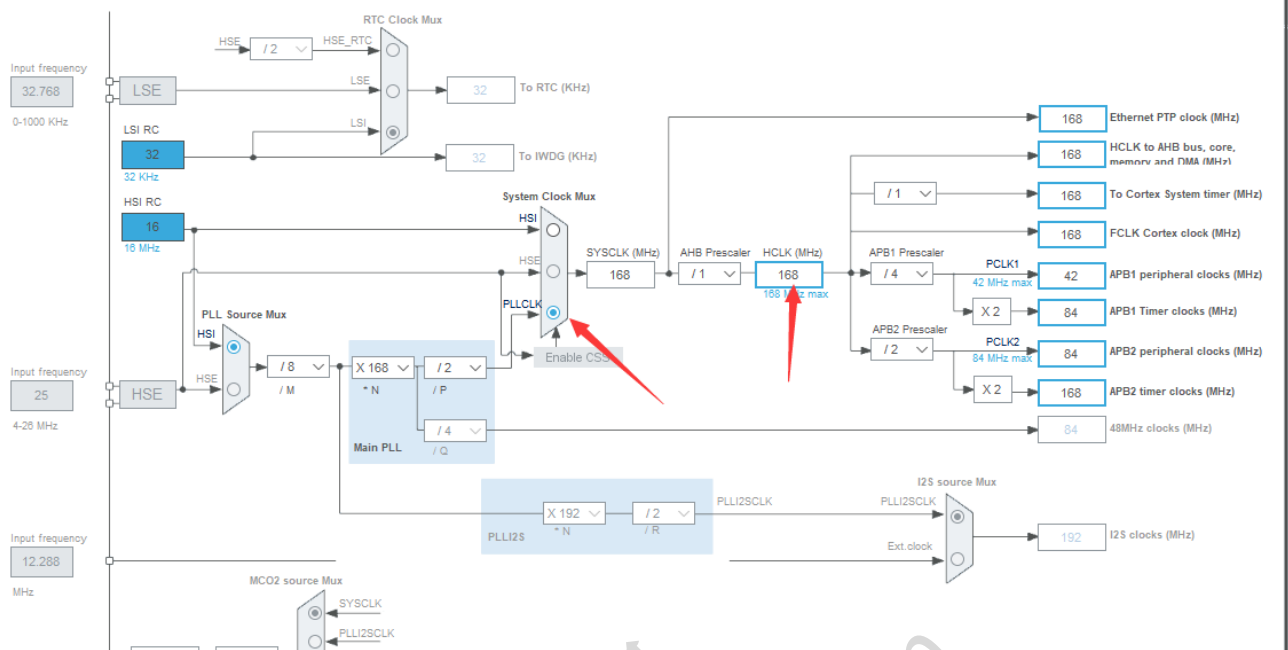
Configuration

☒ Parameter Settings ☒ User Constants ☒ NVIC Settings ☒ GPIO Settings

Configure the below parameters :

Parameter	Value	Notes
Prescaler (PSC - 16 bits value)	84-1	分频84
Counter Mode	Up	
Counter Period (AutoReload Register - 16 bits val...)	20000	重载值20000
Internal Clock Division (CKD)	No Division	
auto-reload preload	Disable	
PWM Generation Channel 1 Mode	PWM mode 1	
Pulse (16 bits value)	1000	
Output compare preload	Enable	
Fast Mode	Disable	
CH Polarity	High	

(5) 点击上方的 Clock Configuration，配置时钟树



- (6) 配置工程名字，工程路径和 IDE 软件（MDK-ARM V5.27），（注意不要有中文路径）。
- (7) 工程软件生成配置，配置工程为 C 文件和 h 文件分开，以及拷贝使用到的 HAL 库文件，其他的文件不拷贝，最后点击右上角的“GENWRTE CODE”，生成工程。
- (8) 生成工程完成后会弹出提示框，选择 Open Project 可直接从 MDK 中打开工程代码。

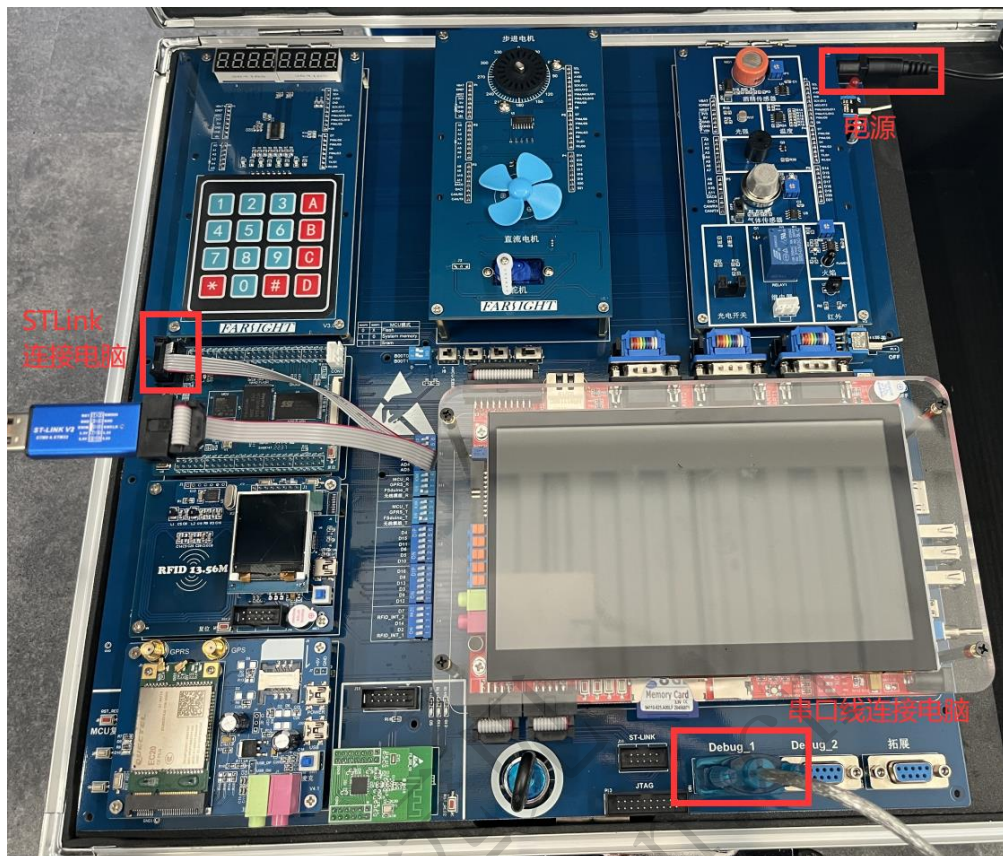
3.4.6 【实验步骤】

- (1) 打开工程文件，路径为

【华清远见-嵌入式 ARM 实验箱资料-III\华清远见-STM 系列资料\Cortex-M4 部分\程序源码

\4_Steering_Engine】

- (2) 对工程进行编译调试
- (3) 开发板连接



- (4) 往开发板下载程序，具体步骤参照 1.4.4 节
- (5) 开发板上电查看实验结果

3.4.7 【实验结果】

将固件程序下载到 STM32F407 核心板后，开发板上的舵机不断的从 0 度到 180 度旋转。

舵机位置如下图红圈所示：

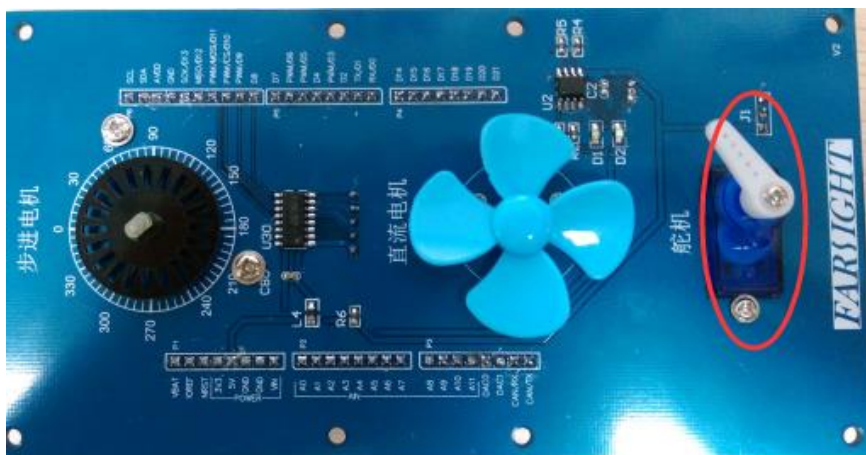




图 实验结果

3.4.8 【实验参考代码】

这里只提供部分重要代码，如想查看部分函数里的详细实现，请打开工程源码，在µVision IDE for ARM 软件里进行查看和操作，这样查找方便清晰、有条理、并可以随意跟进。

C++ Code

```
1  int main(void)
2  {
3
4      /* USER CODE BEGIN 1 */
5      uint16_t i;
6      /* USER CODE END 1 */
7
8      /* MCU Configuration-----*/
9
10     /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
11     HAL_Init();
12
13     /* Configure the system clock */
14     SystemClock_Config();
15
16     /* Initialize all configured peripherals */
17     MX_GPIO_Init();
18     MX_TIM12_Init();
19
20     /* USER CODE BEGIN 2 */
21     HAL_TIM_PWM_Start(&htim12, TIM_CHANNEL_1);
22     /* USER CODE END 2 */
23
24     /* Infinite loop */
25     /* USER CODE BEGIN WHILE */
26     while (1)
27     {
28         /* USER CODE END WHILE */
29
30         /* USER CODE BEGIN 3 */
31         for(i = 600; i < 2400; i++)
32         {
33             __HAL_TIM_SetCompare(&htim12, TIM_CHANNEL_1, i);
```



```

34         HAL_Delay(1);
35     }
36     for(i = 2400; i > 600; i--)
37     {
38         __HAL_TIM_SetCompare(&htim12, TIM_CHANNEL_1, i);
39         HAL_Delay(1);
40     }
41 }
42 /* USER CODE END 3 */
43
44 }
```

3.5 步进电机实验

3.5.1 【实验目的】

了解步进电机驱动原理，并且用软件编程驱动步进电机。

3.5.2 【实验环境】

嵌入式 ARM 实验箱

ST-Link 仿真器

RealView MDK5.21 集成开发软件

PC 机 XP、Window7/8 (32/64bit)

3.5.3 【实验内容】

编写程序，实现嵌入式 ARM 实验箱上的步进电机转动。调试编译好固件程序后，将程序下载到

STM32F407 核心板，按复位键观察实验现象。

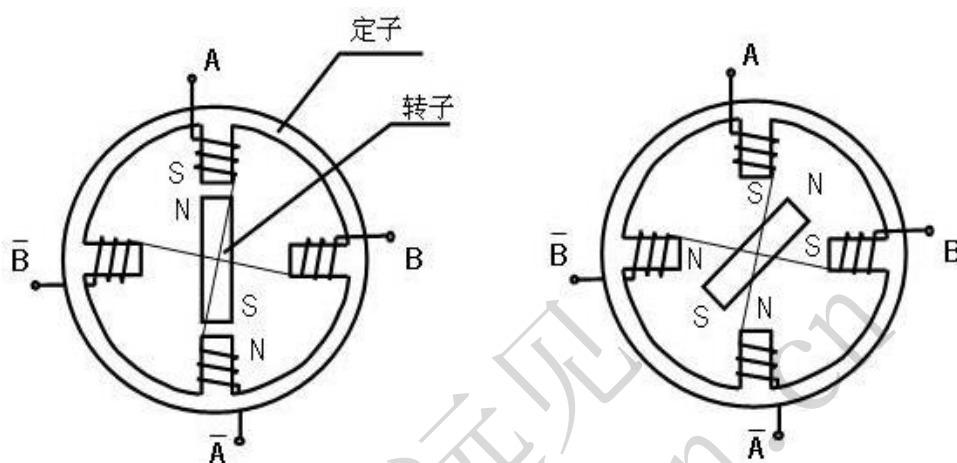
3.5.4 【实验原理】

步进电动机是一种将电脉冲信号转换成角位移或线位移的机电元件。步进电动机的输入量是脉冲序



列，输出量则为相应的增量位移或步进运动。正常运动情况下，它每转一周具有固定的步数；做连续步进运动时，其旋转转速与输入脉冲的频率保持严格的对应关系(所以可以控制每个脉冲的宽度去调节速度每个脉冲转速)，由于步进电动机能直接接受数字量的控制，所以特别适宜采用微机进行控制。

步进电机接线：



8 拍的方式八个状态：

- 1、A 高电平
- 2、A 与 B 高电平
- 3、B 高电平
- 4、B 与 A-高电平
- 5、A-给高电平
- 6、A-与 B-给高电平
- 7、B-给高电平
- 8、B-与 A 给高电平

按以上八个状态轮流供电，控制一下脉宽（用精确延迟就可以控制）就可以了。

4 拍的方式



- 1、A 高电平
- 2、B 高电平
- 3、A-高电平
- 4、B-高电平

如果要想实现反转：只要把时序倒过来就可以实现。

我们所用步进电机型号为 28byj-48，参数如下

主要技术参数

N=64

电机型号	电压 V	相数	相电阻Ω ±10%	步距角度	减速比	起动转矩 100P. P. S g. cm	起动频率 P. P. S	定位转矩 g. cm	摩擦转矩 g. cm	噪声 dB	绝缘介 电强度
28BYJ48	5	4	300	5.625/64	1:64	≥300	≥550	≥300	—	≤35	600VAC 1S

主要技术参数：

1、相数：产生不同对极 N、S 磁场的激磁线圈对数。常用 m 表示。

2、拍数：完成一个磁场周期性变化所需脉冲数或导电状态用 n 表示，或指电机转过一个齿距角所需脉冲数，以四相电机为例，有四相四拍运行方式即 AB-BC-CD-DA-AB，四相八拍运行方式即 A-AB-B-BC-C-CD-D-DA-A。

3、步距角：对应一个脉冲信号，电机转子转过的角位移用 θ 表示。 $\theta = 360 \text{ 度} / (\text{转子齿数} \times \text{运行拍数})$ ，以常规二、四相，转子齿为 50 齿电机为例。四拍运行时步距角为 $\theta = 360 \text{ 度} / (50 \times 4) = 1.8 \text{ 度}$ （俗称整步），八拍运行时步距角为 $\theta = 360 \text{ 度} / (50 \times 8) = 0.9 \text{ 度}$ （俗称半步）。

我们通过控制磁场方向的变化来改变转动方向，这时磁场转动一圈并不代表电机对应要转一圈，这里有一个参数为减速比，通过内部的转子齿来减速，所以磁场转动一圈，电机转动 $360/64 \text{ 度}$ 。

步进电机硬件接线图：

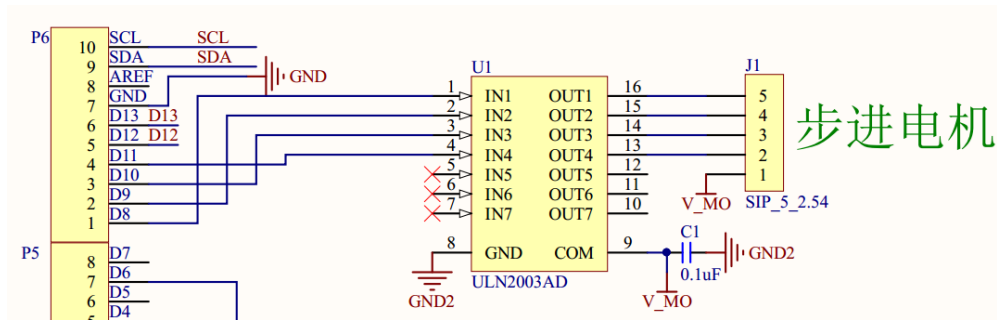
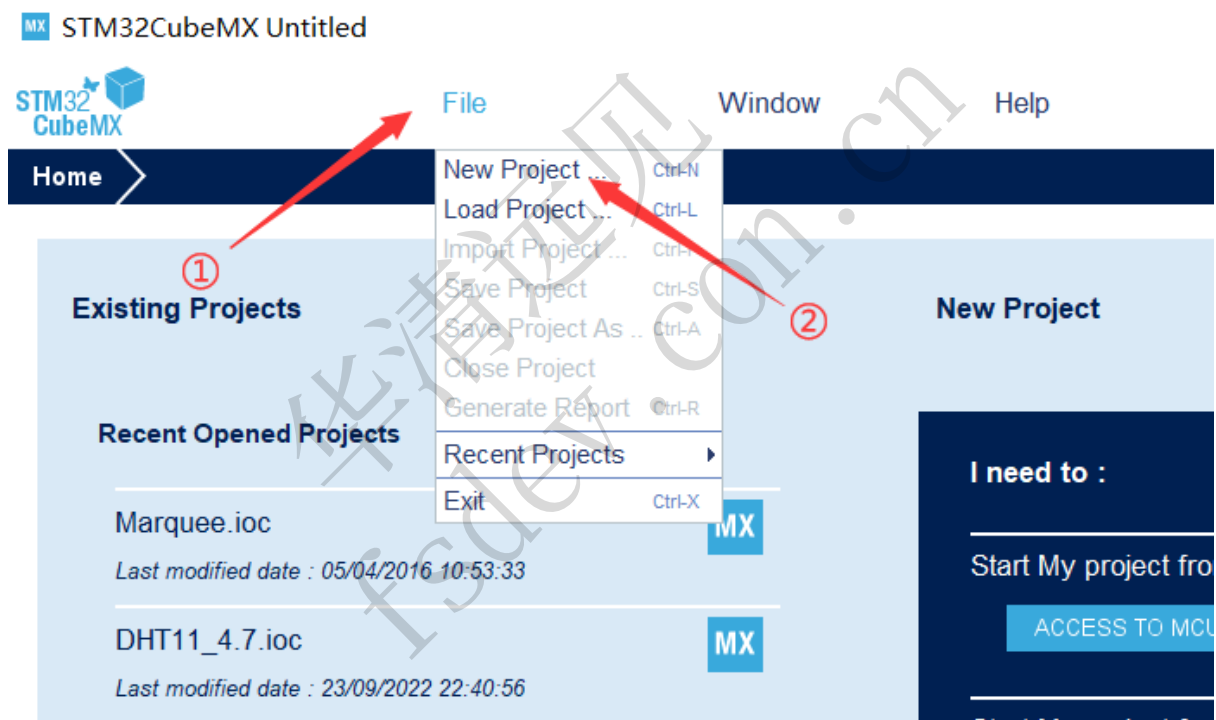


图 原理图

3.5.5 【stm32cubemx 配置介绍】

- (1) 打开 STM32CubeMX->点击左上角的 File->点击 New Project



- (2) 选择芯片，查看芯片的型号，在搜索框输入芯片型号，本实验中选用的芯片是 STM32F407IGHx



New Project

MCU/MPU Selector Board Selector Cross Selector

MCU/MPU Filters

Part Number Search

STM32F407G

Core

Series

Line

Package

Other

Price = 7.577

IO = 140

Espram = 0 (Bytes)

Flash = 1024 (KBytes)

Ram = 192 (KBytes)

Freq = 168 (MHz)

Peripheral

Features

Block Diagram

Docs & Resources

Datasheet

Buy

Start Project

STM32F407G

High-performance foundation line, Arm Cortex-M4 core with DSP and FPU, 1 Mbyte of Flash memory, 168 MHz CPU, ART Accelerator, Ethernet, FSMC

ACTIVE Active

Product is in mass production

Unit Price for 10KU (US\$): 7.577

Board: STM3240G-EVAL

UFPGA176

The STM32F405xx and STM32F407xx family is based on the high-performance Arm® Cortex®-M4 32-bit RISC core operating at a frequency of up to 168 MHz. The Cortex-M4 core features a Floating point unit (FPU) single precision which supports all Arm single-precision data-processing instructions and data types. It also implements a full set of DSP instructions and a memory protection unit (MPU) which enhances application security. The STM32F405xx and STM32F407xx family incorporates high-speed embedded memories (Flash memory up to 1 Mbyte, up to 192 Kbytes of SRAM), up to 4 Kbytes of backup SRAM, and an extensive range of enhanced I/Os and peripherals connected to two APB buses, three AHB buses and a 32-bit multi-AHB bus matrix. All devices offer three 12-bit ADCs, two DACs, a low-power RTC, twelve general-purpose 16-bit timers including two PWM timers for motor control, two general-purpose 32-bit timers, a true random number generator (RNG). They also feature standard and advanced communication interfaces.

Features

Core

MCUs/MPUs List: 2 items

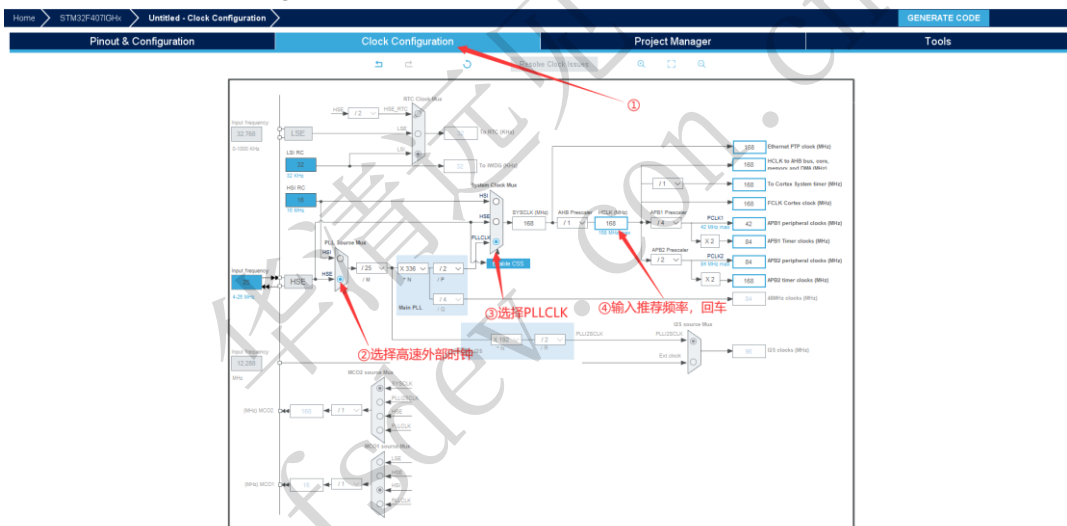
Display similar items

Part No.	Reference	Marketing	Unit Price for 10k	Board	Package	Flash	RAM	IO	Freq	DES/DES	HMAG	MDS	SHA	TRNG
STM32F407G	STM32F407G	Active	7.577	STM3240G-EVAL	UFPGA176	1024 KByte	192 Kbytes	140	168 MHz	0	0	0	0	0
STM32F407G	STM32F407G	Active	7.577		LQFP176	1024 KByte	192 Kbytes	140	168 MHz	0	0	0	0	0

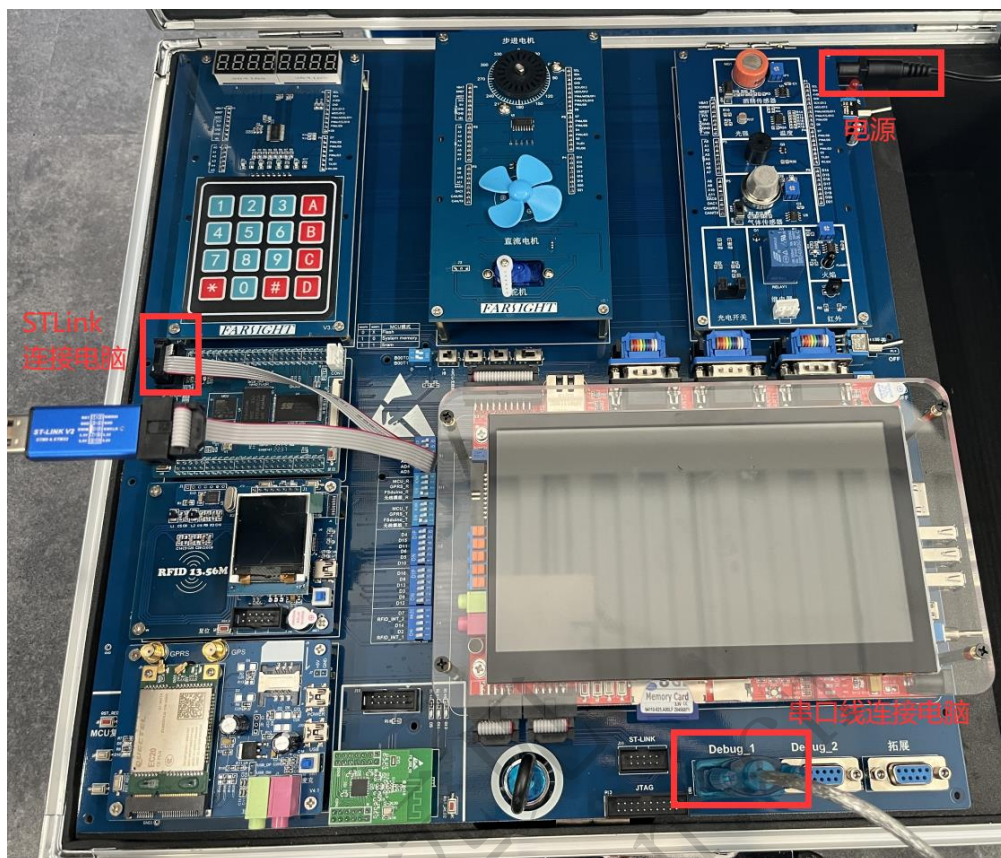
Export

2、双击

(3) 配置 RCC，点击左侧的 RCC，找到 High Speed Clock (HSE)，点击选择 Crystal/Ceramic Resonator，RCC 配置完成，点击上方的 Clock Configuration，配置时钟树



(4) 配置 PD12/PE4/PE6/PH13 引脚为 GPIO_Output 模式。



- (4) 往开发板下载程序，具体步骤参照 1.4.4 节
- (5) 开发板上电查看实验结果

3.5.7 【实验结果】

将固件程序下载到 STM32F407 核心板后，开发板上的步进电机按照一定转速旋转。

步进电机位置如下图所示：

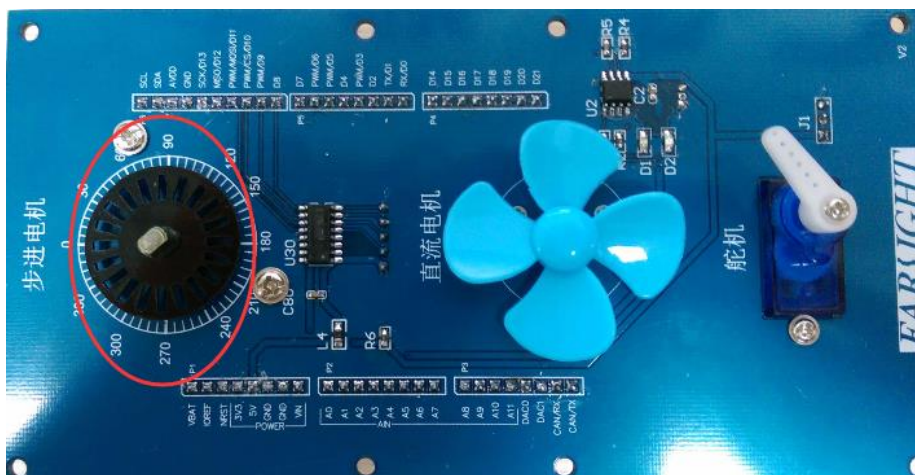




图 步进电机实验结果

3.5.8 【实验参考代码】

这里只提供部分重要代码，如想查看部分函数里的详细实现，请打开工程源码，在µVision IDE for

ARM 软件里进行查看和操作，这样查找方便清晰、有条理、并可以随意跟进。

```

1  int main(void)
2  {
3
4      /* USER CODE BEGIN 1 */
5
6      /* USER CODE END 1 */
7
8      /* MCU Configuration-----*/
9
10     /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
11     HAL_Init();
12
13     /* Configure the system clock */
14     SystemClock_Config();
15
16     /* Initialize all configured peripherals */
17     MX_GPIO_Init();
18
19     /* USER CODE BEGIN 2 */
20
21     /* USER CODE END 2 */
22
23     /* Infinite loop */
24     /* USER CODE BEGIN WHILE */
25     while (1)
26     {
27         /* USER CODE END WHILE */
28
29         /* USER CODE BEGIN 3 */
30         DE_A;
31         HAL_Delay(6); //可进行调速，延时时间不能太短
32         DE_AB;
33         HAL_Delay(6);
34         DE_B;

```



```
35     HAL_Delay(6);
36     DE_BC;
37     HAL_Delay(6);
38     DE_C;
39     HAL_Delay(6);
40     DE_CD;
41     HAL_Delay(6);
42     DE_D;
43     HAL_Delay(6);
44     DE_DA;
45     HAL_Delay(6);
46 }
47 /* USER CODE END 3 */
48
49 }
50
```

3.6 UART 串口 1 数据收发实验

3.6.1 【实验目的】

串口的使用对于我们开发调试过程中的作用是非常之大，可以用来查看，打印以及输入相关信息。

所以对串口的调试使用要熟练掌握。

3.6.2 【实验环境】

嵌入式 ARM 实验箱

ST-Link 仿真器

RealView MDK5.21 集成开发软件

PC 机 XP、Window7/8 (32/64bit)

串口调试工具

串口交叉线



3.6.3 【实验内容】

编写串口 1 数据收发程序，实现串口控制 D4 灯的开关功能。调试编译好固件程序后，将程序下载到 STM32F407 核心板，按复位键观察到串口收到数据后，向串口发送字符或者字符串（在字符后面加回车键），观察实验现象。

3.6.4 【实验原理】

串行通信的基础知识：

串口通信是指外设和计算机间，通过数据信号线、地线、控制线等，按位进行传输数据的一种通讯方式。这种通信方式使用的数据线少，在远距离通信中可以节约通信成本，但其传输速度比并行传输低。

串口是计算机上一种非常通用的设备通信协议。大多数计算机（不包括笔记本电脑）包含两个基于 RS-232 的串口。串口同时也是仪器仪表设备通用的通信协议(串口通信协议也可以用于获取远程采集设备的数据)。

一条信息的各位数据被逐位按顺序传送的通讯方式称为串行通讯。

评价一个通讯是否优质，主要体现在传输的速度，数据的正确性，功耗是否低，布线成本是否低（例如 1 根线收发都能满足就比 8 根线的并行收发要节约成本）；使用是否普及（就好像大家都学英语，世界很大部分的人都可以独立使用英语吗，会英语的人多，就非常普及，可通讯面就非常广；如果你学的鸟语，那就只能跟鸟通信，没有人能听懂）。

串行通讯的特点是：数据位传送，按位顺序进行，最少只需一根传输线即可完成，成本低但传送速度慢。串行通讯的距离可以从几米到几千米。

RS-232（串口的英文代名词）采取不平衡传输方式，即所谓单端通讯。由于其发送电平与接收电平的差仅为 2V 至 3V 左右，所以其共模抑制能力差，再加上双绞线上的分布电容，其传送距离最长



为约 15 米，最高速率为 20kb/s。RS-232 是为点对点（即只用一对收、发设备）通讯而设计的，其驱动器负载为 3~7kΩ。所以 RS-232 适合本地设备之间的通信。

传统的串行接口标准有 22 根线，采用标准 25 芯 D 型插头座（DB25），后来使用简化为 9 芯 D 型插座（DB9），现在应用中 25 芯插头座已很少采用。

像现在所说的几线串口，一般都是指使用了几根线，最初的 RS-232 串口是 25 针的，所有的针脚定义都有用到，后来变成了 9 针的，所谓全功能串口就是所有的针脚定义都使用上了，例如流量控制，握手信号等都有用到，一般来说国外的产品做产品比较规矩，把所有的串口信号都做上去了。但是国内的技术人员发现，其实 RS-232 串口最主要使用的就是 2，3 线，另外的接口如果不使用的话，也不会出现很大的问题，所以，就在 9 针的基础上做精简，所以就有所谓的 2，3，4，5，6，8 线的串口出来了。

2 线串口只有 RXD,TXD 两根基本的收发信号线；3 线串口除了 RXD 和 TXD，还有 GND；所谓 4~9 线只是在 TXD 和 RXD 基础上增加了相应的控制信号线，依据实际需要进行设计。一般来说，使用 5 线的 232 通信，是加了硬件流控的，即 RTS,CTS 信号，主要是为了保证高速通信时的可靠性，如果你的通信速度不是很高，完全可以不用理会。根据信息的传送方向，串行通讯可以进一步分为单工、半双工和全双工三种。信息只能单向传送为单工；信息能双向传送但不能同时双向传送称为半双工；信息能够同时双向传送则称为全双工。

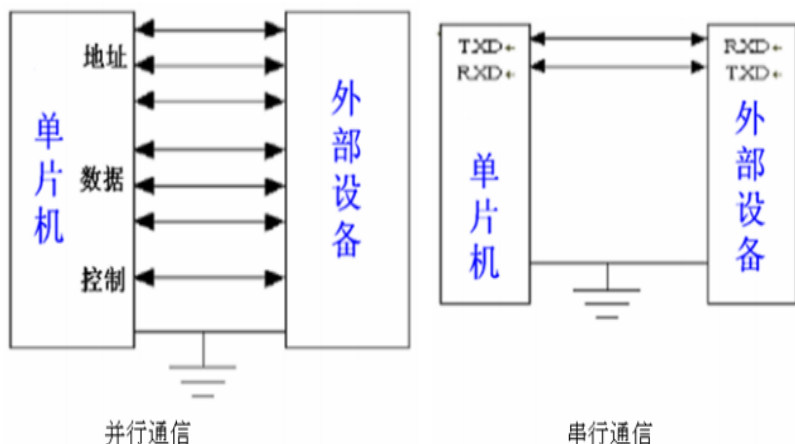
- 如果在通信过程的任意时刻，信息只能由一方 A 传到另一方 B，则称为单工。
- 如果在任意时刻，信息既可由 A 传到 B，又能由 B 传 A，但只能由一个方向上的传输存在，称为半双工传输。
- 如果在任意时刻，线路上存在 A 到 B 和 B 到 A 的双向信号传输，则称为全双工。

并行通信和串行通信：

单片机与外界通信的基本方式有两种：并行通信和串行通信，串口属于串行通信。并行通信是指利用多条数据传输线将一个数据的各位同时发送或接收。串行通信是指利用一条传输线将数据一位位地顺序发送或接收。



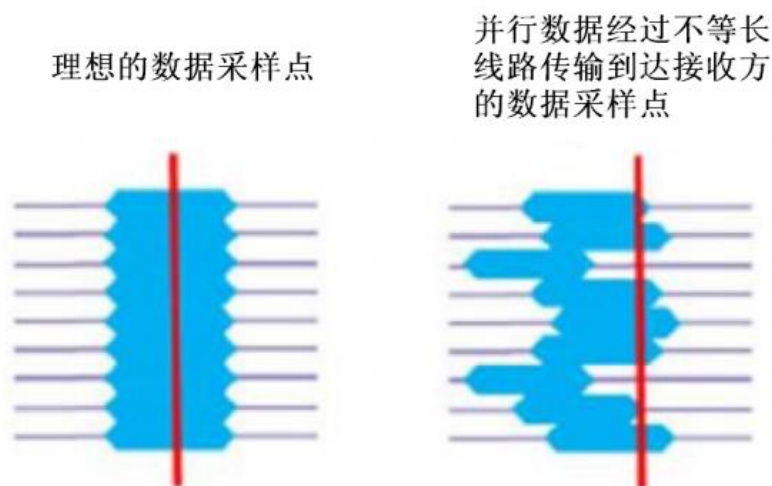
并行通信和串行通信的示意图如下图：



在每一条传输线传输速率相同时，并行通信的传输速度比和串行通信快。然而当传输距离变长时，并行通信的缺点就会凸显，首先是相比于串行通信而言信号易受外部干扰，信号线之间的相互干扰也增加，其次是速率提升之后不能保证每根数据线的的数据同时到达接收方而产生接收错误，而且距离越长布线成本越高。

所以并行通信目前主要用在短距离通信，比如处理器与外部的 flash 以及外部 RAM 以及芯片内部各个功能模块之间的通信。串行通信以其通信速率快和成本低等优点成为了远距离通信的首选。RS232 串口，以及差分串行总线像 RS485 串口、USB 接口、CAN 接口、IEEE-1394 接口、以太网接口、SATA 接口和 PCIE 接口等都属于串行通信的范畴。

下图左侧为每根数据线的的数据同时到达接收方，被正确采样的最理想情况；右侧的图为每根数据线的的数据不能同时到达接收方而产生接收错误情形。



串口的原理和特点：

串口是计算机上一种非常通用设备通信的协议（不要与通用串行总线 Universal Serial Bus 或者 USB 混淆）。大多数计算机包含两个基于 RS232 的串口。串口同时也是仪器仪表设备通用的通信协议；很多 GPIB(通用接口总线)兼容的设备也带有 RS-232 口。同时，串口通信协议也可以用于获取远程采集设备的数据。

串口通信的概念非常简单，串口按位（bit）发送和接收字节。尽管比按字节（byte）的并行通信慢，但是串口可以在使用一根线发送数据的同时用另一根线接收数据。它很简单并且能够实现远距离通信。典型地，串口用于 ASCII 码字符的传输。通信使用 3 根线完成：（1）地线，（2）发送，（3）接收。由于串口通信是异步的，端口能够在在一根线上发送数据同时在另一根线上接收数据。其他线用于握手，但是不是必须的。串口通信最重要的参数是波特率、数据位、停止位和奇偶校验。

对于两个进行通行的端口，这些参数必须匹配：

波特率： 这是一个衡量通信速度的参数。它表示每秒钟传送的 bit 的个数。例如 300 波特表示每秒钟发送 300 个 bit。当我们提到时钟周期时，我们就是指波特率例如如果协议需要 4800 波特率，那么时钟是 4800Hz。这意味着串口通信在数据线上的采样率为 4800Hz。通常电话线的波特率为 14400，28800 和 36600。波特率可以远远大于这些值，但是波特率和距离成反比。高波特率常常用于放置的很近的仪器间的通信，典型的例子就是 GPIB 设备的通信。



数据位： 这是衡量通信中实际数据位的参数。当计算机发送一个信息包，实际的数据不会是 8 位的，标准的值是 5、7 和 8 位。如何设置取决于你想传送的信息。比如，标准的 ASCII 码是 0~127（7 位）。扩展的 ASCII 码是 0~255（8 位）。如果数据使用简单的文本（标准 ASCII 码），那么每个数据包使用 7 位数据。每个包是指一个字节，包括开始/停止位，数据位和奇偶校验位。由于实际数据位取决于通信协议的选取，术语“包”指任何通信的情况。

停止位： 用于表示单个包的最后一位。典型的值为 1，1.5 和 2 位。这里的 1.5 位的数据宽度，就是 1.5 个波特率，由于数据是在传输线上定时的，并且每一个设备有其自己的时钟，很可能在通信中两台设备间出现了小小的不同步。因此停止位不仅仅是表示传输的结束，并且提供计算机校正时钟同步的机会。适用于停止位的位数越多，不同时钟同步的容忍程度越大，但是数据传输率同时也越慢。

奇偶校验位： 在串口通信中一种简单的检错方式。有四种检错方式：偶、奇、高和低。当然没有校验位也是可以的。对于偶和奇校验的情况，串口会设置校验位（数据位后面的一位），用一个值确保传输的数据有偶个或者奇个逻辑高位。例如，如果数据是 011，那么对于偶校验，校验位为 0，保证逻辑高的位数是偶数个。如果是奇校验，校验位为 1，这样就有 3 个逻辑高位。高位和低位不真正的检查数据，简单置位逻辑高或者逻辑低校验。这样使得接收设备能够知道一个位的状态，有机会判断是否有噪声干扰了通信或者是否传输和接收数据是否不同步。

硬件流控制： 硬件流控制常用的有 RTS/CTS 流控制和 DTR/R（数据终端就绪/数据设置就绪）流控制。硬件流控制必须将相应的电缆线连上，用 RTS/CTS（请求发送/清除发送）流控制时，应将通讯两端的 RTS、CTS 线对应相连，数据终端设备（如计算机）使用 RTS 来起始调制解调器或其它数据通讯设备的数据流，而数据通讯设备（如调制解调器）则用 CTS 来起动和暂停来自计算机的数据流。这种硬件握手方式的过程为：我们在编程时根据接收端缓冲区大小设置一个高位标志（可为缓冲区大小的 75%）和一个低位标志（可为缓冲区大小的 25%），当缓冲区内数据量达到高位时，我们在接收端将 CTS 线置低电平（送逻辑 0），当发送端的程序检测到 CTS 为低后，就停止发送数据，直到接收端缓冲区的数据量



低于低位而将 CTS 置高电平。RTS 则用来标明接收设备有没有准备好接收数据。

常用的流控制还有还有 DTR/ R (数据终端就绪/数据设置就绪)。我们在此不再详述。

STM32 串口设置一般可以总结为如下几个步骤：

- 1) 串口时钟使能, GPIO 时钟使能
- 2) 设置引脚复用映射
- 3) GPIO 初始化设置,模式为复用功能
- 4) 串口参数初始化: 设置波特率, 字长, 奇偶校验等参数
- 5) 开启中断并初始化 NVIC, 使能中断 (如果需要开启中断才需要这个步骤)
- 6) 使能串口
- 7) 编写中断处理函数

24.6.1 Status register (USART_SR)

Address offset: 0x00

Reset value: 0x00C0 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved							CTS	LBD	TXE	TC	RXNE	IDLE	ORE	NF	FE
							rc_w0	rc_w0	r	rc_w0	rc_w0	r	r	r	r

寄存器 USART_SR 各位描述这里我们主要关注一下三位, 第 5, 6, 7 位 RXNE、TC 和 TXE。

更加详细寄存器介绍, 请参考《stm32f4xx 中文参考手册》711 页。



位 7 TXE: 发送数据寄存器为空 (Transmit data register empty)

当 TDR 寄存器的内容已传输到移位寄存器时, 该位由硬件置 1。如果 USART_CR1 寄存器中 TXEIE = 1, 则会生成中断。通过对 USART_DR 寄存器执行写入操作将该位清零。

0: 数据未传输到移位寄存器

1: 数据传输到移位寄存器

注意: 单缓冲区发送期间使用该位。

位 6 TC: 发送完成 (Transmission complete)

如果已完成对包含数据的帧的发送并且 TXE 置 1, 则该位由硬件置 1。如果 USART_CR1 寄存器中 TCIE = 1, 则会生成中断。该位由软件序列清零 (读取 USART_SR 寄存器, 然后写入 USART_DR 寄存器)。TC 位也可以通过向该位写入 '0' 来清零。建议仅在多缓冲区通信时使用此清零序列。

0: 传送未完成

1: 传送已完成

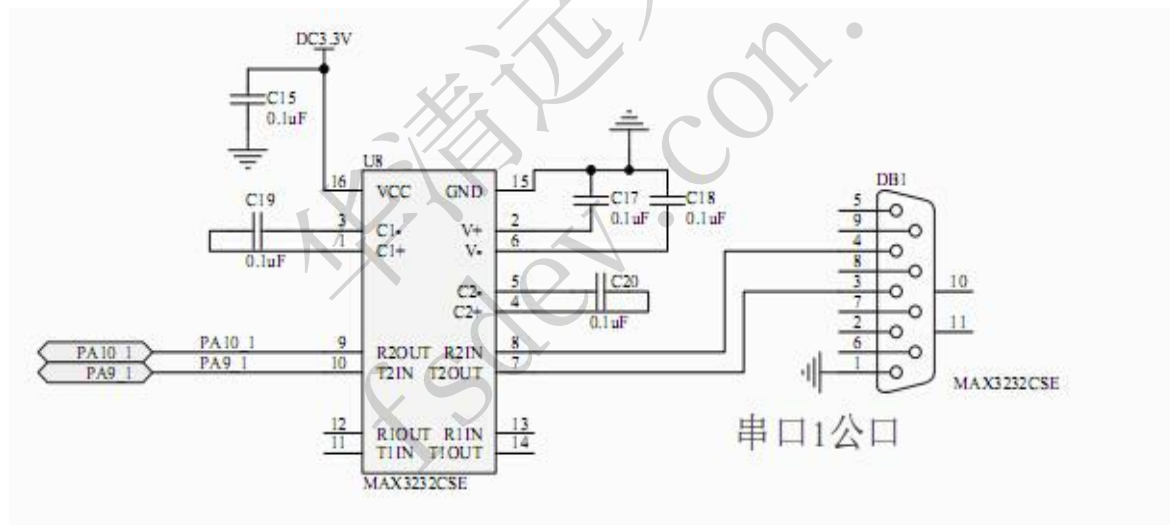
位 5 RXNE: 读取数据寄存器不为空 (Read data register not empty)

当 RDR 移位寄存器的内容已传输到 USART_DR 寄存器时, 该位由硬件置 1。如果 USART_CR1 寄存器中 RXNEIE = 1, 则会生成中断。通过对 USART_DR 寄存器执行读入操作将该位清零。RXNE 标志也可以通过向该位写入零来清零。建议仅在多缓冲区通信时使用此清零序列。

0: 未接收到数据

1: 已准备好读取接收到的数据

FS-STM32F407 中串口的硬件原理图:



HAL 库函数解析:

```
HAL_UART_Transmit(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size, uint32_t Timeout);
HAL_UART_Receive(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size, uint32_t Timeout);
```

这两个函数是轮询方式发送和接收

```
HAL_UART_Transmit_IT(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size);
HAL_UART_Receive_IT(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size);
```

这两个函数是以中断方式发送和接收。具体传参详见工程。

注意: 这两个函数里为指定缓冲区和大小并开启中断。实际上数据还在中断里处理



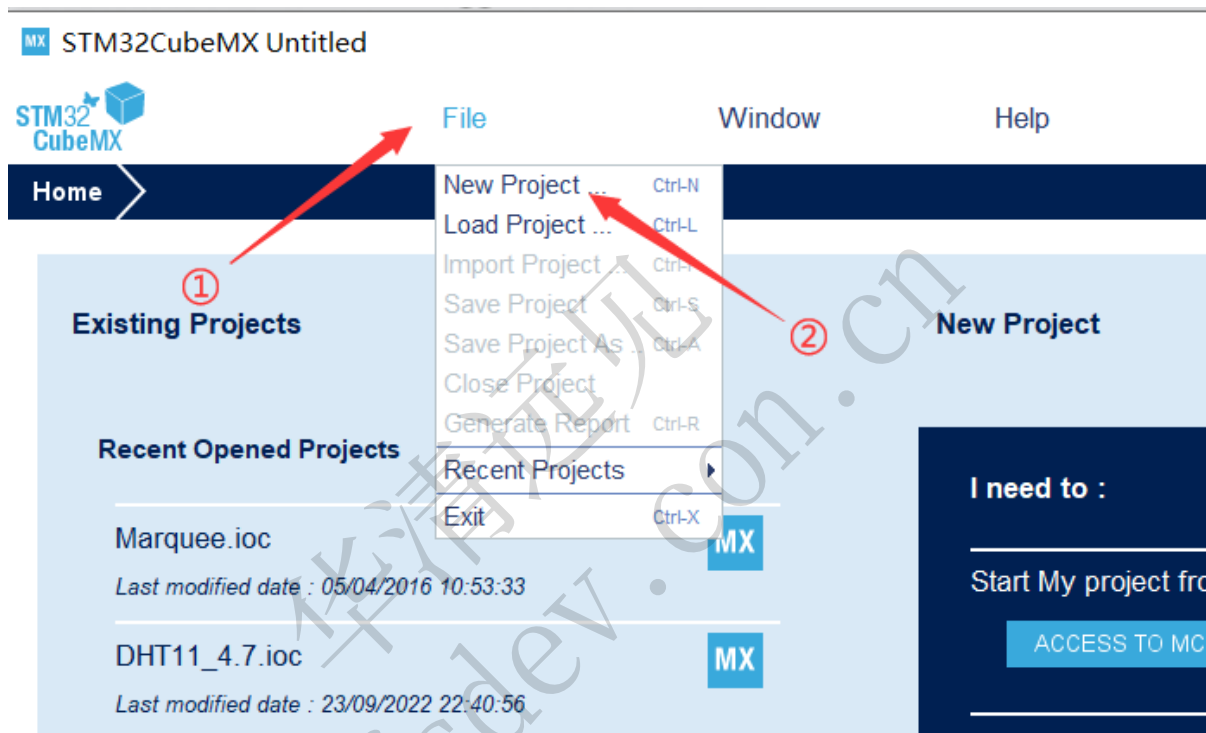
HAL 库提供了用户中断处理函数方式，是调用回调函数。

接收中断在用户代码里重定义 `void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)`

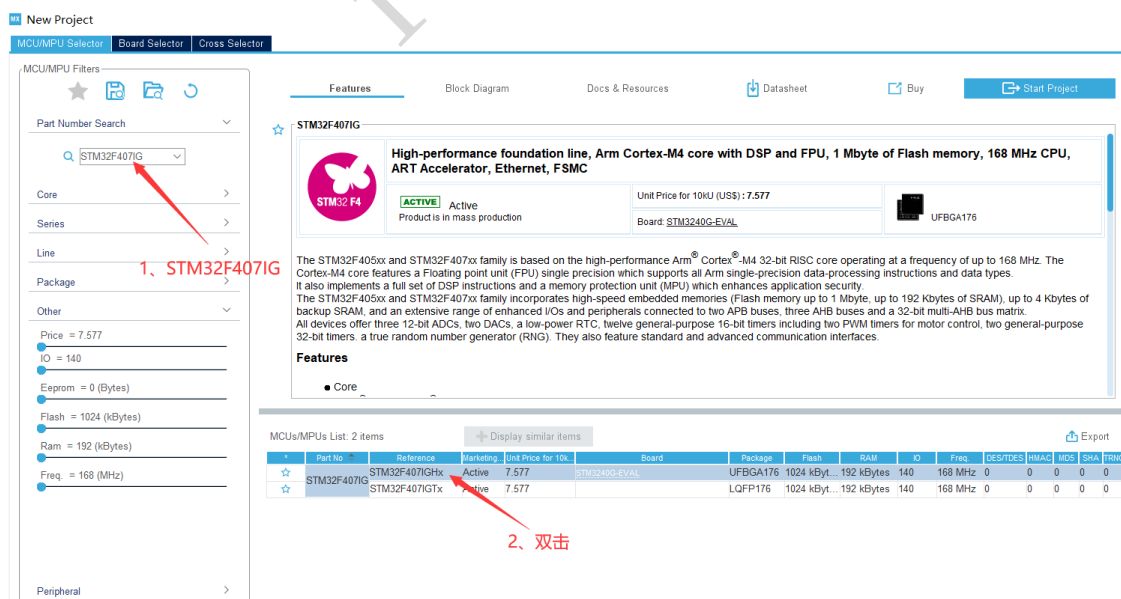
发送中断在用户代码里重定义 `void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart);`

3.6.5 【stm32cubemx 配置介绍】

- (1) 打开 STM32CubeMX->点击左上角的 File->点击 New Project

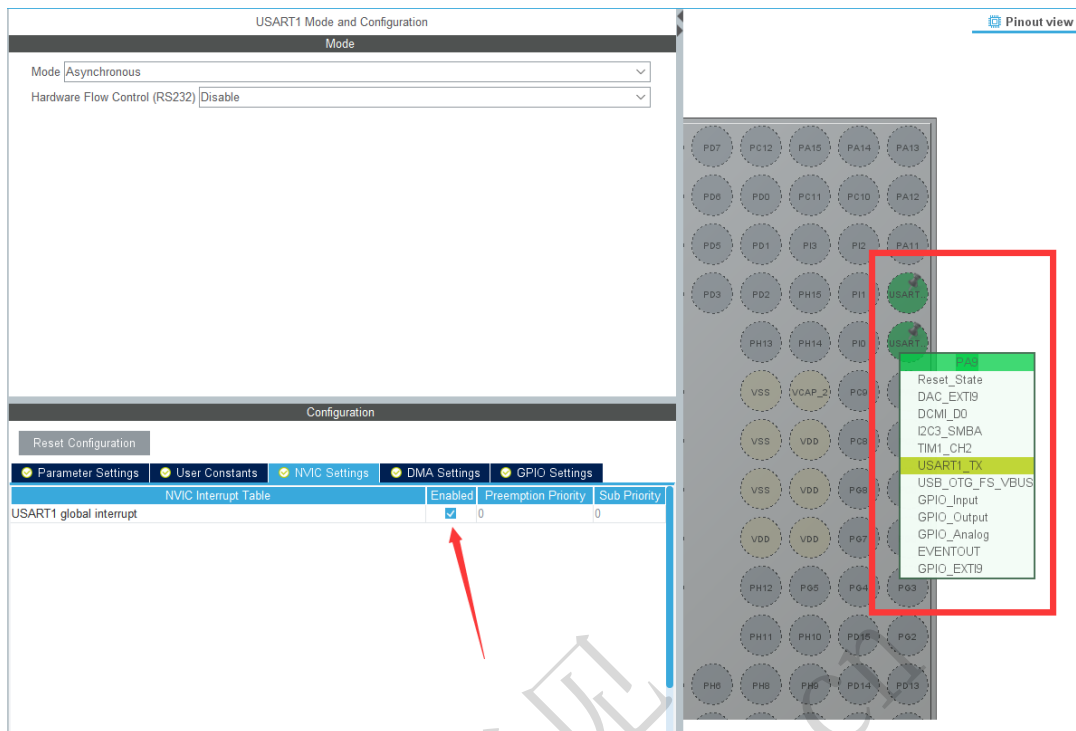


- (2) 选择芯片，查看芯片的型号，在搜索框输入芯片型号，本实验中选用的芯片是 STM32F407IGHX

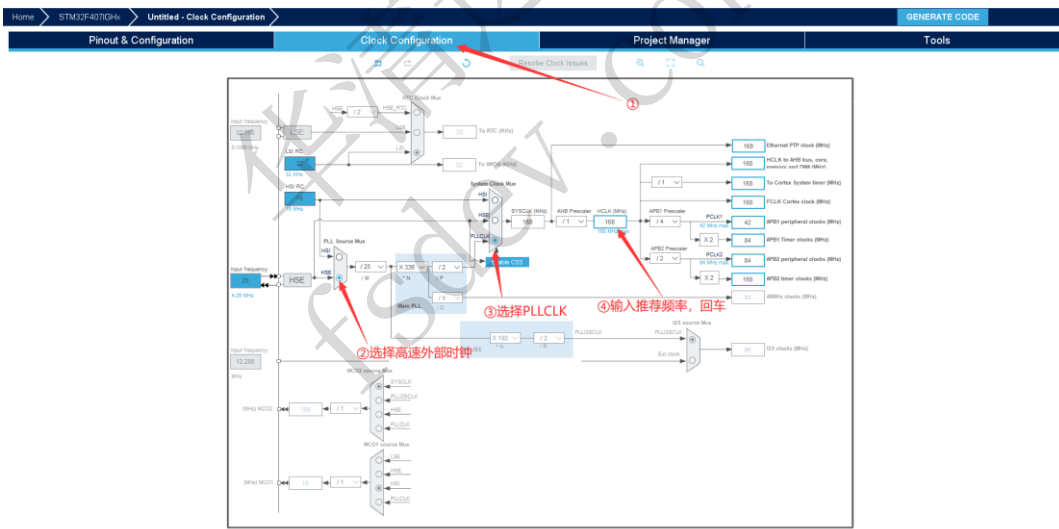




- (3) 配置 PA9 为 USART1_TX, 配置 PA10 为 USART1_RX, 初始化串口。



- (4) 配置 RCC, 点击左侧的 RCC, 找到 High Speed Clock (HSE), 点击选择 Crystal/Ceramic Resonator, RCC 配置完成, 点击上方的 Clock Configuration, 配置时钟树



- (5) 配置工程名字, 工程路径和 IDE 软件 (MDK-ARM V5.27), (注意不要有中文路径)。
 (6) 工程软件生成配置, 配置工程为 C 文件和 h 文件分开, 以及拷贝使用到的 HAL 库文件, 其他的文件不拷贝, 最后点击右上角的 “GENWRTE CODE”, 生成工程。
 (7) 生成工程完成后会弹出提示框, 选择 Open Project 可直接从 MDK 中打开工程代码。

3.6.6 【实验步骤】

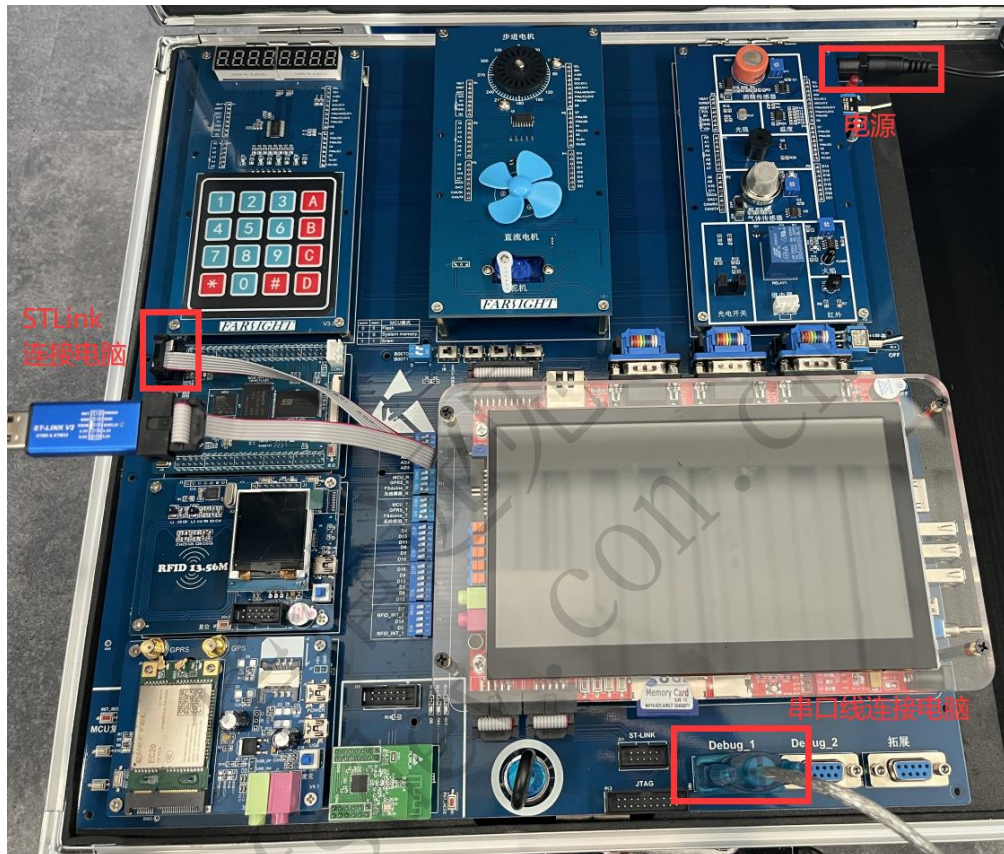
- (1) 打开工程文件, 路径为



【华清远见-嵌入式 ARM 实验箱资料-III\华清远见-STM 系列资料\Cortex-M4 部分\程序源码

\6_USART1】

- (2) 对工程进行编译调试
- (3) 开发板连接



- (4) 往开发板下载程序，具体步骤参照 1.4.4 节
- (5) 开发板上电查看实验结果

3.6.7 【实验结果】

将程序下载到 STM32F407 核心板后，打开两个串口调试工具，分别配置串口信息，向串口输入框中输入 5 个数据。串口 1 接收数据显示：



图 实验结果

1、2、3、4、5 是以中断方式接收并发送的

1.1.2 【实验参考代码】

这里只提供部分重要代码，如想查看部分函数里的详细实现，请打开工程源码，在μVision IDE for ARM 软件里进行查看和操作，这样查找方便清晰、有条理、并可以随意跟进。

```

1  int main(void)
2  {
3
4      /* USER CODE BEGIN 1 */
5      /* USER CODE END 1 */
6
7      /* MCU Configuration-----*/
8
9      /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
10     HAL_Init();

```




```

11
12     /* Configure the system clock */
13     SystemClock_Config();
14
15     /* Initialize all configured peripherals */
16     MX_GPIO_Init();
17     MX_USART1_UART_Init();
18
19     /* USER CODE BEGIN 2 */
20     // __HAL_UART_ENABLE_IT(&huart1, UART_IT_IDLE);
21
22     /* USER CODE END 2 */
23
24     /* Infinite loop */
25     /* USER CODE BEGIN WHILE */
26     while (1)
27     {
28         /* USER CODE END WHILE */
29
30         /* USER CODE BEGIN 3 */
31         HAL_UART_Transmit(&huart1, Buff1, 5, 100); //轮询方式发送
32
33         HAL_UART_Receive_IT(&huart1, Buff, 5); //中断方式接收，中断回调函数里发送 Buff
34         HAL_Delay(500);
35
36     }
37     /* USER CODE END 3 */
38
39 }
```

3.7 按键扫描实验

3.7.1 【实验目的】

熟悉 RealView MDK 工具软件的使用方式。

能够把 ST 官方库文件添加到项目工程。

掌握 RealView MDK 软件的基本设计流程和设计步骤，能够使用工具进行设计、编程、仿真、调试。

掌握如何控制 STM32 的 IO 口输入；



3.7.2 【实验环境】

嵌入式 ARM 实验箱

ST-Link 仿真器

RealView MDK5.21 集成开发软件

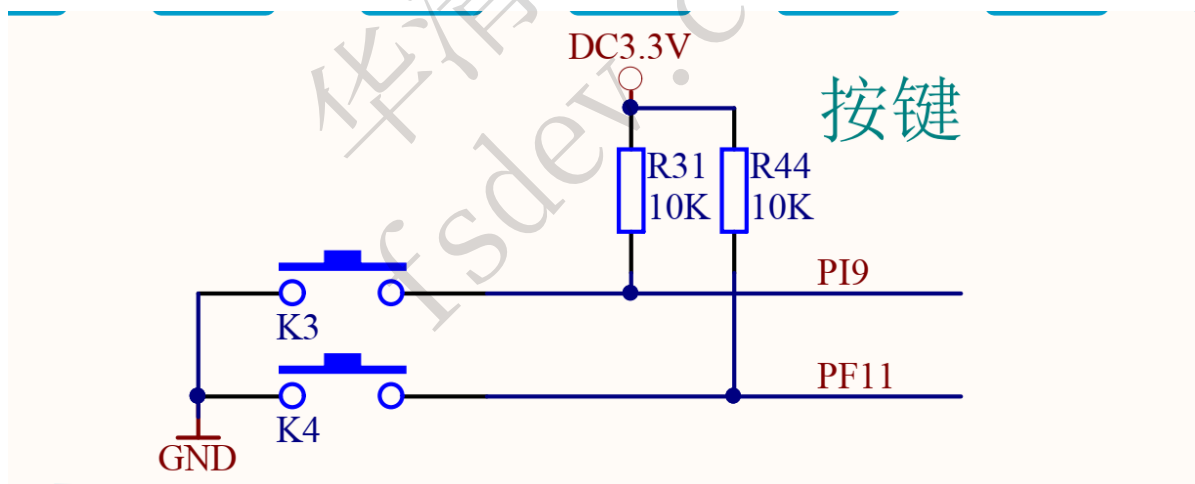
PC 机 XP、Window7/8 (32/64bit)

3.7.3 【实验内容】

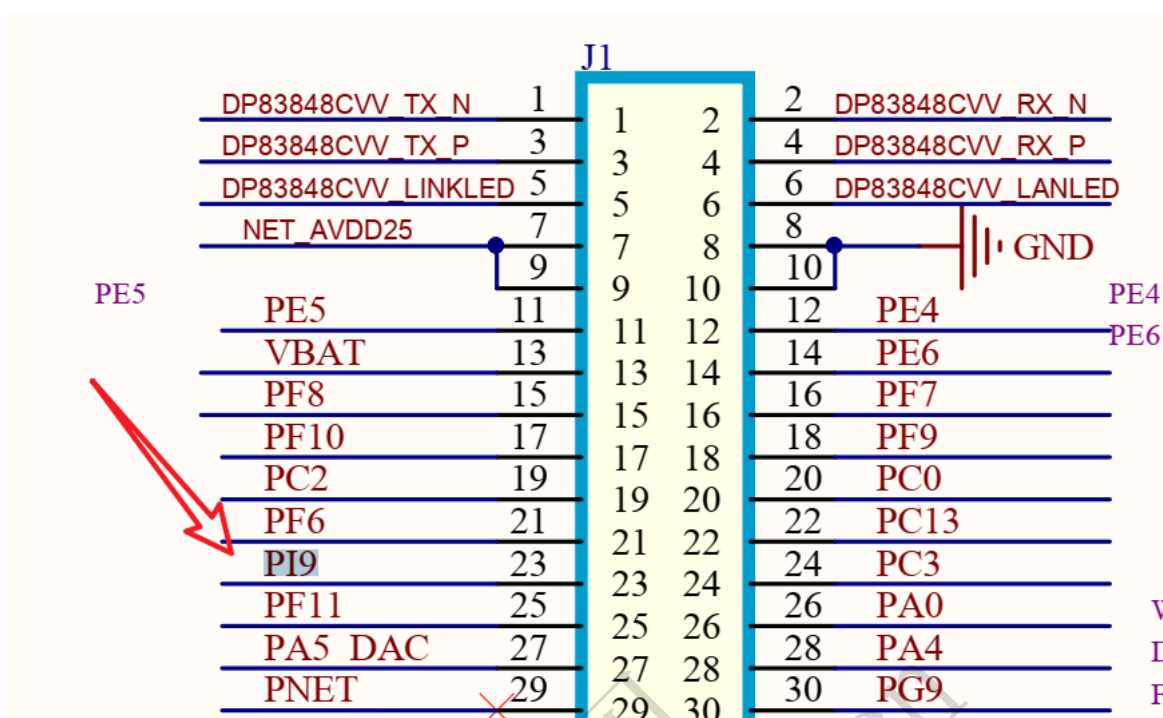
编写按键程序，当检测到按键按下时，串口打印按键被按下；当检测到按键抬起时，串口打印按键抬起。调试编译好程序后，将程序下载到 STM32F407 开发板，按复位键观察实验现象。

3.7.4 【实验原理】

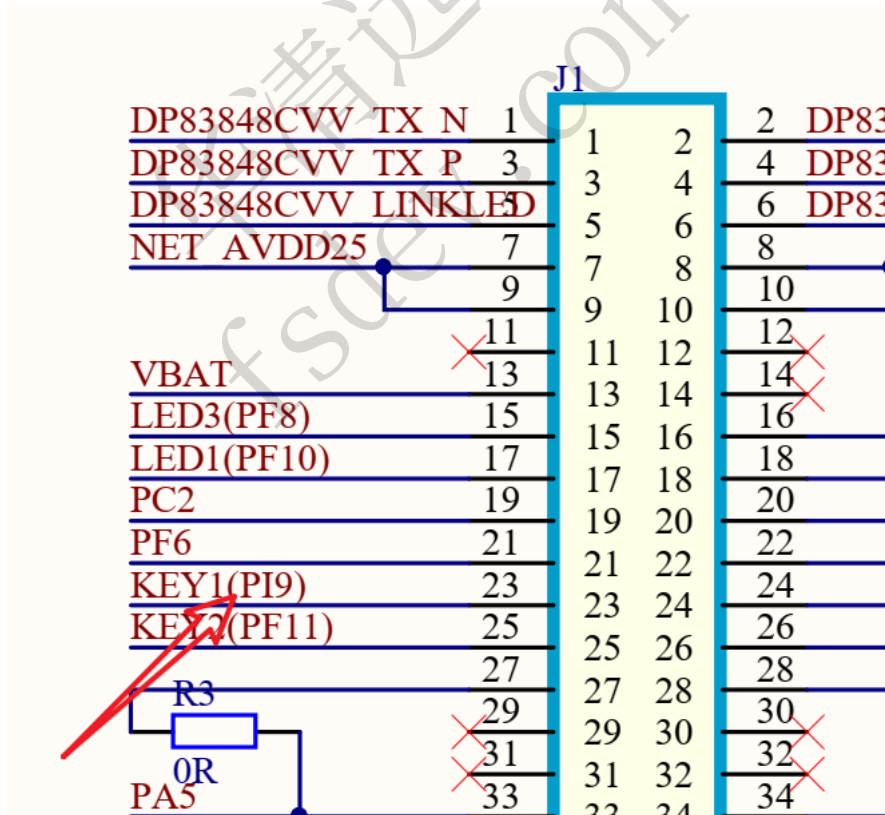
查看 KEY3 和 KEY4 的原理图，如下：



以 KEY3 为例，当按键 KEY3 未被按下时，PI9 为高电平，反之，KEY3 按下时，PI9 为低电平。PI9 在实验箱接 M4 底座 J1 的 23 脚。



打开 M4 开发板原理图，如下：



最终确定 KEY3 所在引脚为 PI9。

本节实验所用到的主要函数：



GPIO_PinState HAL_GPIO_ReadPin(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin)

功能：获取 GPIO 引脚的状态

参数 1：GPIO 所在组

参数 2：引脚号

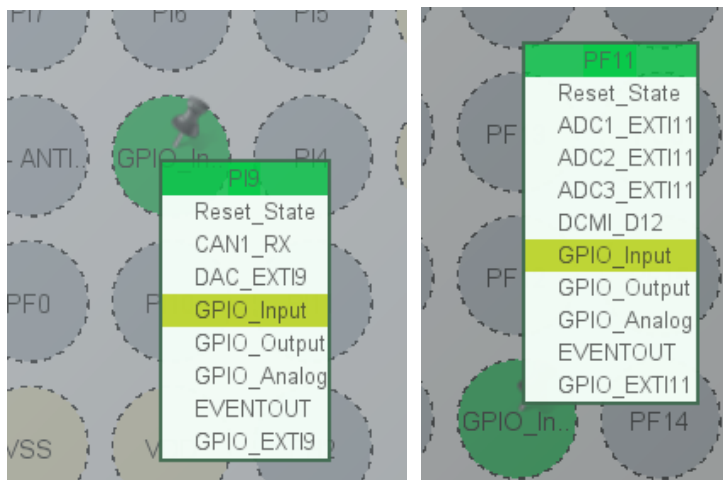
返回值：状态值 0 或 1

3.7.5 【stm32cubemx 配置介绍】

- (1) 打开 STM32CubeMX->点击左上角的 File->点击 New Project
- (2) 选择芯片，查看芯片的型号，在搜索框输入芯片型号，本实验中选用的芯片是 STM32F407IGHx

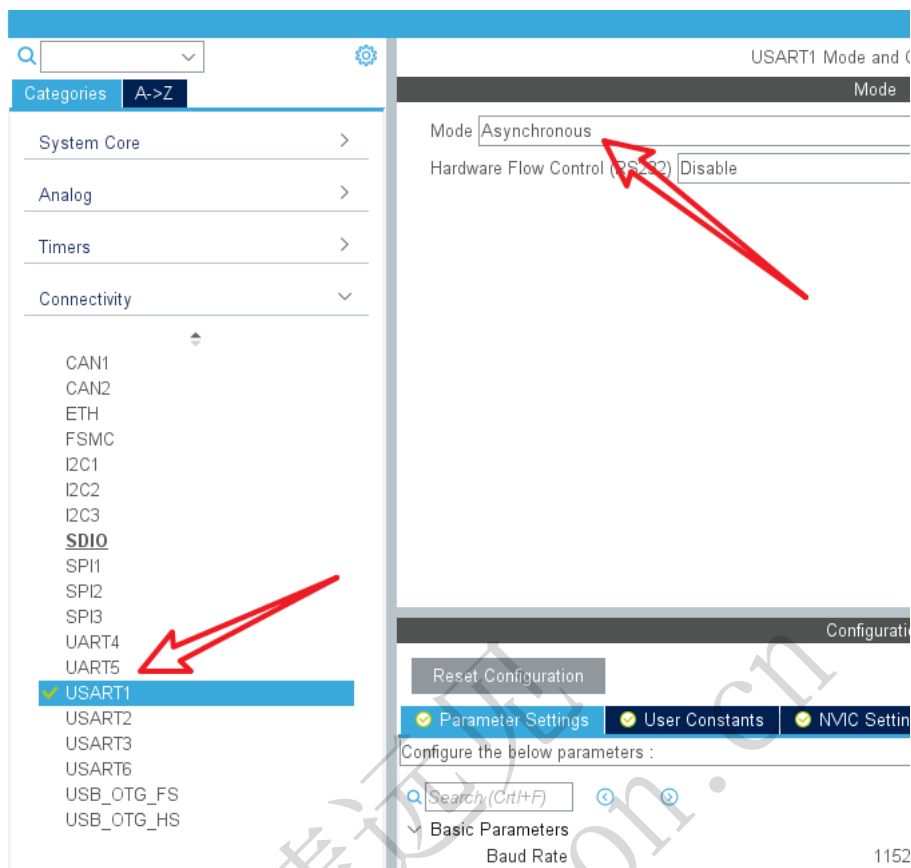


- (3) 配置 PI9 和 PF11 为 GPIO_Input 模式。





- (4) 配置 PA9 为 USART1_TX，配置 PA10 为 USART1_RX，初始化串口。



- (5) 配置工程名字，工程路径和 IDE 软件（MDK-ARM V5.27），（注意不要有中文路径）。
- (6) 工程软件生成配置，配置工程为 C 文件和 h 文件分开，以及拷贝使用到的 HAL 库文件，其他的文件不拷贝，最后点击右上角的“GENWRTE CODE”，生成工程。
- (7) 生成工程完成后会弹出提示框，选择 Open Project 可直接从 MDK 中打开工程代码。

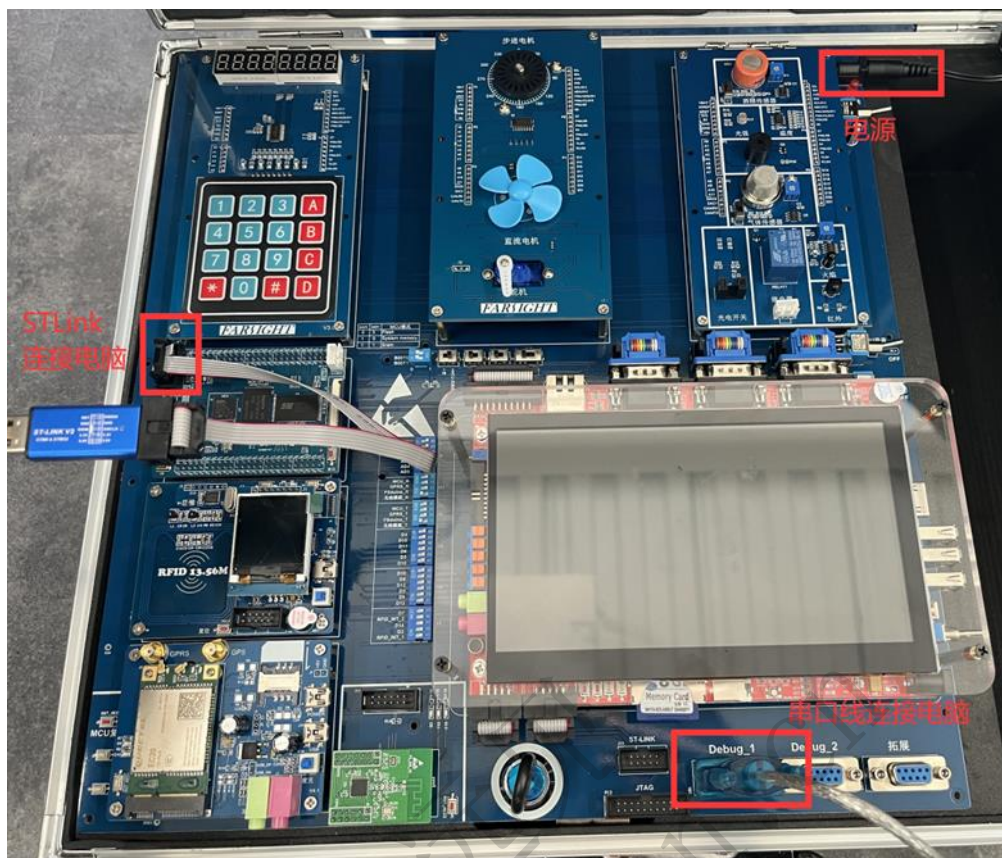
3.7.6 【实验步骤】

- (1) 打开工程文件，路径为

【华清远见-嵌入式 ARM 实验箱资料-III\华清远见-STM 系列资料\Cortex-M4 部分\程序源码

\KEY_POLLING】

- (2) 对工程进行编译调试
- (3) 开发板连接



- (4) 往开发板下载程序，具体步骤参照 1.4.4 节
- (5) 开发板上电查看实验结果

3.7.7 【实验结果】

将程序下载到 FS-STM32F407 后，打开串口调试工具，按复位键，按下 KEY3/或 KEY4，串口打印

KEY3/4 被按下，松开按键，串口打印按键被释放。



3.7.8 【实验参考代码】

这里只提供部分重要代码，如想查看部分函数里的详细实现，请打开工程源码，在µVision IDE for ARM 软件里进行查看和操作，这样查找方便清晰、有条理、并可以随意跟进。

COBOL Code

```
1#define KEY3 HAL_GPIO_ReadPin(GPIOI,GPIO_PIN_9) //获取 GPIO 状态
2#define KEY4 HAL_GPIO_ReadPin(GPIOF,GPIO_PIN_11) //获取 GPIO 状态
3
4int main(void)
5{
6 /* USER CODE BEGIN 1 */
7
8 /* USER CODE END 1 */
9
10 /* MCU Configuration-----*/
11
12 /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
13 HAL_Init();
14
15 /* USER CODE BEGIN Init */
16
```




```

17  /* USER CODE END Init */
18
19  /* Configure the system clock */
20  SystemClock_Config();
21
22  /* USER CODE BEGIN SysInit */
23
24  /* USER CODE END SysInit */
25
26  /* Initialize all configured peripherals */
27  MX_GPIO_Init();
28  MX_USART1_UART_Init();
29  /* USER CODE BEGIN 2 */
30  //打印串口数据
31  printf("-----KEY TEST-----\r\n");
32  /* USER CODE END 2 */
33
34  /* Infinite loop */
35  /* USER CODE BEGIN WHILE */
36  while (1)
37  {
38      //检测 KEY3 是否被按下
39      if(KEY3==0)
40      {
41          HAL_Delay(1);
42          printf("KEY3 is pressed\r\n");
43          //消抖，一直到松开按键
44          while(!KEY3);
45          printf("KEY3 is released\r\n");
46      }
47      if(KEY4==0)
48      {
49          HAL_Delay(1);
50          printf("KEY4 is pressed\r\n");
51          while(!KEY4);
52          printf("KEY3 is released\r\n");
53      }
54  /* USER CODE END WHILE */
55
56  /* USER CODE BEGIN 3 */
57  }
58  /* USER CODE END 3 */
59 }

```



3.8 外部中断实验

3.8.1 【实验目的】

熟悉 RealView MDK 工具软件的使用方式。

能够把 ST 官方库文件添加到项目工程。

掌握 RealView MDK 软件的基本设计流程和设计步骤，能够使用工具进行设计、编程、仿真、调试。

掌握如何控制 STM32 的外部中断；

3.8.2 【实验环境】

嵌入式 ARM 实验箱

ST-Link 仿真器

RealView MDK5.21 集成开发软件

PC 机 XP、Window7/8 (32/64bit)

3.8.3 【实验内容】

编写中断程序，当检测到 KEY3 或 KEY4 被按下时，串口打印按键被按下；当检测到按键抬起时，串口打印按键抬起。调试编译好程序后，将程序下载到 FS-STM4 开发板，按复位键观察实验现象。

3.8.4 【实验原理】

在按键扫描实验章节，我们已经知道了 KEY3 所对应的引脚是 PI9，KEY4 所对应的引脚为 PF11。

不同的计算机其硬件结构和软件指令是不完全相同的，因此，中断系统也是不相同的。计算机的中断系统能够加强 CPU 对多任务事件的处理能力。中断机制是现代计算机系统中的基础设施之一，它在系统中起着通信网络作用，以协调系统对各种外部事件的响应和处理。中断是实现多道程序设计的必要条件。中断是 CPU 对系统发生的某个事件作出的一种反应。引起中断的事件称为中断源。中断源向 CPU



提出处理的请求称为中断请求。发生中断时被打断程序的暂停点称为断点。CPU 暂停现执行程序而转为响应中断请求的过程称为中断响应。处理中断源的程序称为中断处理程序。CPU 执行有关的中断处理程序称为中断处理。而返回断点的过程称为中断返回。中断的实现实行软件和硬件综合完成，硬件部分叫做硬件装置，软件部分称为软件处理程序。

需要注意的是，在工程配置中，我们配置这两个 IO 的外部中断之后，每个引脚对应的中断处理函数都被分到对应的组中，但是不同组的中断处理函数用的是同一个回调函数，所以，当中断发生的时候我们需要根据不同的引脚对产生的中断做一个区分，这样就能具体知道哪个中断是由哪个引脚产生的。

本节实验所用函数：

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
```

功能：当 GPIO 引脚触发外部中断，执行此回调函数。

参数 1：触发中断的 GPIO 引脚

当我们勾选 GPIO 为外部中断时，生成的工程会在 stm32f3xx_it.c 文件下生成对应的函数。



```

in.c* | gpio.c | stm32f4xx_hal_gpio.h | stm32f4xx_it.c | stm32f4xx_hal_gpio.c
/**
 * @brief This function handles EXTI line[9:5] interrupts.
 */
void EXTI9_5_IRQHandler(void)
{
    /* USER CODE BEGIN EXTI9_5_IRQn 0 */

    /* USER CODE END EXTI9_5_IRQn 0 */
    HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_9);
    /* USER CODE BEGIN EXTI9_5_IRQn 1 */

    /* USER CODE END EXTI9_5_IRQn 1 */
}

/**
 * @brief This function handles EXTI line[15:10] interrupts.
 */
void EXTI15_10_IRQHandler(void)
{
    /* USER CODE BEGIN EXTI15_10_IRQn 0 */

    /* USER CODE END EXTI15_10_IRQn 0 */
    HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_11);
    /* USER CODE BEGIN EXTI15_10_IRQn 1 */

    /* USER CODE END EXTI15_10_IRQn 1 */
}

/* USER CODE BEGIN 1 */

```

查看其中的 HAL_GPIO_EXTI_IRQHandler () 函数，当中断触发时会调用

HAL_GPIO_EXTI_Callback(GPIO_Pin).

所以我们修改 HAL_GPIO_EXTI_Callback()函数即可。如下：

COBOL Code

```

1 void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
2 {
3     //检测触发引脚是不是 PF11
4     if(GPIO_Pin == GPIO_PIN_11)
5     {
6         //如果 PF11 是低电平，按键被按下；反之，释放按键
7         if(HAL_GPIO_ReadPin(GPIOF, GPIO_PIN_11) == GPIO_PIN_RESET)/* read PF11 state */
8         /
9             printf("KEY4 is pressed\r\n");
10        else
11            printf("KEY4 is released\r\n");
12    }

```



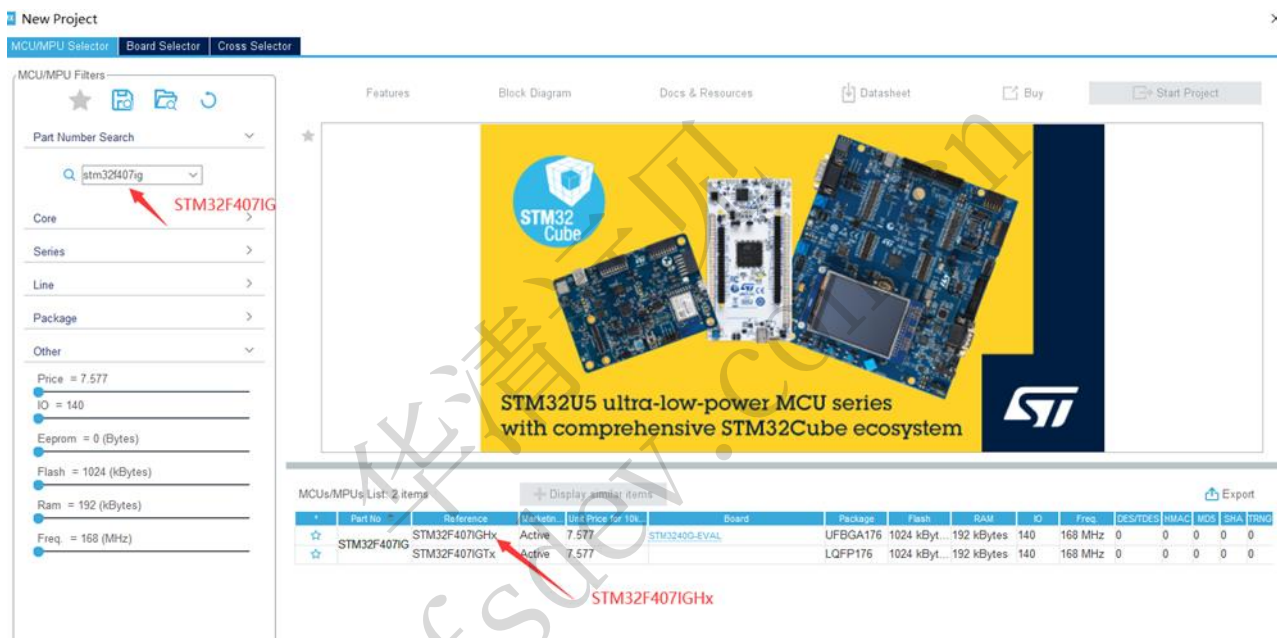
```

13     if(GPIO_Pin == GPIO_PIN_9)
14     {
15         if(HAL_GPIO_ReadPin(GPIOI, GPIO_PIN_9) == GPIO_PIN_RESET) /* read PI9 state */
16             printf("KEY3 is pressed\r\n");
17     }
18     else
19         printf("KEY3 is released\r\n");
20 }

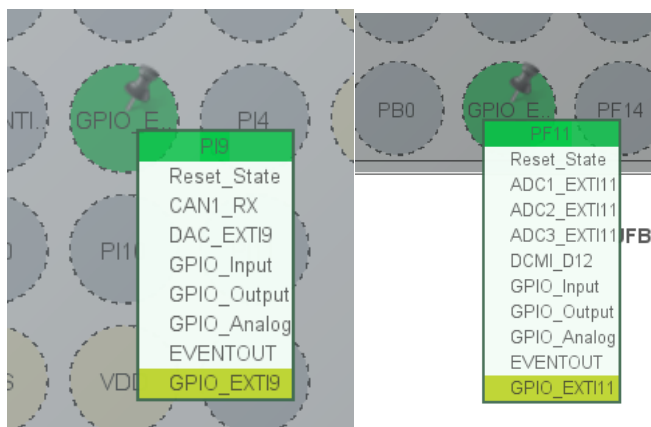
```

3.8.5 【stm32cubemx 配置介绍】

- (1) 打开 STM32CubeMX->点击左上角的 File->点击 New Project
- (2) 选择芯片，查看芯片的型号，在搜索框输入芯片型号，本实验中选用的芯片是 STM32F407IGHx



- (3) 配置 PI9 和 PF11



- (4) 配置 NVIC



NVIC Interrupt Table		Enabled	Preemption Priority	Sub Pr
Non maskable interrupt		☒	0	0
Hard fault interrupt		☒	0	0
Memory management fault		☒	0	0
Pre-fetch fault, memory access fault		☒	0	0
Undefined instruction or illegal state		☒	0	0
System service call via SWI instruction		☒	0	0
Debug monitor		☒	0	0
Pendable request for system service		☒	0	0
Time base: System tick timer		☒	0	0
PVD interrupt through EXTI line 16		☐	0	0
Flash global interrupt		☐	0	0
RCC global interrupt		☐	0	0
EXTI line[9:5] interrupts		☒	0	0
USART1 global interrupt		☐	0	0
EXTI line[15:10] interrupts		☒	0	0
FPU global interrupt		☐	0	0

(5) 配置 PF11 和 PI9 为上升沿和下降沿触发。

PF11 Configuration :

GPIO mode: External Interrupt Mode with Rising/Falling edge trigger detection

GPIO Pull-up/Pull-down: External Interrupt Mode with Rising edge trigger detection

User Label: External Interrupt Mode with Falling edge trigger detection

(6) 配置串口 1

USART1 Mode and Configuration

Mode: Asynchronous

Hardware Flow Control (USART2) Disable

Reset Configuration

Parameter Settings User Constants NVIC Settings

Configure the below parameters :

Basic Parameters

Baud Rate



- (5) 配置工程名字，工程路径和 IDE 软件（MDK-ARM V5.27），（注意不要有中文路径）。
- (6) 工程软件生成配置，配置工程为 C 文件和 h 文件分开，以及拷贝使用到的 HAL 库文件，其他的文件不拷贝，最后点击右上角的“GENWRTE CODE”，生成工程。
- (7) 生成工程完成后会弹出提示框，选择 Open Project 可直接从 MDK 中打开工程代码。

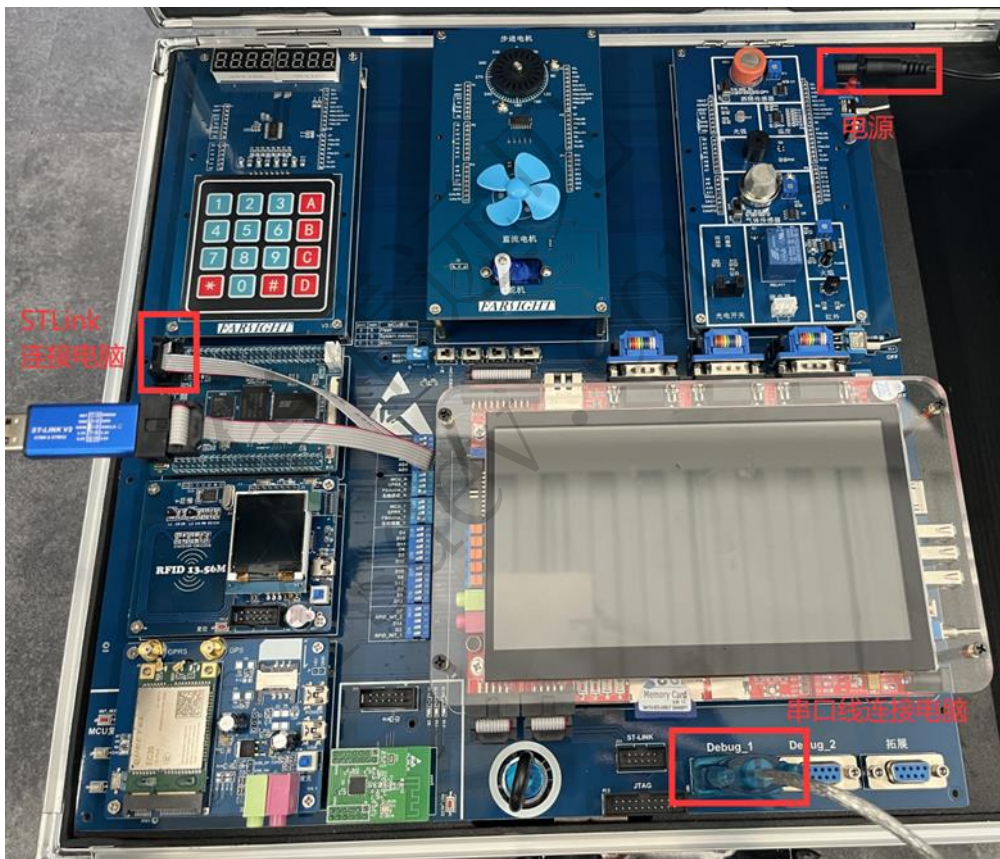
3.8.6 【实验步骤】

- (1) 打开工程文件，路径为

【华清远见-嵌入式 ARM 实验箱资料-III\华清远见-STM 系列资料\Cortex-M4 部分\程序源码

\KEY_POLLING】

- (2) 对工程进行编译调试
- (3) 开发板连接



- (4) 往开发板下载程序，具体步骤参照 1.4.4 节
- (5) 开发板上电查看实验结果。

3.8.7 【实验结果】

将程序下载到 FS-STM32F407 后，打开串口调试工具，按复位键，按下 KEY3/或 KEY4，串口打印

KEY3/4 被按下，松开按键，串口打印按键被释放。



3.8.8 【实验参考代码】

这里只提供部分重要代码，如想查看部分函数里的详细实现，请打开工程源码，在μVision IDE for ARM 软件里进行查看和操作，这样查找方便清晰、有条理、并可以随意跟进。

COBOL Code

```
1  /* USER CODE BEGIN 4 */
2  void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
3  {
4      //检测触发引脚是不是 PF11
5      if(GPIO_Pin == GPIO_PIN_11)
6      {
7          //如果 PF11 是低电平，按键被按下；反之，释放按键
8          if(HAL_GPIO_ReadPin(GPIOF, GPIO_PIN_11) == GPIO_PIN_RESET)/* read PF11 state */
9              printf("KEY4 is pressed\r\n");
10         else
11             printf("KEY4 is released\r\n");
12     }
13     if(GPIO_Pin == GPIO_PIN_9)
14     {
15         if(HAL_GPIO_ReadPin(GPIOI, GPIO_PIN_9) == GPIO_PIN_RESET) /* read PI9 state */
16             printf("KEY3 is pressed\r\n");
```



```

17         else
18             printf("KEY3 is released\r\n");
19     }
20 }
21 //输出重定向到串口1
22 int fputc(int ch, FILE *f)
23 {
24     while((USART1->SR&0X40)==0);
25     USART1->DR = (uint8_t) ch;
26     return ch;
27 }
28 /* USER CODE END 4 */

```

3.9 继电器实验

3.9.1 【实验目的】

掌握上一节有关向串口收发信息的环境配置，并且熟悉串口如何接收数据中断

3.9.2 【实验环境】

嵌入式 ARM 实验箱

ST-Link 仿真器

RealView MDK5.21 集成开发软件

PC 机 XP、Window7/8 (32/64bit)

串口调试工具

串口交叉线

3.9.3 【实验内容】

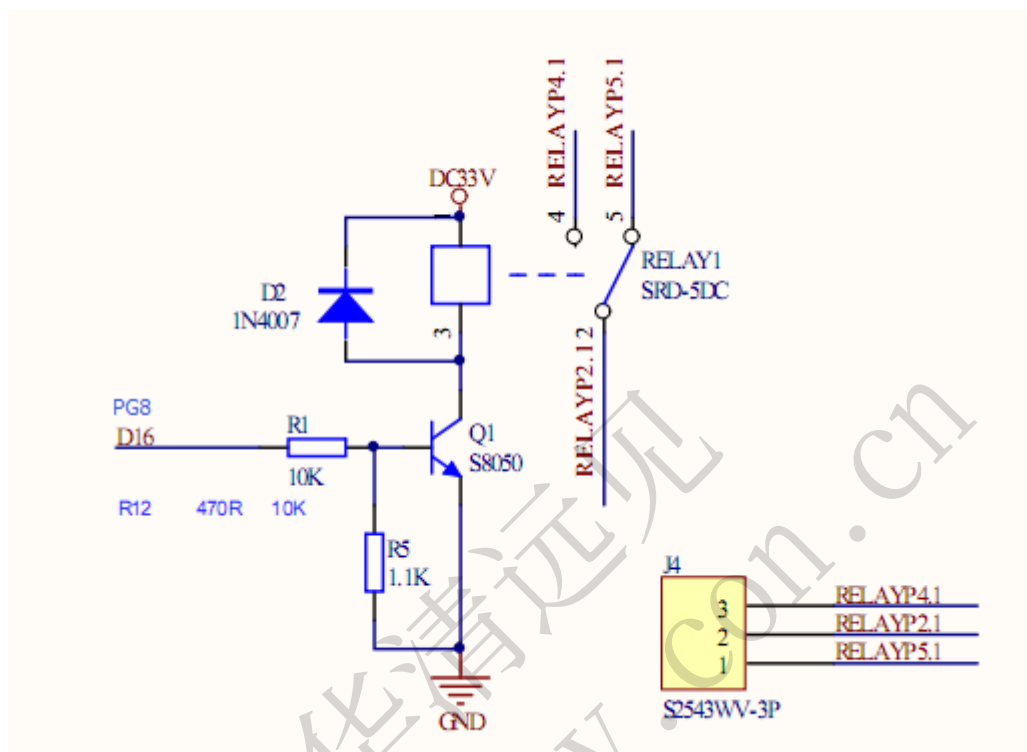
了解了串口中断和继电器的概念及相关原理之后，编写实验程序，实现可以用串口控制继电器开启关闭的功能。调试编译好固件程序后，将程序下载到 STM32F407 核心板，按复位键，并且向串口输入



“开”或者“关”观察实验现象。

3.9.4 【实验原理】

该实验需要使用到开发板上的直流电机，相关硬件电路如下：



继电器连接电路图

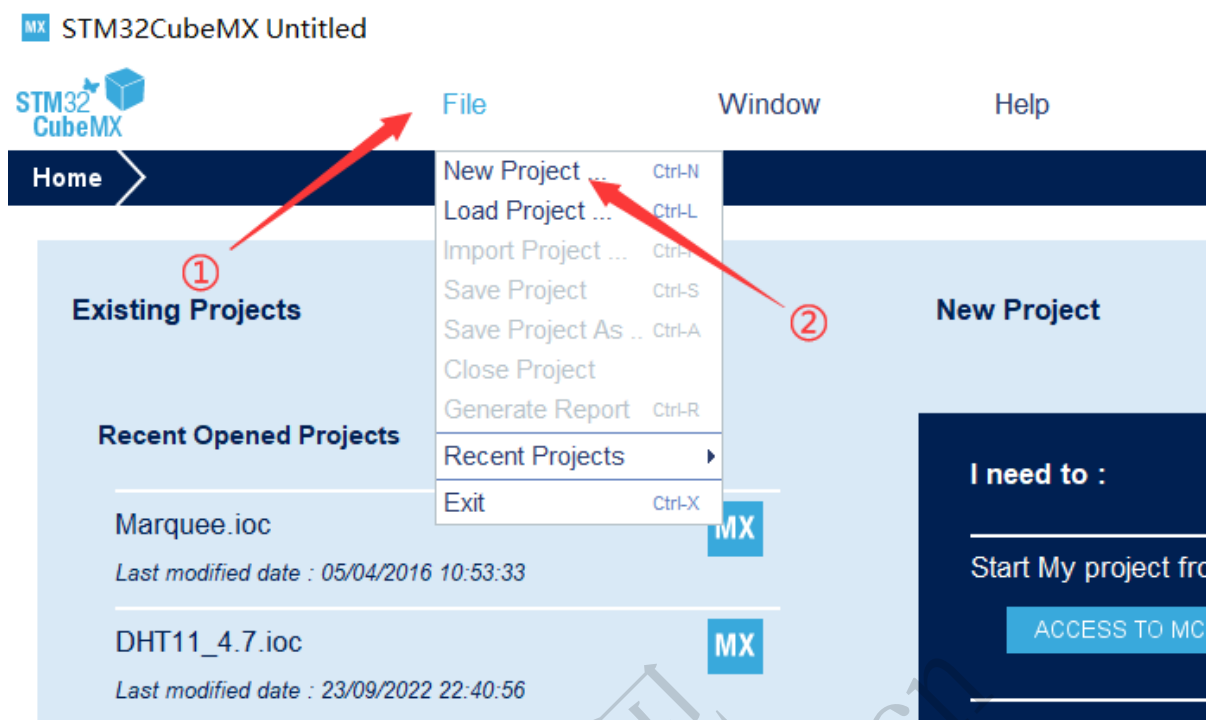
由上图可以看出单片机控制的 IO 口在 PG8 上。继电器原理说明如下：单片机是一个弱电器件，一般情况下它们大都工作在 5V 甚至更低，驱动电流在 mA 级以下而要把它用于一些大功率场合，比如控制电动机，显然是不行的。所以，就要有一个环节来衔接，这个环节就是所谓的“功率驱动”。继电器驱动就是一个典型的简单的功率驱动环节。在这里，继电器驱动含有两个意思：一是对继电器进行驱动，因为继电器本身对于单片机来说就是一个功率器件；还有就是继电器去驱动其他负载，比如继电器可以驱动中间继电器，可以直接驱动接触器，所以，继电器驱动就是单片机与其他大功率负载接口。这个很重要，因为，一直让我们的电气工程师（我指的是那些没有学习过相应的电子技术的）感到迷惑不解的是：一个小小的芯片，怎么会有如此强大的威力来控制像电动机这样强大的东西？



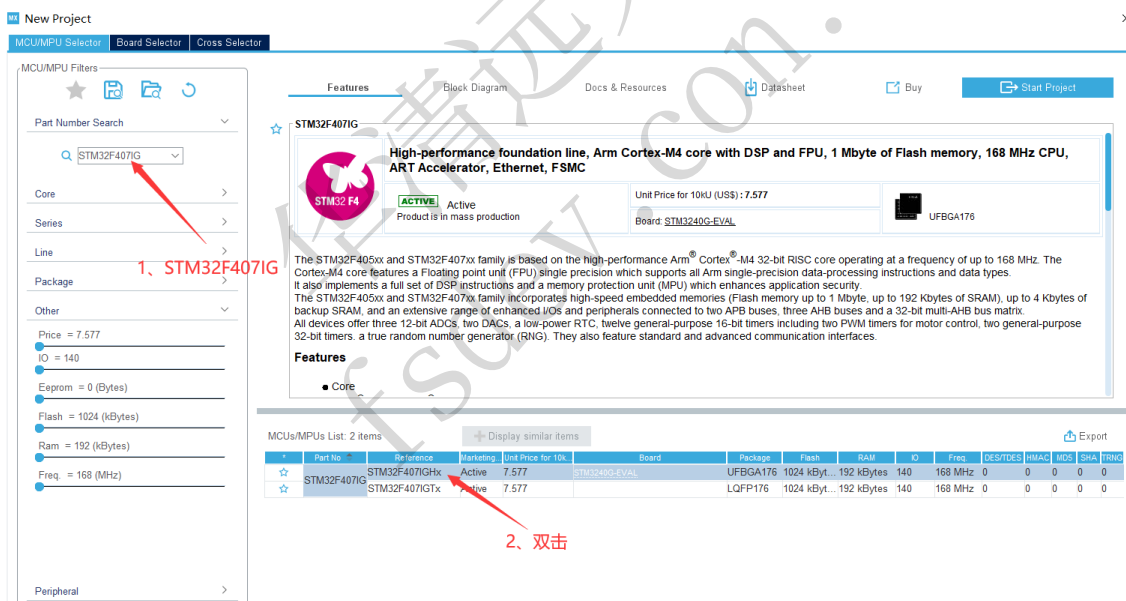
怎么样理解这个电路图? 要理解这个电路,其实也比较容易.那么请您按照我的思路来,应该没有问题: 首先的,里面的三极管很重要三极管是电子电路里很重要的一个元件.怎么样理解三极管呢? 简单的来说三极管有两个作用一个是放大作用,一个是开关作用.(严格来讲开关作用是放大作用的极限情况不过没关系,把两者分开,更便于理解它的工作原理).在这里,我们只了解它跟本电路有关的开关作用. 首先把三极管想成一个水龙头. 上面的 V_{CC} 就是水池,继电器是一个水轮机,下面的 GND 是比水池低的任何一点.刚才说过,三极管就是水龙头,它的把手就是那个带有电阻的引脚. 现在,单片机的某一个需要控制这个继电器电路的输出引脚就是一只"手",当单片机的这个引脚输出低电平的时候,就像"手"在打开三极管"水龙头",水就从上往下流,继电器"水轮机"就开始转起来了.反之,如果是输出高电平,"手"就开始关"水龙头",继电器"水轮机"因为没有水流下来,就会停止. 这就是三极管的开关作用. 简单的理解和记忆就是:三极管是一个开关器件,其实你真的可以将它看成是一个开关,只不过它不是用手来控制,而是用电压(电流)来控制的,因此,三极管有些时候也被称做电子开关(与机械开关相区别). 图上还有一个东西,是保护二极管,如果不需要深入理解的话,你大可不必追究为什么有它存在,但是一定得记住,只要是用三极管驱动继电器的场合,一般都有它的存在.需要特别注意的是它的接法:并联在继电器两端 阴极一定是接 V_{CC} 。

3.9.5 【stm32cubemx 配置介绍】

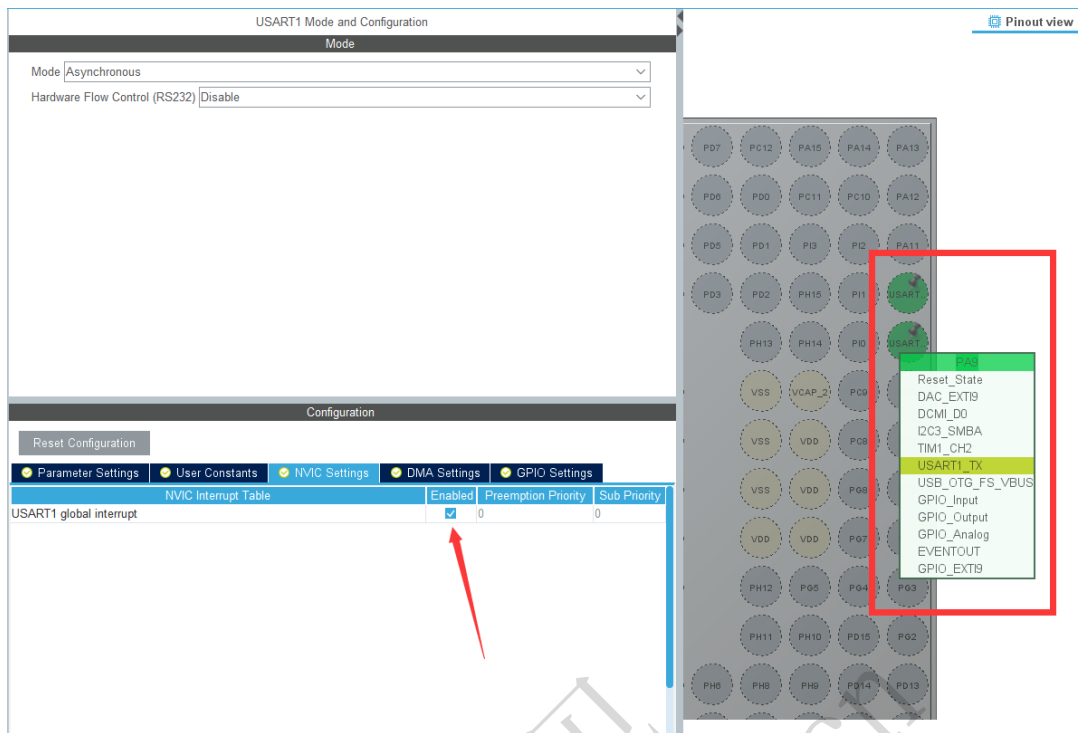
(1) 打开 STM32CubeMX->点击左上角的 File->点击 New Project



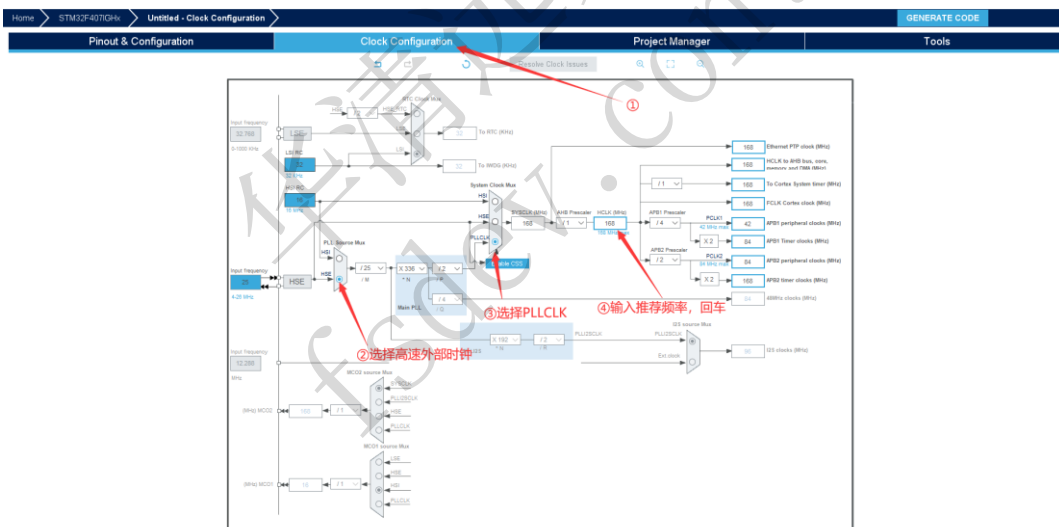
(2) 选择芯片，查看芯片的型号，在搜索框输入芯片型号，本实验中选用的芯片是 STM32F407IGHX



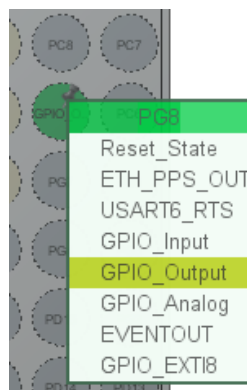
(3) 配置 PA9 为 USART1_TX，配置 PA10 为 USART1_RX，初始化串口。



(4) 配置 RCC，点击左侧的 RCC，找到 High Speed Clock (HSE)，点击选择 Crystal/Ceramic Resonator，RCC 配置完成，点击上方的 Clock Configuration，配置时钟树



(5) 配置 PG8 为 GPIO_Output 模式





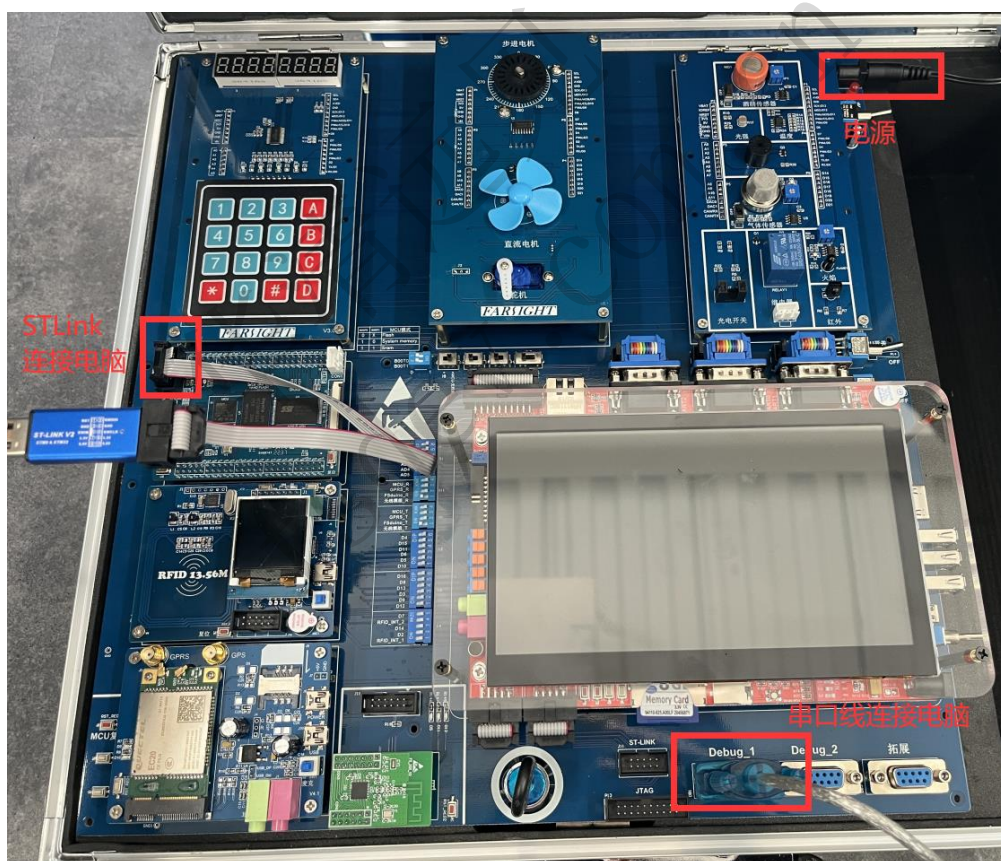
- (6) 配置工程名字，工程路径和 IDE 软件（MDK-ARM V5.27），（注意不要有中文路径）。
- (7) 工程软件生成配置，配置工程为 C 文件和 h 文件分开，以及拷贝使用到的 HAL 库文件，其他的文件不拷贝，最后点击右上角的“GENWRTE CODE”，生成工程。
- (8) 生成工程完成后会弹出提示框，选择 Open Project 可直接从 MDK 中打开工程代码。

3.9.6 【实验步骤】

- (1) 打开工程文件，路径为

【华清远见-嵌入式 ARM 实验箱资料-III\华清远见-STM 系列资料\Cortex-M4 部分\程序源码\7_Relay】

- (2) 对工程进行编译调试
- (3) 开发板连接



- (4) 往开发板下载程序，具体步骤参照 1.4.4 节
- (5) 开发板上电查看实验结果



3.9.7 【实验结果】

将固件程序下载到开发板后，按一下复位键；在串口上显示打印信息，在输入框中输入开或者关，可以听到继电器开启或者关闭的声音。继电器位置如下图红圈所示：



图 继电器示意图

串口输入如下图所示：

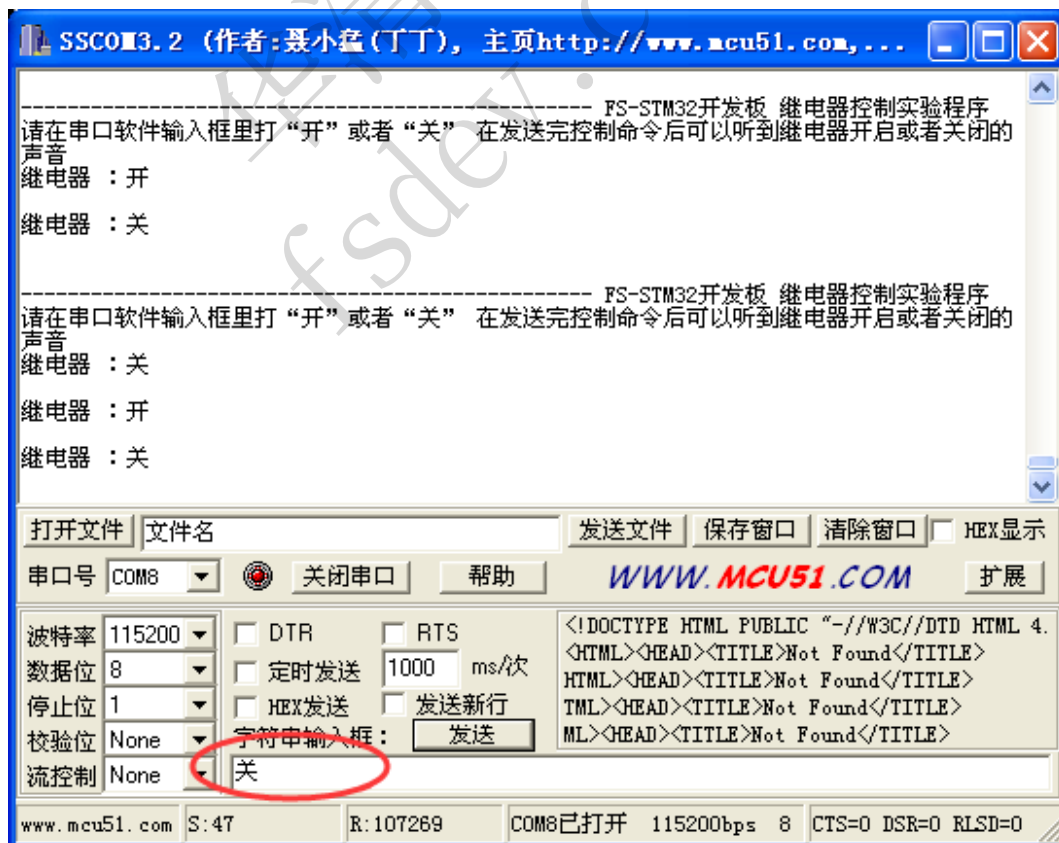




图 实验结果

1.1.3 【实验参考代码】

这里只提供部分重要代码，如想查看部分函数里的详细实现，请打开工程源码，在μVision IDE for ARM 软件里进行查看和操作，这样查找方便清晰、有条理、并可以随意跟进。

```

1      /* Includes -----*/
2      #include "stm32f4xx_hal.h"
3      #include "usart.h"
4      #include "gpio.h"
5
6      /* USER CODE BEGIN Includes */
7      #include "stdio.h"
8      /* USER CODE END Includes */
9
10     /* Private variables -----*/
11
12     /* USER CODE BEGIN PV */
13     /* Private variables -----*/
14     uint8_t Buff[2] = {0};
15     /* USER CODE END PV */
16
17     /* Private function prototypes -----*/
18     void SystemClock_Config(void);
19
20     /* USER CODE BEGIN PFP */
21     /* Private function prototypes -----*/
22
23     /* USER CODE END PFP */
24
25     /* USER CODE BEGIN 0 */
26
27     /* USER CODE END 0 */
28
29     int main(void)
30     {
31
32         /* USER CODE BEGIN 1 */
33
34         /* USER CODE END 1 */
35

```



```

36      /* MCU Configuration-----*/
37
38      /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
39      HAL_Init();
40
41      /* Configure the system clock */
42      SystemClock_Config();
43
44      /* Initialize all configured peripherals */
45      MX_GPIO_Init();
46      MX_USART1_UART_Init();
47
48      /* USER CODE BEGIN 2 */
49      printf("\n\r");
50      printf("\n\r-----");
51      printf("\n\r FS-STM32 开发板 继电器控制实验程序");
52      printf("\n\r 请在串口软件输入框里打“开”或者“关”");
53      printf("\n\r 在发送完控制命令后可以听到继电器开启或者关闭的声音\n\r");
54      /* USER CODE END 2 */
55
56      /* Infinite loop */
57      /* USER CODE BEGIN WHILE */
58      while (1)
59      {
60          /* USER CODE END WHILE */
61          /* USER CODE BEGIN 3 */
62          HAL_UART_Receive_IT(&huart1, Buff, 2);
63      }
64      /* USER CODE END 3 */
65
66  }
67
68  /** System Clock Configuration
69  */
70  void SystemClock_Config(void)
71  {
72
73      RCC_OscInitTypeDef RCC_OscInitStruct;
74      RCC_ClkInitTypeDef RCC_ClkInitStruct;
75
76      __PWR_CLK_ENABLE();
77
78      __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE1);

```



```

79
80     RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
81     RCC_OscInitStruct.HSEState = RCC_HSE_ON;
82     RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
83     RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
84     RCC_OscInitStruct.PLL.PLLM = 25;
85     RCC_OscInitStruct.PLL.PLLN = 336;
86     RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV2;
87     RCC_OscInitStruct.PLL.PLLQ = 4;
88     HAL_RCC_OscConfig(&RCC_OscInitStruct);
89
90     RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_SYSCLK | RCC_CLOCKTYPE_PCLK1
91                                   | RCC_CLOCKTYPE_PCLK2;
92     RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
93     RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
94     RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV4;
95     RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV2;
96     HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_5);
97
98     HAL_SYSTICK_Config(HAL_RCC_GetHCLKFreq() / 1000);
99
100    HAL_SYSTICK_CLKSourceConfig(SYSTICK_CLKSOURCE_HCLK);
101
102 }
103
104 /* USER CODE BEGIN 4 */
105 int fputc(int ch, FILE *f)
106 {
107     while((USART1->SR & 0X40) == 0); //循环发送,直到发送完毕
108     USART1->DR = (uint8_t) ch;
109     return ch;
110 }
111
112 void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
113 {
114     if(Buff[0] == 0xBF && Buff[1] == 0xAA) //检测接收到的数据是不是开
115     {
116         HAL_GPIO_WritePin(GPIOD, GPIO_PIN_8, GPIO_PIN_SET); //开启继电器
117         printf("\n\r 继电器 : 开\r\n");
118     }
119     else if(Buff[0] == 0xB9 && Buff[1] == 0xD8) //检测接收到的数据是不是关
120     {
121         HAL_GPIO_WritePin(GPIOD, GPIO_PIN_8, GPIO_PIN_RESET); //开启继电器

```



```
122     printf("\n\r 继电器 : 关\r\n");
123 }
124 }
125 /* USER CODE END 4 */
```

3.10 SysTick 系统滴答实验

3.10.1 【实验目的】

了解 SysTick 定时器的基本概念和原理；了解与 SysTick 定时器相关的几个寄存器的操作方法。

3.10.2 【实验环境】

嵌入式 ARM 实验箱

ST-Link 仿真器

RealView MDK5.21 集成开发软件

PC 机 XP、Window7/8 (32/64bit)

3.10.3 【实验内容】

编写程序，实现利用 SysTick 定时器实现 LED 灯 1 和 LED 灯 2,一秒闪烁一次。前者是在主函数里实现，后者在 SysTick 中断回调里实现，调试编译好固件程序后，将程序下载到 STM32F407 核心板，按复位键观察实验现象。

3.10.4 【实验原理】

SysTick 定时器被捆绑在 NVIC 中，用于产生 SysTick 异常。在以前，操作系统和所有使用了时基的系统，都必须需要一个硬件定时器来产生需要的“滴答”中断，作为整个系统的时基。滴答中断对操作系统尤其重要。例如，操作系统可以为多个任务许以不同数目的时间片，确保没有一个任务能霸占系统；或者把每个定时器周期的某个时间范围赐予特定的任务等，还有操作系统提供的各种定时功能，都与这



个滴答定时器有关。因此，需要一个定时器来产生周期性的中断，而且最好还让用户程序不能随意访问它的寄存器，以维持操作系统“心跳”的节律。

Cortex-M4 在内核部分 包含了一个简单的定时器——SysTick timer。因为所有的 CM4 芯片都带有这个定时器，软件在不同芯片生产厂商的 CM4 器件间的移植工作就得以化简。该定时器的时钟源可以是内部时钟（FCLK，CM4 上的自由运行时钟），或者是外部时钟（CM4 处理器上的 STCLK 信号）。不过，STCLK 的具体来源则由芯片设计者决定，因此不同产品之间的时钟频率可能会大不相同。因此，需要阅读芯片的使用手册来确定选择什么作为时钟源。在 STM32 中 SysTick 以 HCLK(AHB 时钟)或 HCLK/8 作为运行时钟。

SysTick 定时器：系统滴答定时器是一个非常基本的倒计时定时器，用于在每隔一定的时间产生一个中断，即使是系统在睡眠模式下也能工作。它使得 OS 在各 CM4 器件之间的移植中不必修改系统定时器的代码，移植工作一下子容易多了。

SysTick 定时器能产生中断，CM4 为它专门开出一个异常类型，并且在向量表中有它的一席之地。它使操作系统和其它系统软件在 CM4 器件间的移植变得简单多了，因为在所有 CM4 产品间，SysTick 的处理方式都是相同的。SysTick 定时器除了能服务于操作系统之外，还能用于其它目的：如作为一个闹铃，用于测量时间等。

HAL 库里，其实每次都已经自带配置了滴答定时器，我们调用的函数 HAL_Delay(); 就是通过滴答定时器，这是毫秒级的延迟。实现过程在主函数开始的时候调用了 SystemClock_Config();在函数里面调用了 HAL_SYSTICK_Config(HAL_RCC_GetHCLKFreq()/1000);

该函数就是获得当前 HCLK 的时钟频率，并除以 1000，这就是配置定时器的计数值。计数时间就是计数值乘以计数频率就是一毫秒。看下图 168 前还有一个预分频，要是你配置了 8，那么生成代码的时候，软件会自动把除以 1000 变成除以 8000。

在 SysTick 中断里设一个全局变量自加，而 HAL_Delay()就是判断获得实时的当前的变量值减去之前变



量初始值是否大于等于延迟时间，如果小于，就一直减，直到刚好等于时候才退出。

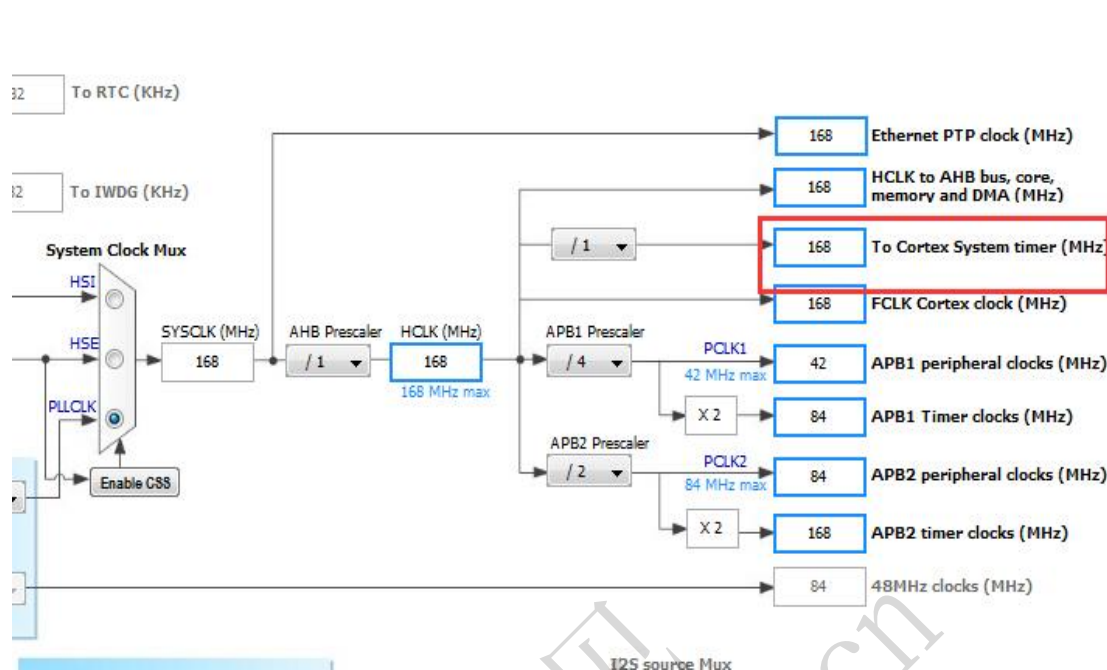


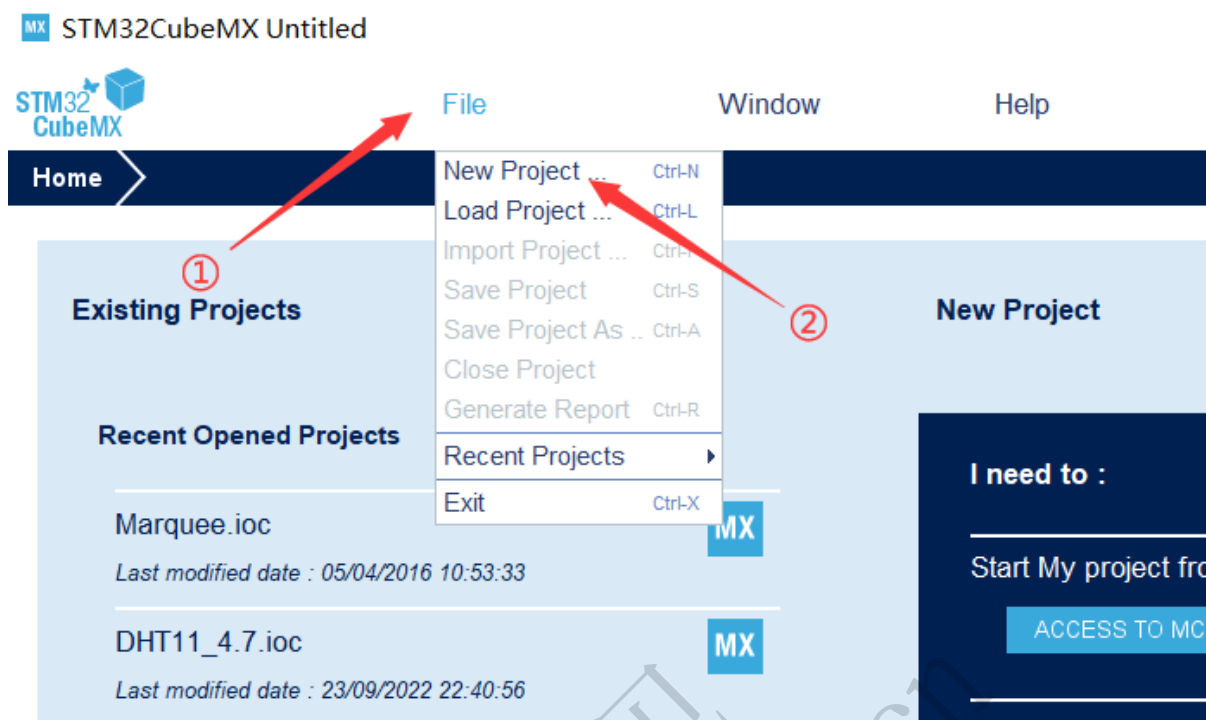
图 时钟树

它使得 SysTick 其实就是一个 24 位的倒计时定时器，当计到 0 时，将从 STK_LOAD 寄存器中自动重装载定时初值。只要不把它在 SysTick 控制及状态寄存器中的使能位清除，就永不停息。

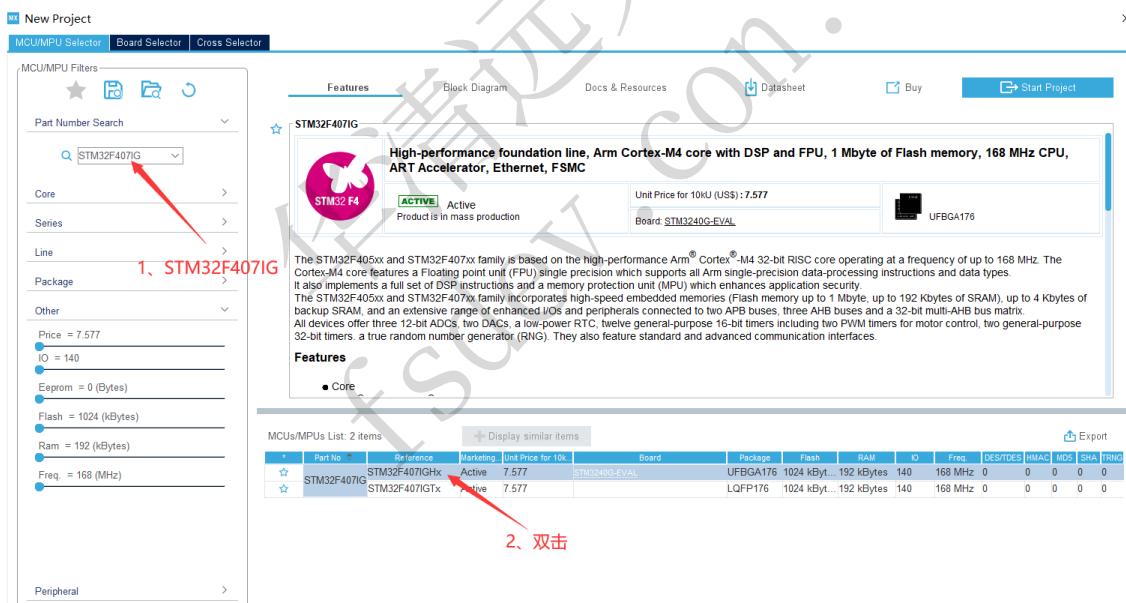
由于 SysTick 是 CortexM4 的标准组件，关于技术细节详细请参见《ARM Cortex-M4 权威指南》文档，“SysTick timer (STK)” 小节。

3.10.5 【stm32cubemx 配置介绍】

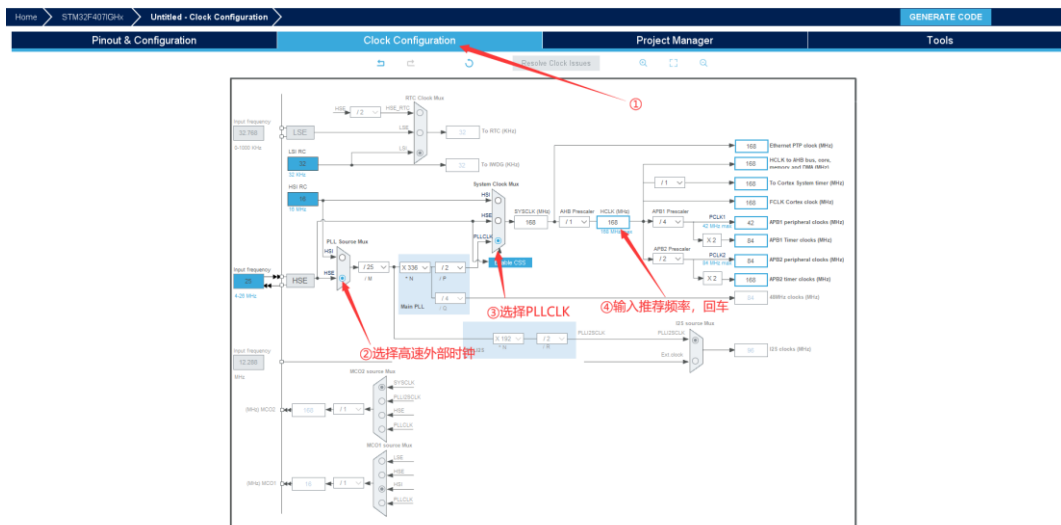
(1) 打开 STM32CubeMX->点击左上角的 File->点击 New Project



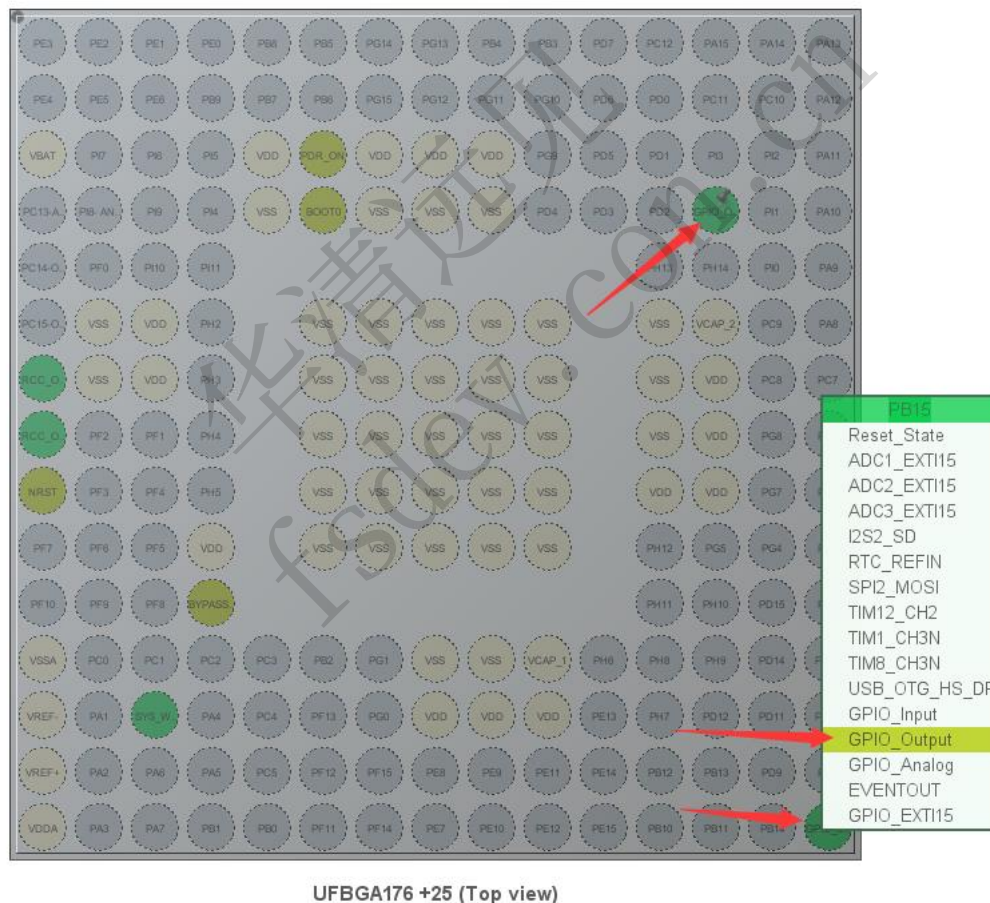
(2) 选择芯片，查看芯片的型号，在搜索框输入芯片型号，本实验中选用的芯片是 STM32F407IGHX



(3) 配置 RCC，点击左侧的 RCC，找到 High Speed Clock (HSE)，点击选择 Crystal/Ceramic Resonator，RCC 配置完成，点击上方的 Clock Configuration，配置时钟树



(4) 配置 PB15 和 PH15 为 GPIO_Output 模式，使用默认配置即可。



UFBGA176 +25 (Top view)

- (5) 配置工程名字，工程路径和 IDE 软件（MDK-ARM V5.27），（注意不要有中文路径）。
- (6) 工程软件生成配置，配置工程为 C 文件和 h 文件分开，以及拷贝使用到的 HAL 库文件，其他的文件不拷贝，最后点击右上角的“GENWRTE CODE”，生成工程。
- (7) 生成工程完成后会弹出提示框，选择 Open Project 可直接从 MDK 中打开工程代码。



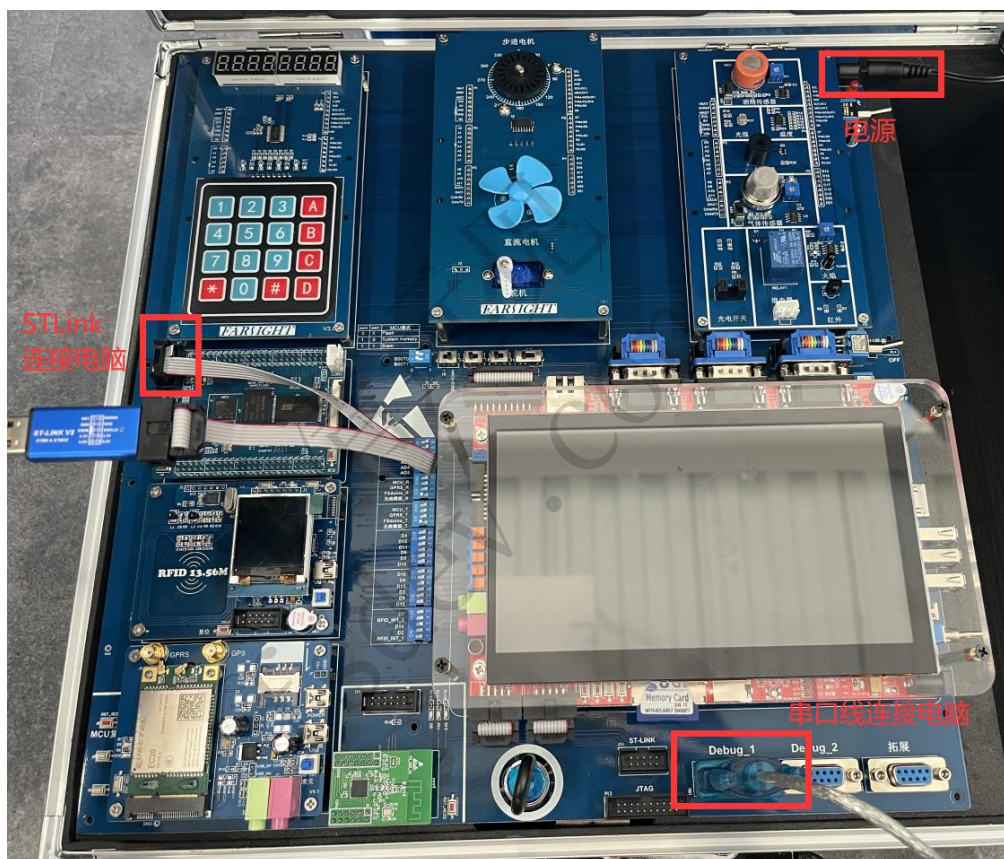
3.10.6 【实验步骤】

(1) 打开工程文件，路径为

【华清远见-嵌入式 ARM 实验箱资料-III\华清远见-STM 系列资料\Cortex-M4 部分\程序源码
\\8_Systick_Led】

(2) 对工程进行编译调试

(3) 开发板连接



(4) 往开发板下载程序，具体步骤参照 1.4.4 节

(5) 开发板上电查看实验结果

3.10.7 【实验结果】

将程序下载到 STM32F407 核心板后，按复位键，正常情况下，开发板上的 LED1 和 LED 灯 2 将按照每秒一次交替闪烁。

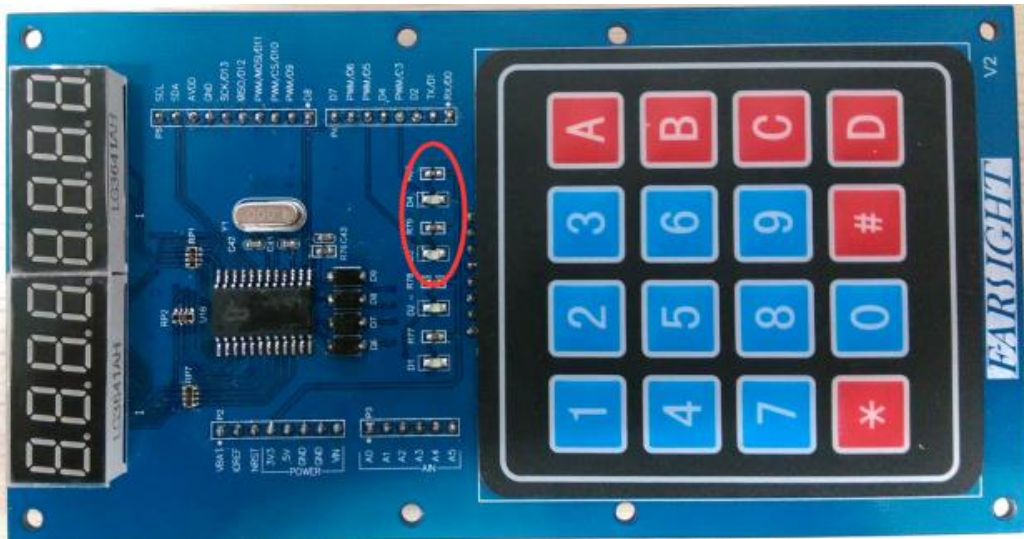


图 示意图

3.10.8 【实验参考代码】

这里只提供部分重要代码，如想查看部分函数里的详细实现，请打开工程源码，在µVision IDE for ARM 软件里进行查看和操作，这样查找方便清晰、有条理、并可以随意跟进。

注意：由于 STM32CubeMX 版本的不同，生成的工程有所差异，需要在工程中添加一行代码

`HAL_SYSTICK_IRQHandler();`



```

main.c  gpio.c  stm32f4xx_hal_msp.c  stm32f4xx_it.c*  stm32f4xx_hal_cortex.c
176  /* USER CODE END PendSV_IRQn 1 */
177  }
178
179  /**
180   * @brief This function handles System tick timer.
181   */
182  void SysTick_Handler(void)
183  {
184      /* USER CODE BEGIN SysTick_IRQn 0 */
185
186      /* USER CODE END SysTick_IRQn 0 */
187      HAL_IncTick();
188      HAL_SYSTICK_IRQHandler();
189      /* USER CODE BEGIN SysTick_IRQn 1 */
190
191      /* USER CODE END SysTick_IRQn 1 */
192  }
193
194  /**
195   * *****
196   * STM32F4xx Peripheral Interrupt Handlers
197   * Add here the Interrupt Handlers for the used peripherals.
198   * For the available peripheral interrupt handler names,
199   * please refer to the startup file (startup_stm32f4xx.s).

```

```

1      #include "stm32f4xx_hal.h"
2
3      /* USER CODE BEGIN Includes */
4      uint16_t i = 0;
5      /* USER CODE END Includes */
6
7      /* Private variables -----*/
8
9      /* USER CODE BEGIN PV */
10     /* Private variables -----*/
11
12     /* USER CODE END PV */
13
14     /* Private function prototypes -----*/
15     void SystemClock_Config(void);
16     static void MX_GPIO_Init(void);
17
18     /* USER CODE BEGIN PFP */
19     /* Private function prototypes -----*/
20
21     /* USER CODE END PFP */
22

```



```

23  /* USER CODE BEGIN 0 */
24
25  /* USER CODE END 0 */
26
27  int main(void)
28  {
29
30      /* USER CODE BEGIN 1 */
31
32      /* USER CODE END 1 */
33
34      /* MCU Configuration-----
35  */
36
37      /* Reset of all peripherals, Initializes the Flash interface and the Systick.
38  */
39      HAL_Init();
40
41      /* Configure the system clock */
42      SystemClock_Config();
43
44      /* Initialize all configured peripherals */
45      MX_GPIO_Init();
46
47      /* USER CODE BEGIN 2 */
48
49      /* USER CODE END 2 */
50
51      /* Infinite loop */
52      /* USER CODE BEGIN WHILE */
53      while (1)
54      {
55          /* USER CODE END WHILE */
56
57          /* USER CODE BEGIN 3 */
58          HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_15); //实现 LED11s 闪烁一次
59          HAL_Delay(1000);
60      }
61      /* USER CODE END 3 */
62
63  }
64
65  /** System Clock Configuration

```



```

66  */
67  void SystemClock_Config(void)
68  {
69
70      RCC_OscInitTypeDef RCC_OscInitStruct;
71      RCC_ClkInitTypeDef RCC_ClkInitStruct;
72
73      __PWR_CLK_ENABLE();
74
75      __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE1);
76
77      RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
78      RCC_OscInitStruct.HSEState = RCC_HSE_ON;
79      RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
80      RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
81      RCC_OscInitStruct.PLL.PLLM = 25;
82      RCC_OscInitStruct.PLL.PLLN = 336;
83      RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV2;
84      RCC_OscInitStruct.PLL.PLLQ = 4;
85      HAL_RCC_OscConfig(&RCC_OscInitStruct);
86
87      RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_SYSCLK | RCC_CLOCKTYPE_PCLK1
88      | RCC_CLOCKTYPE_PCLK2;
89      RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
90      RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
91      RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV4;
92      RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV2;
93      HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_5);
94
95      HAL_SYSTICK_Config(HAL_RCC_GetHCLKFreq() / 8000);
96
97      HAL_SYSTICK_CLKSourceConfig(SYSTICK_CLKSOURCE_HCLK_DIV8);
98
99  }
100
101  /** Configure pins as
102      * Analog
103      * Input
104      * Output
105      * EVENT_OUT
106      * EXTI
107  */
108  void MX_GPIO_Init(void)

```




```

109 {
110
111     GPIO_InitTypeDef GPIO_InitStructure;
112
113     /* GPIO Ports Clock Enable */
114     __GPIOH_CLK_ENABLE();
115     __GPIOF_CLK_ENABLE();
116     __GPIOB_CLK_ENABLE();
117
118     /*Configure GPIO pin : PF14 */
119     GPIO_InitStructure.Pin = GPIO_PIN_14;
120     GPIO_InitStructure.Mode = GPIO_MODE_OUTPUT_PP;
121     GPIO_InitStructure.Pull = GPIO_NOPULL;
122     GPIO_InitStructure.Speed = GPIO_SPEED_LOW;
123     HAL_GPIO_Init(GPIOF, &GPIO_InitStructure);
124
125     /*Configure GPIO pin : PB15 */
126     GPIO_InitStructure.Pin = GPIO_PIN_15;
127     GPIO_InitStructure.Mode = GPIO_MODE_OUTPUT_PP;
128     GPIO_InitStructure.Pull = GPIO_NOPULL;
129     GPIO_InitStructure.Speed = GPIO_SPEED_LOW;
130     HAL_GPIO_Init(GPIOB, &GPIO_InitStructure);
131
132 }
133
134 /* USER CODE BEGIN 4 */
135 void HAL_SYSTICK_Callback(void)
136 {
137     i++;
138     if(i == 1000)
139     {
140         i = 0;
141         HAL_GPIO_TogglePin(GPIOF, GPIO_PIN_14); //实现 LED2,1ms 闪烁一次
142     }
143 }
144 /* USER CODE END 4 */

```



3.11 按键扫描数码管显示实验

3.11.1 【实验目的】

ZLG72128 采用 I2C 总线接口，与微控制器的连接仅需两根信号线，硬件电路比较简单。而且可以驱动 8 位共阴数码管或 64 只独立 LED、64 只独立按键，并可提供自动消除抖动、连击键计数等功能。强大的功能，丰富的资源，良好的接口，使得 zlg72128 比传统的键盘与数码管解决方案且有更大的优越性。上一节实验数字光强采用的是模拟的 I2C。本节将采用 M4 的硬件接口进行 I2C 实验。

3.11.2 【实验环境】

嵌入式 ARM 实验箱

ST-Link 仿真器

RealView MDK5.21 集成开发软件

PC 机 XP、Window7/8 (32/64bit)

串口调试助手

3.11.3 【实验内容】

编写 zlg72128 程序，实现每一次按键，都将在数码管上显示，并且在下一次按键按下时，上一次的按键显示的值全部左移一位。调试编译好固件程序后，将程序下载到 STM32F407 核心板，按下复位键后，按下 zlg72128 连接的按键并且观察实验现象。

3.11.4 【实验原理】

在智能仪表中，经常会用到键盘、数码管等外设。因此，一个稳定、占用系统资源少的人机对话通道设计非常重要。传统的键盘与数码管解决方案，由于键盘与数码管是分离的，因而电路连接比较复杂，不管是独立式键盘还是矩阵式键盘，都会浪费微控制器的端口资源，而且都需要人为进行去抖动处



理，且抗干扰性差。而数码管部分，不管是静态显示方式还是动态显示方式，在不进行锁存器扩展的前提下。仍然要占用 8 根 I/O 端口线，这将严重浪费系统的端口资源。

zlg72128B 是广州周立功单片机发展有限公司自行设计的数码管显示驱动及键盘扫描管理芯片。能够直接驱动 8 位共阴式数码管（或 64 只独立的 LED），同时还可以扫描管理多达 64 只按键。其中有 8 只按键还可以作为功能键使用，就像电脑键盘上的 Ctrl、Shift、Alt 键一样。另外 zlg72128B 内部还设置有连击计数器，能够使某键按下后不松手而连续有效。采用 I2C 总线方式，与微控制器的接口仅需两根信号线。该芯片为工业级芯片，抗干扰能力强，在工业测控中已有大量应用。

zlg72128B 可以扫描管理多达 64 个按键，K1 ~ K56 为普通按键，F0 ~ F7 为功能键。普通按键还有连击检测功能。zlg72128B 内部有 8 个显示缓冲寄存器 DpRam0 ~ DpRam7，（本实验采用的是直接写显示缓冲寄存器，它们地址分别是 10H~17H）它们直接决定数码管显示的内容。zlg72128B 提供有两种显示控制方式，一种是直接向显存写入字型数据，另一种是通过向命令缓冲寄存器写入控制指令实现自动译码显示。访问这些寄存器需要通过 I2C 总线接口来实现。zlg72128B 的 I2C 总线器件地址是 70H（写操作）和 71H（读操作）。访问内部寄存器要通过“子地址”来实现。

在这里介绍本实验用到的几个寄存器作用：

系统寄存器 SystemReg（地址：00H）

系统寄存器的第 0 位（LSB）称作 KeyAvi，标志着按键是否有效，0 - 没有按键被按下，1 - 有某个按键被按下。SystemReg 寄存器的其它位暂时没有定义。当按下某个键时，zlg72128B 的 INT 引脚会产生一个低电平的中断请求信号。当读走键值后，中断信号就会自动撤销。而 KeyAvi 也同时予以反映。正常情况下，微控制器只需要判断 INT 引脚就可以了。通过不断查询 KeyAvi 位也能判断是否有键按下，这样就可以节省微控制器的一根 I/O 口线，但是代价是 I2C 总线处于频繁的活动状态，多消耗电流并且不利于抗干扰。（本节采用外部中断判断 INT 引脚，不读该寄存器）

键值寄存器 Key（地址：01H）



如果某个普通键（图 3.1 中的 K1 ~ K56）被按下，则微控制器可以从键值寄存器 Key 中读取相应的键值 1 ~ 56。如果微控制器发现 zlg72128B 的 INT 引脚产生了中断请求，而从 Key 中读到的键值是 0，则表示按下的可能是功能键。键值寄存器 Key 的值在被读走后自动变成 0。

I2C 总线的定义及特点：

I2C(Inter - Integrated Circuit)总线是一种由 PHILIPS 公司开发的两线式串行总线，用于连接微控制器及其外围设备。I2C 总线产生于在 80 年代，最初为音频和视频设备开发，如今主要在服务器管理中使用，其中包括单个组件状态的通信。例如管理员可对各个组件进行查询，以管理系统的配置或掌握组件的功能状态，如电源和系统风扇。可随时监控内存、硬盘、网络、系统温度等多个参数，增加了系统的安全性，方便了管理。

I2C 总线最主要的优点是其简单性和有效性。由于接口直接在组件之上，因此 I2C 总线占用的空间非常小，减少了电路板的空间和芯片管脚的数量，降低了互联成本。总线的长度可高达 25 英尺，并且能够以 10Kbps 的最大传输速率支持 40 个组件。I2C 总线的另一个优点是，它支持多主控 (multimastering)，其中任何能够进行发送和接收的设备都可以成为主总线。一个主控能够控制信号的传输和时钟频率。当然，在任何时间点上只能有一个主控。它是由数据线 SDA 和时钟 SCL 构成的串行总线，可发送和接收数据。在 CPU 与被控 IC 之间、IC 与 IC 之间进行双向传送，高速 I2C 总线一般可达 400kbps 以上。

I2C 总线的工作原理：

I2C 总线是由一根数据线 (SDA) 和一根时钟线 (SCL) 构成的串行总线，可发送和接收数据。在 CPU 与被控 IC 之间、IC 与 IC 之间进行双向传送，最高传送速率 400kbps。各种被控制模块均并联在这条总线上，每个模块都有唯一的地址标识。在信息的传输过程中，I2C 总线上并接的每一模块电路既可以是主控制器，又可以是发送器，这取决于它所完成的功能。CPU 发出的控制信号分为地址码和控制量两部分，地址码用来选址，即接通需要控制的电路，确定控制的种类；控制量决定该调整类别（如对比



度、亮度等) 及需要调整的量。这样, 各控制电路虽然挂在同一条总线上, 却彼此独立, 互不相关。

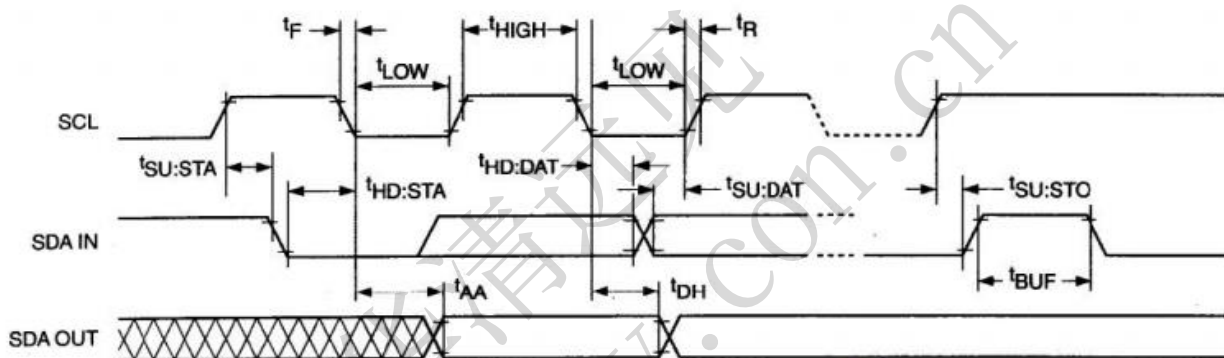
I2C 总线在传送数据过程中共有三种类型信号, 它们分别是: 开始信号、结束信号和应答信号。

开始信号: SCL 为高电平时, SDA 由高电平向低电平跳变, 开始传送数据。

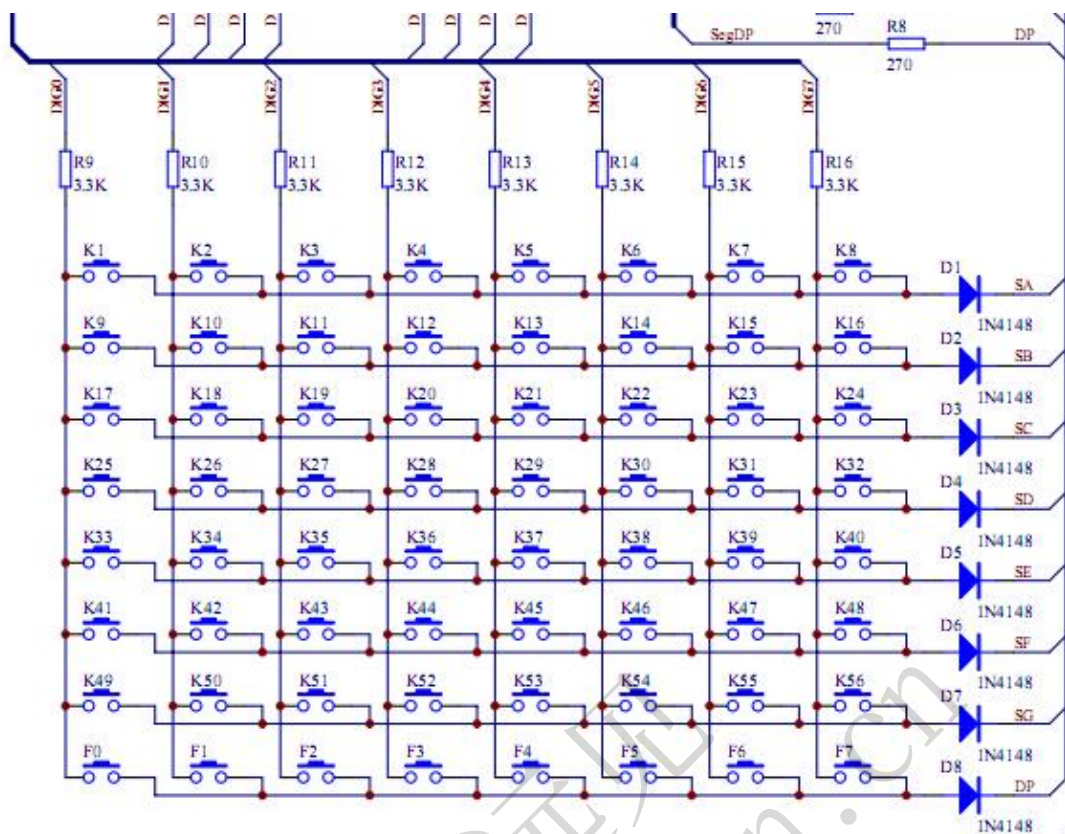
结束信号: SCL 为高电平时, SDA 由低电平向高电平跳变, 结束传送数据。

应答信号: 接收数据的 IC 在接收到 8bit 数据后, 向发送数据的 IC 发出特定的低电平脉冲, 表示已收到数据。CPU 向受控单元发出一个信号后, 等待受控单元发出一个应答信号, CPU 接收到应答信号后, 根据实际情况作出是否继续传递信号的判断。若未收到应答信号, 由判断为受控单元出现故障。

I2C 总线时序图如下:



矩阵键盘扫描原理:



根据电路原理图，键盘扫描方法是：行线为输入线，列线为输出线。一开始单片机将列线全部输出高电平，此时读入行线数据，若行线全为低电平则没有键按下，当行线有出现高电平时调用延时程序以此来去除按键抖动。延时完成后判断是否有高电平，如果此时读入行线数据还是有高电平，则说明确实有键按下。最后一步确定键值。现在我们以第二行的 K10 键为例，若按下 K2 后我们应该怎么得到这个键值呢？当判断确实有键按下之后，列线轮流输出高电平，根据读入行线的数据可以确定键值。首先，芯片将第一列输出为高电平，其它列输出低电平，此时读取行线的数据全为低电平，说明没有在第一行有键按下；其次，单片机将第二列输出高电平，其它仍为低电平，此时再来读取行线数据，发现行线读到的数据有高电平。根据如上判断，就可以得出是 K10 键按下了。当然这些处理扫描是 zlg72128 芯片完成的，我们只要读取键值就行了，在这里我们只是理解下键盘扫描的原理。详细原理图请参考 zlg72128 手册。

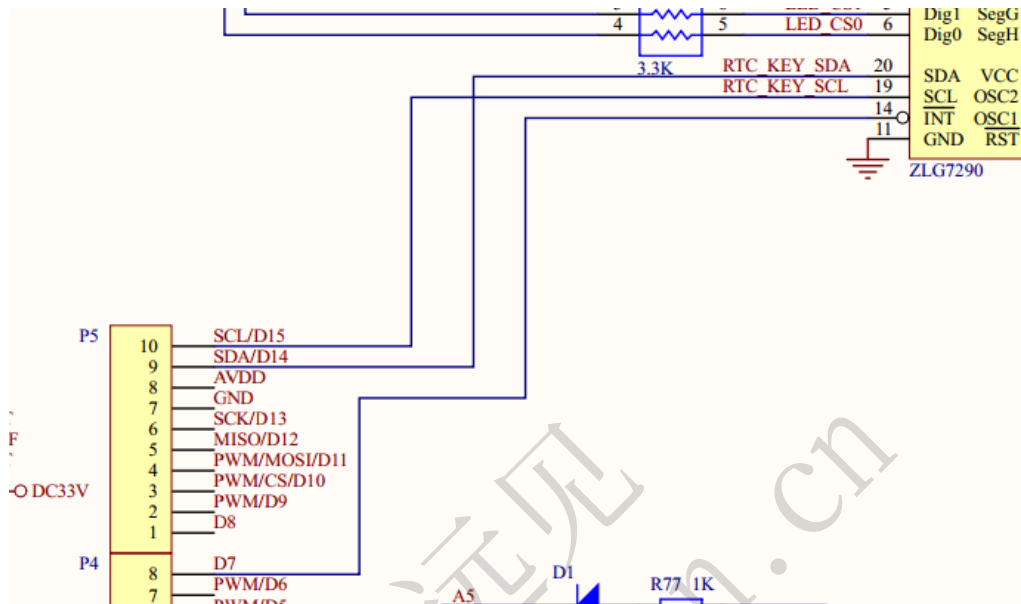
zlg72128 硬件接线如下：



PB6: I2C 的时钟信号线。

PB7: I2C 的数据线。

PD13: 按键的中断引脚。

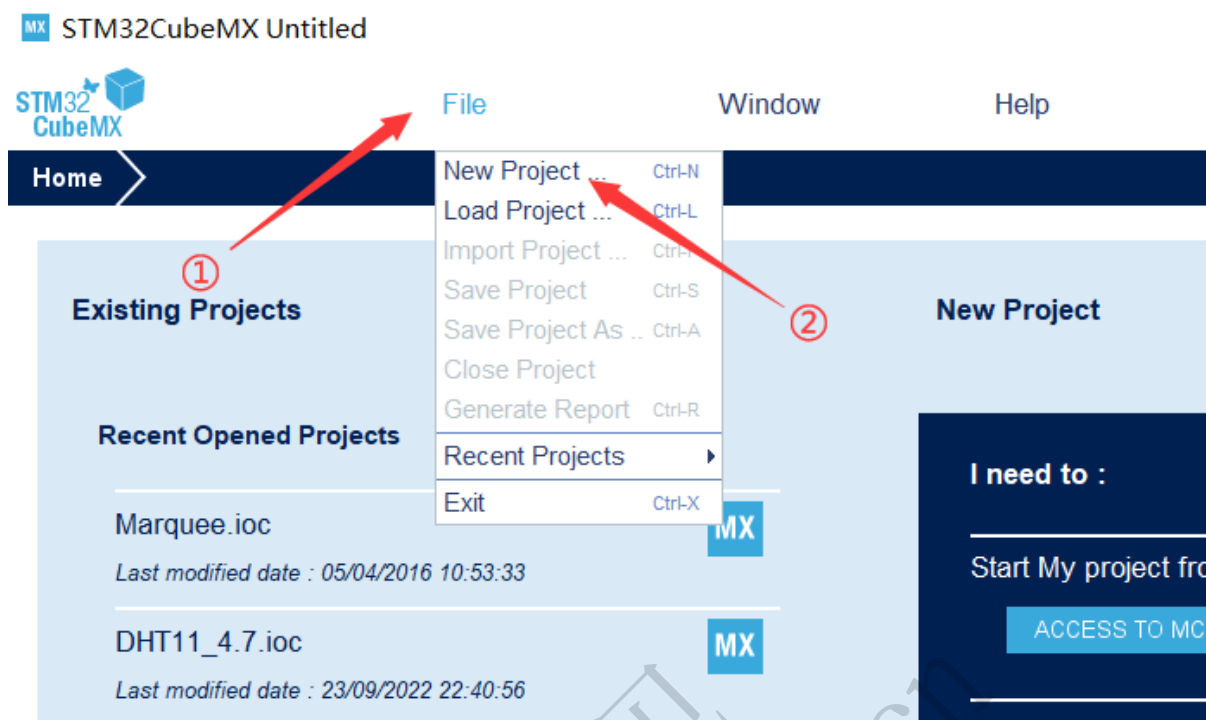


参数设置

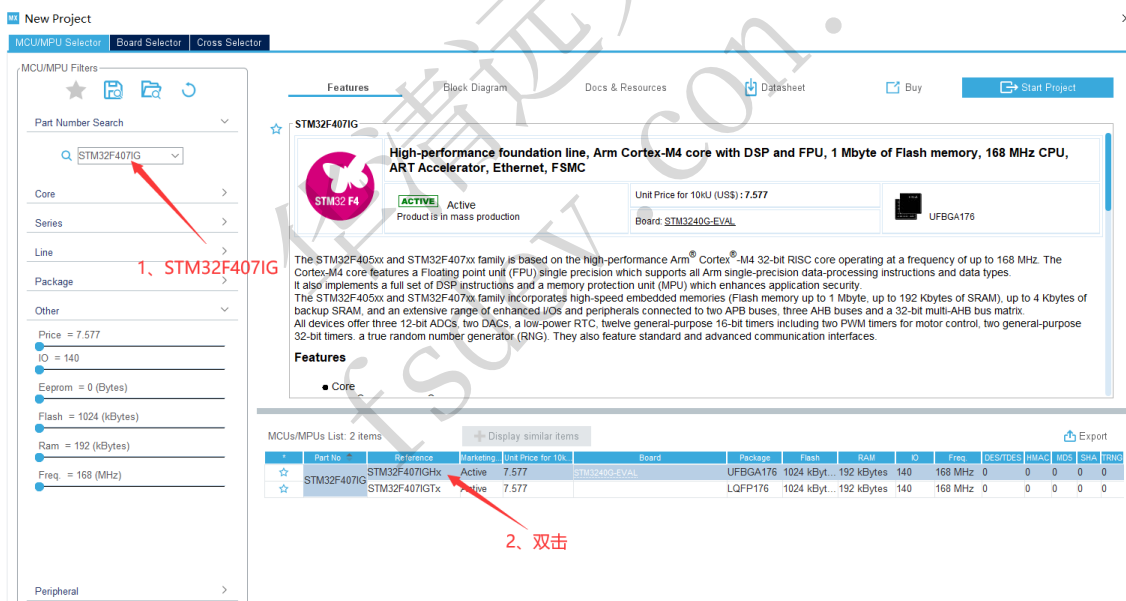
```
/* I2C1 init function */
void MX_I2C1_Init(void)
{
    hi2c1.Instance = I2C1;
    hi2c1.Init.ClockSpeed = 100000; // 标准模式速度
    hi2c1.Init.DutyCycle = I2C_DUTYCYCLE_2; // 快速模式占空比
    hi2c1.Init.OwnAddress1 = 0;
    hi2c1.Init.AddressingMode = I2C_ADDRESSINGMODE_7BIT; // 作为从设备7位寻址
    hi2c1.Init.DualAddressMode = I2C_DUALADDRESS_DISABLED; // 双寻址模式不使能
    hi2c1.Init.OwnAddress2 = 0; // 作为从设备地址
    hi2c1.Init.GeneralCallMode = I2C_GENERALCALL_ENABLED;
    hi2c1.Init.NoStretchMode = I2C_NOSTRETCH_DISABLED; // 作为从设备拉伸
    HAL_I2C_Init(&hi2c1);
}
```

3.11.5 【stm32cubemx 配置介绍】

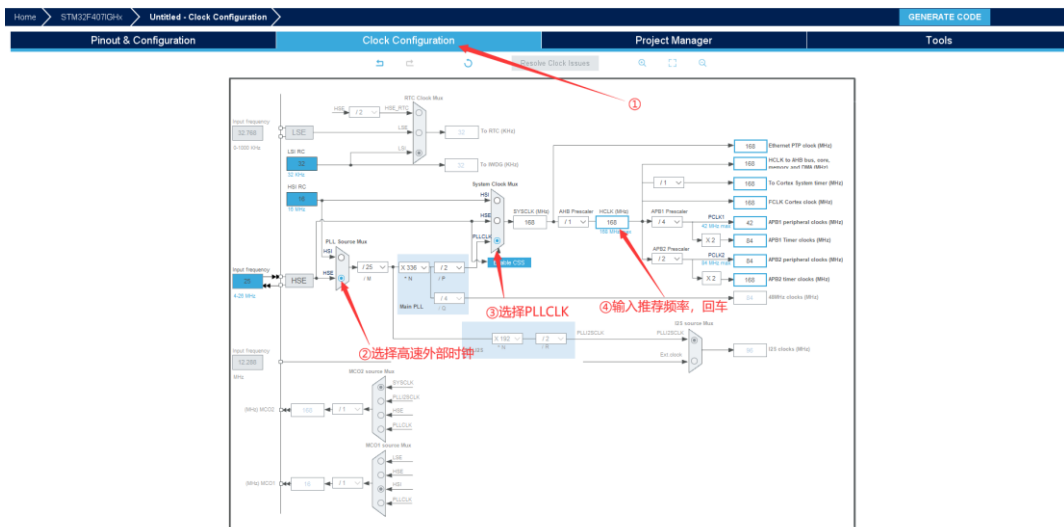
(1) 打开 STM32CubeMX->点击左上角的 File->点击 New Project



(2) 选择芯片，查看芯片的型号，在搜索框输入芯片型号，本实验中选用的芯片是 STM32F407IGHX



(3) 配置 RCC，点击左侧的 RCC，找到 High Speed Clock (HSE)，点击选择 Crystal/Ceramic Resonator，RCC 配置完成，点击上方的 Clock Configuration，配置时钟树



- (4) 配置 PD13 引脚为 GPIO_EXTI13 模式，选择选择外部中断下降沿触发。

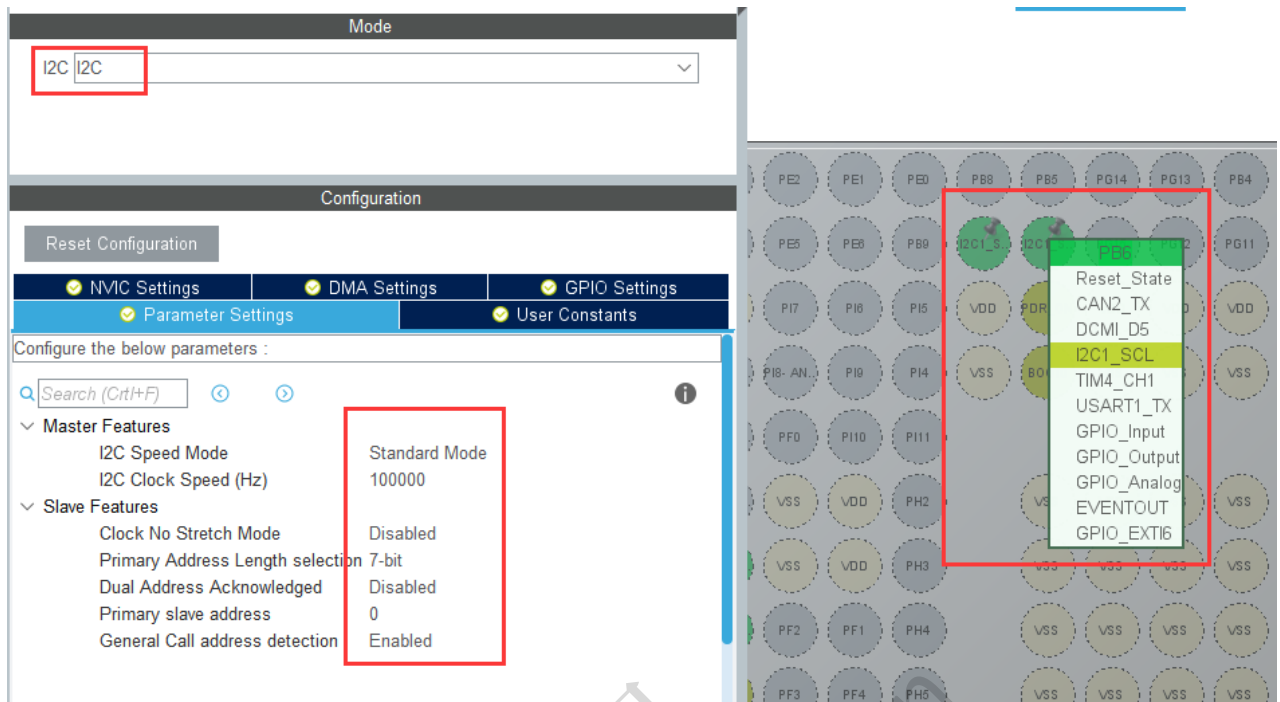
The image shows the configuration for PD13. On the left, the 'Configuration' window is open, showing the 'GPIO' tab. The 'Search Signals' field is empty. The 'Show only Modified Pins' checkbox is checked. The table below shows the configuration for PD13:

Pin N...	Signal o...	GPIO ou...	GPIO m...	GPIO Pu...	Maximu...	User Label	Modified
PD13	n/a	n/a	External ...	No pull-u...	n/a		☒

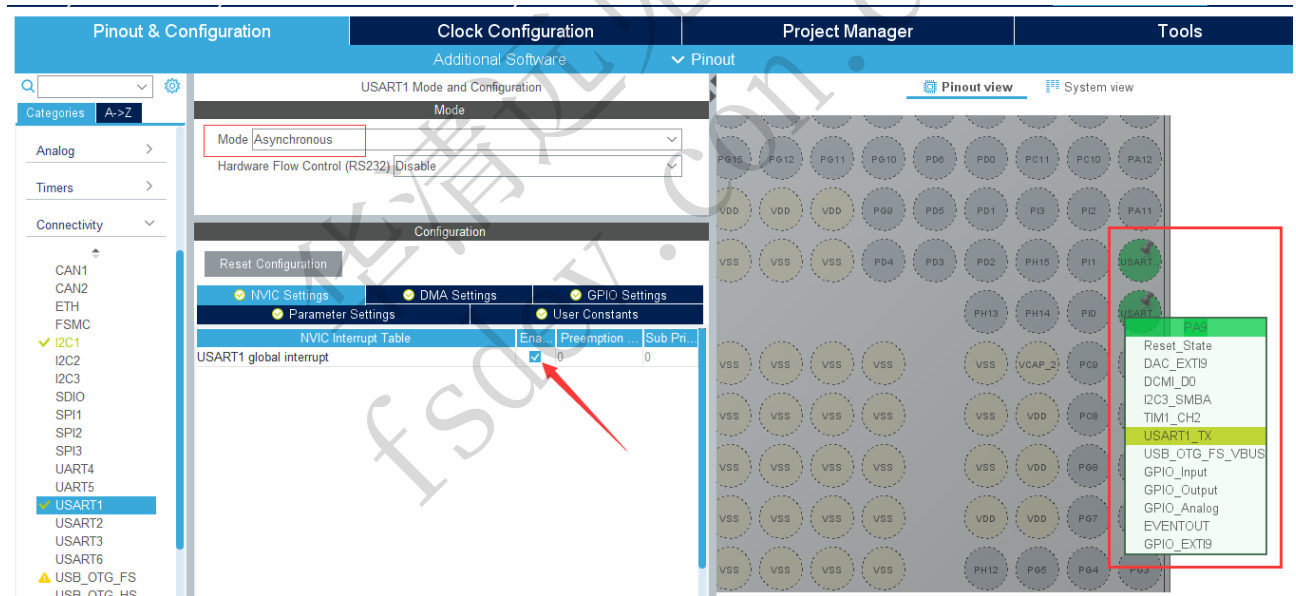
Below the table, the 'PD13 Configuration' is shown. The 'GPIO mode' is set to 'External Interrupt Mode with Falling edge trigger detection'. The 'GPIO Pull-up/Pull...' is set to 'No pull-up and no pull-down'. A red arrow points to the 'External Interrupt Mode with Falling edge trigger detection' option, with the text '下降沿触发' (Falling edge trigger) next to it.

On the right, a pin map of the STM32F407 is shown. A red box highlights the PD13 pin, which is labeled 'PD13' and 'GPIO_EXTI13'. A tooltip for PD13 lists the following signals: Reset_State, FSMC_A18, TIM4_CH2, GPIO_Input, GPIO_Output, GPIO_Analog, EVENTOUT, and GPIO_EXTI13.

- (5) 配置 PB6 为 I2C1_SCL，配置 PB7 为 I2C1_SDA，配置初始化 I2C。



(6) 配置 PA9 为 USART1_TX，配置 PA10 为 USART1_RX，初始化串口。



(7) 配置 NVIC



Configuration

☒ NVIC ☒ Code generation

Priority Group: 4 bits for pre-emption priority 0... ☐ Sort by Preemption Priority and Sub Priority

Search: ☐ Show only enabled interrupts ☒ Force DMA channels Interrupts

NVIC Interrupt Table	Enabled	Preemption Priority	Sub Priority
Non maskable interrupt	☒	0	0
Hard fault interrupt	☒	0	0
Memory management fault	☒	0	0
Pre-fetch fault, memory access fault	☒	0	0
Undefined instruction or illegal state	☒	0	0
System service call via SWI instruction	☒	0	0
Debug monitor	☒	0	0
Pendable request for system service	☒	0	0
Time base: System tick timer	☒	0	0
PVD interrupt through EXTI line 16	☐	0	0
Flash global interrupt	☐	0	0
RCC global interrupt	☐	0	0
I2C1 event interrupt	☐	0	0
I2C1 error interrupt	☐	0	0
USART1 global interrupt	☒	0	0
EXTI line[15:10] interrupts	☒	0	0
FPU global interrupt	☐	0	0

- (8) 配置工程名字，工程路径和 IDE 软件（MDK-ARM V5.27），（注意不要有中文路径）。
- (9) 工程软件生成配置，配置工程为 C 文件和 h 文件分开，以及拷贝使用到的 HAL 库文件，其他的文件不拷贝，最后点击右上角的“GENWRTE CODE”，生成工程。
- (10) 生成工程完成后会弹出提示框，选择 Open Project 可直接从 MDK 中打开工程代码。

3.11.6 【实验步骤】

- (1) 打开工程文件，路径为

【华清远见-嵌入式 ARM 实验箱资料-III\华清远见-STM 系列资料\Cortex-M4 部分\程序源码

\9_ZLG72128】

- (2) 对工程进行编译调试
- (3) 开发板连接



(4) 往开发板下载程序，具体步骤参照 1.4.4 节

(5) 开发板上电查看实验结果

3.11.7 【实验结果】

将固件下载到 FS-STM32F407 核心板后，关闭电源，用随板配置的串口线连接开发板串口 1 与电脑的串口，打开串口调试工具，并如下配置串口参数。打开电源，并案复位键，按下 zlg72128 控制的几个按键，串口调试工具窗口显示如下：





开发板上数码管显示如下图：

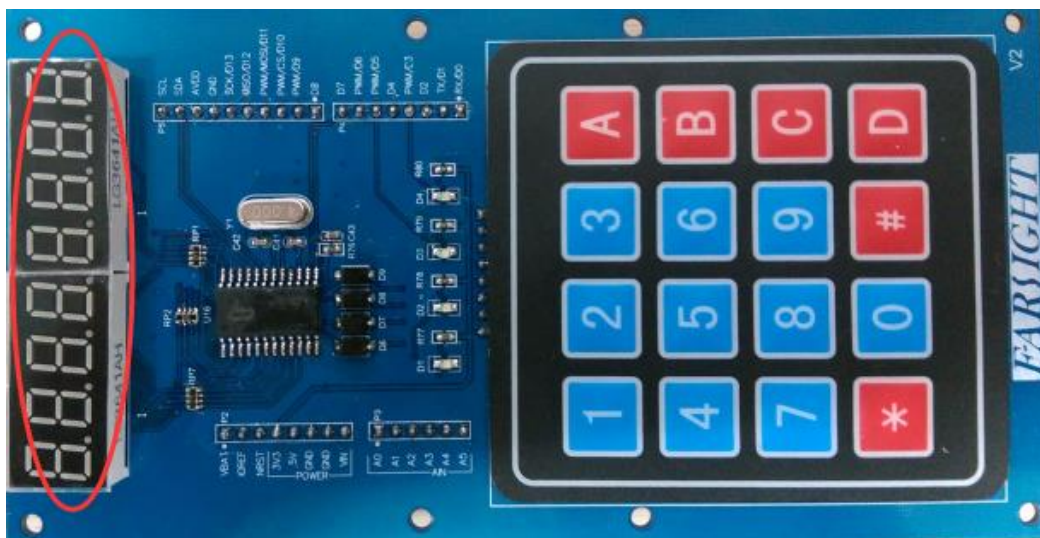


图 示意图

3.11.8 【实验参考代码】

这里只提供部分重要代码，如想查看部分函数里的详细实现，请打开程序源码，在µVision IDE for ARM 软件里进行查看和操作，这样查找方便清晰、有条理、并可以随意跟进。

```

1  int main(void)
2  {
3      /* USER CODE BEGIN 1 */
4
5      /* USER CODE END 1 */
6
7      /* MCU Configuration-----*/
8
9      /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
10     HAL_Init();
11
12     /* USER CODE BEGIN Init */
13
14     /* USER CODE END Init */
15
16     /* Configure the system clock */
17     SystemClock_Config();
18
19     /* USER CODE BEGIN SysInit */

```



```

20
21  /* USER CODE END SysInit */
22
23  /* Initialize all configured peripherals */
24  MX_GPIO_Init();
25  MX_I2C1_Init();
26  MX_USART1_UART_Init();
27  /* USER CODE BEGIN 2 */
28
29  /* USER CODE END 2 */
30
31  /* Infinite loop */
32  /* USER CODE BEGIN WHILE */
33      printf("=====>ZLG72128<=====\n");
34      printf("=====>测试程序<=====\n");
35  while (1)
36  {
37      /* USER CODE END WHILE */
38
39      /* USER CODE BEGIN 3 */
40
41
42      if(flag_key == 1)
43      {
44          flag_key = 0;
45          flag = 0xff;
46          //读键值
47          I2C_ZLG72128_Read(&hi2c1, 0x61, 0x01, Rx1_Buffer_P, 1);
48          //读键值
49          I2C_ZLG72128_Read(&hi2c1, 0x61, 0x03, Rx1_Buffer_T, 1);
50
51          if (Rx1_Buffer_T[0] != 0xff)           //功能按键
52          {
53              printf("\n\r function key value = %#x\r\n", Rx1_Buffer_T[0]);
54              swtich_key_func();
55          }
56          if (Rx1_Buffer_P[0] != 0x0)           //普通按键
57          {
58              printf("\n\r key value = %#x\r\n", Rx1_Buffer_P[0]);
59              swtich_key();
60          }
61
62          printf("\n\r flag = %d\r\n", flag);

```




```

63
64         I2C_ZLG72128_Read(&hi2c1,0x61,0x10,Rx2_Buffer,8);//读 8 位数码管
65         switch_flag();//扫描到相应的按键并且向数码管写进数值
66     }
67 }
68 /* USER CODE END 3 */
69 }
    
```

3.12 ADC 模数转换实验

3.12.1 【实验目的】

日常中，我们直接接触的都是一些模拟的设备，电位器、传感器、语音和视频等等，那么如何将这
些设备采集到的数据进行传输呢？模拟数据在传输过程中，数据量大，占用带宽大，受干扰严重，直接
限制、影响到我们的通信传输质量。因此，在现阶段的做法，在数据进行传输的第一步都会进行模拟/数
字的转换。将模拟信号进行采样、量化、编码等一系列操作后，再传到通信信道中进行通信。因此我们
要了解模拟信号的概念和数字信号的概念，以及模拟信号转换为数字信号的原理。

3.12.2 【实验环境】

嵌入式 ARM 实验箱

J-Link 仿真器

RealView MDK5.21 集成开发软件

PC 机 XP、Window7/8 (32/64bit)

串口调试助手

交叉串口线



3.12.3 【实验内容】

编写 ADC 模数转换程序，获得开发板上传感器的电压值，通过 ADC 的转换得到对应数字值的变化情况从串口显示出来的功能。调试编译好固件程序后，将程序下载到 STM32F407 核心板，按复位键观察实验现象。

3.12.4 【实验原理】

STM32F4 ADC 简介

模数转换器即是 A/D 转换器，或简称 ADC，通常是指一个将模拟信号转变为数字信号电子元件。

STM32F4xx 系列一般都有 3 个 ADC，这些 ADC 可以独立使用，也可以使用双重/三重模式（提高采样率）。STM32F4 的 ADC 是 12 位逐次逼近型的模拟数字转换器。它有 19 个通道，可测量 16 个外部源、2 个内部源和 Vbat 通道的信号。这些通道的 A/D 转换可以单次、连续、扫描或间断模式执行。ADC 的结果可以左对齐或右对齐方式存储在 16 位数据寄存器中。模拟看门狗特性允许应用程序检测输入电压是否超出用户定义的高/低阈值。

STM32F407ZGT6 包含有 3 个 ADC。STM32F4 的 ADC 最大的转换速率为 2.4Mhz，也就是转换时间为 1us（在 ADCCLK=36M,采样周期为 3 个 ADC 时钟下得到），不要让 ADC 的时钟超过 36M，否则将导致结果准确度下降。STM32F4 将 ADC 的转换分为 2 个通道组：规则通道组和注入通道组。规则通道相当于你正常运行的程序，而注入通道呢，就相当于中断。在你程序正常执行的时候，中断是可以打断你的执行的。同这个类似，注入通道的转换可以打断规则通道的转换，在注入通道被转换完成之后，规则通道才得以继续转换。通过一个形象的例子可以说明：假如你在家里的院子内放了 5 个温度探头，室内放了 3 个温度探头；你需要时刻监视室外温度即可，但偶尔你想看看室内的温度；因此你可以使用规则通道组循环扫描室外的 5 个探头并显示 AD 转换结果，当你想看室内温度时，通过一个按钮启动注入转换组(3 个室内探头)并暂时显示室内温度，当你放开这个按钮后，系统又会回到规则通道组继续



检测室外温度。

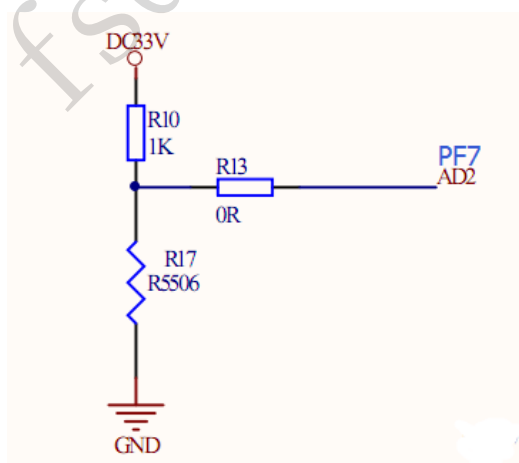
从系统设计上，测量并显示室内温度的过程中断了测量并显示室外温度的过程，但程序设计上可以在初始化阶段分别设置好不同的转换组，系统运行中不必再变更循环转换的配置，从而达到两个任务互不干扰和快速切换的结果。可以设想一下，如果没有规则组和注入组的划分，当你按下按钮后，需要重新配置 AD 循环扫描的通道，然后在释放按钮后需再次配置 AD 循环扫描的通道。

上面的例子因为速度较慢，不能完全体现这样区分(规则通道组和注入通道组)的好处，但在工业应用领域中有很多检测和监视探头需要较快地处理，这样对 AD 转换的分组将简化事件处理的程序并提高事件处理的速度。STM32F4 其 ADC 的规则通道组最多包含 16 个转换，而注入通道组最多包含 4 个通道。关于这两个通道组的详细介绍，请参考《STM32F4xx 中文参考手册》第 250 页，第 11.3.3 节。

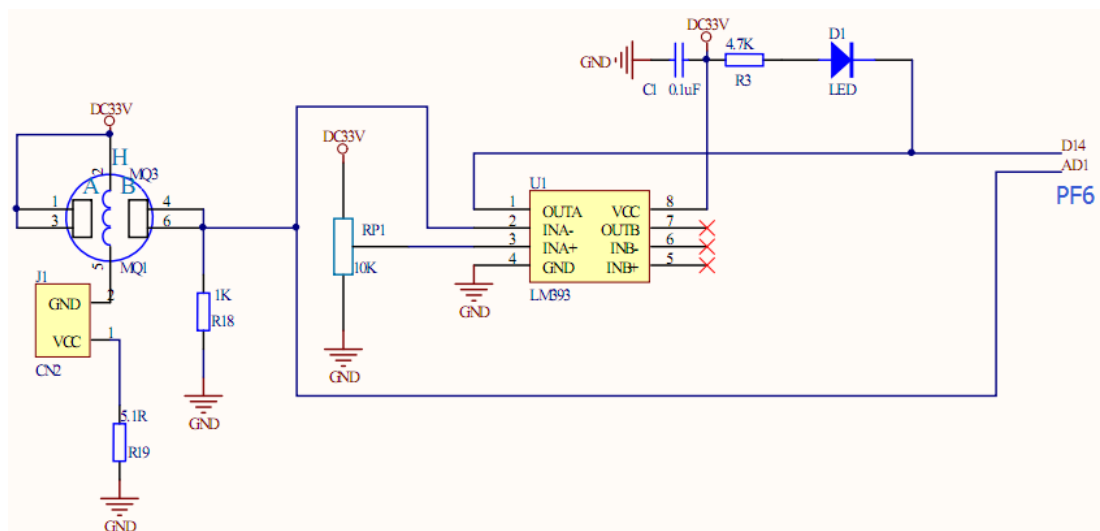
STM32F4 的 ADC 可以进行很多种不同的转换模式，这些模式在《STM32F4xx 中文参考手册》的第 11 章也都有详细介绍，我们这里就不一一列举了。我们本章仅介绍如何使用规则通道的连续转换模式（因为用到 4 个通道 AD 采集）。

需要注意的是，要保证 ADC 时钟频率不超过 36MHz，否则数据不准确。

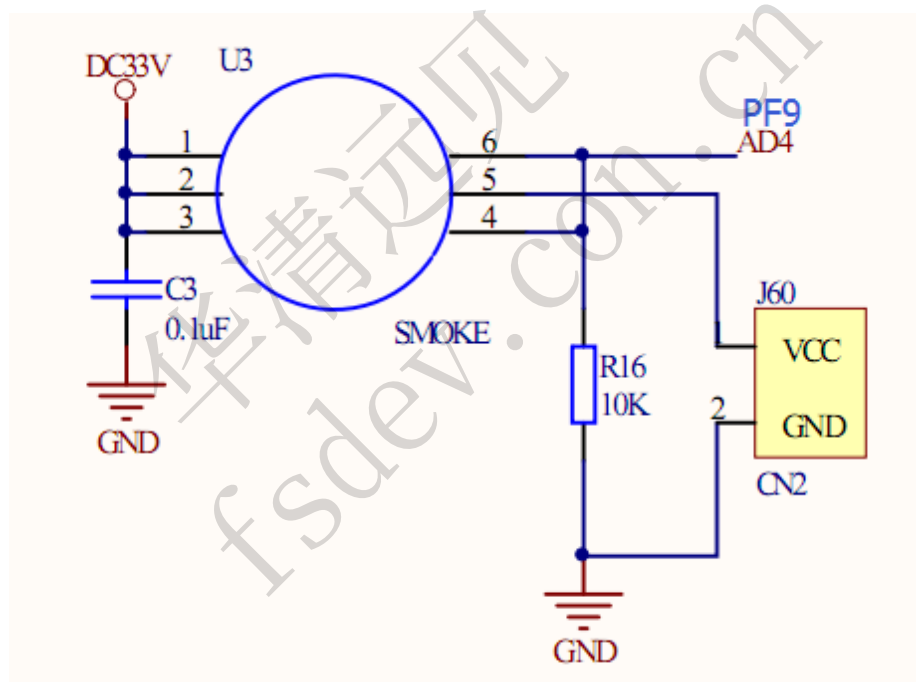
ADC 硬件接口：



光敏传感器接口



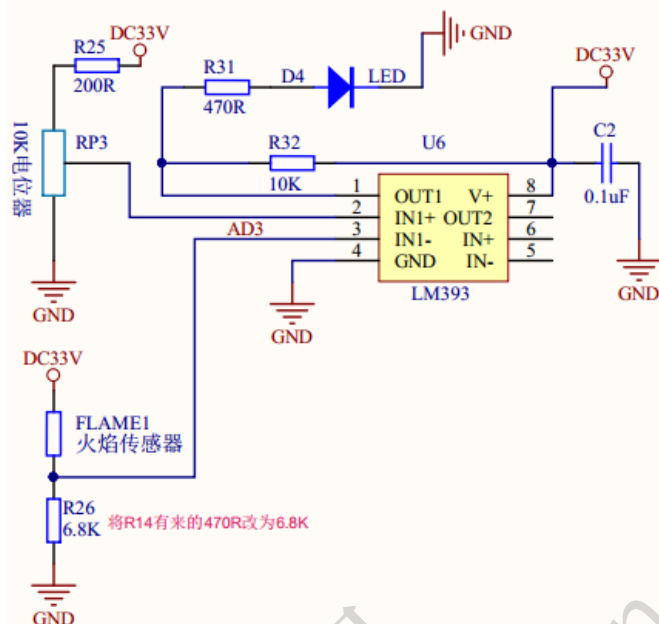
酒精传感器接口



气体传感器接口



火焰传感器

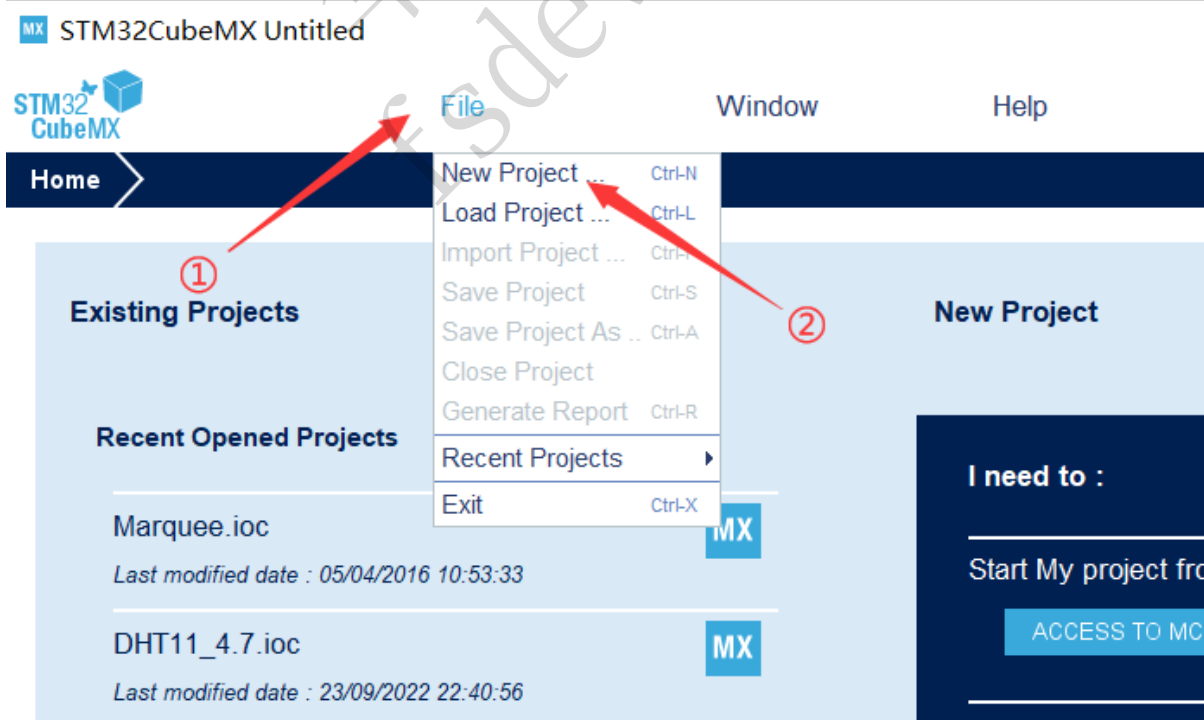


火焰传感器接口

查找数据手册可知这四个传感器的引脚分别对应 ADC3 的通道 4，通道 5，通道 6 和通道 7。

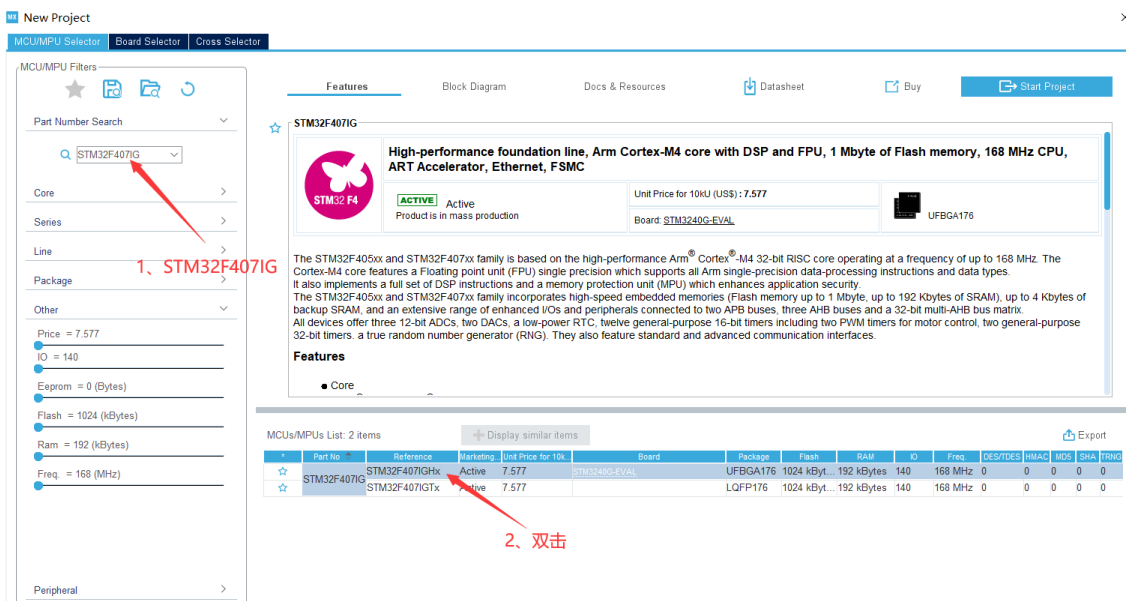
3.12.5 【stm32cubemx 配置介绍】

(1) 打开 STM32CubeMX->点击左上角的 File->点击 New Project

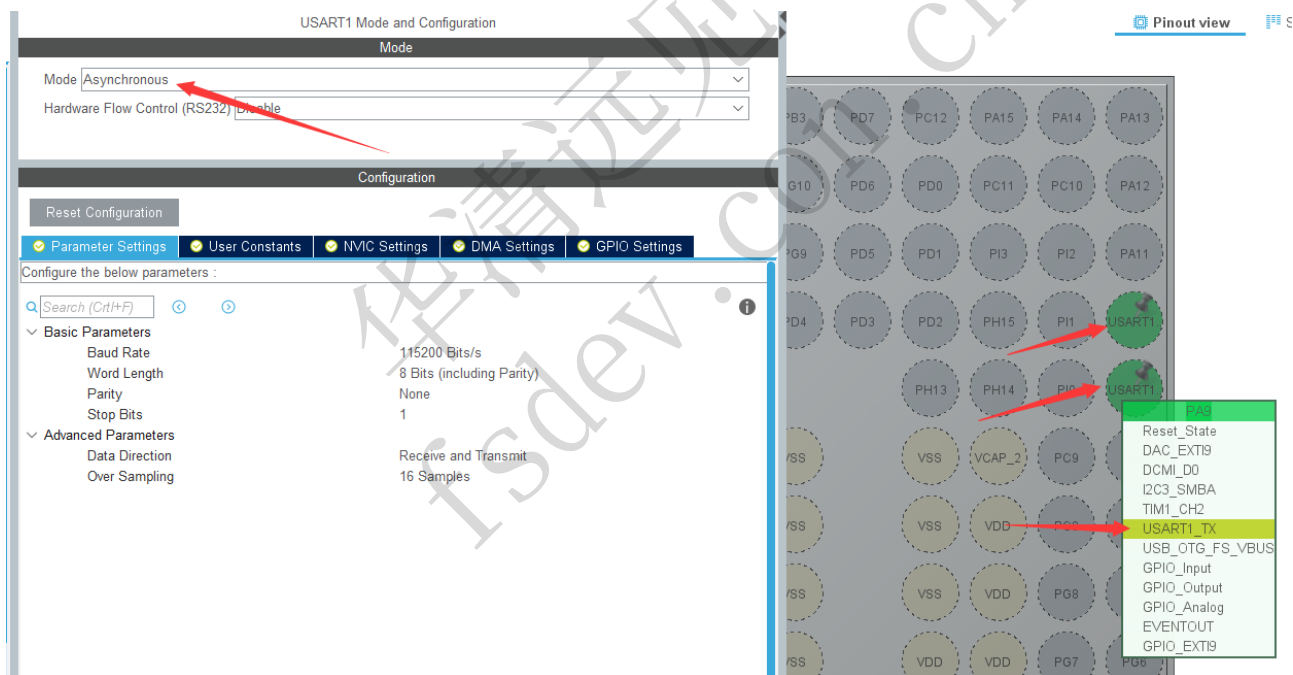




- (2) 选择芯片，查看芯片的型号，在搜索框输入芯片型号，本实验中选用的芯片是 STM32F407IGHx

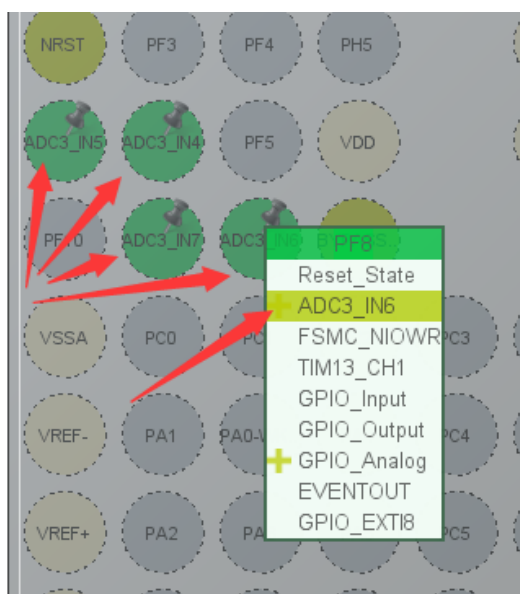


- (3) 配置 PA9 为 USART1_TX，配置 PA10 为 USART1_RX，初始化串口。

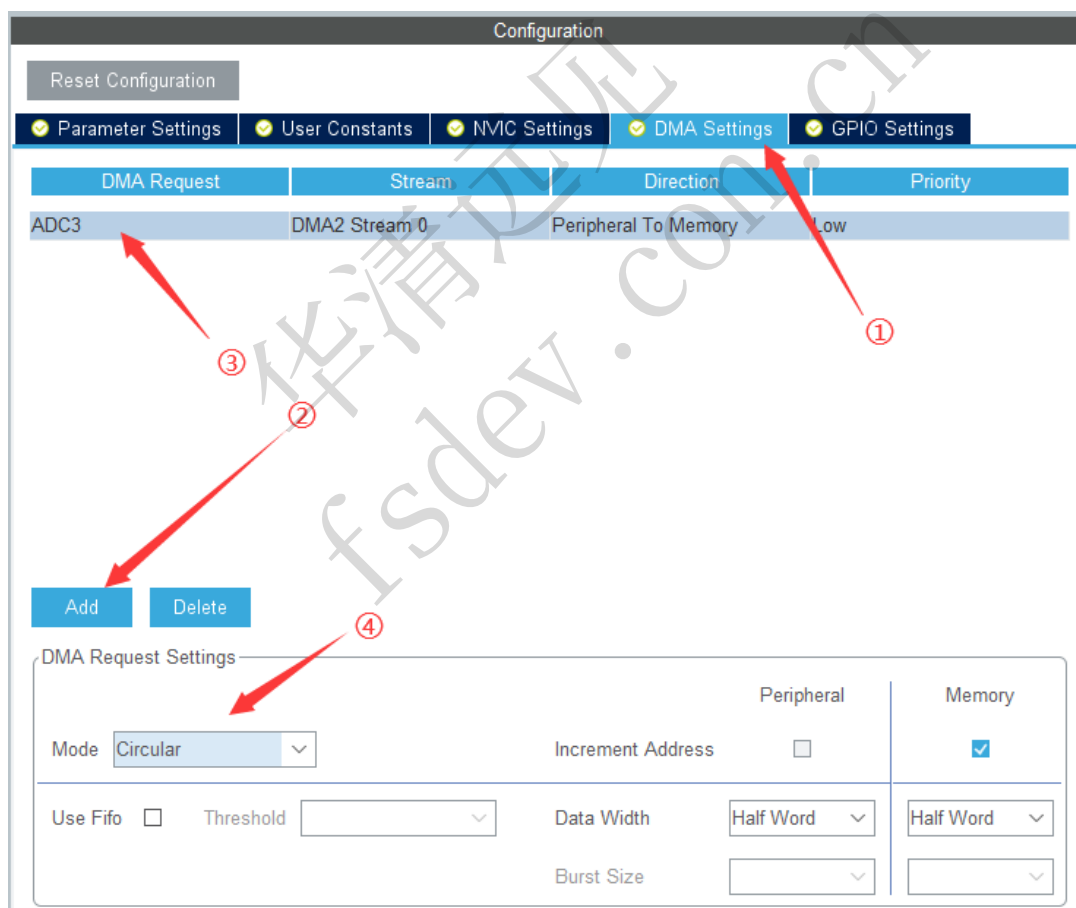


- (4) 本次实验使用高速内部时钟，RCC 和时钟树使用默认配置即可。

- (5) 配置 PF6 为 ADC3_IN4，PF7 为 ADC3_IN5，PF8 为 ADC_IN6，PF9 为 ADC3_IN7



(6) 配置 ADC3





ADC_Settings

- Clock Prescaler
- Resolution
- Data Alignment
- Scan Conversion Mode
- Continuous Conversion Mode
- Discontinuous Conversion Mode
- DMA Continuous Requests
- End Of Conversion Selection

PCLK2 divided by 4
12 bits (15 ADC Clock cycles)
Right alignment
Enabled
Enabled
Disabled
Enabled
EOC flag at the end of single channel conversion

ADC_Regular_ConversionMode

- Number Of Conversion
- External Trigger Conversion Source
- External Trigger Conversion Edge
- Rank
- Channel
- Sampling Time
- Rank
- Channel
- Sampling Time
- Rank
- Channel
- Sampling Time
- Rank
- Channel
- Sampling Time

4
Regular Conversion launched by software
None

1
Channel 4
112 Cycles
2
Channel 5
112 Cycles
3
Channel 6
112 Cycles
4
Channel 7
112 Cycles

- (7) 配置工程名字，工程路径和 IDE 软件（MDK-ARM V5.27），（注意不要有中文路径）。
- (8) 工程软件生成配置，配置工程为 C 文件和 h 文件分开，以及拷贝使用到的 HAL 库文件，其他的文件不拷贝，最后点击右上角的“GENWRTE CODE”，生成工程。
- (9) 生成工程完成后会弹出提示框，选择 Open Project 可直接从 MDK 中打开工程代码。

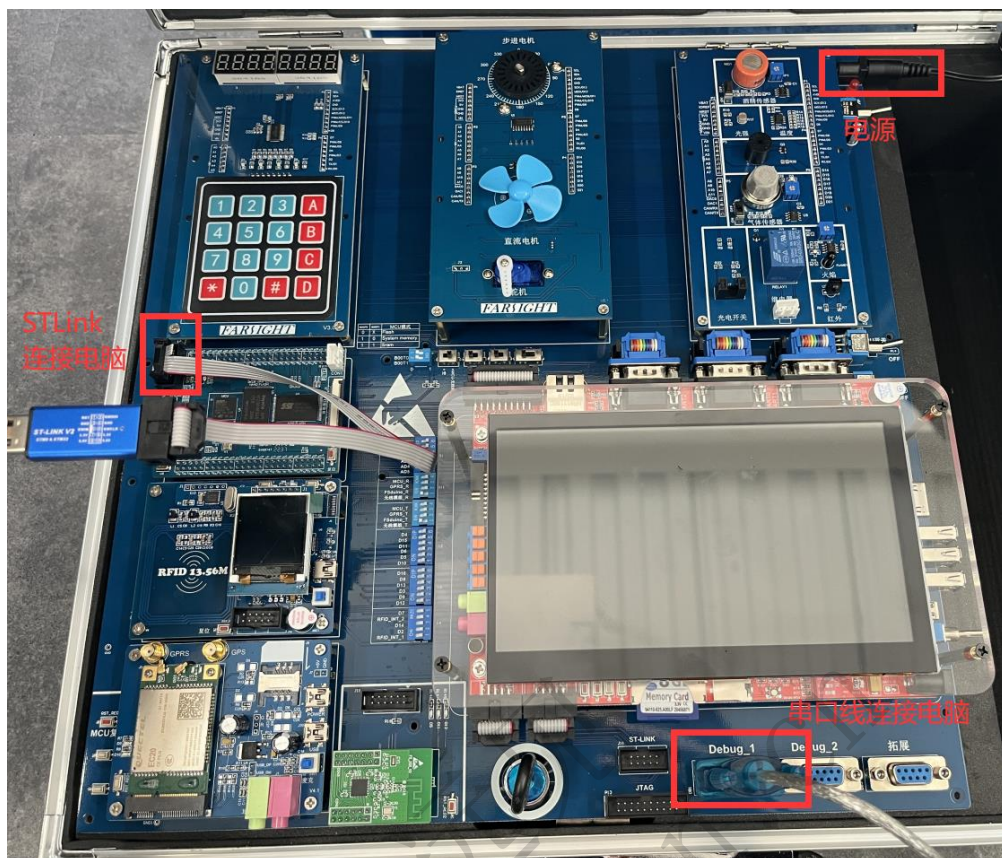
3.12.6 【实验步骤】

- (1) 打开工程文件，路径为

【华清远见-嵌入式 ARM 实验箱资料-III\华清远见-STM 系列资料\Cortex-M4 部分\程序源码\10_ADC】

- (2) 对工程进行编译调试

- (3) 开发板连接



- (4) 往开发板下载程序，具体步骤参照 1.4.4 节
- (5) 开发板上电查看实验结果

3.12.7 【实验结果】

将程序下载到 FS-STM32F407 后，打开串口调试工具，按复位键，正常情况下在串口调试工具上显示其 ADC 的值：



图 实验结果

4 个传感器如下图红圈所示位置：



图 传感器示意图

3.12.8 【实验参考代码】

这里只提供部分重要代码，如想查看部分函数里的详细实现，请打开光盘资料对应的工程【华清远

见-嵌入式 ARM 实验箱资料-III\华清远见-STM 系列资料\Cortex-M4 部分\程序源码\10_ADC】，在µVision



IDE for ARM 软件里进行查看和操作，这样查找方便清晰、有条理、并可以随意跟进。

```

1  int main(void)
2  {
3
4      /* USER CODE BEGIN 1 */
5
6      /* USER CODE END 1 */
7
8      /* MCU Configuration-----
9  */
10
11     /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
12     /
13     HAL_Init();
14
15     /* Configure the system clock */
16     SystemClock_Config();
17
18     /* Initialize all configured peripherals */
19     MX_GPIO_Init();
20     MX_DMA_Init();
21     MX_ADC3_Init();
22     MX_USART1_UART_Init();
23
24     /* USER CODE BEGIN 2 */
25     HAL_ADC_Start_DMA(&hadc3, (uint32_t *)adcx, 4); //开启 ADC 转换
26     /* USER CODE END 2 */
27
28     /* Infinite loop */
29     /* USER CODE BEGIN WHILE */
30     while (1)
31     {
32         /* USER CODE END WHILE */
33
34         /* USER CODE BEGIN 3 */
35         printf("adcx0=%d adc1=%d adc2=%d adc3=%d\n\r", adcx[0], adcx[1], adcx[2], a
36 dcx[3]); //转换得到的 AD 值
37         temp0 = (float)adcx[0] * (3.3 / 4096);
38         temp1 = (float)adcx[1] * (3.3 / 4096);
39         temp2 = (float)adcx[2] * (3.3 / 4096);
40         temp3 = (float)adcx[3] * (3.3 / 4096);
41         printf("酒精传感器=%f\n 光敏传感器=%f\n 火焰传感器=%f\n 气体传感器

```



```
42  =%f\n\n",temp0,temp1,temp2,temp3) //在串口中打印转换到的电压值
43      HAL_Delay(1000);
44      // HAL_ADC_Stop_DMA(&hadc3);
    }
    /* USER CODE END 3 */
}
```

3.13 IR 红外线接收实验

3.13.1 【实验目的】

了解红外的概念；了解红外收发特点与用途；了解红外发送和接收的工作原理。

3.13.2 【实验环境】

嵌入式 ARM 实验箱

ST-Link 仿真器

RealView MDK5.21 集成开发软件

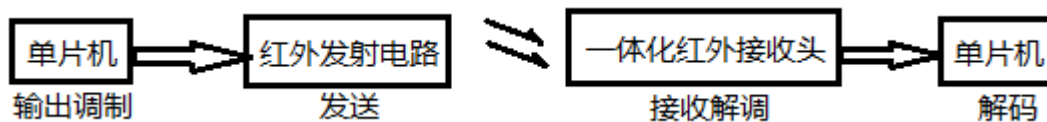
PC 机 XP、Window7/8 (32/64bit)

串口调试工具

交叉串口线

3.13.3 【实验内容】

红外遥控有发送和接收两个组成部分。发送端将待发送的二进制信号编码调制为一系列的脉冲串信号，通过红外发射管发射红外信号。红外接收完成对红外信号的接收、放大、检波、整形，并解调出遥控编码脉冲，流程图如下：





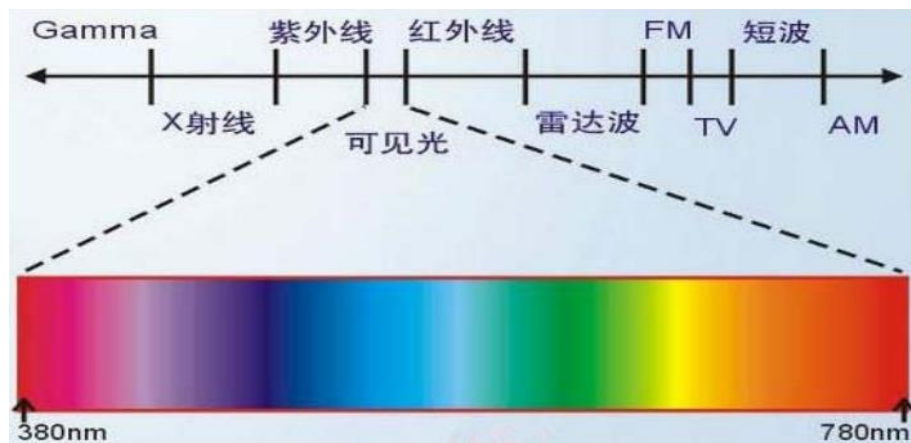
把需要发送的信号编码调制为一系列的脉冲串信号由红外 LED 发送出去。

3.13.4 【实验原理】

在了解红外收发前，我们先来了解一下什么是红外，在日常生活中，我们通过眼睛能看到各种颜色的光。那我们是否了解各种颜色的光之间有什么区别的呢？难道就只是颜色不一样？那为什么光又分为那么多种颜色呢？

其实，光的颜色是由它的波长来决定的，那么波长又是什么呢？为了能让用户更能理解它的定义，我们不是科学家，不深入研究，只是了解一下它的定义就行了，我们前面知道了光的颜色是由波长决定的。光的波长就是光在单位时间内通过的位移（位移是从空间的一个位置运动到另一个位置，它的位置变化叫做在这一运动过程中的位移）。光的本质是电磁波，所以就会有频率的产生。光的频率在传播中保持不变，意思是在光通过不同介质的时候，频率不变而波长发生改变。因此，光的波长由光的频率(颜色)，以及传播的介质决定。

波长的单位我们可以用 nm 或者 μm 来表示，各种颜色有各自的波长，人的眼睛能看到的可见光按波长从长到短排列，依次为红、橙、黄、绿、青、蓝、紫。其中红光的波长范围为 $620 \sim 760\text{nm}$ ；紫光的波长范围为 $380 \sim 440\text{nm}$ 。比紫光波长还短的光叫紫外线，比红光波长还长的光叫红外线。在 1800 年 4 月 24 日英国伦敦皇家学会 (ROYAL SOCIETY) 的威廉·赫歇尔发表太阳光在可见光谱的红光之外还有一种不可见的延伸光谱，具有热效应。他所使用的方法很简单，用一支温度计测量经过棱镜分光后的各色光线温度，由紫到红，发现温度逐渐增加，可是当温度计放到红光以外的部分，温度仍持续上升，因而断定有红外线的存在。红外线遥控就是利用波长为 $760\text{nm} \sim 400\mu\text{m}$ 之间的近红外线来传送控制信号的。下图为各种光的波长示意图，红光外的为红外光，紫色光以外的为紫外光。



红外收发的特点与用途：

1)、红外的特点有：

- 具有很强热效应
- 易于被物体吸收
- 穿透能力比可见光强
- 小角度（30 度锥角以内）
- 点对点直线数据传输，保密性强
- 传输速率较高

2)、 红外的用途有：

通讯技术——常被应用在计算机及其外围设备、移动电话、数码相机、工业设备、网络接入设备，如调制解调器等，像我们这章所用到的红外收发，还有深海探测等无线通讯方面的技术

医疗用途——红外线对人体皮肤、皮下组织具有强烈的穿透力。外界红外线辐射人体产生的一次效应可以使皮肤和皮下组织的温度相应增高，促进血液的循环和新陈代谢，促进人的健康，红外线还有杀菌的能力。

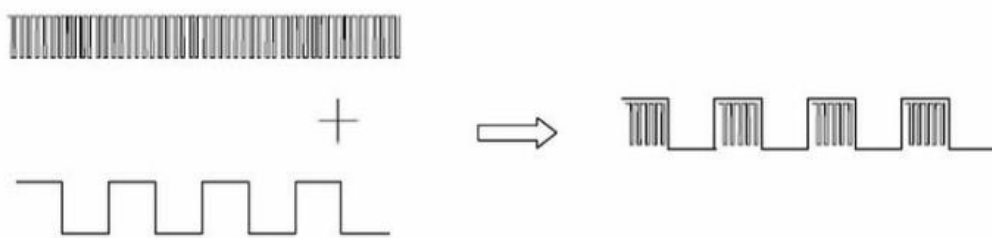
因为我们这章用到的是红外通讯技术，所以它的其他用途我们就不作说明了，主要说下用作红外收发通讯的用途。

红外通讯就是通过红外线传输数据。在电脑技术发展早期，数据都是通过线缆传输的，线缆传输连线麻烦，需要特制接口，颇为不便。无线通信技术逐渐发展起来。

红外的发送工作原理：



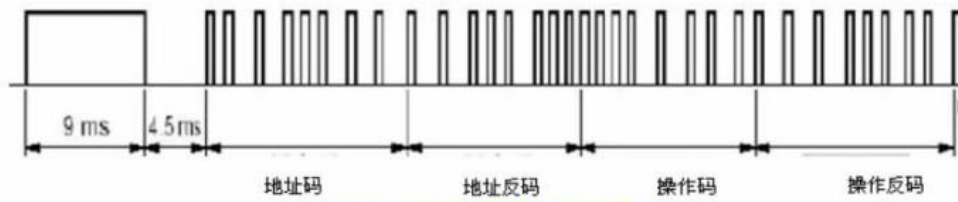
把需要发送的信号编码调制为一系列的脉冲串信号由红外 LED 发送出去。二进制信号的调制由单片机来完成，它把编码后的二进制信号调制成一定频率的间断脉冲串，相当于用二进制信号的编码乘以这个间断脉冲信号得到的间断脉冲串，即是调制后用于红外发射二极管发送的信号，如下图所示，要使红外发光二极管产生调制光，只需在它的驱动管上加上一定频率的脉冲电压。一般由键盘电路、红外编码芯片、电源和红外发射电路组成



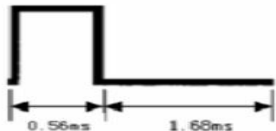
二进制码的调制

在同一个遥控电路中通常要使用实现不同的遥控功能或区分不同的机器类型，这就要求信号按一定的编码传送，编码则会由编码芯片或电路完成。通过对用户码的检验，每个遥控器只能控制一个设备动作，这样可以有效地防止多个设备之间的干扰。就像是电视机、空调有专门的遥控器，遥控器上的按键各有各的功能。

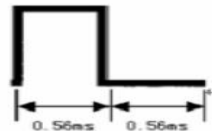
红外遥控发射芯片采用 PPM 编码方式（脉冲位置调制,又称脉位调制，由头码+脉冲数组成,称为 PPM 编码方式），当发射器按键按下后，将发射一组 108ms 的编码脉冲。遥控编码脉冲由前导码、16 位地址码（8 位地址码、8 位地址码的反码）和 16 位操作码（8 位操作码、8 位操作码的反码）组成，反码表示正数的反码与其原码相同；负数的反码是对其原码逐位取反，但符号位除外。由一个 9ms 的高电平（起始码）和一个 4.5ms 的低电平（结果码）组成作为接受数据的准备脉冲。解码的关键是如何识别“0”和“1”，从位的定义我们可以发现“0”、“1”均以脉宽为 0.56ms 的电平开始、周期为 1.12ms 的组合表示二进制的“0”；以脉宽为 1.68ms、周期为 2.24ms 的组合表示二进制的“1”，根据这样的定义，我们就能读出发送的数据是什么了。



发送一组完整的编码脉冲



二进制码'1'



二进制码'0'

下图为我们的红外发送装置，它由键盘电路、红外编码芯片、电源和红外发射电路组成，组合成一起就能把按键编码成一定的数据发送出去给红外接收了

红外遥控图



红外遥控发射头



红外遥控发射头



红外的接收工作原理:

我们首先了解一下红外接收头, 再看红外接收原理:

红外线接收头是目前使用非常广泛的一种通信和遥控器件, 在现实生活中几乎随处可见, 例如电视机、录像机、空调机等等, 都"不约而同"地采用红外线遥控, 它的广泛使用源于它多方面的优点: 抗干扰能力好、编码及解码容易、功耗小、成本低等。我们可以从它的特性、实物图与尺寸及引脚定义和它的接收方式去了解它。

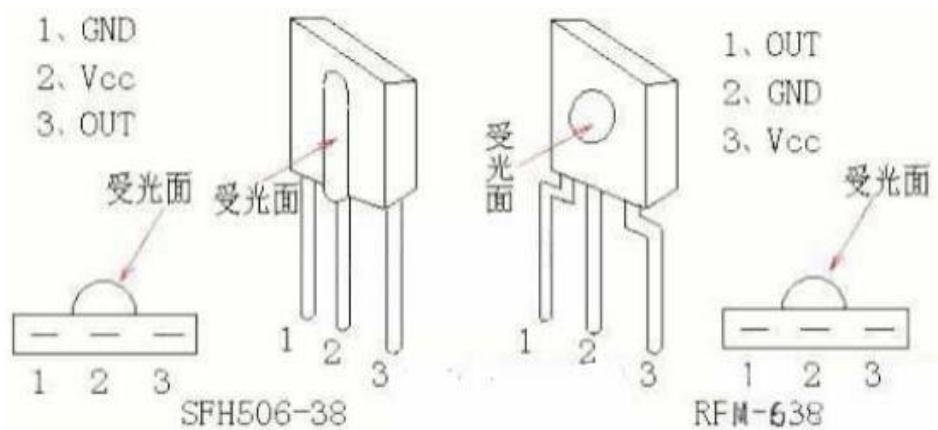
红外接收头的特性:

- 小型设计;
- 内置专用 IC;
- 宽角度及长距离接收;
- 抗干扰能力强;
- 能抵挡环境干扰光线;
- 低电压工作;

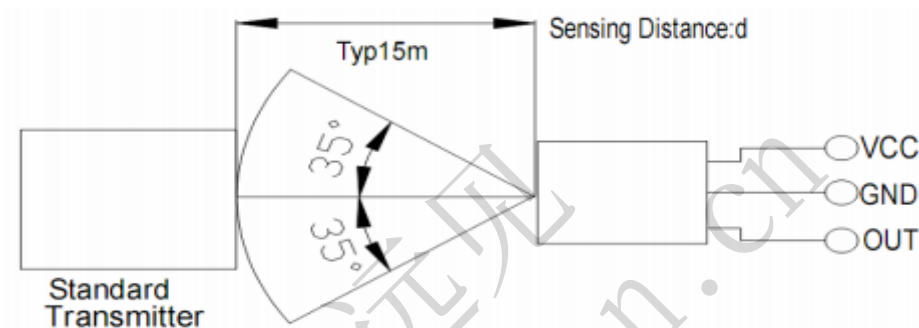
红外接收头实物图:



尺寸及引脚定义结构图:



接收角度:



红外接收原理:

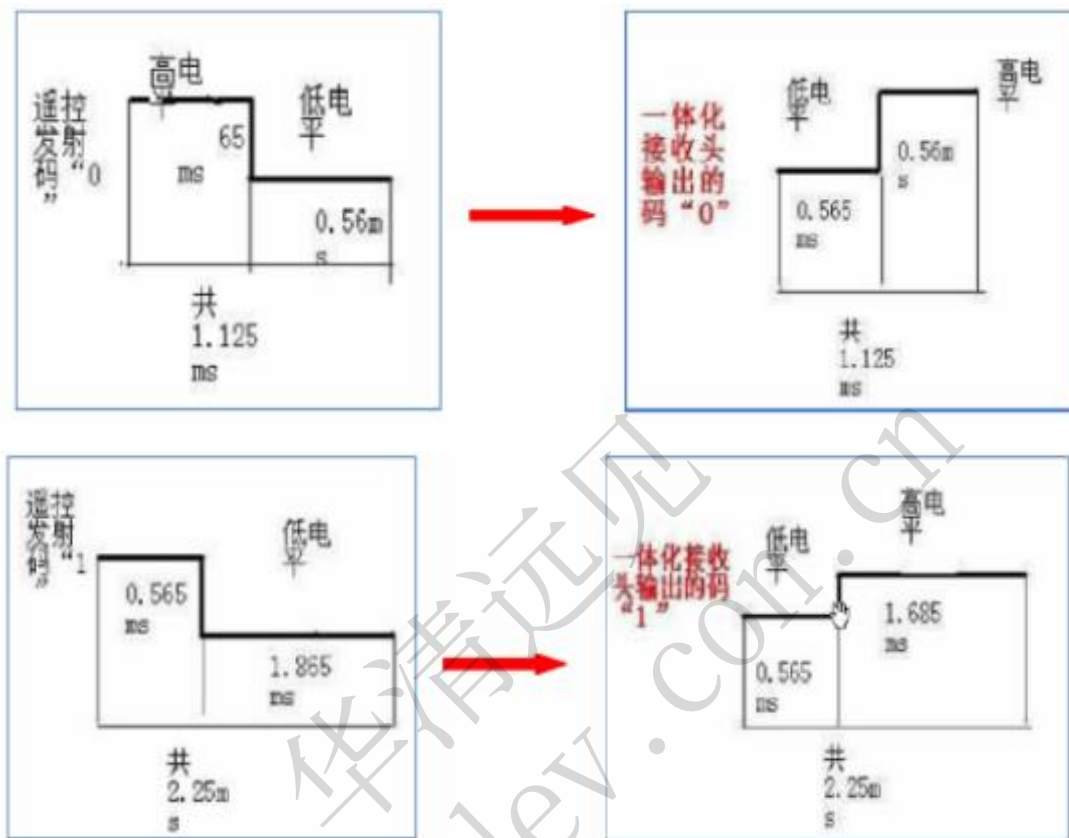
了解了红外接收头后，我们再了解下它是如何工作的，它的原理是什么。红外接收头内置接收管将红外发射管发射出来的光信号转换为微弱的电信号，此信号经由 IC 内部放大器进行放大，然后通过接收管的处理将波形整形后还原为遥控器发射出的原始编码，经由接收头的信号输出脚输入到电器上的编码识别电路，红外解码的关键就是识别 0 和 1。

我们前面介绍过红外接收是完成对红外信号的接收、放大、检波、整形，并解调出遥控编码脉冲的，这个是由接收头内部完成的。经过它的接收放大和解调会在输出端直接输出原始的信号

红外接收头一般是接收、放大、解调一体头，一般红外信号经接收头解调后，数据“0”和“1”的区别通常体现在高低电平的时间长短或信号周期上，这个我们上面有介绍。单片机解码时，通常将接收头输出脚连接到单片机的外部中断，结合定时器判断外部中断间隔的时间从而获取数据。重点是找到数据“0”与“1”间的波形差别。



在这里特别强调：编码与解码是一对逆过程，不仅在原理上是一对逆过程，在码的收发过程也是互反的，即以前发射端原始信号是高电平，那接收头输出的就是低电平，反之亦然。因此为了保证解码过程简单方便，在编码时应该直接换算成其反码。如下图为红外发送与红外接收头接收的电平对比情况



红外接收过程：

接收电路的红外接收管是一种光敏二极管，使用时要给红外接收二极管加反向偏压，它才能正常工作而获得高的灵敏度。红外接收二极管一般有圆形和方形两种。由于红外发光二极管的发射功率较小，红外接收二极管收到的信号较弱，所以接收端就要增加高增益放大电路。然而现在不论是业余制作或正式的产品，大都采用成品的一体化接收头，红外线一体化接收头是集红外接收、放大、滤波和比较器输出等的模块，性能稳定、可靠。所以，有了一体化接收头，人们不再制作接收放大电路，这样红外接收电路不仅简单而且可靠性大大提高。

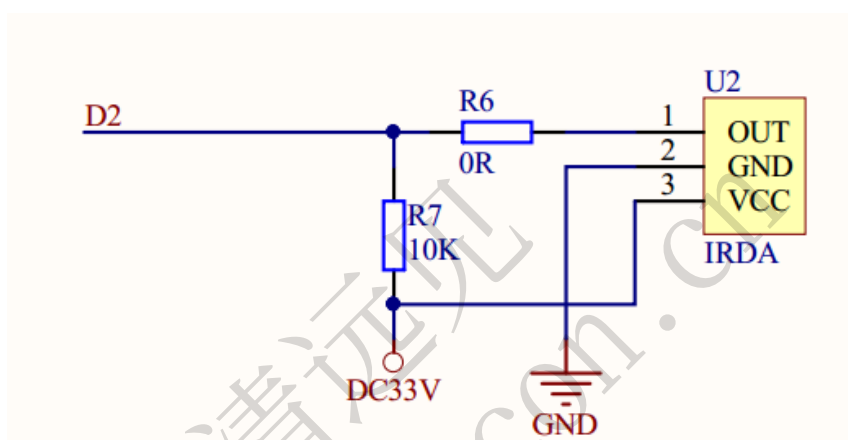
红外接收头包含两个芯片，一个是 PD（即红外接收管），一个是 IC。其中 PD 接收来自发射管的



光信号（该信号已被调制），将光信号转换为电信号，即光电转换，常用于光接收器中。PD 芯片属于典型的 PIN 结构光电二极管。由 PD 接收转换而来的电信号通过 IC 进行放大，自动增益控制，滤波，解调，波形整形，比较器输出交由后面的电路进行识别还原。以上就是红外接收头的接收过程。

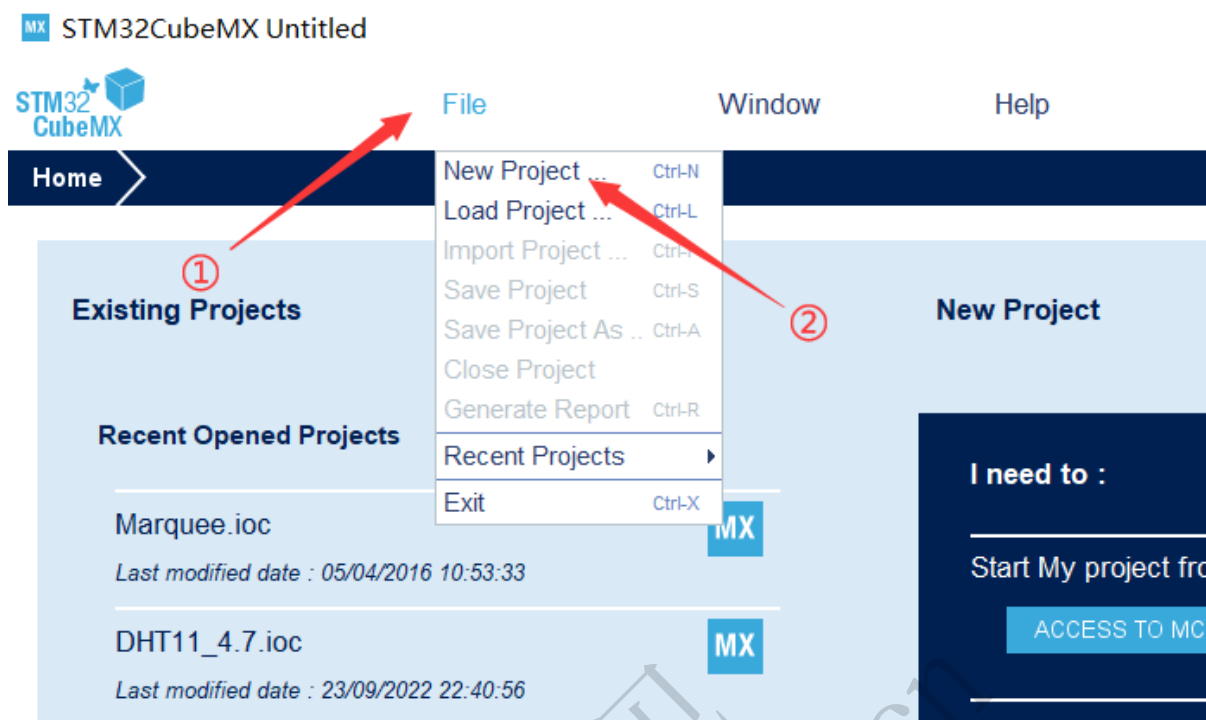
红外接收头在完成解码的时候，通过接收头的信号输出脚输入到电器上的编码识别电路或者是单片机中完成各种操作，在串口上显示接收到的数据。

红外的硬件原理图：

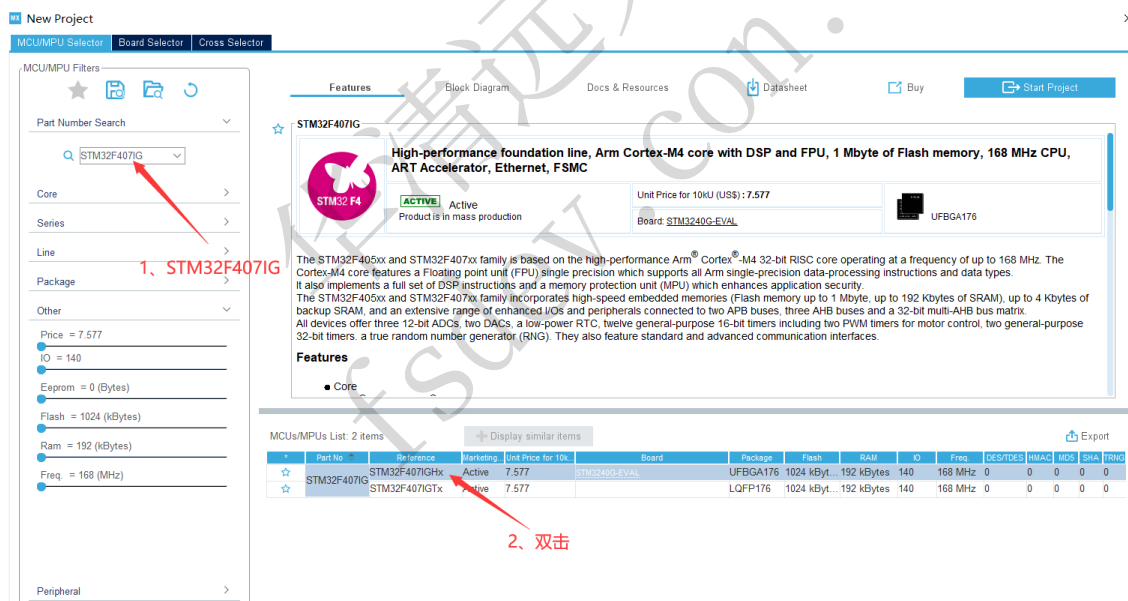


3.13.5 【stm32cubemx 配置介绍】

- (1) 打开 STM32CubeMX->点击左上角的 File->点击 New Project

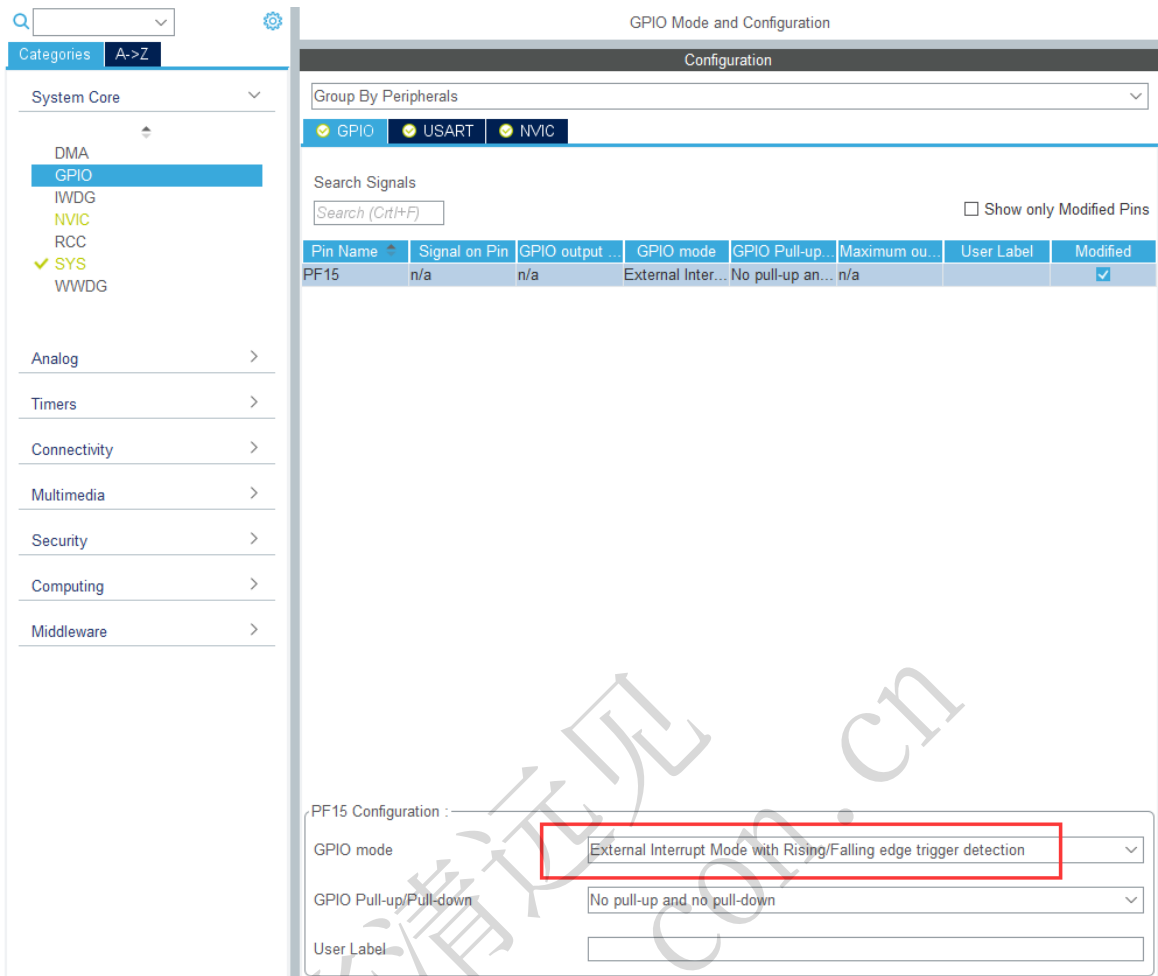


(2) 选择芯片，查看芯片的型号，在搜索框输入芯片型号，本实验中选用的芯片是 STM32F407IGHX

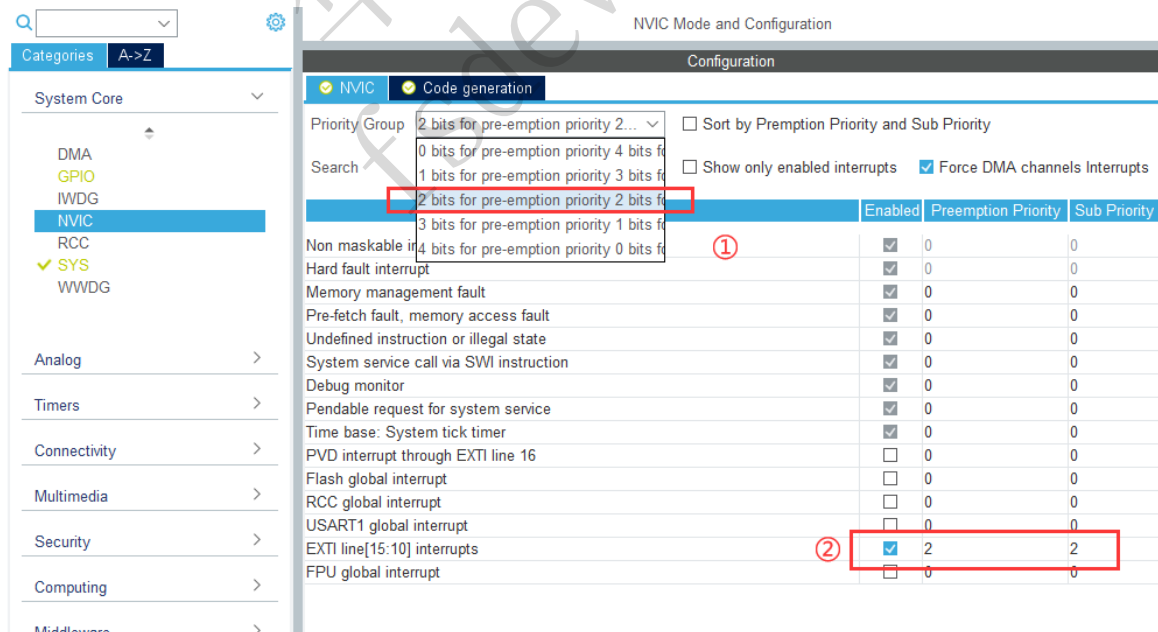


(3) 配置 PA9 为 USART1_TX，配置 PA10 为 USART1_RX，初始化串口。





(6) 配置 NVIC



(7) 配置工程名字，工程路径和 IDE 软件（MDK-ARM V5.27），（注意不要有中文路径）。

(8) 工程软件生成配置，配置工程为 C 文件和 h 文件分开，以及拷贝使用到的 HAL 库文件，其他的文件不拷贝，最后点击右上角的“GENWRTE CODE”，生成工程。



(9) 生成工程完成后会弹出提示框，选择 Open Project 可直接从 MDK 中打开工程代码。

3.13.6 【实验步骤】

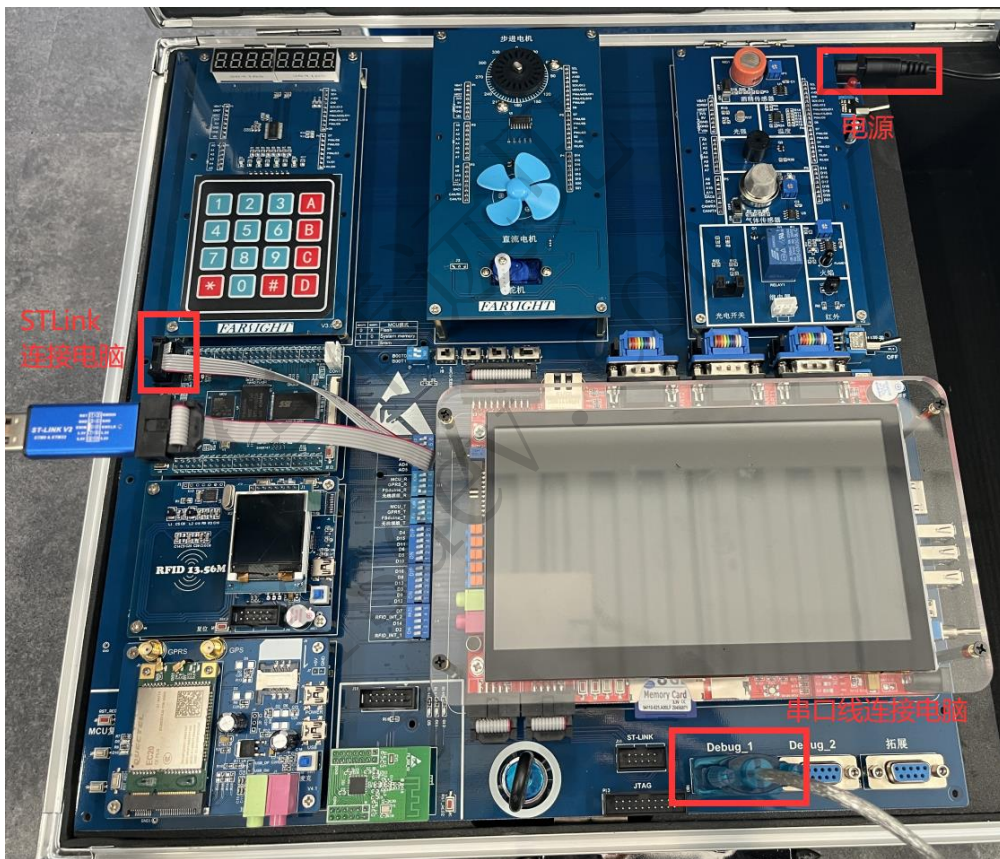
(1) 打开工程文件，路径为

【华清远见-嵌入式 ARM 实验箱资料-III\华清远见-STM 系列资料\Cortex-M4 部分\程序源码

\11_IR_Receive】

(2) 对工程进行编译调试

(3) 开发板连接



(4) 往开发板下载程序，具体步骤参照 1.4.4 节

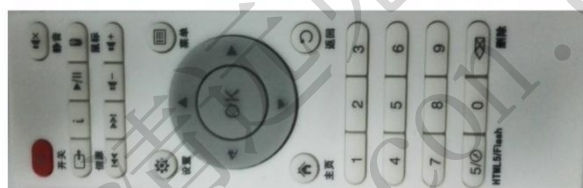
(5) 开发板上电查看实验结果

3.13.7 【实验结果】

红外模块如下图红圈所示位置：



用到外设器件：红外遥控器



串口调试窗口显示串口打印信息如下图：



红外遥控器操作，按下 1、按下 2、按下 3、按下 7、按下 5 显示信息结果如下图：



3.13.8 【实验参考代码】

这里只提供部分重要代码，如想查看部分函数里的详细实现，请打开工程源码，在μVision IDE for ARM 软件里进行查看和操作，这样查找方便清晰、有条理、并可以随意跟进。

注意：由于 STM32CubeMX 版本的不同，生成的工程有所差异，需要在工程中添加几行代码

`HAL_SYSTICK_IRQHandler();`



```

main.c | gpio.c | stm32f4xx_hal_msp.c | stm32f4xx_it.c* | stm32f4xx_hal_cortex.c
176  /* USER CODE END PendSV_IRQn 1 */
177  }
178
179  /**
180   * @brief This function handles System tick timer.
181   */
182  void SysTick_Handler(void)
183  {
184      /* USER CODE BEGIN SysTick_IRQn 0 */
185
186      /* USER CODE END SysTick_IRQn 0 */
187      HAL_IncTick();
188      HAL_SYSTICK_IRQHandler();
189      /* USER CODE BEGIN SysTick_IRQn 1 */
190
191      /* USER CODE END SysTick_IRQn 1 */
192  }
193
194  /**
195   * *****
196   * STM32F4xx Peripheral Interrupt Handlers
197   * Add here the Interrupt Handlers for the used peripherals.
198   * For the available peripheral interrupt handler names,
199   * please refer to the startup file (startup_stm32f4xx.s).
  
```

HAL_SYSTICK_Config(HAL_RCC_GetHCLKFreq()/10000);
 HAL_SYSTICK_CLKSourceConfig(SYSTICK_CLKSOURCE_HCLK);
 HAL_NVIC_SetPriority(SysTick_IRQn, 0, 0);

```

gpio.c | main.c | RemoteInfrared.c | RemoteInfrared.h | usart.c | stm32f4xx_it.c | stm32f4xx_ha
157
158  if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_
159  {
160      Error_Handler();
161  }
162
163  HAL_SYSTICK_Config(HAL_RCC_GetHCLKFreq()/10000);
164  HAL_SYSTICK_CLKSourceConfig(SYSTICK_CLKSOURCE_HCLK);
165
166  /* SysTick_IRQn interrupt configuration */
167  HAL_NVIC_SetPriority(SysTick_IRQn, 0, 0);
168
169
  
```

```

1
2  int main(void)
3  {
4
5      /* USER CODE BEGIN 1 */
6
  
```




```

7      /* USER CODE END 1 */
8
9      /* MCU Configuration-----*/
10
11     /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
12     HAL_Init();
13
14     /* Configure the system clock */
15     SystemClock_Config();
16
17     /* Initialize all configured peripherals */
18     MX_GPIO_Init();
19     MX_USART1_UART_Init();
20
21     /* USER CODE BEGIN 2 */
22     printf("\n\r");
23     printf("\n\r-----");
24     printf("\n\r FS-STM32 开发板 IR 红外线接收实验程序");
25     printf("\n\r 请将红外线接收头连接到开发板的对应接口");
26     printf("\n\r 然后用红外遥控器进行控制，注意串口输出");
27     printf("\n\r-----");
28     printf("\n\r----- 协议如下 -----");
29     printf("\n\r 首先是引导码：开始拉低 9ms,接着一个 4.5ms 的高脉冲.");
30     printf("\n\r      引导码的作用是通知接收方准备接收数据：");
31     printf("\n\r 引导码之后是 4 个字节的二进制码，其中前两个字节是");
32     printf("\n\r      遥控识别码(ID)，第一个为正码,第二个为反码,");
33     printf("\n\r      后两字节是键值，第一个为正码,第二个为反码.");
34     printf("\n\r 最后可能有持续按下通知，上述数据发送完后如果刚才");
35     printf("\n\r      的按键持续被按下，则发送 9ms 低,2ms 高的脉冲。");
36     printf("\n\r----- 载波为 38kHz -----");
37     printf("\n\r 传输一个逻辑 1 需要 2.25ms (560us 低电平+1680us 高电平)");
38     printf("\n\r 传输一个逻辑 0 需要 1.125ms (560us 低电平+560us 高电平)");
39     printf("\n\r-----");
40     printf("\n\r----- 本实验在中断中检测接收 IR 红外线的的数据 -----\n\r");
41     /* USER CODE END 2 */
42
43     /* Infinite loop */
44     /* USER CODE BEGIN WHILE */
45     while (1)
46     {
47         /* USER CODE END WHILE */
48
49         /* USER CODE BEGIN 3 */

```



```
50 Remote_Infrared_KeyDeCode();
51 }
52 /* USER CODE END 3 */
53
54 }
```

3.14 4.3 寸 TFT 彩屏液晶屏实验

3.14.1 【实验目的】

熟悉并了解 TFT 彩屏开发流程，理解彩屏显示图片或者字符的原理。

3.14.2 【实验环境】

嵌入式 ARM 实验箱

ST-Link 仿真器

RealView MDK5.21 集成开发软件

PC 机 XP、Window7/8 (32/64bit)

串口调试助手

3.14.3 【实验内容】

这里我们有两个测试例程，一个是刷屏实验，另一个是利用彩屏绘图 API，绘制椭圆和矩形，并显示图片。

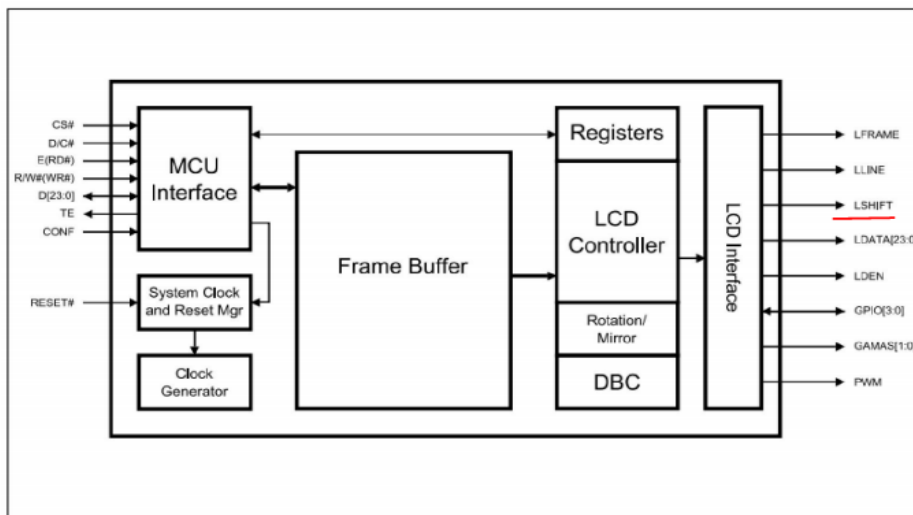
3.14.4 【实验原理】

我们的彩屏使用到的显示控制器是 ssd1963 支持 864 x 480 x 24 位图形内容。它也配有不同宽度并行接口总线来接收图形数据和命令从单片机。它的显示界面，支持常见的内存更少的 LCD 驱动器，每一像素的颜色深度可达 24 比特。

结构图为：



Figure 4-1: SSD1963 Block Diagram



引脚排列:

Figure 5-1: Pinout Diagram –TFBGA (Topview)

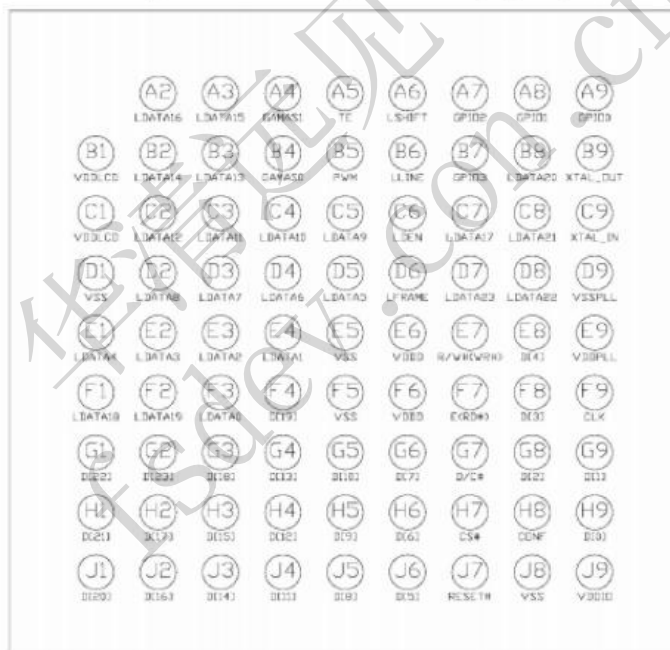


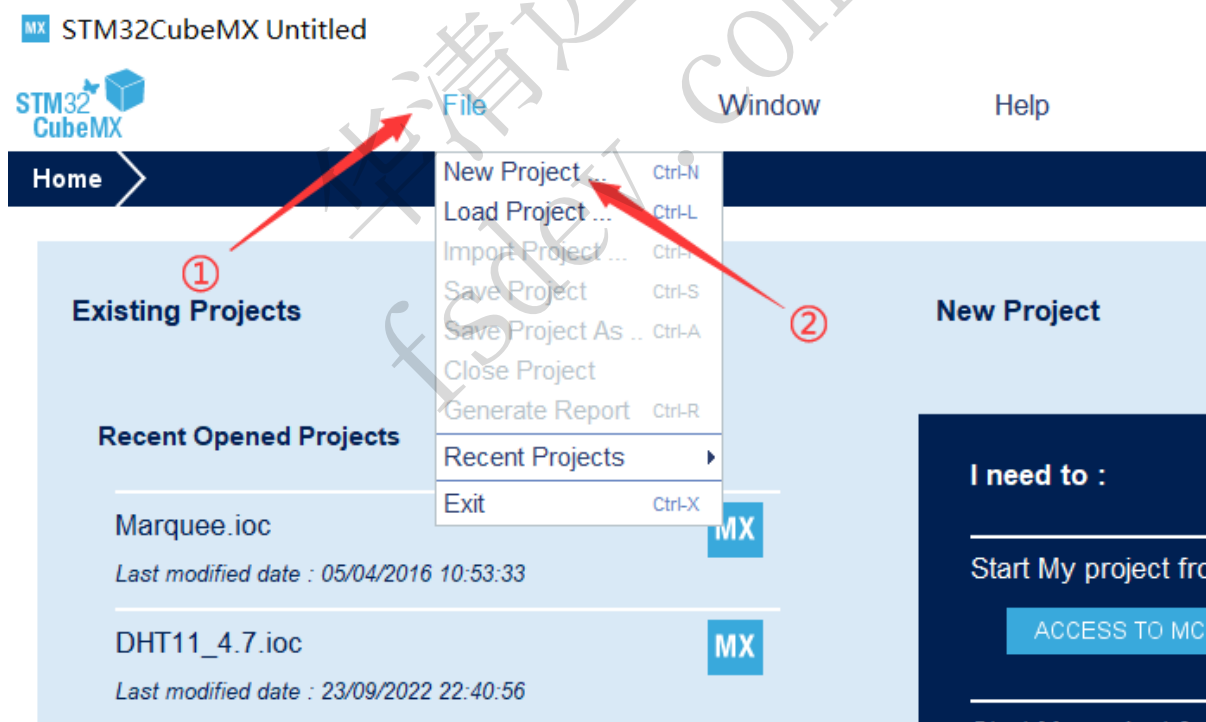


Table 5-1: TFBGA Pin Assignment Table

Pin #	Signal Name	Pin #	Signal Name	Pin #	Signal Name	Pin #	Signal Name	Pin #	Signal Name
A1	-	C1	VDDLCD	E1	LDATA4	G1	D[22]	J1	D[20]
A2	LDATA16	C2	LDATA12	E2	LDATA3	G2	D[23]	J2	D[16]
A3	LDATA15	C3	LDATA11	E3	LDATA2	G3	D[18]	J3	D[14]
A4	GAMAS1	C4	LDATA10	E4	LDATA1	G4	D[13]	J4	D[11]
A5	TE	C5	LDATA9	E5	VSS	G5	D[10]	J5	D[8]
A6	LSHIFT	C6	LDEN	E6	VDDD	G6	D[7]	J6	D[5]
A7	GPIO2	C7	LDATA17	E7	R/W# (WR#)	G7	D/C#	J7	RESET#
A8	GPIO1	C8	LDATA21	E8	D[4]	G8	D[2]	J8	VSS
A9	GPIO0	C9	XTAL_IN	E9	VDDPLL	G9	D[1]	J9	VDDIO
B1	VDDLCD	D1	VSS	F1	LDATA18	H1	D[21]		
B2	LDATA14	D2	LDATA8	F2	LDATA19	H2	D[17]		
B3	LDATA13	D3	LDATA7	F3	LDATA0	H3	D[15]		
B4	GAMAS0	D4	LDATA6	F4	D[19]	H4	D[12]		
B5	PWM	D5	LDATA5	F5	VSS	H5	D[9]		
B6	LLINE	D6	LFRAME	F6	VDDD	H6	D[6]		
B7	GPIO3	D7	LDATA23	F7	E(RD#)	H7	CS#		
B8	LDATA20	D8	LDATA22	F8	D[3]	H8	CONF		
B9	XTAL_OUT	D9	VSSPLL	F9	CLK	H9	D[0]		

3.14.5 【stm32cubemx 配置介绍】

(1) 打开 STM32CubeMX->点击左上角的 File->点击 New Project



(2) 选择芯片，查看芯片的型号，在搜索框输入芯片型号，本实验中选用的芯片是 STM32F407IGHx



New Project

MCU/MPU Selector Board Selector Cross Selector

MCU/MPU Filters

Part Number Search

STM32F407G

Core

Series

Line

Package

Other

Price = 7.577

IO = 140

Expmem = 0 (Bytes)

Flash = 1024 (KBytes)

Ram = 192 (KBytes)

Freq = 168 (MHz)

Peripheral

Features

Block Diagram

Docs & Resources

Datasheet

Buy

Start Project

STM32F407G

High-performance foundation line, Arm Cortex-M4 core with DSP and FPU, 1 Mbyte of Flash memory, 168 MHz CPU, ART Accelerator, Ethernet, FSMC

ACTIVE Active

Product is in mass production

Unit Price for 10KU (US\$): 7.577

Board: STM3240G-EVAL

UFPGA176

The STM32F405x and STM32F407x family is based on the high-performance Arm® Cortex-M4 32-bit RISC core operating at a frequency of up to 168 MHz. The Cortex-M4 core features a Floating point unit (FPU) single precision which supports all Arm single-precision data-processing instructions and data types. It also implements a full set of DSP instructions and a memory protection unit (MPU) which enhances application security. The STM32F405x and STM32F407x family incorporates high-speed embedded memories (Flash memory up to 1 Mbyte, up to 192 Kbytes of SRAM), up to 4 Kbytes of backup SRAM, and an extensive range of enhanced I/Os and peripherals connected to two APB buses, three AHB buses and a 32-bit multi-AHB bus matrix. All devices offer three 12-bit ADCs, two DACs, a low-power RTC, twelve general-purpose 16-bit timers including two PWM timers for motor control, two general-purpose 32-bit timers, a true random number generator (RNG). They also feature standard and advanced communication interfaces.

Features

Core

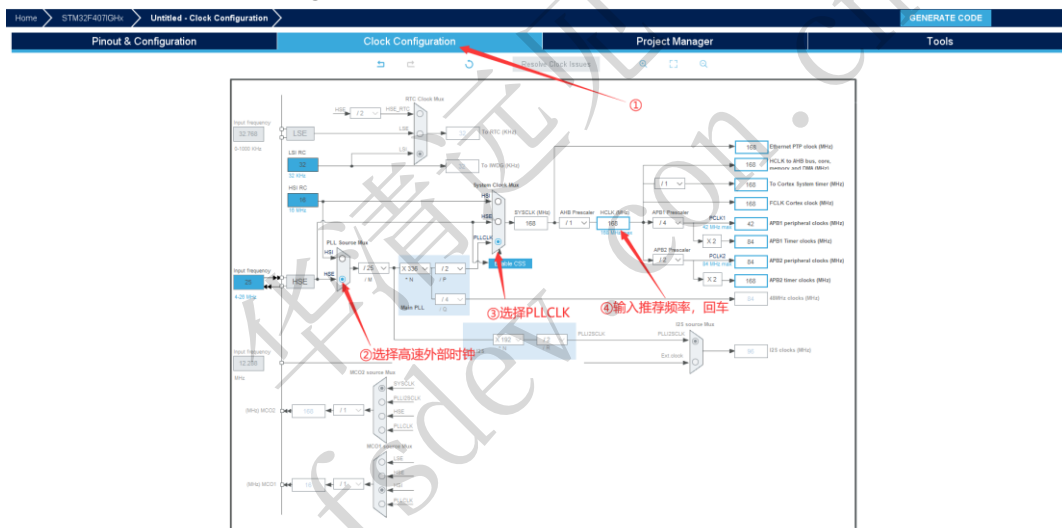
MCUs/MPUs List: 2 items

Display similar items

Part No.	Reference	Marketing	Unit Price for 10k	Board	Package	Flash	RAM	IO	Freq	DES/DES	MAC	MDS	SHA	TRNG
STM32F407G	STM32F407G	Active	7.577	STM3240G-EVAL	UFPGA176	1024 KByte	192 Kbytes	140	168 MHz	0	0	0	0	0
STM32F407G	STM32F407G	Active	7.577		LQFP176	1024 KByte	192 Kbytes	140	168 MHz	0	0	0	0	0

Export

(3) 配置 RCC，点击左侧的 RCC，找到 High Speed Clock (HSE)，点击选择 Crystal/Ceramic Resonator，RCC 配置完成，点击上方的 Clock Configuration，配置时钟树



(4) 配置 FSMC

FSMC Mode and Configuration

Mode

NOR Flash/PSRAM/SRAM/ROM/LCD 1

Chip Select NE4

Memory type LCD Interface

Address Disable

LCD Register Select A0

Data 16 bits

Data/Address Disable

Clock Disable

Address valid

Wait Disable

Byte enable

NOR Flash/PSRAM/SRAM/ROM/LCD 2

NOR Flash/PSRAM/SRAM/ROM/LCD 3

NOR Flash/PSRAM/SRAM/ROM/LCD 4

Configuration

Categories A-Z

ADC3

CAN1

CAN2

CRC

DAC

DCMI

DMA

ETH

FATFS

FREERTOS

FSMC

GPIO

I2C1

I2C2

I2C3

I2S2

I2S3

IWDG

LIBJPEG

LWIP



- (5) 配置工程名字，工程路径和 IDE 软件（MDK-ARM V5.27），（注意不要有中文路径）。
- (6) 工程软件生成配置，配置工程为 C 文件和 h 文件分开，以及拷贝使用到的 HAL 库文件，其他的文件不拷贝，最后点击右上角的“GENWRTE CODE”，生成工程。
- (7) 生成工程完成后会弹出提示框，选择 Open Project 可直接从 MDK 中打开工程代码。

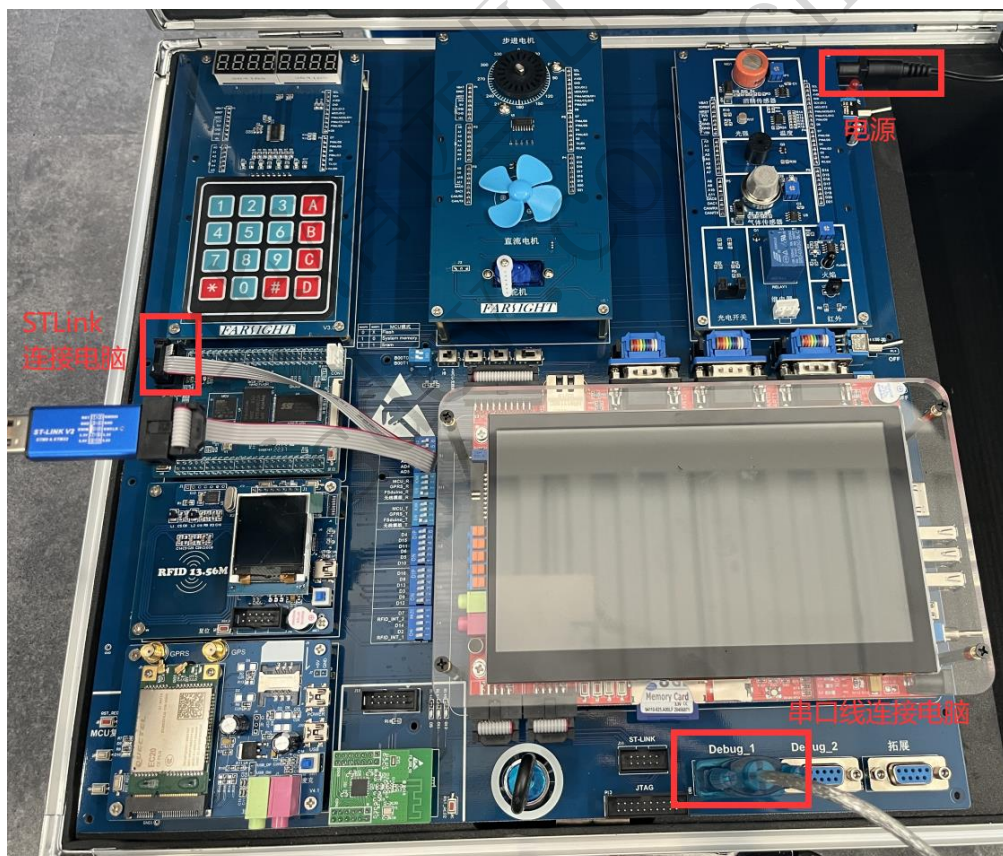
3.14.6 【实验步骤】

- (1) 打开工程文件，路径为

【华清远见-嵌入式 ARM 实验箱资料-III\华清远见-STM 系列资料\Cortex-M4 部分\程序源码

\\12_TFTLCD】

- (2) 对工程进行编译调试
- (3) 开发板连接



- (4) 往开发板下载程序，具体步骤参照 1.4.4 节
- (5) 开发板上电查看实验结果



3.14.7 【实验结果】

将程序下载到开发板后，按复位键运行，正常情况下，我们会看到下面的显示：

TFT 彩屏彩绘 API 测试结果：



3.14.8 【实验参考代码】

这里只提供部分重要代码，如想查看部分函数里的详细实现，请打开工程源码，在μVision IDE for ARM 软件里进行查看和操作，这样查找方便清晰、有条理、并可以随意跟进。

```
1 int main(void)
2 {
3     /* USER CODE BEGIN 1 */
4
5     /* USER CODE END 1 */
6
7     /* MCU Configuration-----*/
```




```

8
9  /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
10 HAL_Init();
11
12 /* USER CODE BEGIN Init */
13
14 /* USER CODE END Init */
15
16 /* Configure the system clock */
17 SystemClock_Config();
18
19 /* USER CODE BEGIN SysInit */
20
21 /* USER CODE END SysInit */
22
23 /* Initialize all configured peripherals */
24 MX_GPIO_Init();
25 MX_FSMC_Init();
26 MX_USART1_UART_Init();
27 /* USER CODE BEGIN 2 */
28     uint8_t lcd_id[12];           //存放 LCD ID 字符串
29     delay_init(168);             //初始化延时函数
30     LCD_Init();                  //LCD 初始化
31     POINT_COLOR=BLUE;
32     sprintf((char*)lcd_id,"ID:%04X",lcddev.id); //将 LCD ID 打印到 lcd_id 数组。
33     LCD_ShowString(30,40,210,24,24,"STM32F407IGT6");
34     LCD_ShowString(30,70,200,16,16,"TFTLCD TEST");
35     LCD_ShowString(170,420,200,24,24,"FS_TFTLCD_V1.0");
36     LCD_ShowString(30,110,200,16,16,lcd_id);    //显示 LCD ID
37     HAL_Delay(2000);
38     LCD_Clear(WHITE);                //清屏
39     LCD_DrawPicture(0,300,480,500,(uint8_t *)gImage_hqyj);
40 /* USER CODE END 2 */
41
42 /* Infinite loop */
43 /* USER CODE BEGIN WHILE */
44 while (1)
45 {
46     /* USER CODE END WHILE */
47
48     /* USER CODE BEGIN 3 */
49
50 }

```



```
51  /* USER CODE END 3 */  
52 }
```

3.15 4.3 寸 TFT 彩屏触摸屏实验

3.15.1 【实验目的】

熟悉并了解 TFT 彩屏开发流程，理解电阻触摸屏的工作原理。

3.15.2 【实验环境】

嵌入式 ARM 实验箱

ST-Link 仿真器

RealView MDK5.21 集成开发软件

PC 机 XP、Window7/8 (32/64bit)

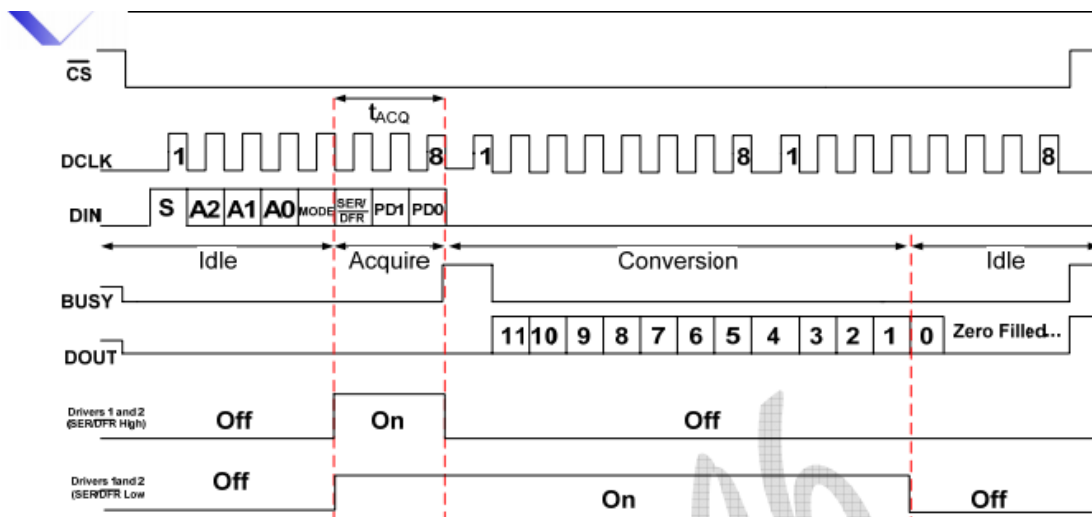
串口调试助手

3.15.3 【实验内容】

这里我们会通过串口打印出我们触摸到的点的坐标，

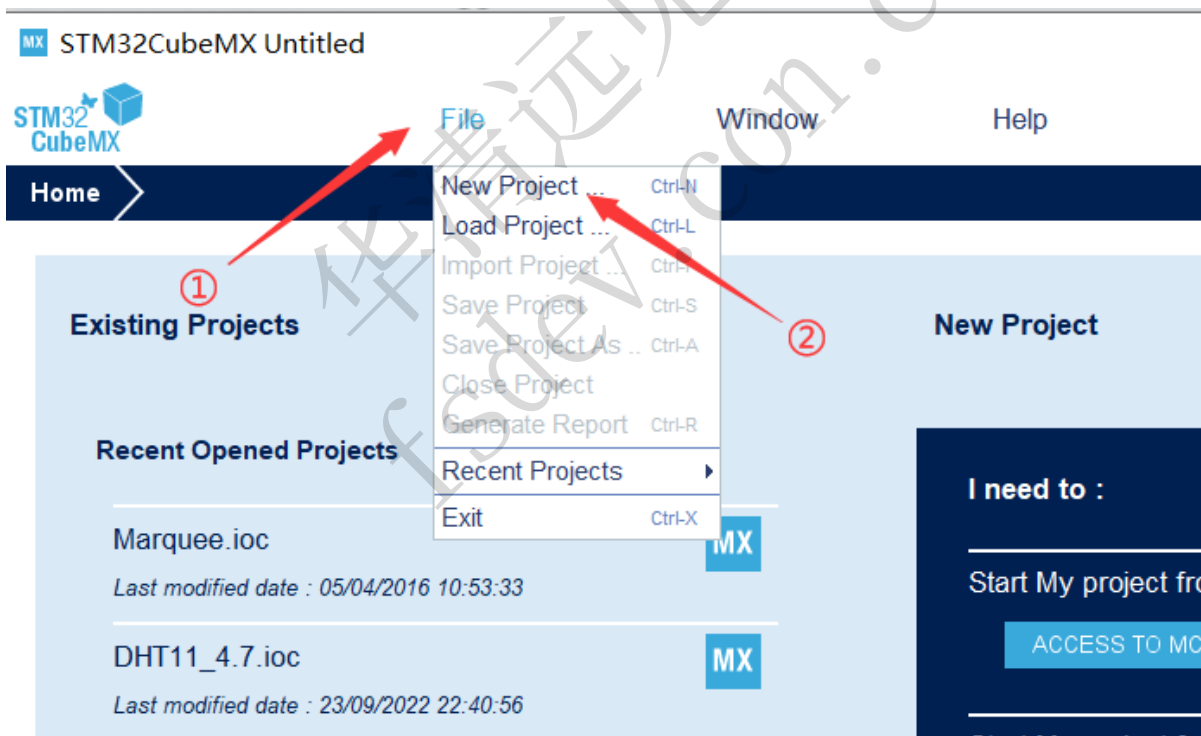
3.15.4 【实验原理】

我们的彩屏使用到的显示控制器是 ssd1963 支持 864 x 480 x 24 位图形内容。它也配有不同宽度并行接口总线来接收图形数据和命令从单片机。它的显示界面，支持常见的内存更少的 LCD 驱动器，每一像素的颜色深度可达 24 比特。触摸屏的点通过 adc 模数转换将我们触摸到的点的电压通过 spi 总线发送给 mcu，然后 mcu 通过一个算法将其坐标值求出来即可。



3.15.5 【stm32cubemx 配置介绍】

(1) 打开 STM32CubeMX->点击左上角的 File->点击 New Project



(2) 选择芯片，查看芯片的型号，在搜索框输入芯片型号，本实验中选用的芯片是 STM32F407IGHx



New Project

MCU/MPU Selector Board Selector Cross Selector

MCU/MPU Filters

Part Number Search

STM32F407G

Core

Series

Line

Package

Other

Price = 7.577

IO = 140

Exprom = 0 (Bytes)

Flash = 1024 (KBytes)

Ram = 192 (KBytes)

Freq = 168 (MHz)

Peripheral

Features

Block Diagram

Docs & Resources

Datasheet

Buy

Start Project

STM32F407G

High-performance foundation line, Arm Cortex-M4 core with DSP and FPU, 1 Mbyte of Flash memory, 168 MHz CPU, ART Accelerator, Ethernet, FSMC

ACTIVE Active

Unit Price for 10KU (US\$): 7.577

Board: STM3240G-EVAL

UFPGA176

The STM32F405xx and STM32F407xx family is based on the high-performance Arm® Cortex®-M4 32-bit RISC core operating at a frequency of up to 168 MHz. The Cortex-M4 core features a Floating point unit (FPU) single precision which supports all Arm single-precision data-processing instructions and data types. It also implements a full set of DSP instructions and a memory protection unit (MPU) which enhances application security. The STM32F405xx and STM32F407xx family incorporates high-speed embedded memories (Flash memory up to 1 Mbyte, up to 192 Kbytes of SRAM), up to 4 Kbytes of backup SRAM, and an extensive range of enhanced I/Os and peripherals connected to two APB buses, three AHB buses, three AHB buses and a 32-bit multi-AHB bus matrix. All devices offer three 12-bit ADCs, two DACs, a low-power RTC, twelve general-purpose 16-bit timers including two PWM timers for motor control, two general-purpose 32-bit timers, a true random number generator (RNG). They also feature standard and advanced communication interfaces.

Features

Core

MCUs/MPUs List: 2 items

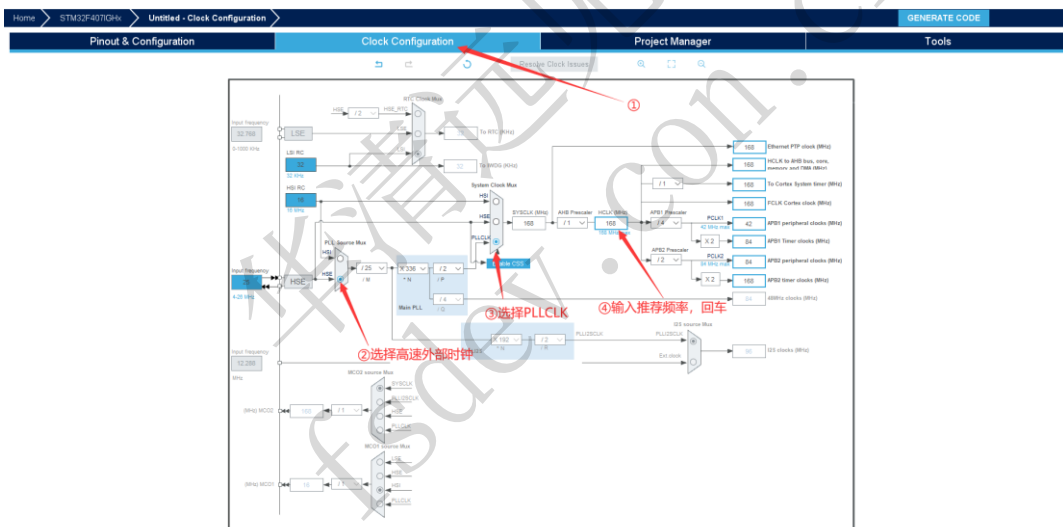
Display similar items

Part No.	Reference	Marketing	Unit Price for 10k	Board	Package	Flash	RAM	IO	Freq	DES/DES	MAC	MDS	SHA	TRNG
STM32F407G	STM32F407G	Active	7.577	STM3240G-EVAL	UFPGA176	1024 KByte	192 Kbytes	140	168 MHz	0	0	0	0	0
STM32F407G	STM32F407G	Active	7.577		LQFP176	1024 KByte	192 Kbytes	140	168 MHz	0	0	0	0	0

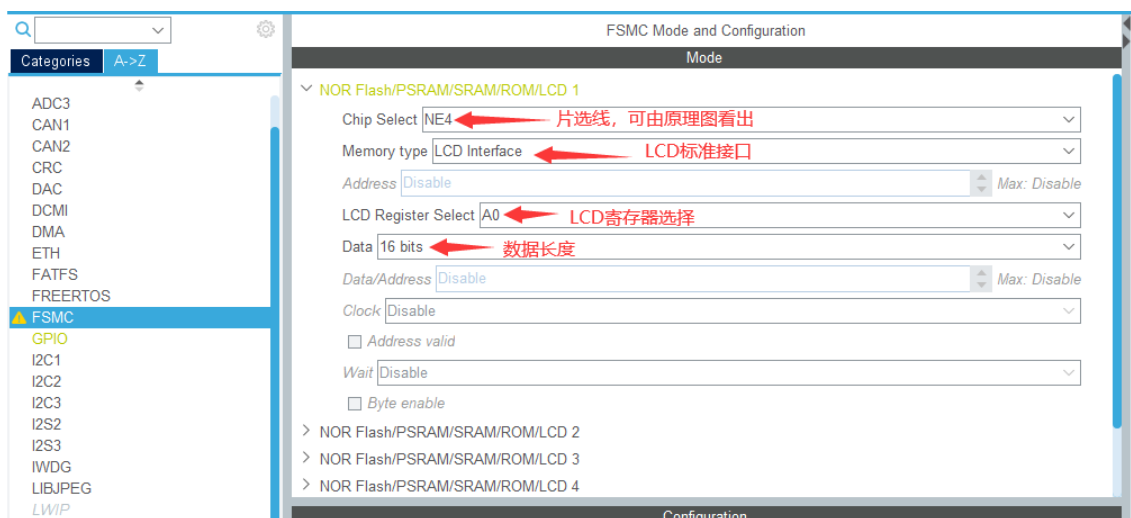
Export

(3) 配置 PA9 为 USART1_TX，配置 PA10 为 USART1_RX，初始化串口。

(4) 配置 RCC，点击左侧的 RCC，找到 High Speed Clock (HSE)，点击选择 Crystal/Ceramic Resonator，RCC 配置完成，点击上方的 Clock Configuration，配置时钟树



(5) 配置 FSMC



- (6) 配置工程名字，工程路径和 IDE 软件（MDK-ARM V5.27），（注意不要有中文路径）。
- (7) 工程软件生成配置，配置工程为 C 文件和 h 文件分开，以及拷贝使用到的 HAL 库文件，其他的文件不拷贝，最后点击右上角的“GENWRTE CODE”，生成工程。
- (8) 生成工程完成后会弹出提示框，选择 Open Project 可直接从 MDK 中打开工程代码。

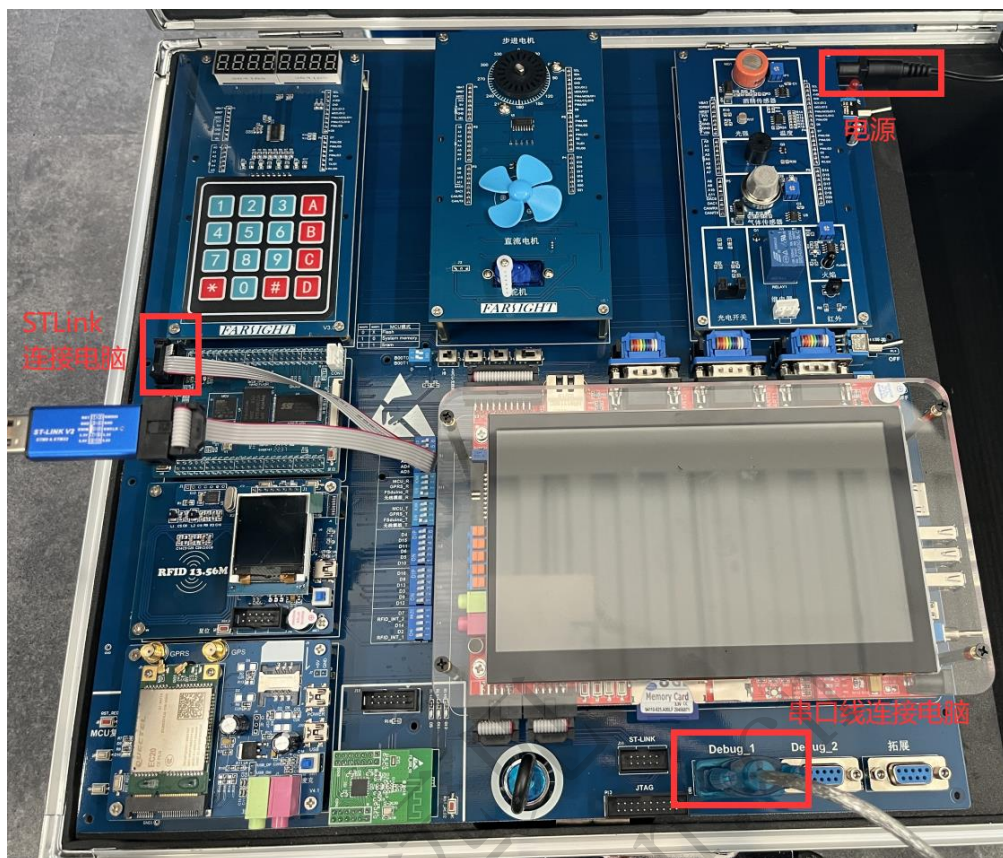
3.15.6 【实验步骤】

- (1) 打开工程文件，路径为

【华清远见-嵌入式 ARM 实验箱资料-III\华清远见-STM 系列资料\Cortex-M4 部分\程序源码

\13_Touch_Lcd】

- (2) 对工程进行编译调试
- (3) 开发板连接



- (4) 往开发板下载程序，具体步骤参照 1.4.4 节
- (5) 开发板上电查看实验结果

3.15.7 【实验结果】

将程序下载到开发板后，打开串口调试工具，按复位键运行，正常情况下，我们会看到下面的显

示：



图 实验结果

3.15.8 【实验参考代码】

这里只提供部分重要代码，如想查看部分函数里的详细实现，请打开工程源码，在µVision IDE for ARM 软件里进行查看和操作，这样查找方便清晰、有条理、并可以随意跟进。

```

1      /* Includes ----- */
2      #include "stm32f4xx_hal.h"
3      #include "spi.h"
4      #include "usart.h"
5      #include "gpio.h"
6      #include "fsmc.h"
7
8      /* USER CODE BEGIN Includes */
9      #include "lcd.h"
10     #include "Touch.h"
11     #include "stdio.h"

```



```

12  /* USER CODE END Includes */
13
14  /* Private variables -----*/
15
16  /* USER CODE BEGIN PV */
17  /* Private variables -----*/
18  extern uint8_t CMD_X[3];
19  extern uint8_t CMD_Y[3];
20  ///uint8_t CMD_X[6] ={0xD0,0x00,0x00,0x00,0x00,0x00};
21  ///uint8_t CMD_Y[6] ={0x90,0x00,0x00,0x00,0x00,0x00};
22  //uint8_t pseudo_data[2]={0x00,0x00};
23  extern uint8_t SPI_RE[3];
24
25  extern uint8_t Cmd_Buf[15];
26  extern uint8_t Cmdcont;
27  extern unsigned char gImage_menu[];
28  uint8_t aaa[16] = {0};
29  uint8_t recv_end_flag = 0;
30  uint8_t rx_len = 0;
31  uint8_t Down_flag = 0; //1 表示已经按下
32  uint16_t line_x = 0; //存储当前坐标
33  uint16_t line_y = 0;
34  uint16_t Date = 0;
35
36
37  uint8_t j = 0;
38  /* USER CODE END PV */
39
40  /* Private function prototypes -----*/
41  void SystemClock_Config(void);
42
43  /* USER CODE BEGIN PFP */
44  /* Private function prototypes -----*/
45  void lcd_init(void);
46  void Touch_Init(void);
47  //void Refreshes_Screen(void);
48  void lcd_clear(unsigned short Color);
49  void LCD_DrawPicture(uint16_t StartX, uint16_t StartY, uint16_t Xend, uint16_t Yend
50  , uint8_t *pic);
51  /* USER CODE END PFP */
52
53  /* USER CODE BEGIN 0 */
54  Pen_Holder Pen_Point; /* 定义笔实体 */

```



```

55  /* USER CODE END 0 */
56
57  int main(void)
58  {
59
60      /* USER CODE BEGIN 1 */
61
62      /* USER CODE END 1 */
63
64      /* MCU Configuration-----
65  */
66
67      /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
68  /
69      HAL_Init();
70
71      /* Configure the system clock */
72      SystemClock_Config();
73
74      /* Initialize all configured peripherals */
75      MX_GPIO_Init();
76      MX_FSMC_Init();
77      MX_SPI2_Init();
78      MX_USART1_UART_Init();
79
80      /* USER CODE BEGIN 2 */
81      // __SPI2_CLK_ENABLE();
82      //printf("=====");
83      lcd_init();
84      // Touch_Init();
85      printf("====init====\n");
86      LCD_DrawPicture(0, 0, 480, 272, gImage_menu);           //480、272 为图片
87  的高度和宽度
88      printf("init ok!!!!!!\n");
89      /* USER CODE END 2 */
90
91      /* Infinite loop */
92      /* USER CODE BEGIN WHILE */
93      while (1)
94      {
95          /* USER CODE END WHILE */
96
97          /* USER CODE BEGIN 3 */

```



```

98
99     if( Pen_Point.Key_Sta == Key_Down )    //触摸屏被按下
100     {
101         //          HAL_NVIC_DisableIRQ(EXTI9_5_IRQn);          //关闭中断
102         HAL_Delay(20);
103         if(!HAL_GPIO_ReadPin(GPIOG, GPIO_PIN_7))
104         {
105             HAL_NVIC_DisableIRQ(EXTI9_5_IRQn); //关闭中断
106             printf("+++++intter++++\r\n");
107
108             if( Convert_Pos() )                //存放点击的 X,Y 的坐标值 成功转换 返
109 回 1 失败返回 0
110             {
111                 line_x = Pen_Point.X0;
112                 line_y = Pen_Point.Y0;
113                 HAL_Delay(200);
114                 //          while(!HAL_GPIO_ReadPin(GPIOG,GPIO_PIN_7)
115 );          //判断 P_7 是否还在按下，直到抬起来止 按下为 0，抬起为 1
116                 printf("x = %d,y = %d\r\n", line_x, line_y);
117                 Pen_Point.X0 = 0;
118                 Pen_Point.Y0 = 0;
119                 Pen_Point.Key_Sta = Key_Up;          //处理数据后，回复到没有按
120 下的状态
121             }
122
123             Pen_Point.Key_Sta = Key_Up;          //处理数据后，回复到没有按下的
124 状态
125             printf("%%%%%%%%%\r\n");
126             HAL_NVIC_EnableIRQ(EXTI9_5_IRQn);          //开启中断
127             LCD_Fill(250, 65, 480, 65 + 32, Blue);
128
129         }
130     else
131     {
132         HAL_NVIC_EnableIRQ(EXTI9_5_IRQn);
133         Pen_Point.Key_Sta = Key_Up;
134         continue ;
135     }
136 }
137 }
138 /* USER CODE END 3 */
139

```



```
140
141  /* USER CODE BEGIN 4 */
    int fputc(int ch, FILE *f)
    {
        while((USART1->SR & 0X40) != 0); //循环发送,直到发送完毕
        USART1->DR = (uint8_t) ch;
        return ch;
    }
    /* USER CODE END 4 */
```

3.16 温度采集实验

3.16.1 【实验目的】

熟悉 LM75 温度传感器的使用方法;

掌握 I2C 总线工作原理及使用方法;

3.16.2 【实验环境】

嵌入式 ARM 实验箱

ST-Link 仿真器

RealView MDK5.21 集成开发软件

PC 机 XP、Window7/8 (32/64bit)

串口调试助手

3.16.3 【实验内容】

本次实验主要通过 LM75 温湿度模块采集当前环境中的温度值。

3.16.4 【实验原理】

LM75A 是一个使用了内置带隙温度传感器和 Σ - Δ 模数转换技术的温度-数字转换器。它也是一个温度检测器, 可提供一个过热检测输出。 LM75A 包含许多数据寄存器: 配置寄存器 (Conf) , 用来存储器



件的某些配置，如器件的工作模式、OS 工作模式、OS 极性和 OS 故障队列等（在功能描述一节中有详细描述）；温度寄存器（Temp），用来存储读取的数字温度；设定点寄存器（Tos & Thyst），用来存储可编程的过热关断和滞后限制，器件通过 2 线的串行 I²C 总线接口与控制器通信。LM75A 还包含一个开漏输出（OS），当温度超过编程限制的值时该输出有效。LM75A 有 3 个可选的逻辑地址管脚，使得同一总线上可同时连接 8 个器件而不发生地址冲突。

LM75A 可配置成不同的工作条件。它可设置成在正常工作模式下周期性地对环境温度进行监控或进入关断模式来将器件功耗降至最低。OS 输出有 2 种可选的工作模式：OS 比较器模式和 OS 中断模式。OS 输出可选择高电平或低电平有效。故障队列和设定点限制可编程，为了激活 OS 输出，故障队列定义了许多连续的故障。

温度寄存器通常存放着一个 11 位的二进制数的补码，用来实现 0.125°C 的精度。这个高精度在需要精确地测量温度偏移或超出限制范围的应用中非常有用。

正常工作模式下，当器件上电时，OS 工作在比较器模式，温度阈值为 80°C，滞后 75°C，这时，LM75A 就可用作一个具有以上预定义温度设定点的独立的温度控制器。

特性：

- 器件可以完全取代工业标准的 LM75，并提供了良好的温度精度（0.125°C），单个器件的电源可超出 2.8V~5.5V 的范围。
- 小型 8 脚封装：SO8 和 TSSOP8。
- 具有 I²C 总线接口，同一总线上可连接多达 8 个器件。
- 电源电压范围：2.8V~5.5V。
- 温度范围：-55°C~+125°C。
- 提供 0.125°C 的精度 11 位的 ADC。
- 温度精度：-25°C~+100°C 时为 ±2°C；-55°C~+125°C 时为 ±3°C
- 可编程温度阈值和滞后设定点。
- 为了降低功耗，关断模式下消耗的电流仅为 3.5uA



- 上电时器件可用作一个独立的温度控制器

下面是 LM75 的硬件原理图：

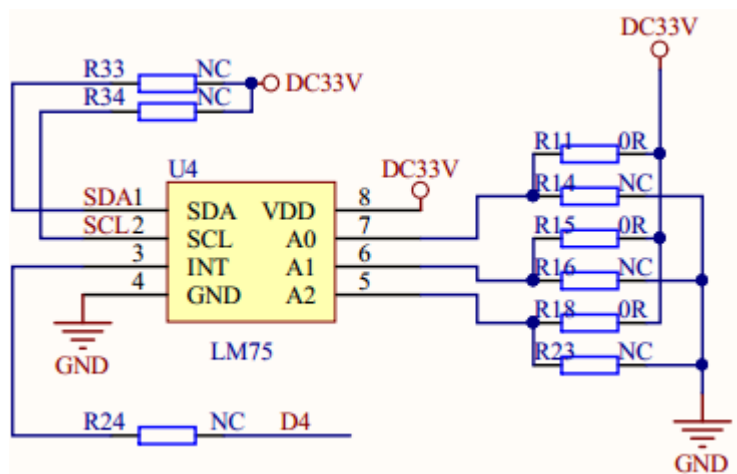
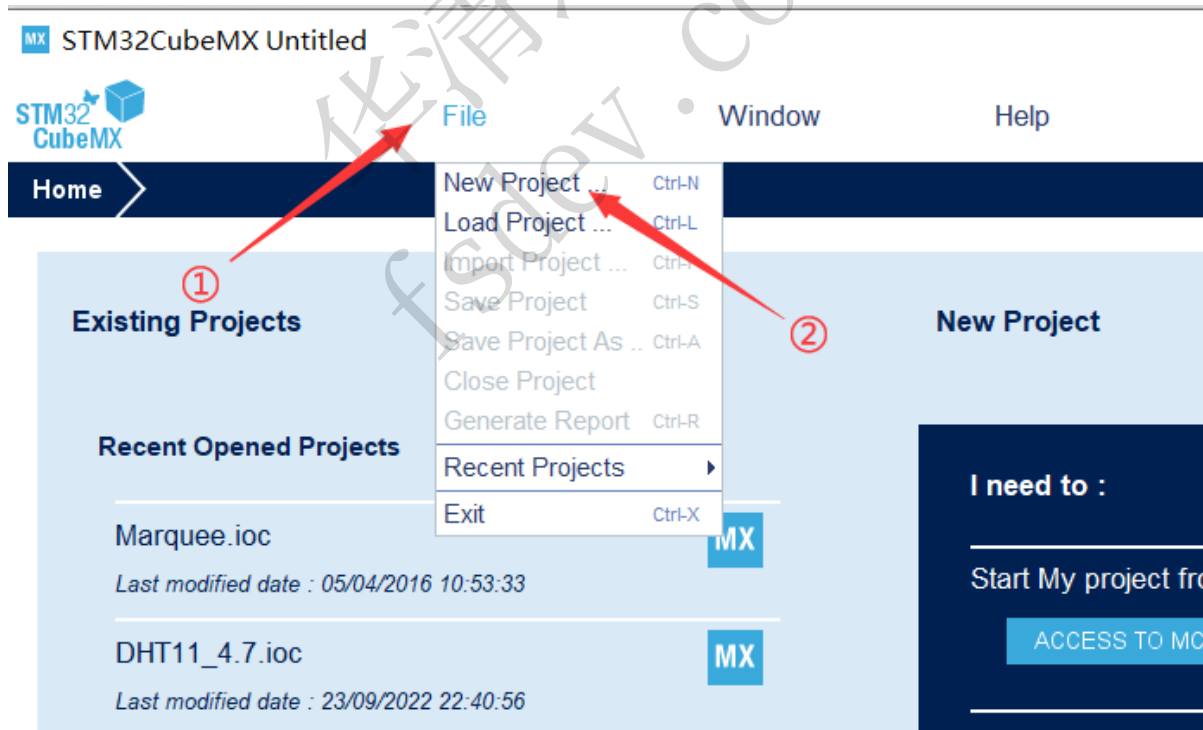


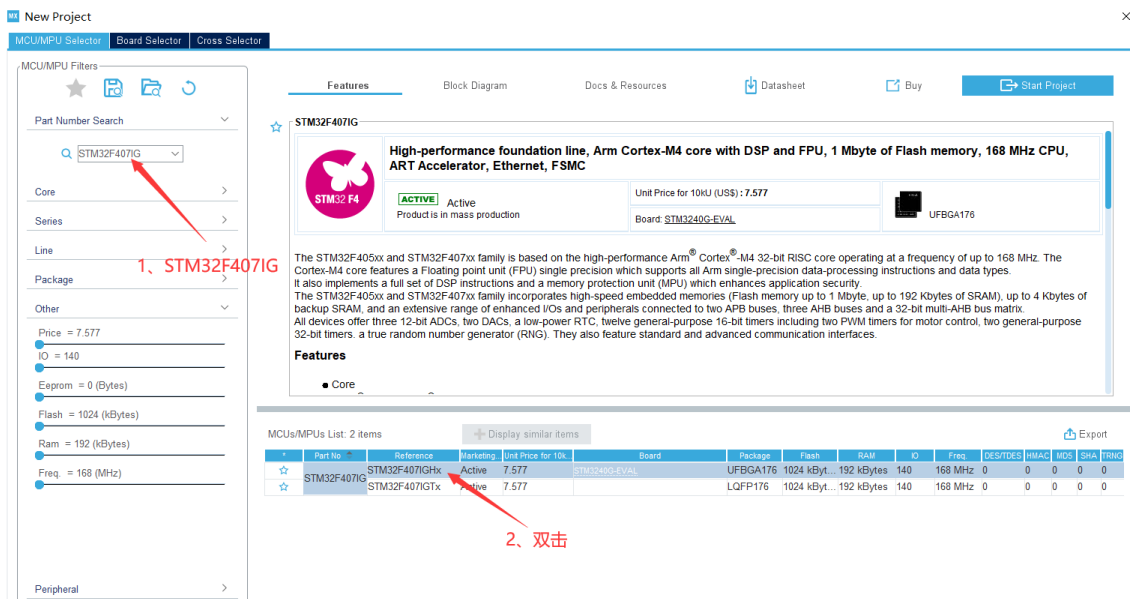
图 LM75 硬件原理图

3.16.5 【stm32cubemx 配置介绍】

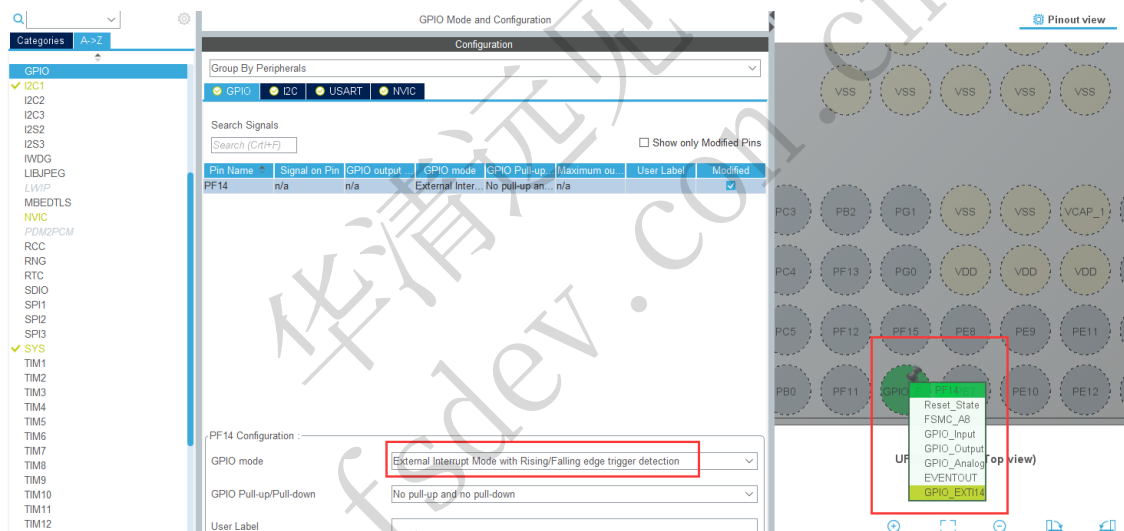
- (1) 打开 STM32CubeMX->点击左上角的 File->点击 New Project



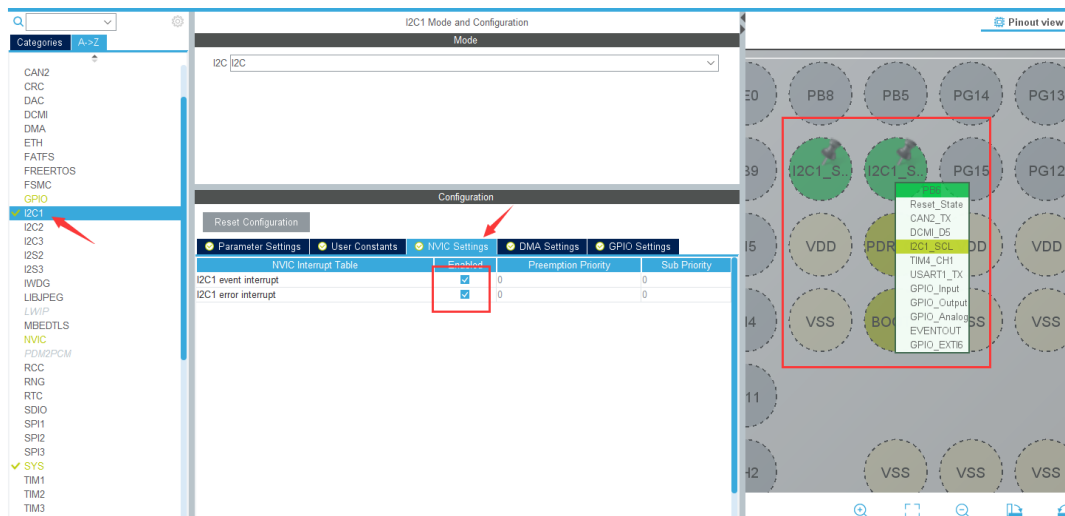
- (2) 选择芯片，查看芯片的型号，在搜索框输入芯片型号，本实验中选用的芯片是 STM32F407IGHx



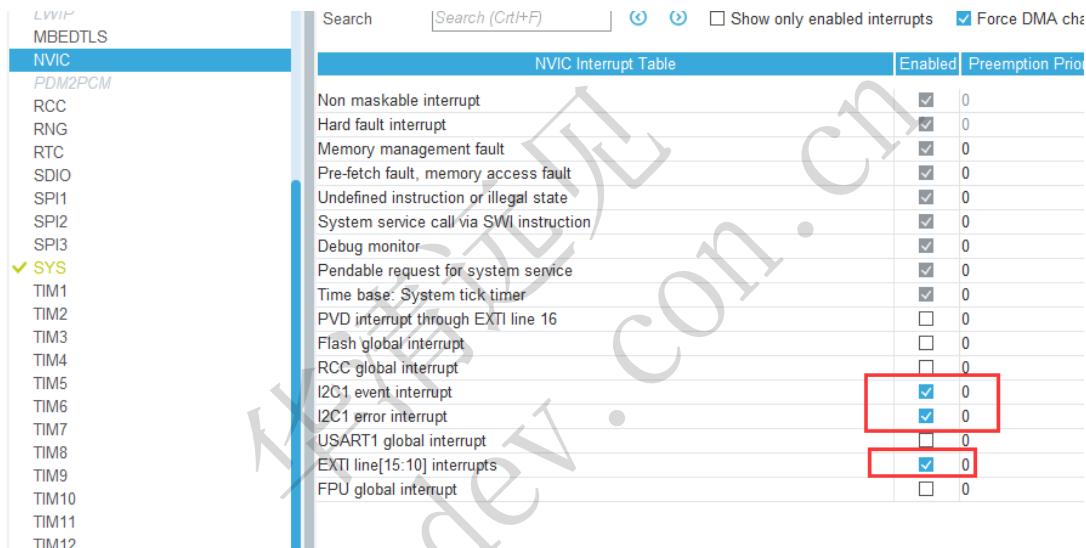
- (3) 配置 PA9 为 USART1_TX，配置 PA10 为 USART1_RX，初始化串口。
- (4) 配置 PF14 为 GPIO_EXTI14 模式，配置为上升沿和下降沿触发。



- (5) 本次实验采用内部时钟，不对设备树进行配置。
- (6) 配置 PB6 为 I2C1_SCL，配置 PB7 为 I2C1_SDA，初始化 I2C1，使能中断。



(7) 配置 NVIC



(8) 配置工程名字，工程路径和 IDE 软件（MDK-ARM V5.27），（注意不要有中文路径）。

(9) 工程软件生成配置，配置工程为 C 文件和 h 文件分开，以及拷贝使用到的 HAL 库文件，其他的文件不拷贝，最后点击右上角的“GENWRTE CODE”，生成工程。

(10) 生成工程完成后会弹出提示框，选择 Open Project 可直接从 MDK 中打开工程代码。

3.16.6 【实验步骤】

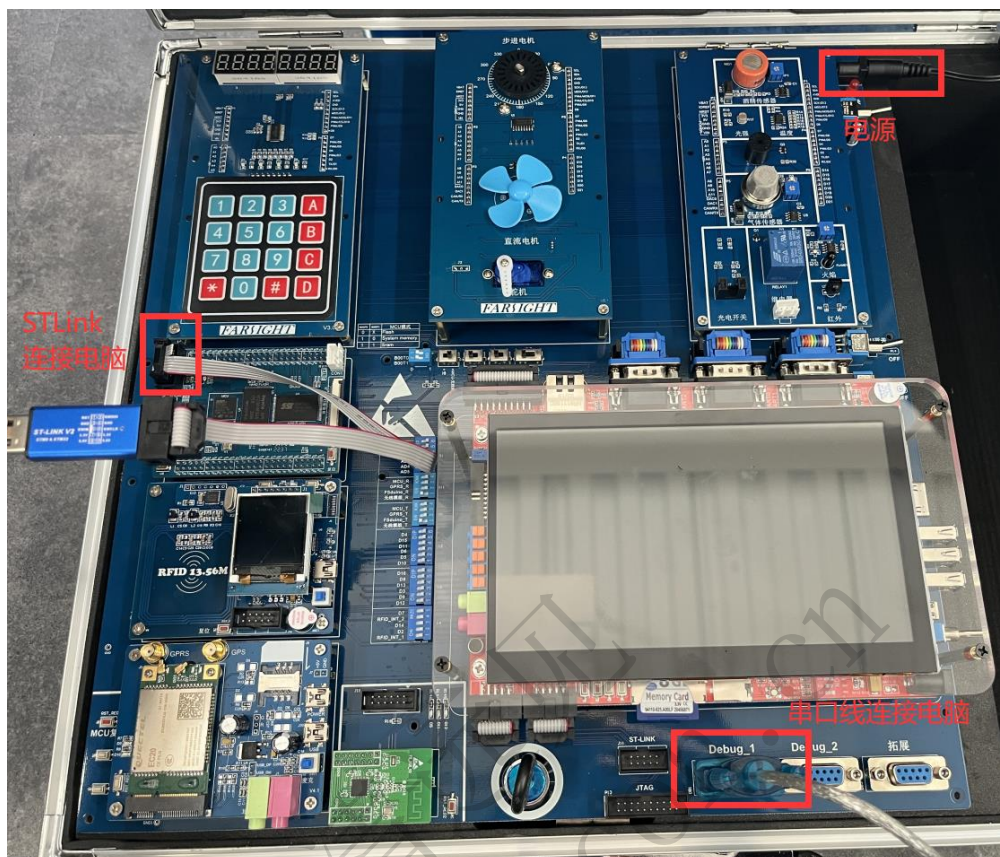
(1) 打开工程文件，路径为

(2) 【华清远见-嵌入式 ARM 实验箱资料-III\华清远见-STM 系列资料\Cortex-M4 部分\程序源码
\14_Temp】

(3) 对工程进行编译调试



(4) 开发板连接



(5) 往开发板下载程序，具体步骤参照 1.4.4 节

(6) 开发板上电查看实验结果

3.16.7 【实验结果】

通过串口调试工具可以看到采集到的温度值：



3.16.8 【实验参考代码】

这里只提供部分重要代码，如想查看部分函数里的详细实现，请打开工程源码，在µVision IDE for ARM 软件里进行查看和操作，这样查找方便清晰、有条理、并可以随意跟进。

```
1  int main(void)
2  {
3
4      /* USER CODE BEGIN 1 */
5
6      /* USER CODE END 1 */
7
8      /* MCU Configuration-----*/
9
10     /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
11     HAL_Init();
12
13     /* Configure the system clock */
14     SystemClock_Config();
15
16     /* Initialize all configured peripherals */
```



```

17  MX_GPIO_Init();
18  MX_I2C1_Init();
19  MX_USART1_UART_Init();
20
21  /* USER CODE BEGIN 2 */
22  LM75SetMode(CONF_ADDR, NORMOR_MODE);
23
24  /* USER CODE END 2 */
25
26  /* Infinite loop */
27  /* USER CODE BEGIN WHILE */
28  while (1)
29  {
30      if((Temp = LM75GetTempReg()) != EVL_ER)
31      {
32          LM75GetTempValue(Temp);
33      }
34      HAL_Delay(1000);
35      /* USER CODE END WHILE */
36
37      /* USER CODE BEGIN 3 */
38
39  }
40  /* USER CODE END 3 */
41
42}
43double LM75GetTempValue(uint16_t tempreg)
44{
45    double TempValue;
46    if(tempreg & (0x01 << 16))
47    {
48        TempValue = ((!tempreg) + 1) * 0.125;
49        printf("TempValue: %lf`C\n", TempValue);
50    }
51    else
52    {
53        TempValue = tempreg * 0.125;
54        printf("TempValue: %lf`C\n", TempValue);
55    }
56    return TempValue;
57}
58uint16_t LM75GetTempReg(void)
59{

```



```

60     uint8_t tempreg[2];
61     uint16_t temp;
62     if(HAL_I2C_Mem_Read(&hi2c1, 0x9F, TEMP_ADDR, 2, tempreg, 2, 100) == HAL_OK)
63     {
64         temp = ((tempreg[0] << 8) | tempreg[1]) >> 5;
65 #if DEBUG
66         printf("current Temp: %04x\n", temp);
67 #endif
68         return temp;
69     }
70     return EVL_ER;
71 }

```

3.17 光电开关实验

3.17.1 【实验目的】

熟悉并掌握光电开关的工作原理，学会在后续的项目开发中使用类似的器件以达到需求的功能。

3.17.2 【实验环境】

嵌入式 ARM 实验箱

ST-Link 仿真器

RealView MDK5.21 集成开发软件

PC 机 XP、Window7/8 (32/64bit)

串口调试助手

3.17.3 【实验内容】

读取光电开关的状态，在串口中实时显示。

3.17.4 【实验原理】

光电传感器是采用光电元件作为检测元件的传感器。它首先把被测量的变化转换成光信号的变化，



然后借助光电原件进一步将光信号转换成电信号。光电传感器一般由光源、光学通路和光电原件三部分组成。

光电传感器是通过把光强度的变化转换成电信号的变化来实现控制的，光电传感器在一般情况下，有三部分构成，它们分为：发送器、接收器和检测电路。

发送器对准目标发射光束，发射的光束一般来源于半导体光源，发光二极管(LED)、激光二极管及红外发射二极管。光束不间断地发射，或者改变脉冲宽度。接收器有光电二极管、光电三极管、光电池组成。在接收器的前面，装有光学元件如透镜和光圈等。在其后面是检测电路，它能滤出有效信号和应用该信号。

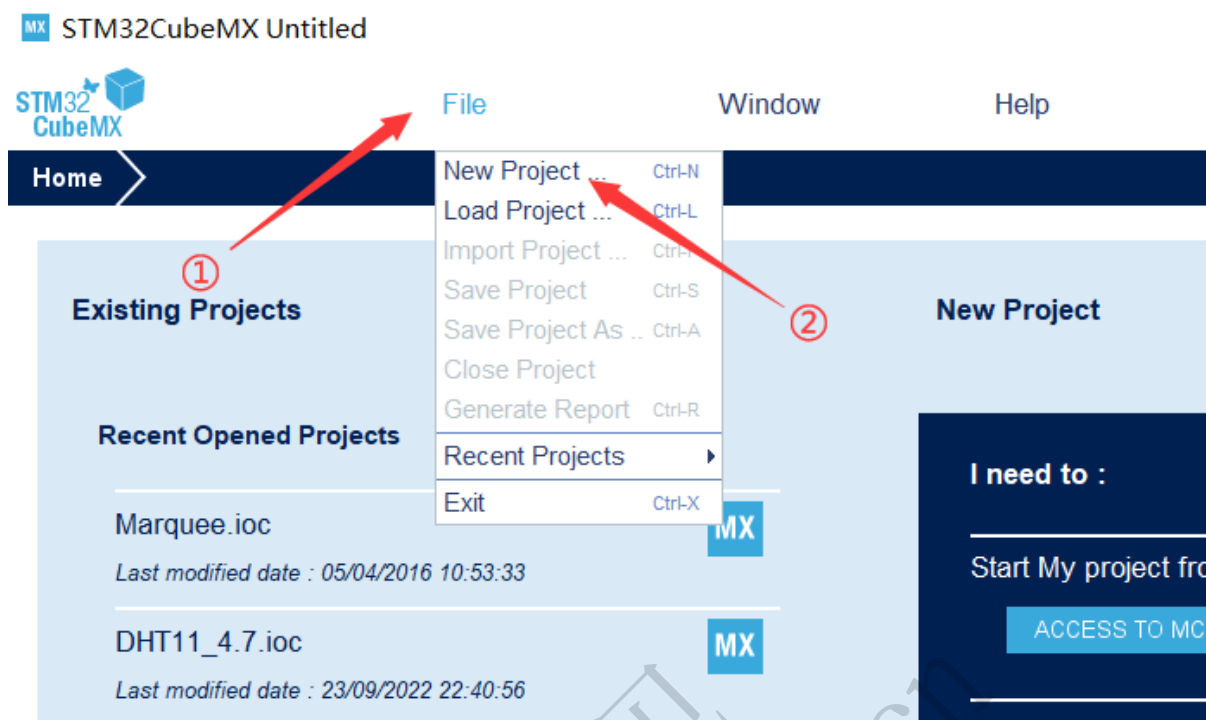
此外，光电开关的结构元件中还有发射板和光导纤维。

三角反射板是结构牢固的发射装置。它由很小的三角锥体反射材料组成，能够使光束准确地从反射板中返回，具有实用意义。它可以在与光轴 0 到 25 的范围改变发射角，使光束几乎是从一根发射线，经过反射后，还是从这根反射线返回。

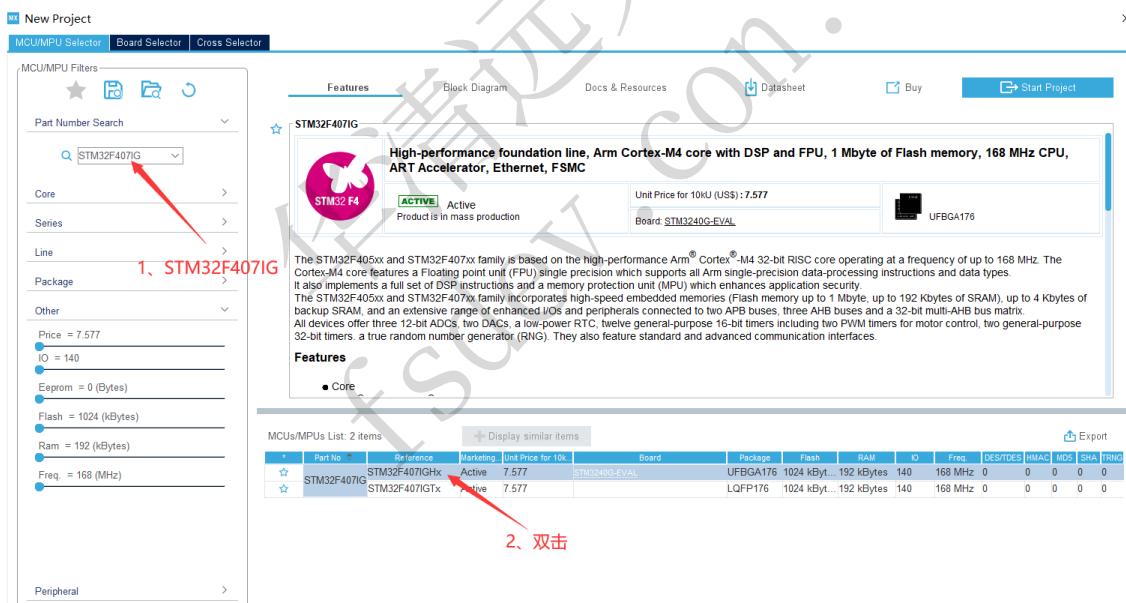


3.17.5 【stm32cubemx 配置介绍】

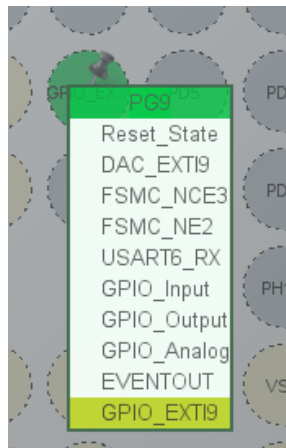
(1) 打开 STM32CubeMX->点击左上角的 File->点击 New Project



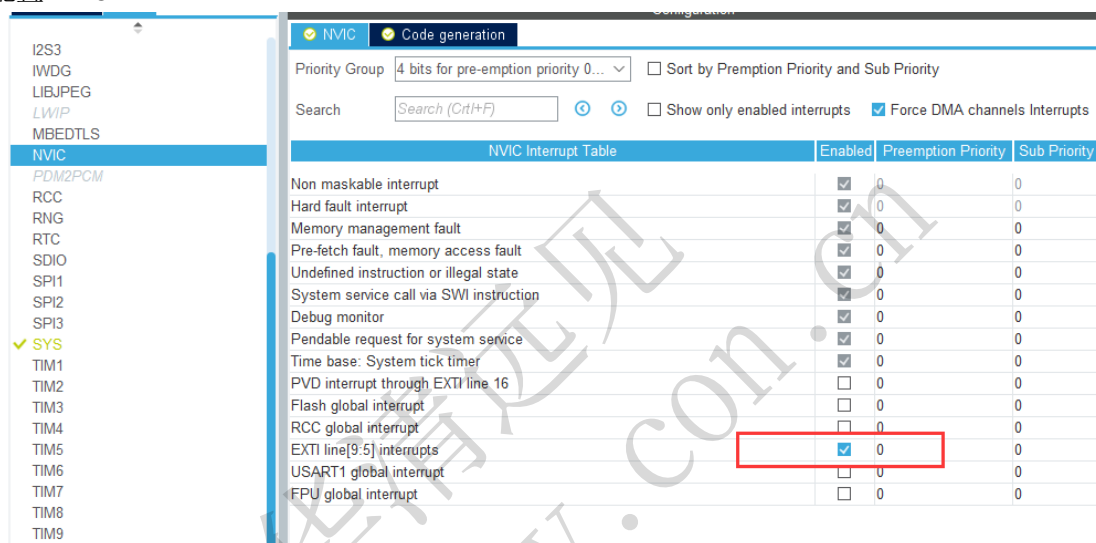
(2) 选择芯片，查看芯片的型号，在搜索框输入芯片型号，本实验中选用的芯片是 STM32F407IGHX



- (3) 配置 PA9 为 USART1_TX，配置 PA10 为 USART1_RX，初始化串口。
- (4) 配本次实验采用内部时钟，不对设备树进行配置。
- (5) 配置 PG9 为 GPIO_EXTI9 模式



(6) 配置 NVIC



(7) 配置工程名字，工程路径和 IDE 软件（MDK-ARM V5.27），（注意不要有中文路径）。

(8) 工程软件生成配置，配置工程为 C 文件和 h 文件分开，以及拷贝使用到的 HAL 库文件，其他的文件不拷贝，最后点击右上角的“GENWRTE CODE”，生成工程。

(9) 生成工程完成后会弹出提示框，选择 Open Project 可直接从 MDK 中打开工程代码。

3.17.6 【实验步骤】

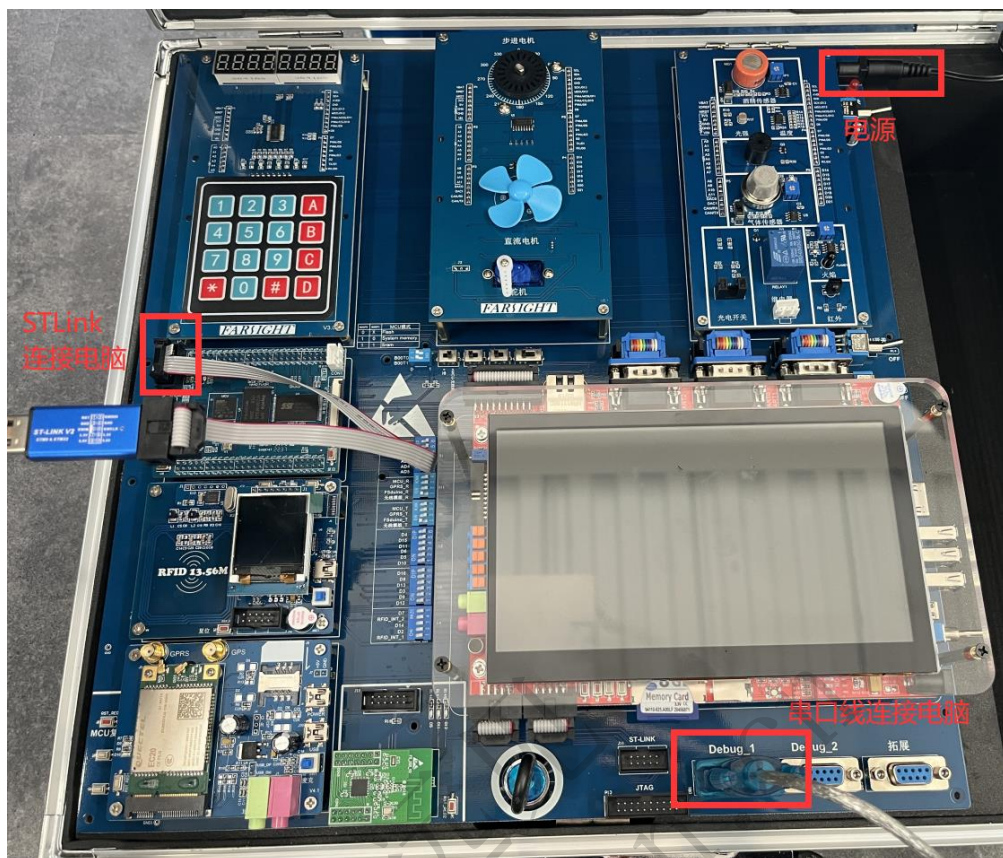
(1) 打开工程文件，路径为

【华清远见-嵌入式 ARM 实验箱资料-III\华清远见-STM 系列资料\Cortex-M4 部分\程序源码

\15_lightelectric】

(2) 对工程进行编译调试

(3) 开发板连接



- (4) 往开发板下载程序，具体步骤参照 1.4.4 节
- (5) 开发板上电查看实验结果

3.17.7 【实验结果】

光电开关的位置如下：





图 光电开关位置标注图

打开串口调试助手，设置波特率为 115200，使用薄纸片在光电开关的凹槽处遮挡，每次遮挡在串口中都会计数并打印一次，如下：



图 串口数据显示图

3.17.8 【实验参考代码】

这里只提供部分重要代码，如果想查看详细源码，请打开对应工程文件，可以对代码进行追踪查看，更容易理解程序的逻辑关系。

```
1  int main(void)
2  {
3
4      /* USER CODE BEGIN 1 */
5
6      /* USER CODE END 1 */
7      HAL_Init();
8
```



```

9      /* Configure the system clock */
10     SystemClock_Config();
11
12     /* Initialize all configured peripherals */
13     MX_GPIO_Init();
14     MX_USART1_UART_Init();
15
16     /* USER CODE BEGIN 2 */
17     printf("\n\r-----\r\n");
18     printf("\n\r FS-STM32 开发板 光电开关实验\r\n");
19     /* USER CODE END 2 */
20
21     /* Infinite loop */
22     /* USER CODE BEGIN WHILE */
23     while (1)
24     {
25         /* USER CODE END WHILE */
26
27         /* USER CODE BEGIN 3 */
28         if(state == 1)
29         {
30             state = 0;
31             count++;
32             printf("计数值: %d\r\n", count);
33         }
34
35     }

```

3.18 UCOSII 操作系统创建 1 个任务实验

3.18.1 【实验目的】

通过本次实验了解 UCOSII 这种可以基于 ROM 运行的、可裁剪的、抢占式、实时多任务内核；熟悉怎么通过 UCOSII 操作系统创建一个简单任务的流程。

3.18.2 【实验环境】

嵌入式 ARM 实验箱



ST-Link 仿真器

RealView MDK5.21 集成开发软件

PC 机 XP、Window7/8 (32/64bit)

串口调试助手

3.18.3 【实验内容】

本次实验通过 UCOSII 操作系统创建一个简单任务，这个任务的工作就是实现 LED 灯闪烁操作。

3.18.4 【实验原理】

通过 UCOSII 操作系统创建一个简单任务，这个任务的工作就是实现 LED 灯闪烁操作。

3.18.5 【实验步骤】

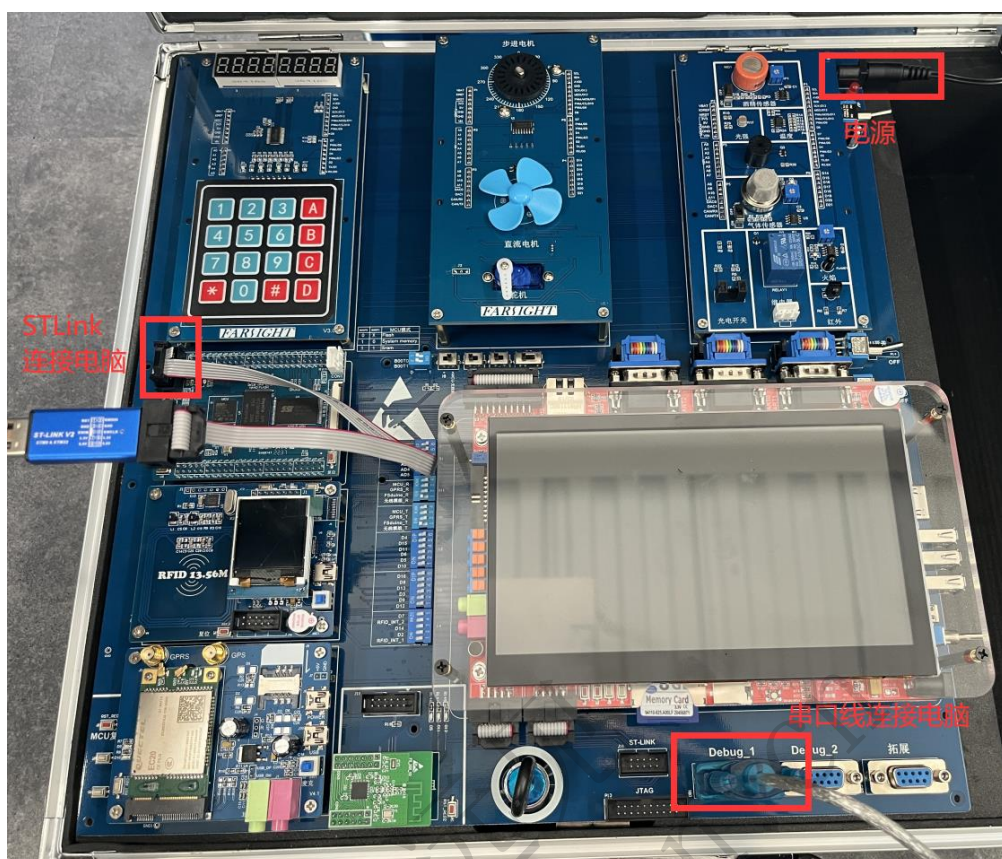
(1) 打开工程文件，路径为

【[华清远见-嵌入式 ARM 实验箱资料-III\华清远见-STM 系列资料\Cortex-M4 部分\程序源码](#)

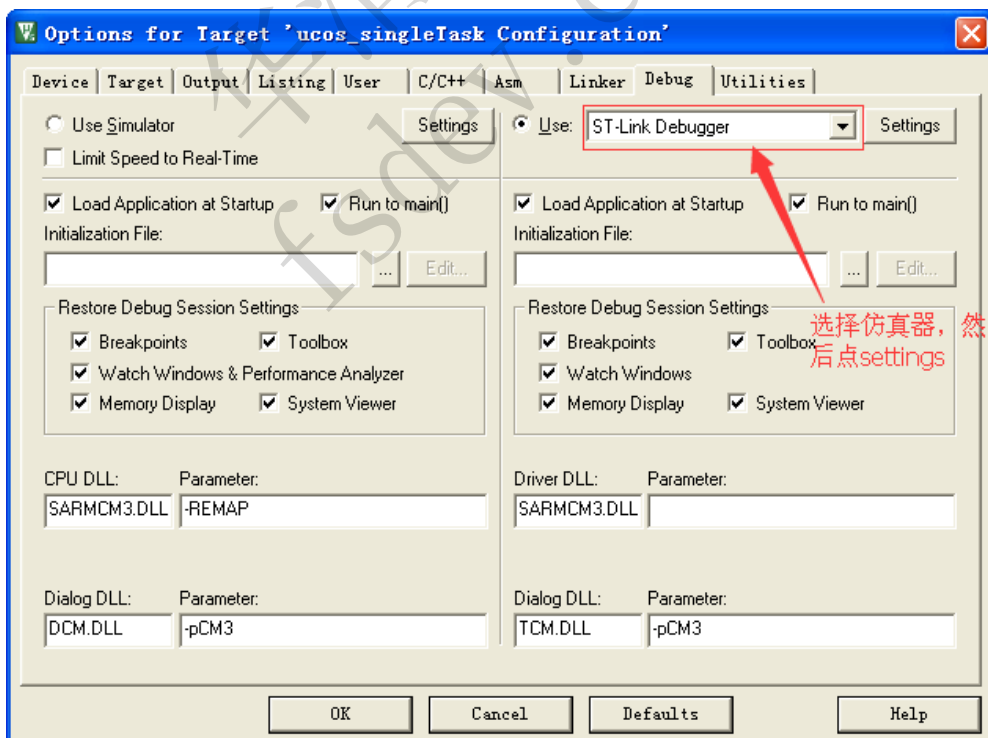
[\ucos_singleTask】](#)

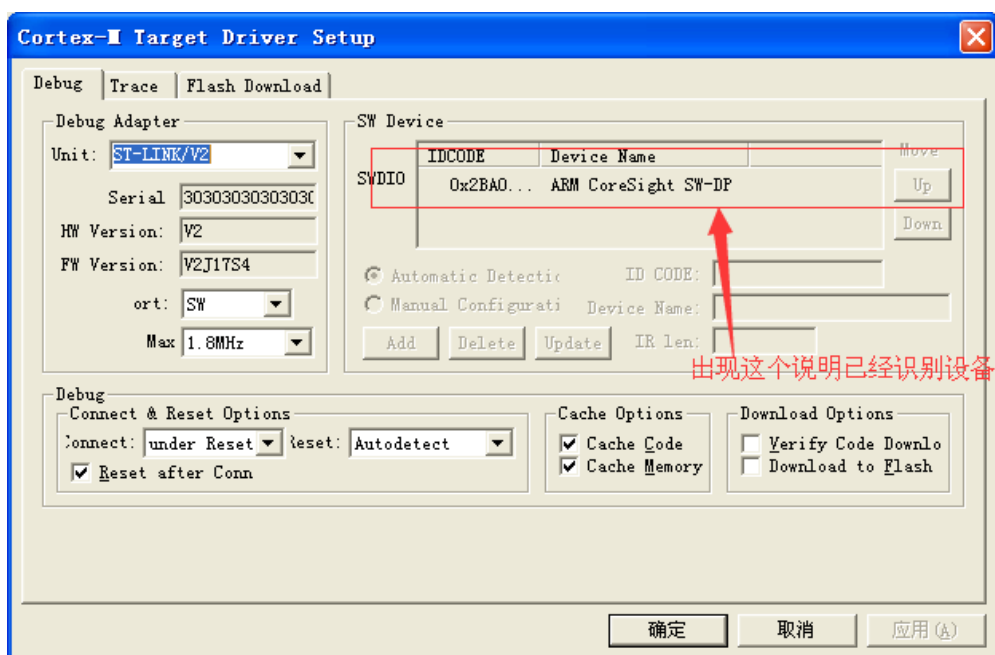
(2) 对工程进行编译调试

(3) 开发板连接



(4) 往开发板下载程序，具体步骤参照 1.4.4 节





(5) 开发板上电查看实验结果

3.18.6 【实验结果】

在程序编译成功之后，将程序下载到开发板，可以看到 D1 灯以固定的频率闪烁，说明任务

Task_farsight_LED () 已经正常运行了！

3.18.7 【实验参考代码】

这里只提供部分重要代码，如想查看部分函数里的详细实现，请打开工程源码，在µVision IDE for

ARM 软件里进行查看和操作，这样查找方便清晰、有条理、并可以随意跟进。

```

1 #define STARTUP_TASK_PRIO      4
2 #define STARTUP_TASK_STK_SIZE  80
3 static OS_STK startup_task_stk[STARTUP_TASK_STK_SIZE];
4 /* USER CODE END PV */
5
6 /* Private function prototypes -----*/
7 void SystemClock_Config(void);
8 static void Task_farsight_LED(void *p_arg);
9 /* USER CODE BEGIN PFP */
10 /* Private function prototypes -----*/
11

```



```

12  /* USER CODE END PFP */
13
14  /* USER CODE BEGIN 0 */
15
16  /* USER CODE END 0 */
17
18  int main(void)
19  {
20
21      /* USER CODE BEGIN 1 */
22      OSInit();
23      /* USER CODE END 1 */
24
25      /* MCU Configuration-----*/
26
27      /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
28      HAL_Init();
29
30      /* Configure the system clock */
31      SystemClock_Config();
32
33      /* Initialize all configured peripherals */
34      MX_GPIO_Init();
35
36      /* USER CODE BEGIN 2 */
37      OSTaskCreate(Task_farsight_LED,(void *) 0,
38                  (OS_STK *) &startup_task_stk[STARTUP_TASK_STK_SIZE-
39 1],
40                  (INT8U) STARTUP_TASK_PRIO);
41      OSStart();
42      /* USER CODE END 2 */
43
44  }
45  /* USER CODE BEGIN 4 */
46  static void Task_farsight_LED(void *p_arg)
47  {
48      while(1)
49      {
50          HAL_GPIO_WritePin(GPIOF,GPIO_PIN_8,GPIO_PIN_SET);
51          OSTimeDly(500);
52          HAL_GPIO_WritePin(GPIOF,GPIO_PIN_8,GPIO_PIN_RESET);;
53          OSTimeDly(500);
54      }

```



```
54 }  
55 /* USER CODE END 4 */
```

3.19 UCOSII 操作系统创建多任务实验

3.19.1 【实验目的】

通过本次实验了解 UCOSII 这种可以基于 ROM 运行的、可裁剪的、抢占式、实时多任务内核；熟悉怎么通过 UCOSII 操作系统创建多个简单任务的流程。

3.19.2 【实验环境】

嵌入式 ARM 实验箱

ST-Link 仿真器

RealView MDK5.21 集成开发软件

PC 机 XP、Window7/8 (32/64bit)

串口调试助手

3.19.3 【实验内容】

通过 UCOSII 操作系统创建 2 个简单任务，每个任务分别让不同的 LED 灯以不同的频率闪烁。

3.19.4 【实验原理】

SYSTICK 时钟介绍：

运行多个任务的时候，需要用到时间片切换，也就是说，一个时间片用完之后，就切换到另外一个任务执行一个时间片的时间，那么这里时间片该如何确定呢？在 UCOSII 中，我们使用了 systick 定时器和 UCOSII 共用。首先我们简单介绍下 ucos 的时钟，ucos 运行需要一个系统时钟节拍（类似“心跳”），而这个节拍是固定的（由 OS_TICKS_PER_SEC 设置），比如 10ms（设置：OS_TICKS_PER_SEC



= 100 即可)，在 STM32 下面，一般是由 systick 来提供这个节拍，也就是 systick 要设置为 10ms 中断一次，为 UCOSII 提供时钟节拍，而且这个时钟一般是不能被打断的（否则就不准了）。

CM3 内核的处理器，内部包含了一个 SysTick 定时器，SysTick 是一个 24 位的倒计数定时器，当计到 0 时，将从 RELOAD 寄存器中自动重装载定时初值。只要不把它在 SysTick 控制及状态寄存器中的使能位清除，就永不停息。SysTick 在《STM32 的参考手册》（V10.0 版本）里面介绍的很简单，其详细介绍，请参阅《Cortex-M3 权威指南》第 133 页。在以下的代码中我们就利用 STM32 的内部 SysTick 来实现延时的，这样既不占用中断，也不占用系统定时器。

因为在 UCOSII 下 systick 一旦设定之后就不能再被随意更改，对延时的设计来说，如果利用 systick 来做延时微秒或者毫秒的延时，就必须想办法进行整合和计算了，要凑足我们想延时的单位数值；以延时 us 为例，比如希望延时 100 个 us，先计算好这段延时需要等待的 systick 计数次数，这里为 100/9（假设系统时钟为 72Mhz，那么 systick 是 24 位的，用 24 位无法表示 72M 的主频，计算了一下，每 systick 增加 1 相当于主频次数运

3.19.5 【实验步骤】

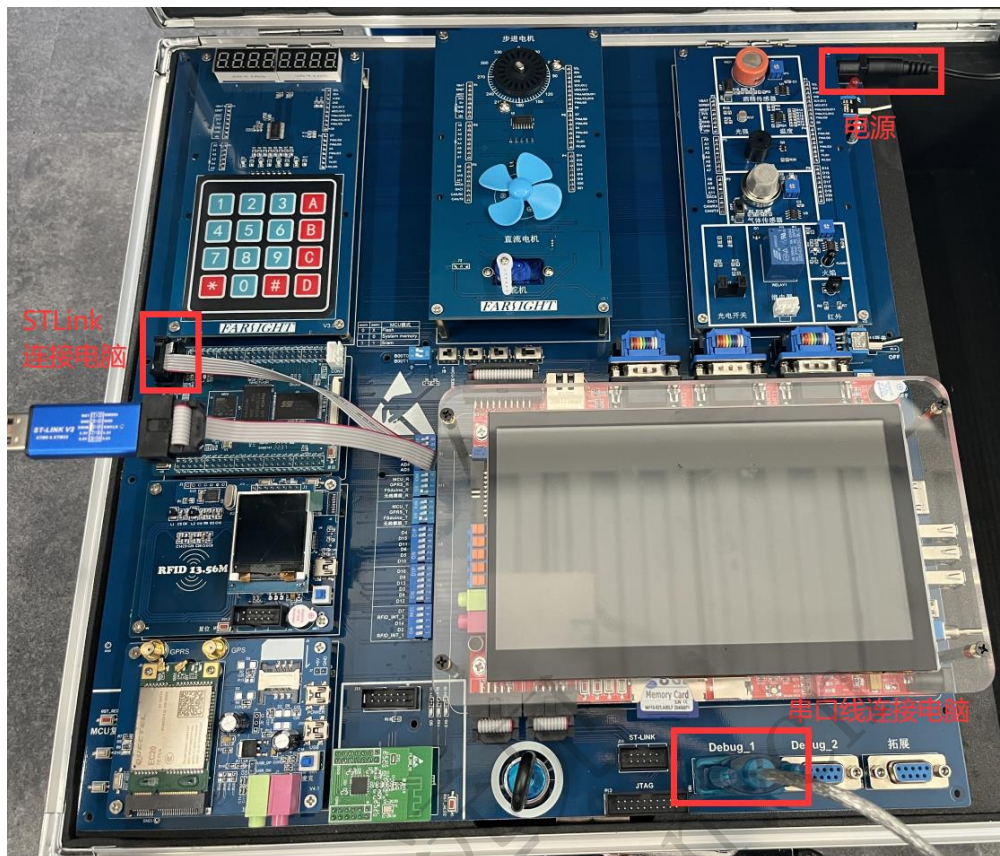
(1) 打开工程文件，路径为

【[华清远见-嵌入式 ARM 实验箱资料-III\华清远见-STM 系列资料\Cortex-M4 部分\程序源码](#)

[\ucosii_mulit_task】](#)

(2) 对工程进行编译调试

(3) 开发板连接



- (4) 往开发板下载程序，具体步骤参照 1.4.4 节
- (5) 开发板上电查看实验结果

3.19.6 【实验结果】

在代码编译成功之后，我们通过下载代码到开发板，可以看到 2 个 LED 灯都是以不同的频率闪烁，说明 2 个任务在分别控制各自的 LED 灯。

3.19.7 【实验参考代码】

这里只提供部分重要代码，如想查看部分函数里的详细实现，请打开工程源码，在 μ Vision IDE for ARM 软件里进行查看和操作，这样查找方便清晰、有条理、并可以随意跟进。

```
1      /* USER CODE BEGIN Includes */
2      #include "ucos_ii.h"
3      /* USER CODE END Includes */
4
```



```

5  /* Private variables -----*/
6
7  /* USER CODE BEGIN PV */
8  /* Private variables -----*/
9  #define APP_TASK_START_PRIO          4
10 #define TASK1_PRIO                    5
11 #define TASK2_PRIO                    6
12
13 #define APP_TASK_START_STK_SIZE       64
14 #define TASK1_STK_SIZE                 128
15 #define TASK2_STK_SIZE                 128
16
17 static OS_STK App_TaskStartStk[APP_TASK_START_STK_SIZE];
18 static OS_STK task1_stk[TASK1_STK_SIZE];
19 static OS_STK task2_stk[TASK2_STK_SIZE];
20
21 #define D1_RCC                        RCC_AHB1Periph_GPIOF
22 #define D1_GPIO                       GPIOF
23 #define D1_GPIO_PIN                   GPIO_PIN_7
24 #define D1_ONOFF(x)                   HAL_GPIO_WritePin(GPIOF,GPIO_PIN_7,x)
25
26 #define D2_RCC                        RCC_AHB1Periph_GPIOF
27 #define D2_GPIO                       GPIOF
28 #define D2_GPIO_PIN                   GPIO_PIN_8
29 #define D2_ONOFF(x)                   HAL_GPIO_WritePin(GPIOF,GPIO_PIN_8,x)
30 static void App_TaskStart (void *p_arg);
31 static void task1(void *p_arg);
32 static void task2(void *p_arg);
33
34 OS_EVENT *Sem1;
35 /* USER CODE END PV */
36
37 /* Private function prototypes -----*/
38 void SystemClock_Config(void);
39
40 /* USER CODE BEGIN PFP */
41 /* Private function prototypes -----*/
42
43 /* USER CODE END PFP */
44
45 /* USER CODE BEGIN 0 */
46
47 /* USER CODE END 0 */

```



```

48
49 int main(void)
50 {
51
52     /* USER CODE BEGIN 1 */
53     OSInit();
54     /* USER CODE END 1 */
55
56     /* MCU Configuration-----
57 --*/
58
59     /* Reset of all peripherals, Initializes the Flash interface and the Systick
60 . */
61     HAL_Init();
62
63     /* Configure the system clock */
64     SystemClock_Config();
65
66     /* Initialize all configured peripherals */
67     MX_GPIO_Init();
68     /* USER CODE BEGIN 2 */
69     D1_ONOFF(GPIO_PIN_SET);
70     D2_ONOFF(GPIO_PIN_RESET);
71     OSTaskCreate ((void ( *) (void *)) App_TaskStart,                //???è??
72 ?'ú??µ?????
73                 (void *) 0,                //è???aê??'
74 DDê±£?'?µY??è???µ?2?êyµ?????
75                 (OS_STK *) &App_TaskStartStk[APP_TASK_START_STK_SIZE - 1],
76 //·????è???µ????µ????¥???? 'ó¥?ò??µY??
77                 (INT8U) APP_TASK_START_PRIO);
78     OSStart();
79     /* USER CODE END 2 */
80
81 }
82
83 /** System Clock Configuration
84 */
85 void SystemClock_Config(void)
86 {
87
88     RCC_OscInitTypeDef RCC_OscInitStruct;
89
90     __PWR_CLK_ENABLE();

```




```

91
92     __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE1);
93
94     RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
95     RCC_OscInitStruct.HSIState = RCC_HSI_ON;
96     RCC_OscInitStruct.HSICalibrationValue = 16;
97     RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;
98     HAL_RCC_OscConfig(&RCC_OscInitStruct);
99
100    HAL_SYSTICK_Config(HAL_RCC_GetHCLKFreq() / 1000);
101
102    HAL_SYSTICK_CLKSourceConfig(SYSTICK_CLKSOURCE_HCLK);
103
104    /* SysTick_IRQn interrupt configuration */
105    HAL_NVIC_SetPriority(SysTick_IRQn, 0, 0);
106
107 }
108
109 /* USER CODE BEGIN 4 */
110 static void App_TaskStart (void *p_arg)
111 {
112     (void) p_arg;
113     \
114     /*Ë1?üucos μ?í3??è???*/
115     #if (OS_TASK_STAT_EN > 0)
116         /*í3??è???3?è??~o~*/
117         OSStatInit();
118     #endif
119     #if (OS_SEM_EN > 0)
120         Sem1 = OSSemCreate(1);
121         if(Sem1 != (void *)0)
122         {
123
124         }
125     #endif
126     /*?`á? ???μ?è???*/
127     OSTaskCreate(task1, (void *)0, &task1_stk[TASK1_STK_SIZE - 1], TASK1_PRI0);
128     OSTaskCreate(task2, (void *)0, &task2_stk[TASK2_STK_SIZE - 1], TASK2_PRI0);
129     OSTaskDel(OS_PRI0_SELF);
130 }
131 static void task1(void *p_arg)
132 {
133     INT8U err;

```



```

134     while(1)
135     {
136         /* apply for sem*/
137         OSSemPend(Sem1, 0, &err);
138         D1_ONOFF(GPIO_PIN_RESET);
139         OSTimeDlyHMSM(0, 0, 0, 200);
140         D1_ONOFF(GPIO_PIN_SET);
141         OSTimeDlyHMSM(0, 0, 0, 200);
142         /*free sem*/
143         OSSemPost(Sem1);
144     }
145 }
146
147 static void task2(void *p_arg)
148 {
149     INT8U err;
150     while(1)
151     {
152         /* apply for sem*/
153         OSSemPend(Sem1, 0, &err);
154         D2_ONOFF(GPIO_PIN_RESET);
155         OSTimeDlyHMSM(0, 0, 0, 1000);
156         D2_ONOFF(GPIO_PIN_SET);
157         OSTimeDlyHMSM(0, 0, 0, 1000);
158         /*free sem*/
159         OSSemPost(Sem1);
160     }
161 }
162
163 /* USER CODE END 4 */

```

3.20 UCOSII 操作系统-信号量

3.20.1 【实验目的】

了解信号量的概念，熟悉使用信号量的资源。

3.20.2 【实验环境】

嵌入式 ARM 实验箱



ST-Link 仿真器

RealView MDK5.21 集成开发软件

PC 机 XP、Window7/8 (32/64bit)

串口调试助手

3.20.3 【实验内容】

编写 UCOSII 操作系统使用信号量控制任务的执行程序，程序调试编译好之后，下载到开发板，按复位键运行，观察实验现象。

3.20.4 【实验原理】

信号量介绍：

1965 年，荷兰学者 Dijkstra 提出了利用信号量机制解决进程同步问题，信号量正式成为有效的进程同步工具，现在信号量机制被广泛的用于单处理机和多处理机系统以及计算机网络中。信号量 (Semaphore)，有时被称为信号灯，是在多线程环境下使用的一种设施，它负责协调各个线程，以保证它们能够正确、合理的使用公共资源。

信号量通过一个计数器控制对共享资源的访问，信号量的值是一个非负整数，所有通过它的线程都会将该整数减一。如果计数器大于 0，则访问被允许，计数器减 1；如果为 0，则访问被禁止，所有试图通过它的线程都将处于等待状态。计数器计算的结果是允许访问共享资源的通行证。因此，为了访问共享资源，线程必须从信号量得到通行证，如果该信号量的计数大于 0，则此线程获得一个通行证，这将导致信号量的计数递减，否则，此线程将阻塞直到获得一个通行证为止。当此线程不再需要访问共享资源时，它释放该通行证，这导致信号量的计数递增，如果另一个线程等待通行证，则那个线程将在那时获得通行证。

信号量的 PV 操作：



P 原语操作的动作

- (1) S 减 1;
- (2) 若 S 减 1 后仍大于或等于零, 则进程继续执行;
- (3) 若 S 减 1 后小于零, 则该进程被阻塞后进入与该信号相对应的队列中, 然后转进程调度。

V 原语操作的动作

- (1) S 加 1;
- (2) 若相加结果大于零, 则进程继续执行;

信号量 S 是一个整数, S 大于等于零是代表可供并发进程使用的资源实体数, 当 S 小于零时表示正在等待使用临界区的进程数。Dijkstra 同时提出了对信号量操作的 PV 原语, PV 操作对于每一个进程来说, 都只能进行一次, 而且必须成对使用。在 PV 原语执行期间不允许有中断的发生。3) 若相加结果小于或等于零, 则从该信号的等待队列中唤醒一等待进程, 然后再返回原进程继续执行或转进程调度。

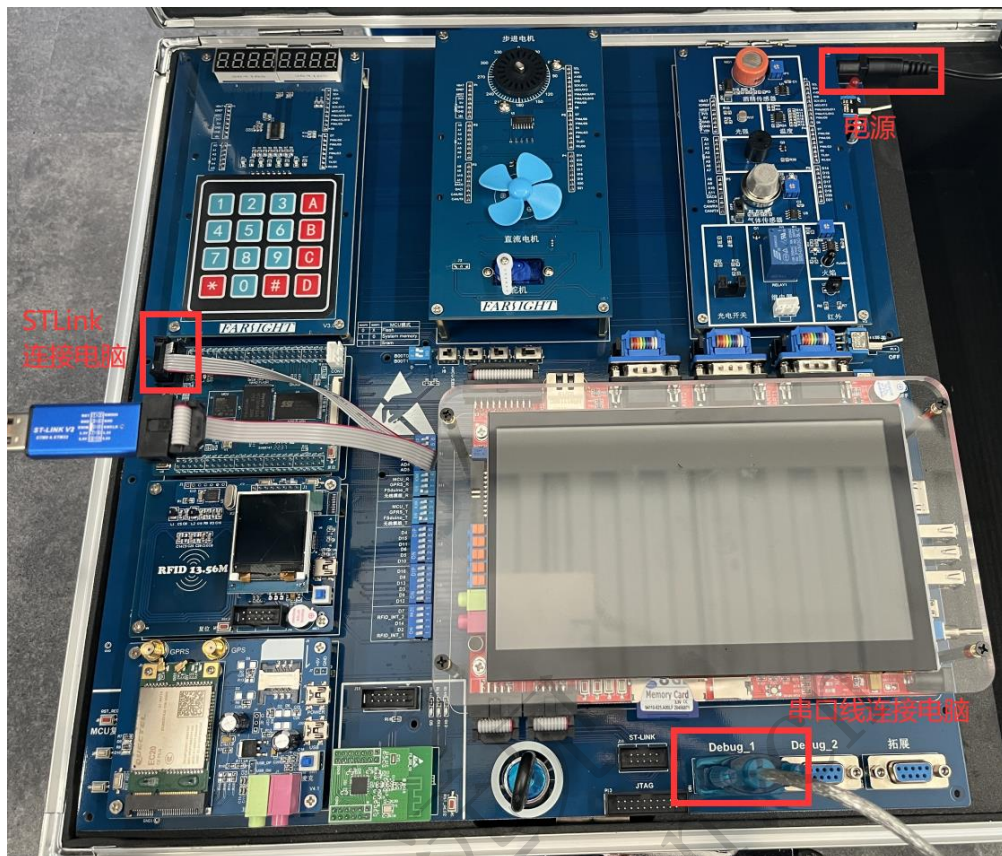
3.20.5 【实验步骤】

- (1) 打开工程文件, 路径为

【[华清远见-嵌入式 ARM 实验箱资料-III\华清远见-STM 系列资料\Cortex-M4 部分\程序源码](#)

[\ucosii_sem】](#)

- (2) 对工程进行编译调试
- (3) 开发板连接



- (4) 往开发板下载程序，具体步骤参照 1.4.4 节
- (5) 开发板上电查看实验结果

3.20.6 【实验结果】

程序调试编译好之后，将程序下载到开发板，按复位键运行程序，观察实验现象。正常情况下，2 个 L E D 灯不停循环亮灭。

3.20.7 【实验参考代码】

这里只提供部分重要代码，如想查看部分函数里的详细实现，请打开工程文件，在 μ Vision IDE for ARM 软件里进行查看和操作，这样查找方便清晰、有条理、并可以随意跟进。

```

1      /* USER CODE BEGIN Includes */
2      #include "ucos_ii.h"
3      /* USER CODE END Includes */
4

```



```

5  /* Private variables ----- */
6
7  /* USER CODE BEGIN PV */
8  /* Private variables ----- */
9  #define APP_TASK_START_PRI0          4
10 #define TASK1_PRI0                    5
11 #define TASK2_PRI0                    6
12
13 #define APP_TASK_START_STK_SIZE       64
14 #define TASK1_STK_SIZE                 128
15 #define TASK2_STK_SIZE                 128
16
17 static OS_STK App_TaskStartStk[APP_TASK_START_STK_SIZE];
18 static OS_STK task1_stk[TASK1_STK_SIZE];
19 static OS_STK task2_stk[TASK2_STK_SIZE];
20
21 #define D1_RCC            RCC_AHB1Periph_GPIOF
22 #define D1_GPIO           GPIOF
23 #define D1_GPIO_PIN       GPIO_PIN_7
24 #define D1_ONOFF(x)       HAL_GPIO_WritePin(GPIOF,GPIO_PIN_7,x)
25
26 #define D2_RCC            RCC_AHB1Periph_GPIOF
27 #define D2_GPIO           GPIOF
28 #define D2_GPIO_PIN       GPIO_PIN_8
29 #define D2_ONOFF(x)       HAL_GPIO_WritePin(GPIOF,GPIO_PIN_8,x)
30 static void App_TaskStart (void *p_arg);
31 static void task1(void *p_arg);
32 static void task2(void *p_arg);
33
34 OS_EVENT *Sem1;
35 /* USER CODE END PV */
36
37 /* Private function prototypes ----- */
38 void SystemClock_Config(void);
39
40 /* USER CODE BEGIN PFP */
41 /* Private function prototypes ----- */
42
43 /* USER CODE END PFP */
44
45 /* USER CODE BEGIN 0 */
46
47 /* USER CODE END 0 */

```



```

48
49 int main(void)
50 {
51
52     /* USER CODE BEGIN 1 */
53     OSInit();
54     /* USER CODE END 1 */
55
56     /* MCU Configuration-----*/
57
58     /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
59     HAL_Init();
60
61     /* Configure the system clock */
62     SystemClock_Config();
63
64     /* Initialize all configured peripherals */
65     MX_GPIO_Init();
66     /* USER CODE BEGIN 2 */
67     D1_ONOFF(GPIO_PIN_SET);
68     D2_ONOFF(GPIO_PIN_RESET);
69     OSTaskCreate ((void ( *) (void *)) App_TaskStart, (void *) 0,
70     (OS_STK *) &App_TaskStartStk[APP_TASK_START_STK_SIZE - 1],
71     (INT8U) APP_TASK_START_PRIO);
72     OSStart();
73     /* USER CODE END 2 */
74
75 }
76
77 /** System Clock Configuration
78 */
79 void SystemClock_Config(void)
80 {
81
82     RCC_OscInitTypeDef RCC_OscInitStruct;
83
84     __PWR_CLK_ENABLE();
85
86     __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE1);
87
88     RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
89     RCC_OscInitStruct.HSISState = RCC_HSI_ON;
90     RCC_OscInitStruct.HSICalibrationValue = 16;

```




```

91     RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;
92     HAL_RCC_OscConfig(&RCC_OscInitStruct);
93
94     HAL_SYSTICK_Config(HAL_RCC_GetHCLKFreq() / 1000);
95
96     HAL_SYSTICK_CLKSourceConfig(SYSTICK_CLKSOURCE_HCLK);
97
98     /* SysTick_IRQn interrupt configuration */
99     HAL_NVIC_SetPriority(SysTick_IRQn, 0, 0);
100
101 }
102
103 /* USER CODE BEGIN 4 */
104 static void App_TaskStart (void *p_arg)
105 {
106     (void) p_arg;
107     #if (OS_TASK_STAT_EN > 0)
108         OSStatInit();
109     #endif
110     #if (OS_SEM_EN > 0)
111         Sem1 = OSSemCreate(1);
112         if(Sem1 != (void *)0)
113         {
114         }
115     #endif
116     OSTaskCreate(task1, (void *)0, &task1_stk[TASK1_STK_SIZE - 1], TASK1_PRIO);
117     OSTaskCreate(task2, (void *)0, &task2_stk[TASK2_STK_SIZE - 1], TASK2_PRIO);
118     OSTaskDel(OS_PRIO_SELF);
119 }
120 static void task1(void *p_arg)
121 {
122     INT8U err;
123     while(1)
124     {
125         /* apply for sem*/
126         OSSemPend(Sem1, 0, &err);
127         D1_ONOFF(GPIO_PIN_RESET);
128         OSTimeDlyHMSM(0, 0, 0, 200);
129         D1_ONOFF(GPIO_PIN_SET);
130         OSTimeDlyHMSM(0, 0, 0, 200);
131         /*free sem*/
132         OSSemPost(Sem1);
133     }

```



```

134 }
135
136 static void task2(void *p_arg)
137 {
138     INT8U err;
139     while(1)
140     {
141         /* apply for sem*/
142         OSSemPend(Sem1, 0, &err);
143         D2_ONOFF(GPIO_PIN_RESET);
144         OSTimeDlyHMSM(0, 0, 0, 1000);
145         D2_ONOFF(GPIO_PIN_SET);
146         OSTimeDlyHMSM(0, 0, 0, 1000);
147         /*free sem*/
148         OSSemPost(Sem1);
149     }
150 }
151 /* USER CODE END 4 */
152
153
154
155
156
157

```

3.21 UCOSIII 操作系统移植

3.21.1 【实验目的】

了解 ucosiii 的移植过程，了解 ucosii 与 ucosiii 的工作机制区别。

3.21.2 【实验环境】

嵌入式 ARM 实验箱

ST-Link 仿真器

RealView MDK5.21 集成开发软件



PC 机 XP、Window7/8 (32/64bit)

串口调试助手

3.21.3 【实验内容】

编写 UCOSIII 操作系统使用信号量控制任务的执行程序，程序调试编译好之后，下载到开发板，按复位键运行，观察实验现象，移植过程和 ucossii 类似。

3.21.4 【实验原理】

实时多任务操作系统 uCOS-III 的特点

- uCOS-III 是一个全新的实时内核，源于世界上最流行的实时内核 uC/OS-II，除了提供熟悉的一系列系统服务，全面修订了 API 接口，使 uC/OS-III 更直观，更容易使用。该产品可以广泛应用于通信，工业控制，仪器仪表，汽车电子，消费电子，办公自动化设备等的设计开发。
- uCOS-III 是一个抢占的多任务内核，支持优先级相同的任务轮询调度。它可以移植到许多不同的 CPU 架构。uC/OS-III 是专为嵌入式系统设计，可以与应用程序代码一起固化到 ROM 中。
- uCOS-III 可在运行时配置实时操作系统。所有内核对象，如任务，堆栈，信号量，事件标志组，消息队列，消息数量，互斥信号量，内存分区和定时器，由用户在运行时进行分配。这可以防止在编译的时候分配过多资源。
- uCOS-III 允许有任意数量的任务，信号量，互斥信号量，事件标志，消息队列，定时器和内存分区（仅受限于处理器可用的 RAM 大小）。
- uCOS-III 添加了许多非常实用的功能，如：可嵌套互斥信号量，可嵌套任务暂停，不需要信号量可发信号给任务，不需要消息队列可发送消息给任务，等待多个内核对象，针对'errno'或其他任务的特定状况的任务注册，内置的性能测量，死锁预防，用户定义的钩函数等。
- uCOS-III 还内置了支持内核感知调试。允许内核感知调试器以用户友好的方式检测和显示



- uC/OS-III 的变量和数据结构，也允许 uC/Probe 在运行时显示和改变变量。
- μ COS-III 是可以抢占的多任务内核，始终运行进入就绪态的最重要的任务。 μ C/OS-III 支持无限数量的任务，并允许在运行时，监测堆栈增长的任务。它还支持无限数量的优先级。然而，通常情况下，对于大多数应用，32 至 256 个不同的优先级是足够的。

对于今天的设计，特别有用的是具有同等优先级的轮转调度的任务。 μ C/OS-III 允许多个任务运行在同一优先级，每一个任务运行由用户指定的时间片。每个任务可以定义自己的时间单元，如果其完整的时间单元并不是必需的，每个任务可以放弃时间片。 μ C/OS-III 还允许无限数量的内核对象，如任务，信号量，互斥，信号旗，消息队列，计时器和内存分区。 μ C/OS-III 大部分是运行时可以配置。

μ C/OS-III 提供接近零的中断停用时间。 μ C/OS-III 有一些内部数据结构和变量，需要获得原子访问权（不能够被打断的）。这些关键区域的保护由锁调度，而不是由禁用中断实现。中断被禁用的时钟周期几乎为零，确保了实时操作系统将能够响应一些最快的中断源。

允许任务使用同一优先级。对同一优先级的多个任务，采用时间片调度法。

可以有任意多的任务，任意多的信号量（semaphore）、互斥型信号量（mutex）、事件标志（event flag）、消息队列（queue）、定时器（timer）和任意分配的存储块容量（仅受限于用户 CPU 可以使用的 RAM 量）

可嵌套的互斥型信号量，可嵌套的任务挂起。

向无信号量请求的任务发送信号量，向无消息队列请求的任务发送消息。

任务可被内核的多个元素挂起（多重挂起）。

增加为其他任务的状态或“出错代码”服务的任务。

内在的性能测试。不仅能得到每个任务的最长关中断时间和最长禁止调度时间，还能得到系统的最长关中断时间和最长禁止调度时间。

选择和确定优先级的算法可以用汇编语言写，以发挥一些有特殊指令的 CPU 的优势，如：置位和复



位指令；计数器清零（CLZ）；找出第一个不为零位（FF1）指令等。

访问临界资源的方法由关中断改为给调度器上锁的方式，使得内核关中断的时钟周期数几乎为零，保证了 $\mu\text{C}/\text{OS-III}$ 能以最快的速度响应中断。

Ucosii 和 ucosiii 的区别：

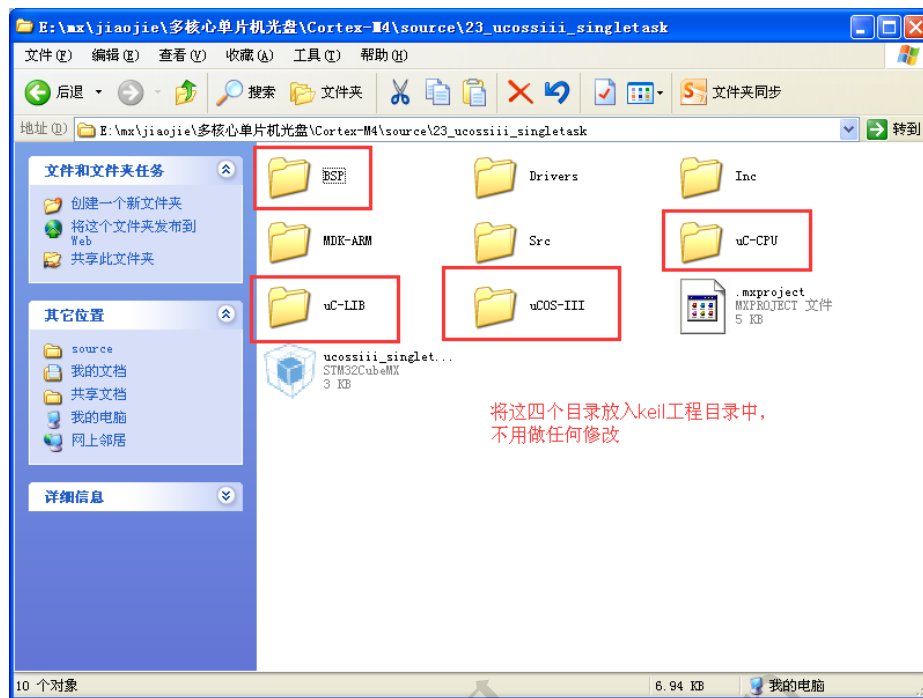
ucos ii 是原来只有 0~63 个优先级，而且优先级不能重复，ucos iii 允许几个任务使用同一个优先级，在同一个优先级里面，支持时间片调度法；

ucos iii 允许用户在程序运行中动态配置实时操作系统内核资源，比如，任务、任务栈、信号量、事件标志组、消息队列、消息数、互斥型信号量、存储块划分和定时器，可以在程序运行中变更。这样，用户可以避免在程序编译过程中出现资源不够分配的问题。在资源复用上，也做了一些改进。

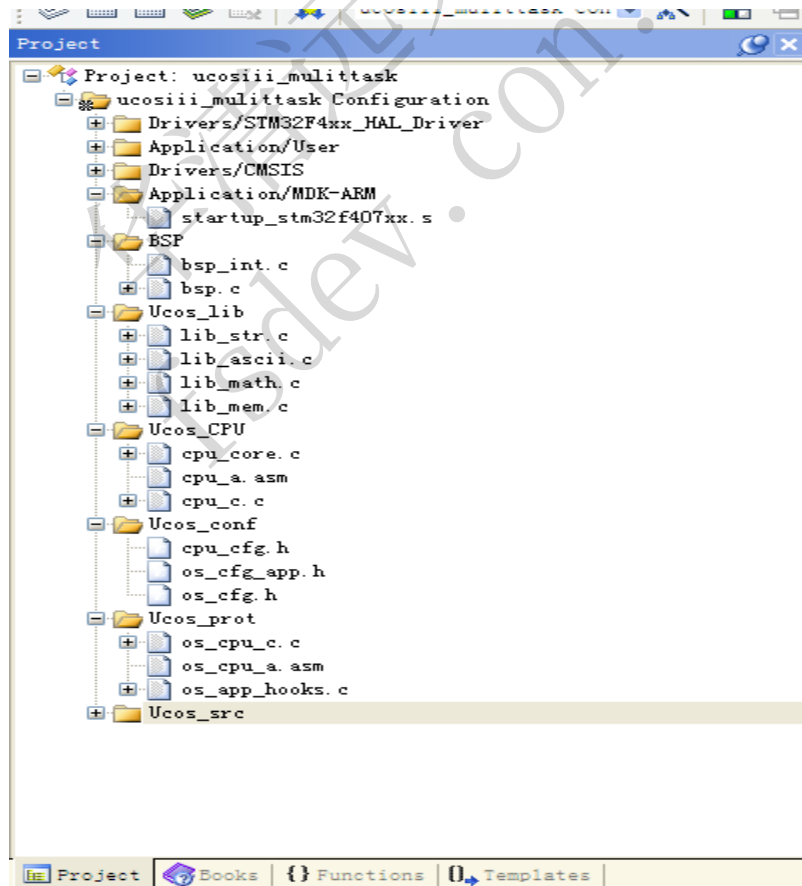
$\mu\text{C}/\text{OS-II}$ 中，最多任务数有 64 个，到了版本 2.82 以后是 256 个， $\mu\text{C}/\text{OS-III}$ 中，用户可以由任意多的任务、任意多的信号量、互斥型信号量、事件标志、消息列表、定时器和任意分配的存储块容量，仅受限于用户 CPU 可以使用的 RAM 量。

3.21.5 【实验步骤】

- (1) 新建一个 keil 工程。
- (2) 将我们的 Ucos_III 源码放入工程目录中。



- (3) 将 Ucos_III 源码文件加入工程中，工程目录如下图。



- (4) 修改部分为 打开 startup_stm32f10xmd.s 文件



```
DCD PendSV_Handler ; PendSV Handler
DCD SysTick_Handler ; SysTick Handler
```

改为

```
DCD OS_CPU_PendSVHandler ; PendSV Handler
DCD OS_CPU_SysTickHandler ; SysTick Handler
```

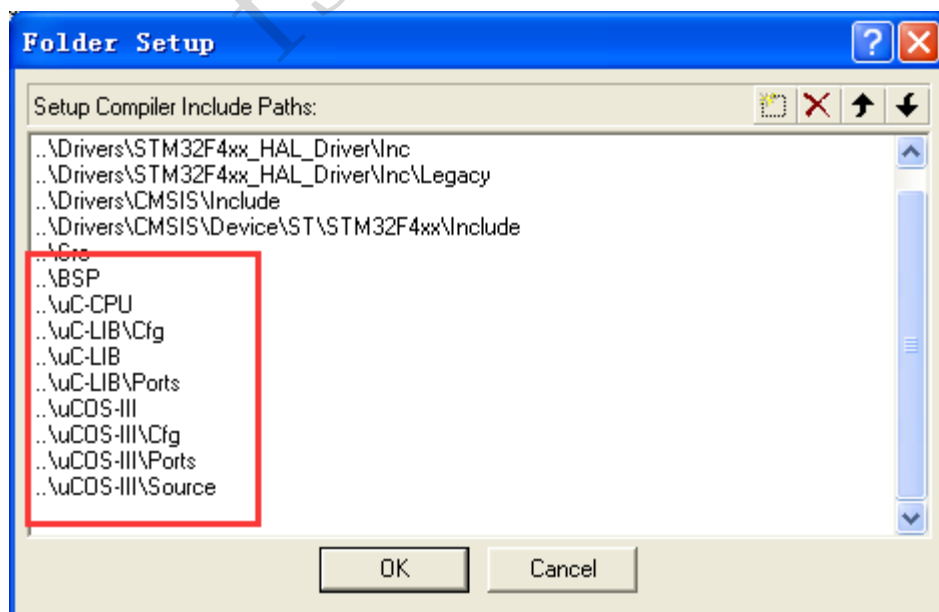
以及

```
ENDP
PendSV_Handler PROC
EXPORT PendSV_Handler [WEAK]
B .
ENDP
SysTick_Handler PROC
EXPORT SysTick_Handler [WEAK]
B .
ENDP
```

改为

```
OS_CPU_PendSVHandler PROC
EXPORT OS_CPU_PendSVHandler [WEAK]
B .
ENDP
OS_CPU_SysTickHandler PROC
EXPORT OS_CPU_SysTickHandler [WEAK]
B .
ENDP
```

- (5) 将头文件目录指定, 打开 option for target/C/C++/Include Path.





3.21.6 【实验结果】

编译成功，无报错，并且在调试的时候，不会死卡在一块不动。

3.22 UCOSIII 操作系统创建 1 个任务实验

3.22.1 【实验目的】

通过本次实验了解 UCOSIII 这种可以基于 ROM 运行的、可裁剪的、抢占式、实时多任务内核；熟悉怎么通过 UCOSII 操作系统创建一个简单任务的流程。

3.22.2 【实验环境】

嵌入式 ARM 实验箱

ST-Link 仿真器

RealView MDK5.21 集成开发软件

PC 机 XP、Window7/8 (32/64bit)

串口调试助手

3.22.3 【实验内容】

本次实验通过 UCOSIII 操作系统创建一个简单任务，这个任务的工作就是实现 LED 灯闪烁操作。

3.22.4 【实验原理】

通过 UCOSIII 操作系统创建一个简单任务，这个任务的工作就是实现 LED 灯闪烁操作。

3.22.5 【实验步骤】

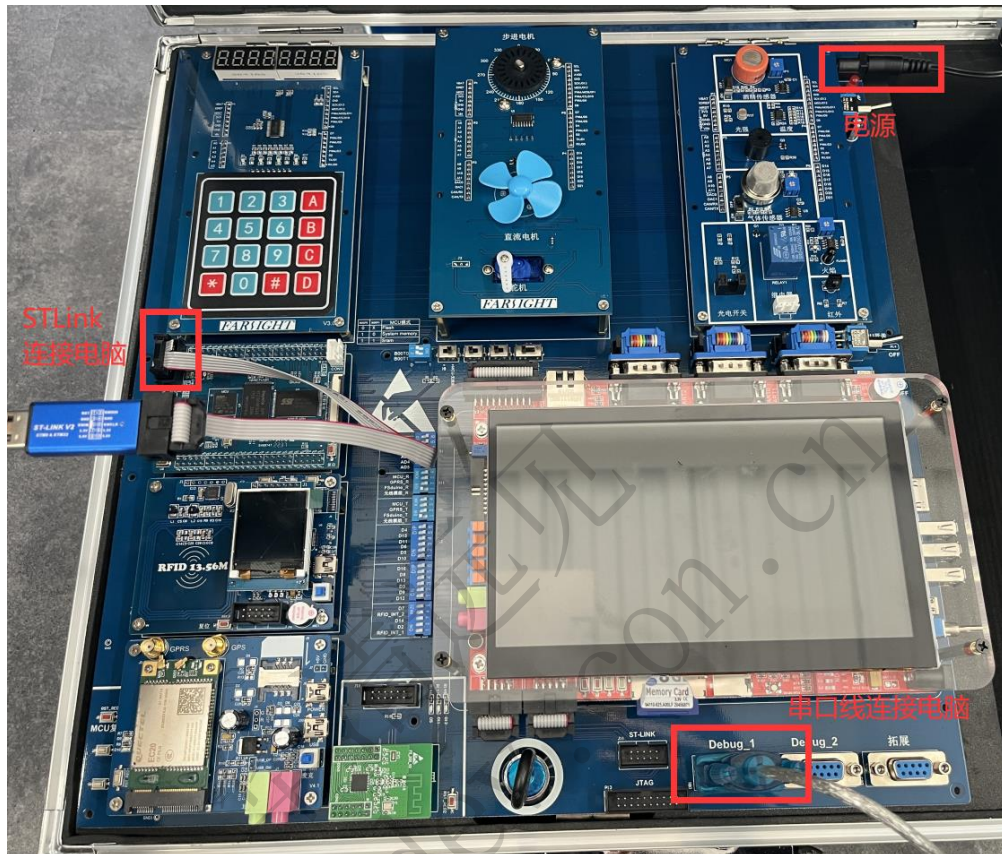
(1) 打开工程文件，路径为

[【华清远见-嵌入式 ARM 实验箱资料-III\华清远见-STM 系列资料\Cortex-M4 部分\程序源码](#)



`\ucossiii_singletask`】

- (2) 对工程进行编译调试
- (3) 开发板连接



- (4) 往开发板下载程序，具体步骤参照 1.4.4 节
- (5) 开发板上电查看实验结果

3.22.6 【实验结果】

在程序编译成功之后，将程序下载到开发板，可以看到 D1 灯以固定的频率闪烁，说明任务

`Task_farsight_LED ()` 已经正常运行了！

3.22.7 【实验参考代码】

这里只提供部分重要代码，如想查看部分函数里的详细实现，请打开工程源码，在 `µVision IDE for`



ARM 软件里进行查看和操作，这样查找方便清晰、有条理、并可以随意跟进。

```

1      /* USER CODE BEGIN Includes */
2      #include "includes.h"
3      /* USER CODE END Includes */
4
5      /* Private variables -----*/
6
7      /* USER CODE BEGIN PV */
8      /* Private variables -----*/
9      static OS_TCB starup_task_TCB;
10     #define STARTUP_TASK_PRIO      4
11     #define STARTUP_TASK_STK_SIZE 128
12     static CPU_STK starup_task_stk[STARTUP_TASK_STK_SIZE];
13
14     OS_ERR err;
15     /* USER CODE END PV */
16
17     /* Private function prototypes -----*/
18     void SystemClock_Config(void);
19
20     /* USER CODE BEGIN PFP */
21     /* Private function prototypes -----*/
22     static void Task_farsight_LED(void *p_arg);
23     /* USER CODE END PFP */
24
25     /* USER CODE BEGIN 0 */
26
27     /* USER CODE END 0 */
28
29     int main(void)
30     {
31
32         /* USER CODE BEGIN 1 */
33         OSInit(&err);
34         /* USER CODE END 1 */
35
36         /* MCU Configuration-----*/
37
38         /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
39         HAL_Init();
40
41         /* Configure the system clock */

```



```

42     SystemClock_Config();
43
44     /* Initialize all configured peripherals */
45     MX_GPIO_Init();
46
47     /* USER CODE BEGIN 2 */
48     OSTaskCreate((OS_TCB *)&starup_task_TCB,
49                 (CPU_CHAR *)"Task1 Start",
50                 (OS_TASK_PTR )Task_farsight_LED,
51                 (void *)0,
52                 (OS_PRIO      ) STARTUP_TASK_PRIO,
53                 (CPU_STK *)&starup_task_stk[0],
54                 (CPU_STK_SIZE)12,
55                 (CPU_STK_SIZE)128,
56                 (OS_MSG_QTY  )0,
57                 (OS_TICK     )0,
58                 (void *)0,
59                 (OS_OPT      )(OS_OPT_TASK_STK_CHK | OS_OPT_TASK_STK_CLR),
60                 (OS_ERR *)&err);
61
62     OSStart(&err);
63     /* USER CODE END 2 */
64
65     /* Infinite loop */
66
67 }
68
69 /** System Clock Configuration
70 */
71 void SystemClock_Config(void)
72 {
73
74     RCC_OscInitTypeDef RCC_OscInitStruct;
75
76     __PWR_CLK_ENABLE();
77
78     __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE1);
79
80     RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
81     RCC_OscInitStruct.HSIState = RCC_HSI_ON;
82     RCC_OscInitStruct.HSICalibrationValue = 16;
83     RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;
84     HAL_RCC_OscConfig(&RCC_OscInitStruct);

```



```

85
86     HAL_SYSTICK_Config(HAL_RCC_GetHCLKFreq() / 1000);
87
88     HAL_SYSTICK_CLKSourceConfig(SYSTICK_CLKSOURCE_HCLK);
89
90     /* SysTick_IRQn interrupt configuration */
91     HAL_NVIC_SetPriority(SysTick_IRQn, 0, 0);
92 }
93
94 /* USER CODE BEGIN 4 */
95 static void Task_farsight_LED(void *p_arg)
96 {
97     while(1)
98     {
99         HAL_GPIO_WritePin(GPIOF, GPIO_PIN_8, GPIO_PIN_SET);
100        OSTimeDly(500, OS_OPT_TIME_DLY, &err);
101        HAL_GPIO_WritePin(GPIOF, GPIO_PIN_8, GPIO_PIN_RESET);
102        OSTimeDly(500, OS_OPT_TIME_DLY, &err);
103    }
104 }
105 /* USER CODE END 4 */
106

```

3.23 UCOSIII 操作系统创建多任务实验

3.23.1 【实验目的】

通过本次实验了解 UCOSIII 这种可以基于 ROM 运行的、可裁剪的、抢占式、实时多任务内核；熟悉怎么通过 UCOSII 操作系统创建多个简单任务的流程。

3.23.2 【实验环境】

嵌入式 ARM 实验箱

ST-Link 仿真器

RealView MDK5.21 集成开发软件

PC 机 XP、Window7/8 (32/64bit)



串口调试助手

3.23.3 【实验内容】

通过 UCOSIII 操作系统创建 2 个简单任务，每个任务分别让不同的 LED 灯以不同的频率闪烁。

3.23.4 【实验原理】

这里的多任务实验和 ucossii 原理一样，只不过所用的 api 函数有所改变。

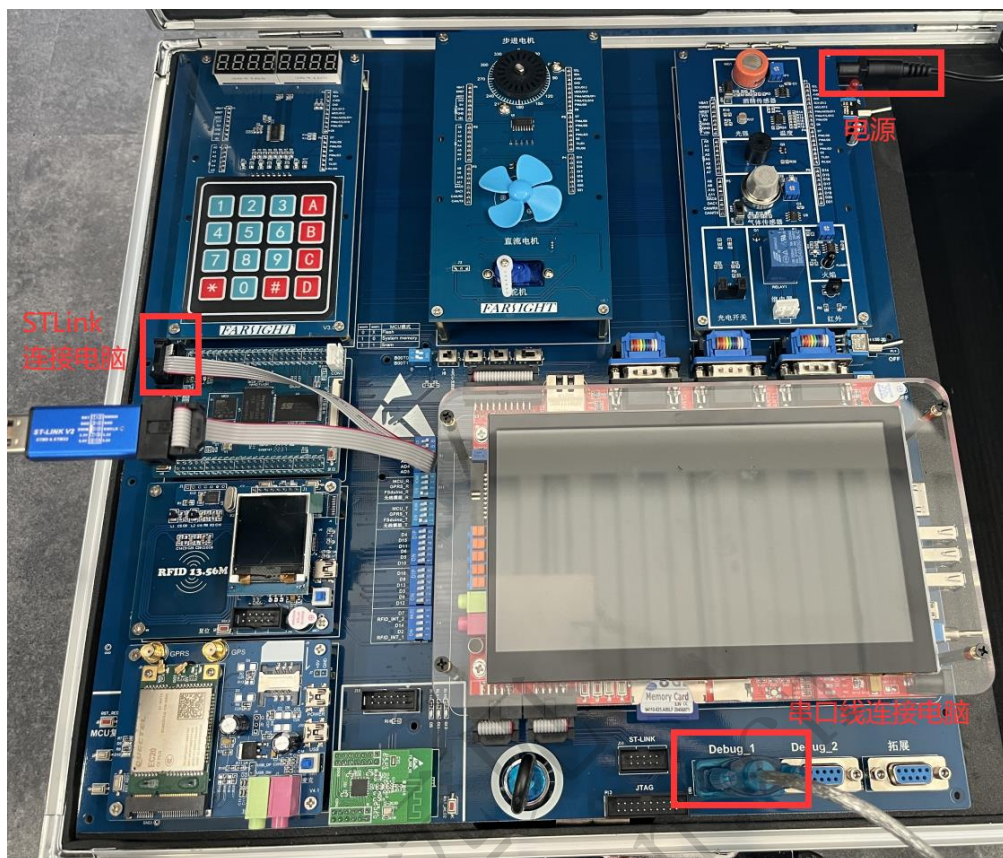
3.23.5 【实验步骤】

- (1) 打开工程文件，路径为

【[华清远见-嵌入式 ARM 实验箱资料-III\华清远见-STM 系列资料\Cortex-M4 部分\程序源码](#)

[\20_ucosiii_mulittask】](#)

- (2) 对工程进行编译调试
- (3) 开发板连接



- (4) 往开发板下载程序，具体步骤参照 1.4.4 节
- (5) 开发板上电查看实验结果

3.23.6 【实验结果】

程序调试编译好之后，将程序下载到开发板，按复位键运行程序，观察实验现象。正常情况下，2 个 L E D 灯不停循环亮灭。与 uc0sii 多任务实验结果一样说明成功。

3.23.7 【实验参考代码】

这里只提供部分重要代码，如想查看部分函数里的详细实现，请打开工程源码，在 μ Vision IDE for ARM 软件里进行查看和操作，这样查找方便清晰、有条理、并可以随意跟进。

```
1      /* USER CODE BEGIN Includes */
2      #include "includes.h"
3      /* USER CODE END Includes */
4
```




```

5  /* Private variables -----*/
6
7  /* USER CODE BEGIN PV */
8  /* Private variables -----*/
9  #define D1_ONOFF(x)      HAL_GPIO_WritePin(GPIOF,GPIO_PIN_7,x)
10 #define D2_ONOFF(x)      HAL_GPIO_WritePin(GPIOF,GPIO_PIN_8,x)
11
12 static OS_TCB  starup_task_TCB;
13 static OS_TCB  task1_TCB;
14 static OS_TCB  task2_TCB;
15
16 #define TASK_START_PRIO          4
17 #define TASK1_PRIO              5
18 #define TASK2_PRIO              6
19
20 #define TASK_START_STK_SIZE      128
21 #define TASK1_STK_SIZE          128
22 #define TASK2_STK_SIZE          128
23
24 static CPU_STK App_TaskStartStk[TASK_START_STK_SIZE];
25 static CPU_STK task1_stk[TASK1_STK_SIZE];
26 static CPU_STK task2_stk[TASK2_STK_SIZE];
27
28 OS_ERR err;
29 /* USER CODE END PV */
30
31 /* Private function prototypes -----*/
32 void SystemClock_Config(void);
33
34 /* USER CODE BEGIN PFP */
35 /* Private function prototypes -----*/
36 static void App_TaskStart (void *p_arg);
37 static void task1(void *p_arg);
38 static void task2(void *p_arg);
39 /* USER CODE END PFP */
40
41 /* USER CODE BEGIN 0 */
42
43 /* USER CODE END 0 */
44
45 int main(void)
46 {
47

```



```

48      /* USER CODE BEGIN 1 */
49      OSInit(&err);
50      /* USER CODE END 1 */
51
52      /* MCU Configuration-----
53      */
54
55      /* Reset of all peripherals, Initializes the Flash interface and the Systick.
56      */
57      HAL_Init();
58
59      /* Configure the system clock */
60      SystemClock_Config();
61
62      /* Initialize all configured peripherals */
63      MX_GPIO_Init();
64
65      /* USER CODE BEGIN 2 */
66      D1_ONOFF(GPIO_PIN_SET);
67      D2_ONOFF(GPIO_PIN_RESET);
68      OSTaskCreate((OS_TCB *)&starup_task_TCB,
69                  (CPU_CHAR *)"Taskall Start",
70                  (OS_TASK_PTR )App_TaskStart,
71                  (void *)0,
72                  (OS_PRIO      ) TASK_START_PRIO,
73                  (CPU_STK *)&App_TaskStartStk[0],
74                  (CPU_STK_SIZE)12,
75                  (CPU_STK_SIZE)128,
76                  (OS_MSG_QTY  )0,
77                  (OS_TICK     )0,
78                  (void *)0,
79                  (OS_OPT       )(OS_OPT_TASK_STK_CHK | OS_OPT_TASK_STK_CLR),
80                  (OS_ERR *)&err);
81      OSStart(&err);
82      /* USER CODE END 2 */
83      while (1)
84      {
85          /* USER CODE END WHILE */
86
87          /* USER CODE BEGIN 3 */
88
89      }
90      /* USER CODE END 3 */

```



```

91
92 }
93
94 /** System Clock Configuration
95 */
96 void SystemClock_Config(void)
97 {
98
99     RCC_OscInitTypeDef RCC_OscInitStruct;
100
101     __PWR_CLK_ENABLE();
102
103     __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE1);
104
105     RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
106     RCC_OscInitStruct.HSIState = RCC_HSI_ON;
107     RCC_OscInitStruct.HSICalibrationValue = 16;
108     RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;
109     HAL_RCC_OscConfig(&RCC_OscInitStruct);
110
111     HAL_SYSTICK_Config(HAL_RCC_GetHCLKFreq() / 1000);
112
113     HAL_SYSTICK_CLKSourceConfig(SYSTICK_CLKSOURCE_HCLK);
114
115     /* SysTick_IRQn interrupt configuration */
116     HAL_NVIC_SetPriority(SysTick_IRQn, 0, 0);
117 }
118
119 /* USER CODE BEGIN 4 */
120 static void App_TaskStart (void *p_arg)
121 {
122     (void) p_arg;
123     /*3?ê??~ucosê±?ó?ú??*/
124     // OS_CPU_SysTickInit();
125     /*ê1?üucos μ?í3??è??*/
126     #if (OS_TASK_STAT_EN > 0)
127     /*í3??è???3?ê??~o~*/
128     OSStatInit();
129     #endif
130     /*?`á? ???μ?è??*/
131     OSTaskCreate((OS_TCB *)&task1_TCB,
132                 (CPU_CHAR *)"Task1 Start",
133                 (OS_TASK_PTR )task1,

```



```

134         (void *)0,
135         (OS_PRIO      ) TASK1_PRIO,
136         (CPU_STK *)&task1_stk[0],
137         (CPU_STK_SIZE)12,
138         (CPU_STK_SIZE)128,
139         (OS_MSG_QTY  )0,
140         (OS_TICK     )0,
141         (void *)0,
142         (OS_OPT      )(OS_OPT_TASK_STK_CHK | OS_OPT_TASK_STK_CLR),
143         (OS_ERR *)&err);
144 OSTaskCreate((OS_TCB *)&task2_TCB,
145             (CPU_CHAR *)"Task2 Start",
146             (OS_TASK_PTR )task2,
147             (void *)0,
148             (OS_PRIO      ) TASK2_PRIO,
149             (CPU_STK *)&task2_stk[0],
150             (CPU_STK_SIZE)12,
151             (CPU_STK_SIZE)128,
152             (OS_MSG_QTY  )0,
153             (OS_TICK     )0,
154             (void *)0,
155             (OS_OPT      )(OS_OPT_TASK_STK_CHK | OS_OPT_TASK_STK_CLR),
156             (OS_ERR *)&err);
157 OSTaskDel(&starup_task_TCB, &err);
158 }
159 static void task1(void *p_arg)
160 {
161     while(1)
162     {
163         D1_ONOFF(GPIO_PIN_RESET);
164         OSTimeDly(200, OS_OPT_TIME_DLY, &err);
165         D1_ONOFF(GPIO_PIN_SET);
166         OSTimeDly(200, OS_OPT_TIME_DLY, &err);
167     }
168 }
169
170 static void task2(void *p_arg)
171 {
172     while(1)
173     {
174         D2_ONOFF(GPIO_PIN_RESET);
175         OSTimeDly(500, OS_OPT_TIME_DLY, &err);
176         D2_ONOFF(GPIO_PIN_SET);

```



```
177     OSTimeDly(500, OS_OPT_TIME_DLY, &err);  
178 }  
179 }  
/* USER CODE END 4 */
```

3.24 UCOSIII 操作系统-信号量

3.24.1 【实验目的】

了解信号量的概念，熟悉使用信号量的资源。

3.24.2 【实验环境】

嵌入式 ARM 实验箱

ST-Link 仿真器

RealView MDK5.21 集成开发软件

PC 机 XP、Window7/8 (32/64bit)

串口调试助手

3.24.3 【实验内容】

编写 UCOSIII 操作系统使用信号量控制任务的执行程序，程序调试编译好之后，下载到开发板，按复位键运行，观察实验现象。

3.24.4 【实验原理】

这里的信号量工作原理与 ucousii 不同，分为阻塞和非阻塞模型，我们需选择阻塞模型，这样才能实现两个任务轮流运行的现象。

3.24.5 【实验步骤】

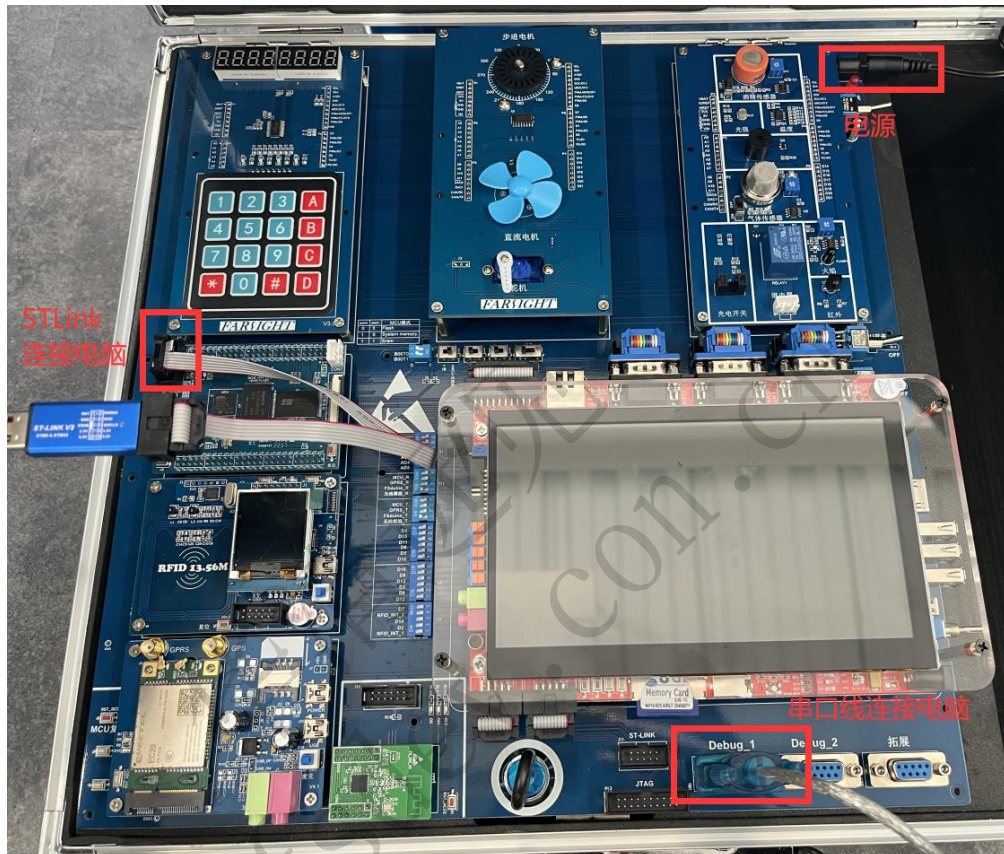
(1) 打开工程文件，路径为



【华清远见-嵌入式 ARM 实验箱资料-III\华清远见-STM 系列资料\Cortex-M4 部分\程序源码

\22_ucosiii_sem】

- (2) 对工程进行编译调试
- (3) 开发板连接



- (4) 往开发板下载程序，具体步骤参照 1.4.4 节
- (5) 开发板上电查看实验结果

3.24.6 【实验结果】

程序调试编译好之后，将程序下载到开发板，按复位键运行程序，观察实验现象。正常情况下，2 个 L E D 灯不停循环亮灭。与 ucossii 信号量实验结果一样说明成功。



3.24.7 【实验参考代码】

这里只提供部分重要代码，如想查看部分函数里的详细实现，请打开工程源码，在μVision IDE for

ARM 软件里进行查看和操作，这样查找方便清晰、有条理、并可以随意跟进。

```

1      /* USER CODE BEGIN Includes */
2      #include "includes.h"
3      /* USER CODE END Includes */
4
5      /* Private variables -----*/
6
7      /* USER CODE BEGIN PV */
8      /* Private variables -----*/
9      #define D1_ONOFF(x)      HAL_GPIO_WritePin(GPIOF,GPIO_PIN_7,x)
10     #define D2_ONOFF(x)      HAL_GPIO_WritePin(GPIOF,GPIO_PIN_8,x)
11
12     static OS_TCB  starup_task_TCB;
13     static OS_TCB  task1_TCB;
14     static OS_TCB  task2_TCB;
15
16     #define TASK_START_PRI0      4
17     #define TASK1_PRI0          5
18     #define TASK2_PRI0          6
19
20     #define TASK_START_STK_SIZE  128
21     #define TASK1_STK_SIZE       128
22     #define TASK2_STK_SIZE       128
23
24     static CPU_STK App_TaskStartStk[TASK_START_STK_SIZE];
25     static CPU_STK task1_stk[TASK1_STK_SIZE];
26     static CPU_STK task2_stk[TASK2_STK_SIZE];
27
28     OS_ERR err, err_sem;
29     OS_SEM sem1;
30     /* USER CODE END PV */
31
32     /* Private function prototypes -----*/
33     void SystemClock_Config(void);
34
35     /* USER CODE BEGIN PFP */
36     /* Private function prototypes -----*/
37     static void App_TaskStart (void *p_arg);

```




```

38 static void task1(void *p_arg);
39 static void task2(void *p_arg);
40 /* USER CODE END PFP */
41
42 /* USER CODE BEGIN 0 */
43
44 /* USER CODE END 0 */
45
46 int main(void)
47 {
48
49     /* USER CODE BEGIN 1 */
50     OSInit(&err);
51     /* USER CODE END 1 */
52
53     /* MCU Configuration-----
54 */
55
56     /* Reset of all peripherals, Initializes the Flash interface and the Systick.
57 */
58     HAL_Init();
59
60     /* Configure the system clock */
61     SystemClock_Config();
62
63     /* Initialize all configured peripherals */
64     MX_GPIO_Init();
65
66     /* USER CODE BEGIN 2 */
67     D1_ONOFF(GPIO_PIN_SET);
68     D2_ONOFF(GPIO_PIN_RESET);
69     OSTaskCreate((OS_TCB *)&starup_task_TCB,
70                 (CPU_CHAR *)"Taskall Start",
71                 (OS_TASK_PTR )App_TaskStart,
72                 (void *)0,
73                 (OS_PRIO    ) TASK_START_PRIO,
74                 (CPU_STK *)&App_TaskStartStk[0],
75                 (CPU_STK_SIZE)12,
76                 (CPU_STK_SIZE)128,
77                 (OS_MSG_QTY  )0,
78                 (OS_TICK     )0,
79                 (void *)0,
80                 (OS_OPT      )(OS_OPT_TASK_STK_CHK | OS_OPT_TASK_STK_CLR),

```



```

81         (OS_ERR *)&err);
82     OSStart(&err);
83     /* USER CODE END 2 */
84 }
85
86 /** System Clock Configuration
87 */
88 void SystemClock_Config(void)
89 {
90
91     RCC_OscInitTypeDef RCC_OscInitStruct;
92
93     __PWR_CLK_ENABLE();
94
95     __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE1);
96
97     RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
98     RCC_OscInitStruct.HSIState = RCC_HSI_ON;
99     RCC_OscInitStruct.HSICalibrationValue = 16;
100    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;
101    HAL_RCC_OscConfig(&RCC_OscInitStruct);
102
103    HAL_SYSTICK_Config(HAL_RCC_GetHCLKFreq() / 1000);
104
105    HAL_SYSTICK_CLKSourceConfig(SYSTICK_CLKSOURCE_HCLK);
106
107    /* SysTick_IRQn interrupt configuration */
108    HAL_NVIC_SetPriority(SysTick_IRQn, 0, 0);
109 }
110
111 /* USER CODE BEGIN 4 */
112 static void App_TaskStart (void *p_arg)
113 {
114     (void) p_arg;
115     /*3?ê??~ucosê±?ó?ú??*/
116     // OS_CPU_SysTickInit();
117     /*ê1?üucos μ?í3??è???*/
118     #if (OS_TASK_STAT_EN > 0)
119     /*í3??è???3?è??~o~*/
120     OSStatInit();
121     #endif
122     #if (OS_CFG_SEM_EN > 0)
123     OSSemCreate(&sem1, (CPU_CHAR *)"sem1", (OS_SEM_CTR)1, &err_sem);

```



```

124     if(err_sem != OS_ERR_NONE)
125     {
126         return ;
127     }
128 #endif
129 /*"áÇ????µ?è???*/
130 OSTaskCreate((OS_TCB *)&task1_TCB,
131             (CPU_CHAR *)"Task1 Start",
132             (OS_TASK_PTR )task1,
133             (void *)0,
134             (OS_PRIO    ) TASK1_PRIO,
135             (CPU_STK *)&task1_stk[0],
136             (CPU_STK_SIZE)12,
137             (CPU_STK_SIZE)128,
138             (OS_MSG_QTY )0,
139             (OS_TICK    )0,
140             (void *)0,
141             (OS_OPT      )(OS_OPT_TASK_STK_CHK | OS_OPT_TASK_STK_CLR),
142             (OS_ERR *)&err);
143 OSTaskCreate((OS_TCB *)&task2_TCB,
144             (CPU_CHAR *)"Task2 Start",
145             (OS_TASK_PTR )task2,
146             (void *)0,
147             (OS_PRIO    ) TASK2_PRIO,
148             (CPU_STK *)&task2_stk[0],
149             (CPU_STK_SIZE)12,
150             (CPU_STK_SIZE)128,
151             (OS_MSG_QTY )0,
152             (OS_TICK    )0,
153             (void *)0,
154             (OS_OPT      )(OS_OPT_TASK_STK_CHK | OS_OPT_TASK_STK_CLR),
155             (OS_ERR *)&err);
156 OSTaskDel(&starup_task_TCB, &err);
157 }
158 static void task1(void *p_arg)
159 {
160     while(1)
161     {
162         OSSemPend(&sem1, 0, OS_OPT_PEND_BLOCKING, NULL, &err_sem);
163
164         D1_ONOFF(GPIO_PIN_SET);
165         OSTimeDly(200, OS_OPT_TIME_DLY, &err);
166         D1_ONOFF(GPIO_PIN_RESET);

```



```

167     OSTimeDly(200, OS_OPT_TIME_DLY, &err);
168
169     OSSemPost(&sem1, OS_OPT_POST_ALL, &err_sem);
170 }
171 }
172
173 static void task2(void *p_arg)
174 {
175     while(1)
176     {
177         OSSemPend(&sem1, 0, OS_OPT_PEND_BLOCKING, NULL, &err_sem);
178
179         D2_ONOFF(GPIO_PIN_SET);
180         OSTimeDly(1000, OS_OPT_TIME_DLY, &err);
181         D2_ONOFF(GPIO_PIN_RESET);
182         OSTimeDly(1000, OS_OPT_TIME_DLY, &err);
183
184         OSSemPost(&sem1, OS_OPT_POST_ALL, &err_sem);
185     }
186 }
187
188 /* USER CODE END 4 */

```

3.25 综合测试例程

3.25.1 【实验目的】

熟悉并掌握，这个工程的实现逻辑，并能够自己创建并开发出更有效率的综合例程

3.25.2 【实验环境】

嵌入式 ARM 实验箱

ST-Link 仿真器

RealView MDK5.21 集成开发软件

PC 机 XP、Window7/8 (32/64bit)

串口调试助手



3.25.3 【实验内容】

通过 LCD 触摸屏控制，监测所有外部模块及器件。

3.25.4 【实验原理】

本实验为综合实验，是将之前我们学到的单独例程尽量集合在一起，考验大家对外设的综合使用能力。

这里使用到的外设，GPIO，定时器，串口，液晶屏，pwm，iic，ADC，SPI，等等，具体的功能实现上面都有例程，这里不再赘述，这里简单介绍一下软件设计结构。

这里使用了多级菜单的设计方法，通过判断触摸屏的位置来给函数指针赋值，从而决定函数的跳转位置。

本程序由于图片内容较多，故将部分图片链接到了外置的 NOR FLASH 中，具体烧写方法见 1.5 STM32F407 外部 NOR FLASH 下载。初始化时候，执行函数名为 desc 的函数，也就是当前的一个源目录画面，如下





当按下的时候,

COBOL Code

```

1  if(( (1  < line_x)  && (line_x < 47) ) && ( (95 < line_y) && (line_y < 150) ))
2
3  {
4
5      touchflag = STEP;
6
7  }
8  switch(touchflag)
9  {
10
11      case BEEP  :
12          BEEP_Process();
13          break;
14
15      case RELAY  :
16          RELAY_Process();
17          break;
18
19      case DCMOTOR  :
20          DCMOTOR_Process();
21          break;
22
23      case STREE  :
24          STREE_Process();
25          break;
26
27      case STEP  :
28          STEP_Process();
29          break;
30
31      default :
32          break ;
33
34  }
35
36  touchflag = 0;

```

以上代码就是改变图标索引值，然后在 main.c 文件的主函数里去修改函数指针，并且执行函数指针



指向的函数。

COBOL Code

```
1   while (1)
2   {
3   /* USER CODE END WHILE */
4
5   /* USER CODE BEGIN 3 */
6       KeyFunPtr = KeyTab[KeyFunIndex].CurrentOperate;
7       (*KeyFunPtr)();
8   /*******/
9   }
```

索引值变为 DCMOTOR，执行函数名为 DCMOTOR_Process()的函数，执行 DCMOTOR_Process()函数，

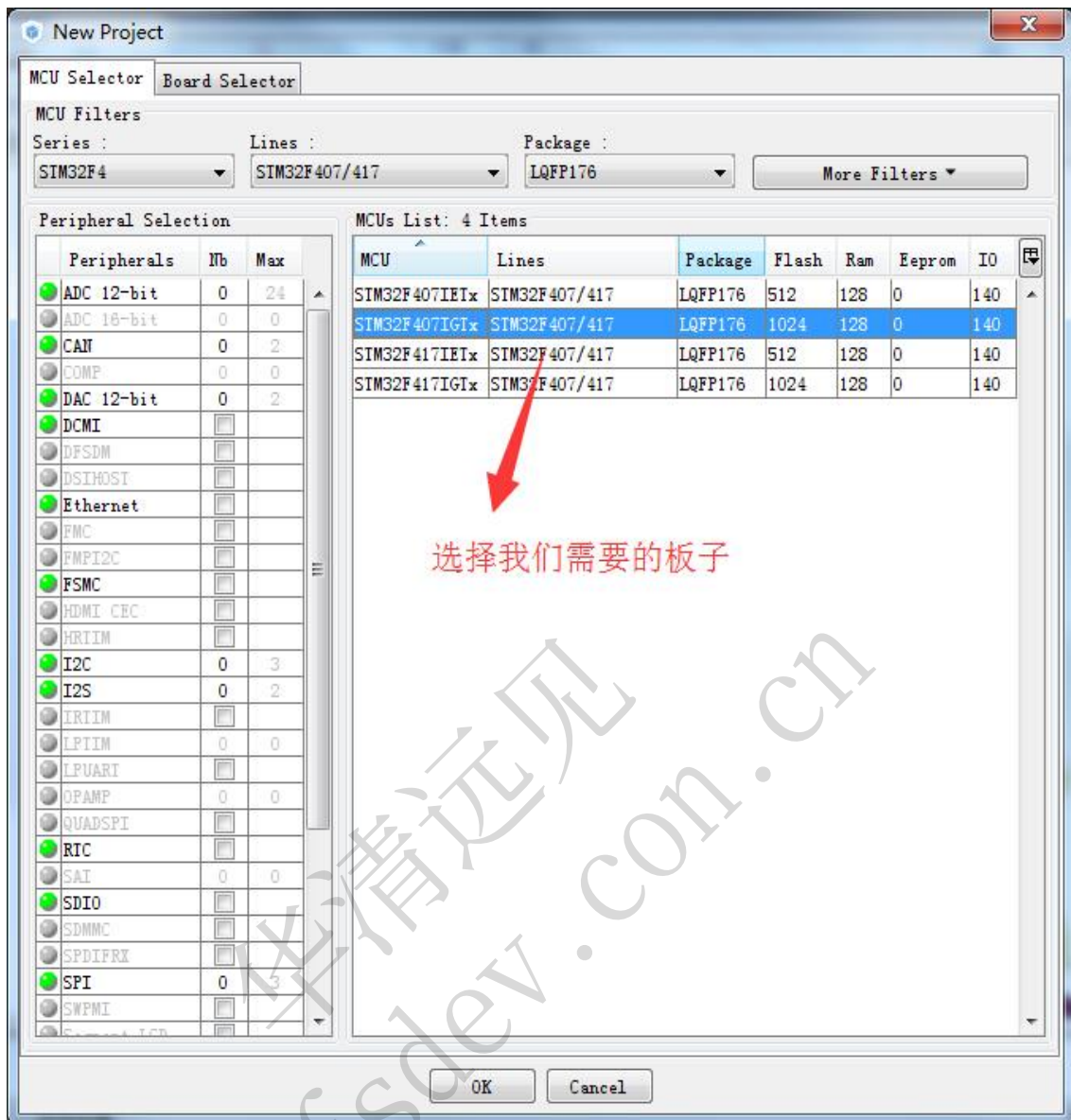
如下图：



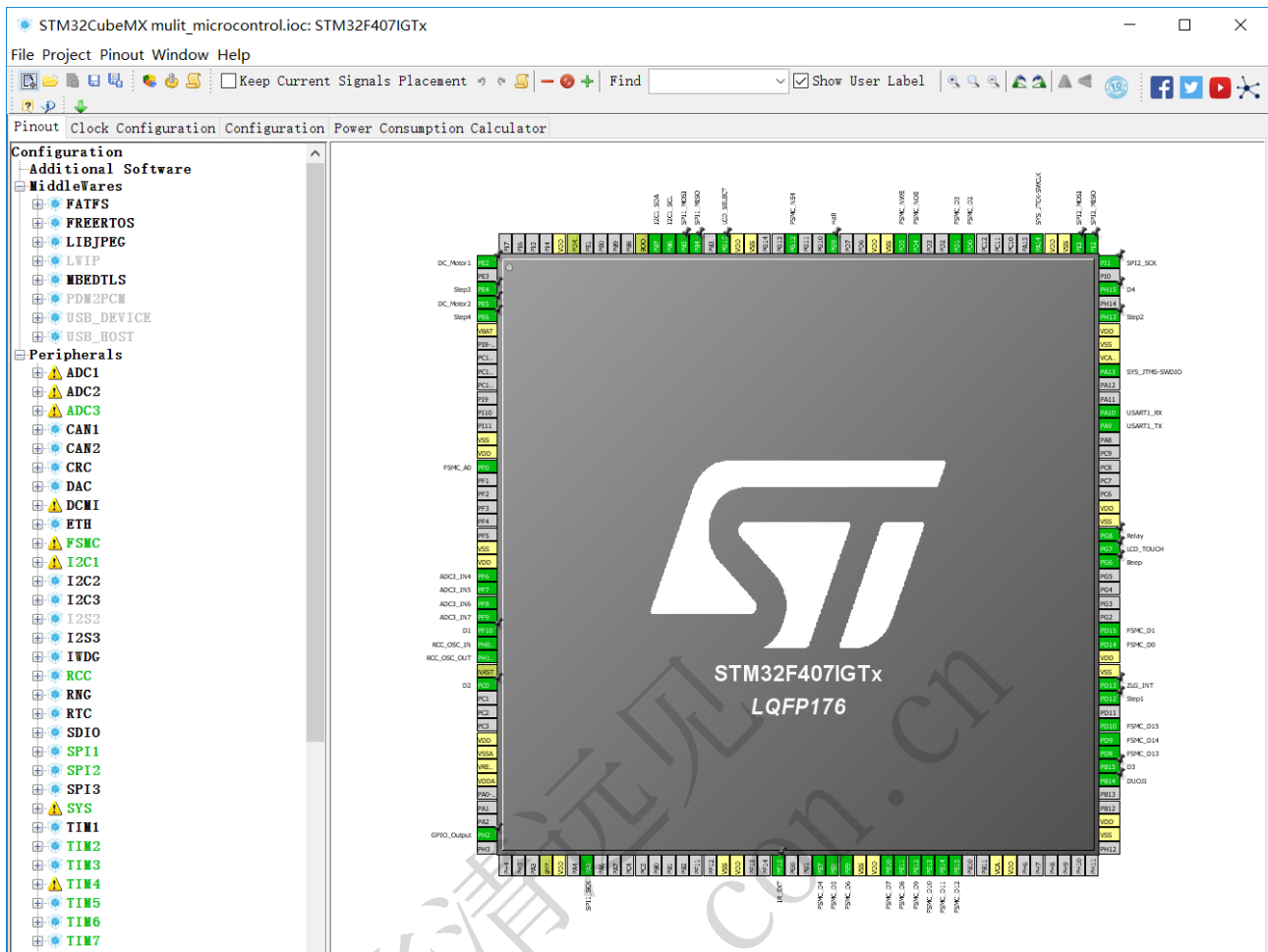
RELAY_Process()函数的执行同理，具体可参考源代码中的内容。

3.25.5 【stm32cubemx 配置介绍】

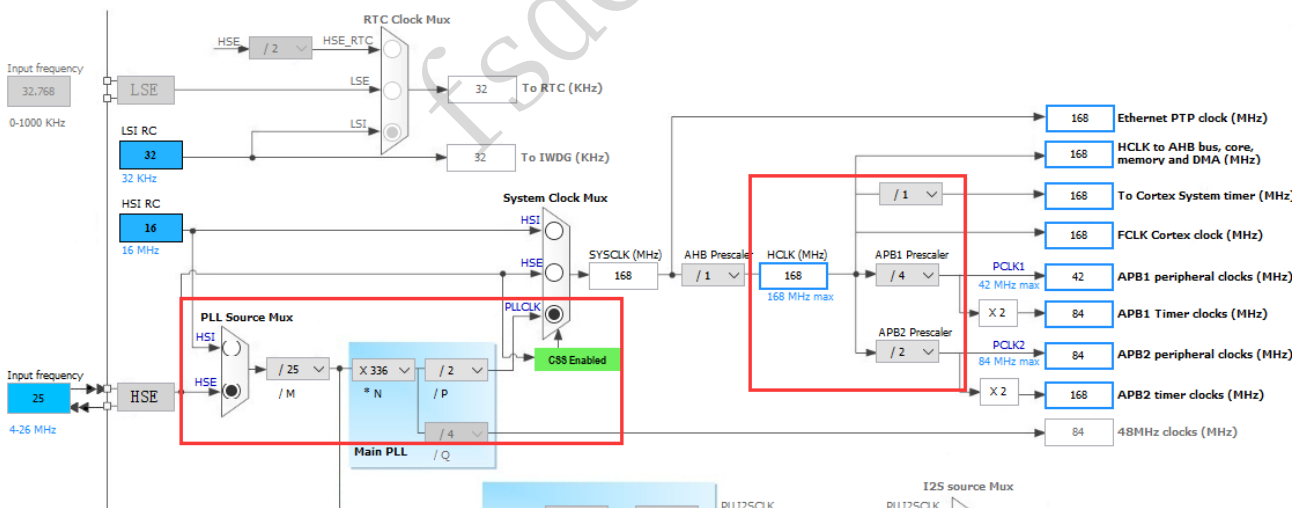
(1) 选择芯片



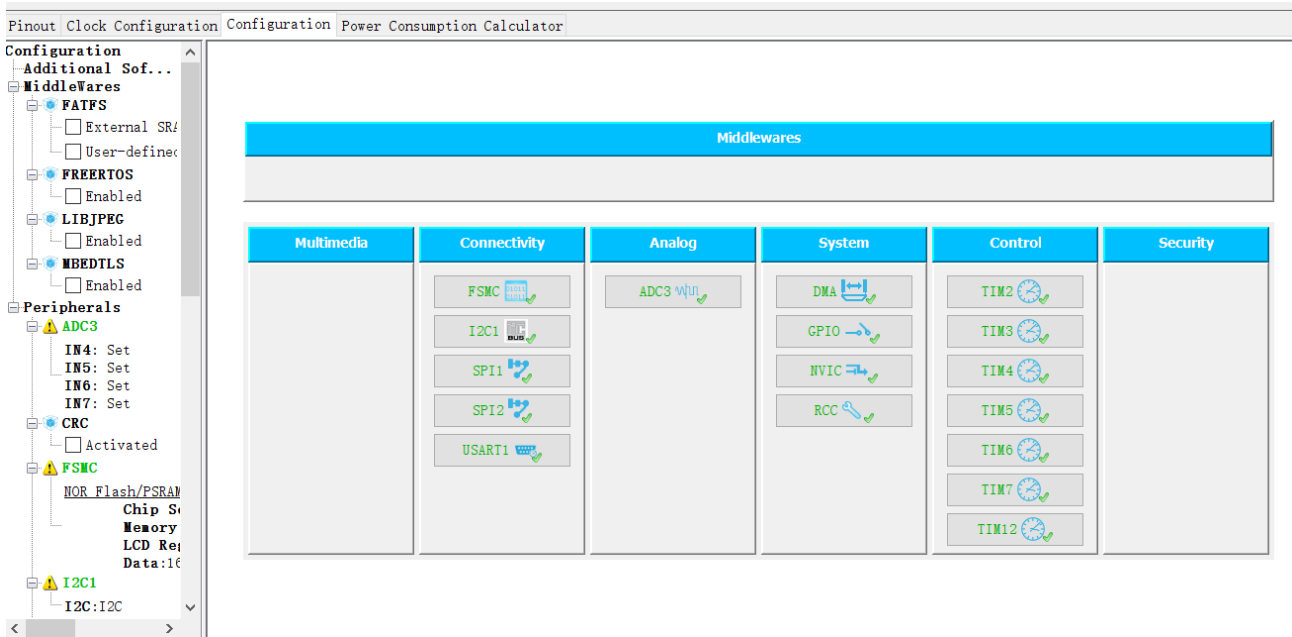
- (2) 对管脚进行配置，具体管脚配置可参照源码工程。



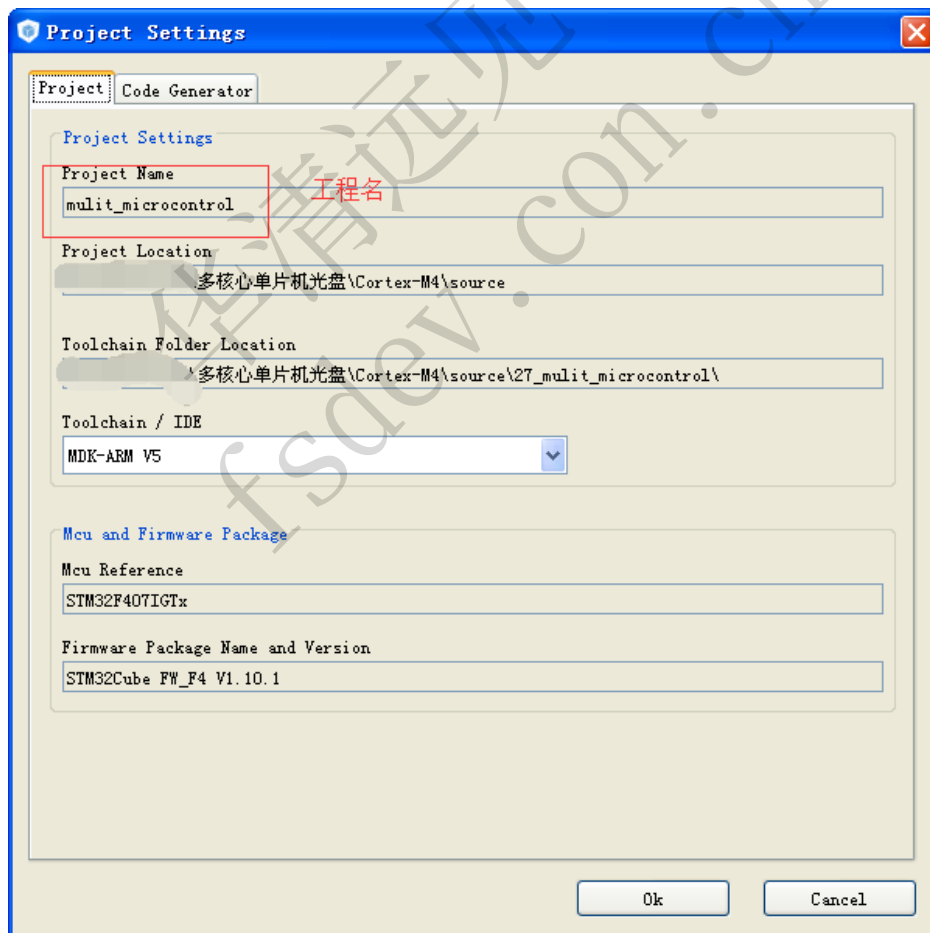
(3) 点击 CLOCK CONFIGURATION 对时钟进行设置



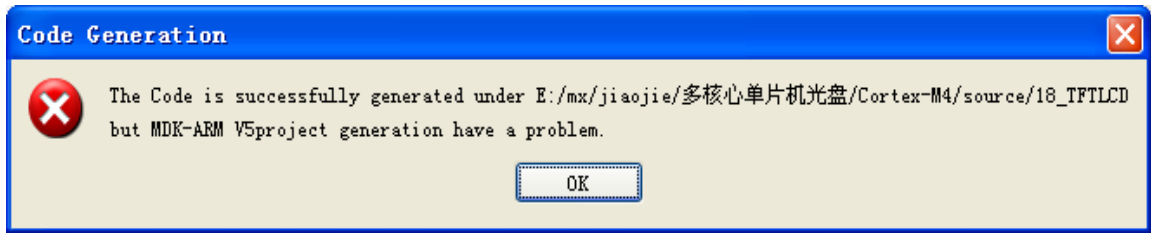
(4) 点击 CONFIGURATION, 详细设置可参照源码工程。



(5) 点击生成工程按钮后，如下进行配置



生成工程，如果路径带中文的话会报如下错误



解决方法就是拷贝到全英文目录下即可。

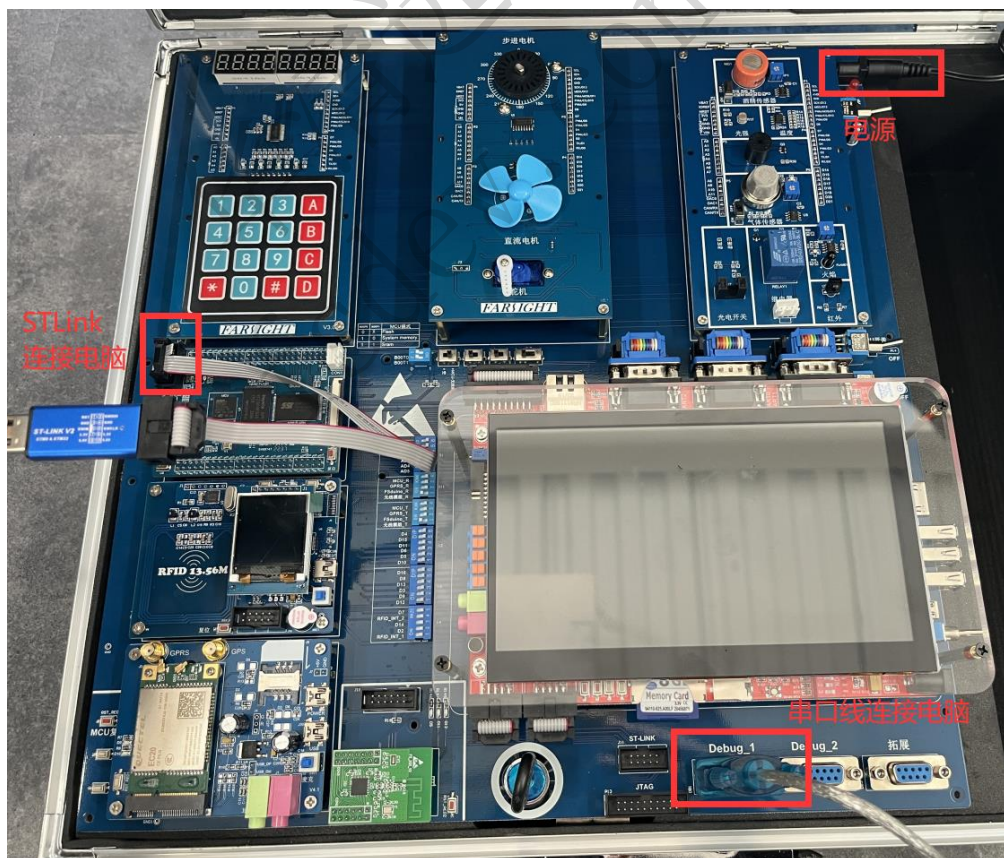
3.25.6 【实验步骤】

- (1) 打开工程文件，路径为

【华清远见-嵌入式 ARM 实验箱资料-III\华清远见-STM 系列资料\Cortex-M4 部分\程序源码

\23_mulit_microcontrol】

- (2) 对工程进行编译调试
- (3) 开发板连接



- (4) 往开发板下载程序，具体步骤参照 1.4.4 节



(5) 开发板上电查看实验结果

3.25.7 【实验结果】

如果成功下载程序后，可以看到如下现象：



需要注意的是，在程序下载后，按下复位键，如果要测试矩阵键盘或者 RFID 模块，那么请先确认底板上的 SW8 拨码开关是否拨到了 M4，如果没有请拨到 M4。因为该开关是切换 M4 和 4412 I2C 总线的开关。

该界面中步进电机、直流电机、舵机、继电器、蜂鸣器等设备需要在触摸屏上点击对应的图标进行控制。其他的设备无需点击，直接显示对应数据。

3.25.8 【实验参考代码】

这里只粘贴部分程序，如果想查看具体的程序实现，请打开工程源码查看。

```
1 int main(void)
2 {
3
```



```

4  /* USER CODE BEGIN 1 */
5
6  /* USER CODE END 1 */
7
8  /* MCU Configuration-----*/
9
10 /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
11 HAL_Init();
12
13 /* Configure the system clock */
14 SystemClock_Config();
15
16 /* Initialize all configured peripherals */
17 MX_GPIO_Init();
18 MX_DMA_Init();
19 MX_ADC3_Init();
20 MX_FSMC_Init();
21 MX_I2C1_Init();
22 MX_SPI1_Init();
23 MX_SPI2_Init();
24 MX_TIM2_Init();
25 MX_TIM3_Init();
26 MX_TIM4_Init();
27 MX_TIM5_Init();
28 MX_TIM6_Init();
29 MX_TIM7_Init();
30 MX_TIM12_Init();
31 MX_USART1_UART_Init();
32
33 /* USER CODE BEGIN 2 */
34
35     lcd_init();
36     HAL_TIM_PWM_Start(&htim12, TIM_CHANNEL_1);
37
38     HAL_TIM_Base_Start_IT(&htim6);
39     desc();
40
41 /* USER CODE END 2 */
42
43 /* Infinite loop */
44 /* USER CODE BEGIN WHILE */
45 while (1)
46 {

```



```
47  /* USER CODE END WHILE */
48
49  /* USER CODE BEGIN 3 */
50
51      KeyFunPtr = KeyTab[KeyFunIndex].CurrentOperate;
52      (*KeyFunPtr)();
53
54  }
55  /* USER CODE END 3 */
56
57 }
```

华清远见
fsdev.con.cn